DSA with Java

Pattern by Pattern

Learn data structures and algorithms the way interviews test them: one pattern at a time, with the idea, a step-by-step dry run, Java code, the questions to expect, and a verified LeetCode practice list after every pattern.

Arrays · Two Pointers · Sliding Window · Binary Search · Linked Lists · Stacks
Trees · Backtracking · Heaps · Graphs · Dynamic Programming · Greedy · Bits

```java
int[] twoSum(int[] a, int target) {
    Map<Integer, Integer> seen = new HashMap<>();
    for (int i = 0; i < a.length; i++) {
        if (seen.containsKey(target - a[i])) return ...;
        seen.put(a[i], i);
    }
```

sreeram

Version basis

All code is Java and was compiled and run on JDK 21; outputs in dry runs are real
Every LeetCode number in the practice lists was checked against the problem page
Prepared for Sreeram

How to use this guide

- **Who this is for.** You know core Java and want to learn data structures and algorithms from the basics up to the advanced patterns that interviews use. The guide is organised by **pattern**, not by data structure.
- **Every pattern chapter follows the same steps:** (1) the idea and how to recognise it, (2) a **dry run** by hand, step by step, (3) the Java code, (4) complexity and common mistakes, (5) **questions to expect** in interviews, (6) a **LeetCode practice list** with problem numbers.
- **How to practise.** Read the concept, do the dry run on paper yourself before reading the code, type the code, then solve the practice problems. Try each problem for 20-30 minutes, then read the idea, close it and solve again the next day. Mark the pattern and the clue that gave it away.
- **Running the code.** Save a listing as `Main.java` and run `java Main.java` (Java 11 or newer). Dry-run tables and terminal output were produced by running the code on Java 21.
- **LeetCode numbers.** Each problem was checked against its LeetCode page. LeetCode renumbers rarely, but if a number does not match, search by the title.

Contents

Part 1: Foundations 4

1.1 How to use this guide and how interviews work 4
1.2 Complexity analysis 6
1.3 The dry-run method 9
1.4 Java toolkit for DSA 12
1.5 Core data structures you must know 18
1.6 Pattern map and how to choose 20

Part 2: Arrays and Strings Patterns 23

2.1 Prefix Sum and Difference Array 23
2.2 Two Pointers 28
2.3 Sliding Window: Fixed Size 34
2.4 Sliding Window: Variable Size 38
2.5 Hash Map and Frequency Counting 43
2.6 Binary search on sorted arrays 48
2.7 Binary search on the answer 53
2.8 Sorting-based problems and merge intervals 58
2.9 Kadane, greedy scans and in-place array tricks 62
2.10 Matrix and string patterns 68

Part 3: Linked Lists, Stacks and Queues 75

3.1 Linked list basics, dummy node and reversal 75
3.2 Fast and slow pointers 81
3.3 Stack patterns 87
3.4 Monotonic stack 92
3.5 Queue, deque and monotonic deque 98

Part 4: Recursion, Backtracking and Trees 105

sreeram

4.1 Recursion and Divide and Conquer . . . 105
4.2 Backtracking: Subsets, Permutations, Combinations . . . 110
4.3 Backtracking on Grids and Constraints . . . 116
4.4 Binary Tree DFS . . . 122
4.5 Binary Tree BFS and Level Order . . . 128
4.6 BST and Tree Construction . . . 133
4.7 Tries . . . 139

Part 5: Heaps and Graphs . . . 146

5.1 Heaps and Top-K . . . 146
5.2 Graph Representation, DFS and BFS . . . 152
5.3 Topological Sort . . . 159
5.4 Union-Find (Disjoint Set Union) . . . 164
5.5 Shortest Paths . . . 170

Part 6: Dynamic Programming, Greedy and Bits . . . 177

6.1 Dynamic programming: the method . . . 177
6.2 DP on grids and paths . . . 183
6.3 Knapsack patterns . . . 188
6.4 DP on strings and sequences . . . 193
6.5 DP on intervals, trees and states . . . 199
6.6 Greedy . . . 204
6.7 Bit manipulation . . . 210

Part 7: Revision and Interview Playbook . . . 216

7.1 Pattern recognition cheat sheet . . . 216
7.2 Mixed interview questions . . . 218
7.3 Mock interview walkthroughs . . . 220
7.4 Complexity and Java cheat sheet . . . 224
7.5 A 14-week study plan . . . 226
7.6 Final checklist and how to practise on LeetCode . . . 228

sreram

PART 1

Foundations

The ground rules: how interviews work, how to count cost, how to trace code by hand, and the required Java subset.

In this part

1.1 How to use this guide and how interviews work
1.2 Complexity analysis
1.3 The dry-run method
1.4 Java toolkit for DSA
1.5 Core data structures you must know
1.6 Pattern map and how to choose

1.1 How to use this guide and how interviews work

Failure in coding interviews is seldom caused by unfamiliarity with a problem; it is more often caused by reliance on memorised solutions, which fail when a single constraint changes. This book is organised around **patterns**: a small set of reusable techniques (two pointers, sliding window, binary search, graph traversal, dynamic programming and others) that together cover the large majority of interview problems. The reader learns to identify the characteristic clue in a problem statement, select the corresponding pattern, and derive the code from a template whose correctness is understood.

Every pattern chapter in the later Parts has the same seven-part structure: the idea and its recognition criteria, a hand-executed dry run, runnable Java code, complexity and common mistakes, expected questions, and a LeetCode practice list. The present Part supplies the shared prerequisites: cost measurement, tracing technique, the Java library subset used throughout, and the underlying data structures.

1.1.1 How to study with this guide

The following procedure is recommended for each pattern chapter.

1. Read "The idea" and the recognition clues, then state the pattern aloud in two sentences without consulting the text.
2. Execute the dry-run table with pencil and paper. Cover the right-hand columns and predict each row before comparing.
3. Type the Java program manually (not paste it), run it, and compare the result with the printed output.
4. Solve at least four problems from the practice table, easiest first. If no progress is made within 25 minutes, read the key idea and repeat the problem from scratch on the following day.
5. Maintain a mistakes log with one line per defect (for example "empty array not handled", "`==` applied to `Integer`"). Defects recur, and the log identifies which ones.

All code in this book targets Java 21 and was executed on that version. Output that depends on the machine (timings, stack depth) is labelled as typical.

1.1.2 The interview, step by step

A coding interview lasts 35 to 45 minutes, and the interviewer evaluates the reasoning process at least as much as the final code. A seven-step structure is the most reliable.

Step	Activity	Statement or artefact	Minutes
1. Clarify	Restate the problem; ask about input size, duplicates, negative numbers, empty input, sorted or not, what to return if no answer	"Can the array be empty? Are values only positive? Is the answer unique?"	3 to 5
2. Examples	Make one normal example and one edge case; work them out by hand	"For [2,7,11,15] and target 9 the answer is indices 0 and 1."	3

freeran

Step	Activity	Statement or artefact	Minutes
3. Brute force	State the obvious solution and its cost, even if it is too slow	"Try every pair: O(n^2) time, O(1) space."	3
4. Optimise	Find the bottleneck, name the pattern or data structure that removes it, state the new cost	"The inner loop only asks 'have I seen target minus x?'. A hash map answers that in O(1)."	5 to 8
5. Code	Write clean code with meaningful names; talk while you type	Narrate the invariant, not the syntax	12 to 15
6. Test	Dry run on the example, then the edge cases; correct defects	"Empty array returns... one element returns..."	5
7. Compl exity	State time and space and say which part dominates	"O(n) time, O(n) space for the map."	2

The seven steps and a time budget for a 45-minute interview.

Coding is only the fifth step. Starting to code early commonly yields a solution to the wrong problem, and the remaining time is spent on repairs. Omitting step 6 leaves defects for the interviewer to discover, which is judged more negatively than discovering them independently.

1.1.3 A complete example of the seven steps

The Two Sum problem illustrates the seven steps. The input is an integer array and a target; the output is the pair of indices of two distinct elements whose sum equals the target. After clarification (exactly one solution exists; an element may not be used twice), the brute force examines every pair with two nested loops. The bottleneck is the inner loop, which evaluates a single predicate: whether the value `target - nums[i]` occurs among the earlier elements. A `HashMap` from value to index evaluates this predicate in O(1) expected time.

Main.java JAVA

```java
import java.util.*;

public class Main {
    // Step 3: brute force, O(n^2) time, O(1) space
    static int[] twoSumBrute(int[] nums, int target) {
        for (int i = 0; i < nums.length; i++)
            for (int j = i + 1; j < nums.length; j++)
                if (nums[i] + nums[j] == target) return new int[]{i, j};
        return new int[]{-1, -1};
    }

    // Steps 4 and 5: one pass with a hash map, O(n) time, O(n) space
    static int[] twoSum(int[] nums, int target) {
        Map<Integer, Integer> seen = new HashMap<>(); // value -> index
        for (int i = 0; i < nums.length; i++) {
            Integer j = seen.get(target - nums[i]);
            if (j != null) return new int[]{j, i};
            seen.put(nums[i], i);
        }
        return new int[]{-1, -1};
    }

    public static void main(String[] args) {
        int[][] inputs = {{2, 7, 11, 15}, {3, 2, 4}, {3, 3}, {5}, {},
                {1_500_000_000, 1_500_000_000}};
        int[] targets = {9, 6, 6, 5, 0, -1_294_967_296};
        for (int k = 0; k < inputs.length; k++) {
            System.out.println(Arrays.toString(inputs[k]) + " target " + targets[k] + " -> brute "
                    + Arrays.toString(twoSumBrute(inputs[k], targets[k])) + ", map "
                    + Arrays.toString(twoSum(inputs[k], targets[k])));
        }
    }
}
```

Run it TERMINAL

```
$ java Main.java
[2, 7, 11, 15] target 9 -> brute [0, 1], map [0, 1]
[3, 2, 4] target 6 -> brute [1, 2], map [1, 2]
[3, 3] target 6 -> brute [0, 1], map [0, 1]
[5] target 5 -> brute [-1, -1], map [-1, -1]
[] target 0 -> brute [-1, -1], map [-1, -1]
[1500000000, 1500000000] target -1294967296 -> brute [0, 1], map [0, 1]
```

Observations on the listing. The map is queried *before* the current element is inserted, which guarantees that an element cannot pair with itself: the single-element input `{5}` returns "not found" although 5 equals the target. The duplicate input `{3, 3}` is handled by the same ordering, since the first 3 is already present when the second is read. The final test exposes an overflow: `1_500_000_000 + 1_500_000_000` exceeds the `int` range and wraps to `-1294967296`, so both implementations report a pair that is mathematically invalid. Published constraints normally exclude this case, but the assumption should be stated explicitly.

1.1.4 Communicating

- **Verbalise reasoning in short statements.** Extended silence conveys no information; when progress stalls, state what has been attempted and what will be tried next.
- **Propose before implementing.** For example: "A hash map is proposed; the extra space is O(n). Is this acceptable?" The interviewer frequently gives a hint at this point, which should be taken.
- **Use descriptive identifiers.** Names such as `left`, `right`, `seen` and `windowSum` are preferable to `a`, `b`, `x` and `tmp`.
- **Reduce the problem when blocked.** Solve a restricted version (sorted input, one dimension, no duplicates), then remove the restriction.
- **Treat hints as part of the exchange.** Hints are normal; the evaluation concerns whether they are used.

INTERVIEW ANGLE

"Walk me through your approach before you write any code." The answer should contain, in order: the brute force and its cost; the observation that exposes the redundant work; the data structure or pattern that removes it; the resulting time and space complexity; and the edge cases to be tested. This takes about two minutes and demonstrates structure.

KEY POINTS

- The method is pattern-based: each chapter gives the clue, a template, a hand trace, Java code, costs and mistakes.
- The interview procedure is: clarify, examples, brute force, optimise, code, test, complexity.
- Coding occupies about one third of the available time; explanation and testing occupy the remainder.
- The brute force is stated first even when it is inefficient; it establishes understanding of the problem and a baseline for improvement.
- A mistakes log records recurring defects and is the most relevant revision material.

1.2 Complexity analysis

Two programs that produce the same correct output may differ in running time by a factor of 10^6. **Complexity analysis** describes this difference without measurement, so that algorithms can be compared before implementation. In interviews it is the principal criterion for judging a proposed solution: the cost of the brute force is compared with the limit implied by the constraints, and the gap determines how much algorithmic improvement is required.

1.2.1 Big-O, Omega and Theta

Cost is expressed as a function of the input size **n** (array length, node count, number of digits). The unit is the basic step (comparison, assignment, array read) rather than seconds, and the quantity of interest is the asymptotic growth of the step count as n increases.

DEFINITION

Big-O, Omega, Theta **O(f(n))**: an asymptotic upper bound. The cost is bounded above by a constant multiple of f(n) for sufficiently large n. **Omega(f(n))**: an asymptotic lower bound. The cost is bounded below by a constant multiple of f(n). **Theta(f(n))**: both bounds hold; f(n) is the exact order of growth.

Two rules convert a step count into a Big-O expression. Constant factors are discarded: `3n + 5` is O(n). Only the dominant term is retained: `n^2 + 100n + 7` is O(n^2). In informal usage O(n) frequently denotes Theta(n), and this is accepted in interviews. The distinction matters for best, average and worst cases: linear search is Omega(1) (match at the first position) but O(n) in the worst case. Unless stated otherwise, the **worst case** is reported.

The cost can be derived from the structure of the code as follows.

- A single loop over n items with O(1) work per iteration: O(n).
- Two consecutive loops: O(n) + O(n) = O(n). Costs of sequential statements add.
- A loop nested in a loop, both over n: O(n^2). Costs of nested statements multiply.
- A loop variable that is halved or doubled in each iteration (`i *= 2`, `n /= 2`): O(log n).
- A loop of n iterations containing a binary search or a sort: O(n log n).
- A recursion that makes two calls on size n-1: O(2^n). A recursion that halves the input and does linear work per level: O(n log n).

The bounds of the inner loop require care. The nested loop in which the inner variable starts at `i + 1` executes n-1, n-2, ..., 0 inner iterations, a total of n(n-1)/2, which is O(n^2). Measurement confirmed this: for n = 1000 the body executed exactly 499500 times, equal to 1000 x 999 / 2. The factor 1/2 vanishes in Big-O notation but remains in the actual running time.

1.2.2 The growth rates you must know

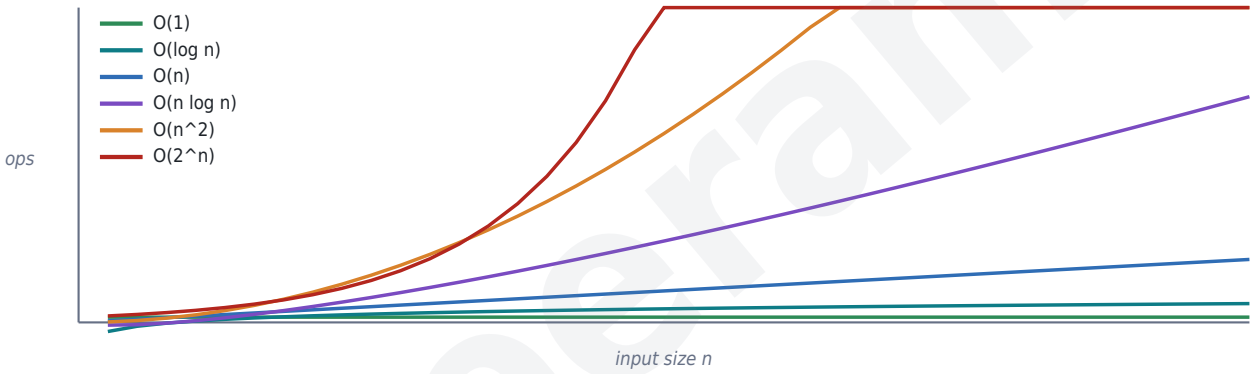


Figure 1.1. The curves separate already at small n. The O(n log n) curve remains close to O(n), whereas O(n^2) and O(2^n) leave the plotted range almost immediately.

Name	Notation	Doubling n does this	Typical source
Constant	O(1)	nothing	array index, hash lookup, stack push
Logarithmic	O(log n)	adds one step	binary search, halving loop, balanced tree walk
Linear	O(n)	doubles the work	one pass, linear search, building a prefix array
Linearithmic	O(n log n)	a bit more than doubles	sorting, heap-based algorithms, divide and conquer
Quadratic	O(n^2)	quadruples	nested loops over all pairs, simple DP over two sequences
Cubic	O(n^3)	8 times	three nested loops, Floyd-Warshall
Exponential	O(2^n)	squares the work	all subsets, naive recursive Fibonacci
Factorial	O(n!)	explodes	all permutations

Growth rates, with typical sources in code.

The table above is complemented by a measured comparison. All figures were obtained in one run on the JDK 21 build machine after a warm-up, taking the best of five repetitions; absolute values differ between machines, but the ratios do not. The linear row sums an `int[]`; the n log n row is `Arrays.sort` on random ints; the log n row is a single binary search on a sorted array (below measurement resolution); the quadratic row is a double loop over all n x n index pairs.

n	O(log n) one search	O(n) sum	O(n log n) sort	O(n^2) double loop
1,000	under 0.001 ms	0.002 ms	0.04 ms	2 ms

sreeram

n	O(log n) one search	O(n) sum	O(n log n) sort	O(n^2) double loop
10,000	under 0.001 ms	0.02 ms	0.8 ms	58 ms
100,000	under 0.001 ms	0.05 ms	7.8 ms	5.6 s
1,000,000	under 0.001 ms	0.19 ms	80 ms	too slow to run
10,000,000	under 0.001 ms	4.1 ms	1.07 s	too slow to run

Measured running time (typical, JDK 21) by input size n.

When n increases from 10,000 to 100,000 (a factor of 10), the quadratic loop slows by a factor of about 100, as O(n^2) predicts. A separate run of the same double loop took 558 ms at n = 30,000 and 6.7 s at n = 100,000. The exponential case is more severe: naive recursive Fibonacci, which makes two recursive calls per invocation, took 6 ms for `fib(30)`, 65 ms for `fib(35)`, 636 ms for `fib(40)` and 7.2 s for `fib(45)`. Each increment of 5 in n multiplies the time by about 10, so `fib(60)` would require hours. This growth motivates dynamic programming, which reduces the recursion to O(n) by memoisation (treated in a later Part).

1.2.3 Reading the constraints

Problem statements usually specify the maximum n. A typical machine performs roughly 10^8 to 10^9 simple operations per second, and this budget indicates the intended complexity. The following correspondence is a heuristic for deciding whether a brute force is acceptable or a better algorithm is required.

Max n	Target	Typical approaches
`n <= 10`	O(n!) or O(n * 2^n)	backtracking over all permutations, bitmask DP
`n <= 20`	O(2^n)	all subsets, meet in the middle, bitmask
`n <= 500`	O(n^3)	triple loops, Floyd-Warshall, interval DP
`n <= 5,000`	O(n^2)	double loops, 2D DP, all-pairs checks
`n <= 10^5 or 10^6`	O(n log n)	sorting, heap, binary search, sweep line, segment tree
`n <= 10^7`	O(n)	one pass, prefix sums, hashing, two pointers
`n up to 10^18`	O(log n) or O(1)	binary search on the answer, math, fast exponentiation

Constraint to target complexity (rule of thumb, assuming about 10^8 simple steps).

Example 1: for an array of up to 10^5 integers and a request to count pairs with a given sum, the O(n^2) brute force requires 10^10 steps, about a hundred times over budget. An O(n log n) or O(n) algorithm is therefore required: sorting with two pointers, or a hash map. Example 2: for an 8 x 8 board with 8 queens, n is small and exponential backtracking is the intended solution. The constraint is a hint embedded in the statement and is read before a solution is designed.

> **TIP**
> When n reaches 10^5 and the answer is a sum of values up to 10^9, the result can exceed the `int` range and `long` is required. Constraints indicate overflow risk as well as time limits.

1.2.4 Amortised analysis

Some operations are inexpensive in most invocations and expensive in a few. The **amortised cost** is the cost per operation averaged over a worst-case sequence of operations; it is a deterministic guarantee and involves no probability. The standard example is the dynamic array (`ArrayList`). An append is O(1) while spare capacity exists; when the array is full, a larger array is allocated and all elements are copied at cost O(size). If the capacity grows by a constant factor (about 1.5 in Java, 2 in the textbook version), the copy costs form a geometric series whose sum is a small multiple of n.

The doubling variant was instrumented: n = 1,000,000 appends from initial capacity 1 caused 20 resizes and 1,048,575 element copies in total, about 1.05 copies per append. The total cost of n appends is thus O(n), and each append is **amortised O(1)**, although a single append may cost 524,288 copies.

Output of the counting program (n = 1,000,000)

```
n=1000000 resizes=20 copies=1048575 copies/n=1.048575
```

The same argument establishes the linear time of several patterns presented later. The inner `while` loop of a sliding window or of a monotonic stack appears to give quadratic cost, but each element is pushed once and popped at most

once, so the total is O(n) amortised. The statement "every element enters and leaves at most once" is a sufficient proof of linear time.

1.2.5 Space complexity and the recursion stack

Space complexity measures the **additional** memory used beyond the input. A loop with a fixed number of variables uses O(1). A hash map holding n entries uses O(n). Memory that is not allocated explicitly is easily omitted: **each recursive call occupies a stack frame**, so a recursion of depth d uses O(d) space.

Algorithm	Time	Extra space	Why
Iterative sum of an array	O(n)	O(1)	a few variables
Recursive sum `sumTo(n)`	O(n)	O(n)	n frames on the call stack
Binary search, iterative	O(log n)	O(1)	two indexes
Binary search, recursive	O(log n)	O(log n)	log n frames
Merge sort	O(n log n)	O(n)	temporary array for merging
DFS on a tree	O(n)	O(h)	h = tree height; up to n for a skewed tree
BFS on a tree	O(n)	O(w)	w = widest level

Space used by common shapes.

The Java thread stack is small. A typical default size is about 1 MB, and a trivial recursive method overflowed after roughly 22,000 calls on the build machine (typical; dependent on frame size and JVM flags). Recursive code applied to a skewed tree or to a linked list of 100,000 nodes can therefore throw `StackOverflowError`. When the depth may reach about 10^5, the recursion should be replaced by an explicit `Deque` stack or a loop, and this should be stated to the interviewer.

GOTCHA

A single `HashMap.put` that triggers a resize costs O(n), and a poor `hashCode` can degrade one bucket to O(log n) or worse. Hash operations are reported as O(1) expected (amortised over resizes), with the justification available on request.

INTERVIEW ANGLE

"What is the time complexity of building a string with `s += c` in a loop of n characters?" O(n^2) in total. Strings are immutable, so each `+=` allocates a new string and copies all existing characters, giving 1 + 2 + ... + n copies. `StringBuilder.append` is amortised O(1) per character, giving O(n) overall.

KEY POINTS

- Big-O describes asymptotic growth: constants and lower-order terms are discarded, and the worst case is reported.
- The growth ladder is 1, log n, n, n log n, n^2, n^3, 2^n, n!; the effect of doubling n identifies the rung.
- The constraint table is consulted before design: n = 10^5 requires approximately O(n log n) or better.
- Amortised O(1) (dynamic array append, pops of a monotonic stack) means O(n) in total over n operations.
- Recursion frames count as space, and Java recursion deeper than about 10^4 to 10^5 may overflow the stack.

1.3 The dry-run method

A **dry run** is the execution of an algorithm by hand on a small input, recording the state after every step. It serves three purposes: it detects off-by-one errors that inspection of the code does not reveal, it validates an idea before implementation, and in an interview it constitutes step 6, which the interviewer expects. Every pattern chapter contains a dry-run table; this chapter specifies how to construct one.

1.3.1 How to trace by hand

1. **Select a small input** that exercises the relevant path: 5 to 10 elements, and include a case in which the answer lies in the interior.
2. **Enumerate the state variables**: all variables modified in the loop (indexes, running sum, stack contents, map contents, result). Each becomes a column.
3. **Record one row per iteration**, with the values at the start of the iteration and the action taken.
4. **Apply only the updates the code performs.** Expected behaviour is not substituted for actual behaviour; the discrepancy between the two is the defect.
5. **Terminate when the loop condition fails**, and record the final state and the return value.
6. **Repeat for the edge cases** in the checklist below.

1.3.2 Worked example: binary search

The input is the sorted array `{2, 5, 8, 12, 16, 23, 38, 56, 72, 91}` (indexes 0 to 9). The search keys are 23 (present) and 6 (absent). The state variables are `lo`, `hi` and `mid`. The tables below were obtained by adding one print statement to the loop and running it on JDK 21.

Main.java — JAVA

```java
public class Main {
    public static void main(String[] args) {
        int[] a = {2, 5, 8, 12, 16, 23, 38, 56, 72, 91};
        int key = 23;
        int lo = 0, hi = a.length - 1, step = 0;
        while (lo <= hi) {
            int mid = lo + (hi - lo) / 2;
            step++;
            String act = a[mid] == key ? "found" : a[mid] < key ? "lo = mid + 1" : "hi = mid - 1";
            System.out.println(step + " | " + lo + " | " + hi + " | " + mid + " | " + a[mid]
                    + " | " + act);
            if (a[mid] == key) break;
            if (a[mid] < key) lo = mid + 1; else hi = mid - 1;
        }
        System.out.println("end: lo=" + lo + " hi=" + hi);
    }
}
```

Trace for key 23 — TERMINAL

```
$ java Main.java
1 | 0 | 9 | 4 | 16 | lo = mid + 1
2 | 5 | 9 | 7 | 56 | hi = mid - 1
3 | 5 | 6 | 5 | 23 | found
end: lo=5 hi=6
```

Step	lo	hi	mid	a[mid]	Action
1	0	9	4	16	16 < 23, so the key is to the right: lo = 5
2	5	9	7	56	56 > 23, so the key is to the left: hi = 6
3	5	6	5	23	equal: return index 5

Dry run, key = 23. Each row is the state at the top of the iteration.

Step	lo	hi	mid	a[mid]	Action
1	0	9	4	16	16 > 6: hi = 3
2	0	3	1	5	5 < 6: lo = 2
3	2	3	2	8	8 > 6: hi = 1
end	2	1	none	none	lo > hi, loop stops, return -1

Dry run, key = 6 (not present).

In the last row, the loop terminates with `lo` equal to 2, which is the index at which 6 would be inserted to keep the array sorted (after 5, before 8). This follows from the loop invariant and explains why `Arrays.binarySearch` returns `-(insertion point) - 1` for an absent key; this was verified by searching for 6 in `[1, 2, 5, 7, 9]`, which returns `-4` (insertion point 3).

1.3.3 Invariants: the reason code is right

An **invariant** is a predicate that holds at the start of every iteration. Stating the invariant permits a proof of correctness and determines the update of each variable, which eliminates most off-by-one errors. The invariant, together with the negated loop condition, yields the postcondition.

Algorithm	Invariant (true before each iteration)	What the update preserves
Binary search (closed range)	if the key is in the array, it is in a[lo..hi]	lo = mid + 1 or hi = mid - 1 discards only indexes that cannot hold the key
Two pointers on sorted array	any valid pair lies inside [l, r]	moving the side that makes the sum wrong discards no valid pair
Sliding window	the window [left, right] satisfies the constraint	shrink from the left until it holds again
Kadane's max subarray	best ending at i is max(a[i], best ending at i-1 + a[i])	carries only the useful prefix
Insertion sort	a[0..i-1] is sorted	inserting a[i] extends the sorted prefix by one

Invariants of common loops.

A second dry run illustrates the invariant concretely: the two-pointer search for a pair with a given sum in a sorted array. The input is `{1, 3, 4, 6, 8, 11}` and the target is 10. The invariant is: *if any valid pair exists, both of its indexes lie in* `[l, r]`. If `a[l] + a[r]` exceeds the target, `a[r]` cannot be part of a valid pair with any element at or above index `l`, since every such partner is at least `a[l]`; it is discarded by `r--`. If the sum is smaller than the target, `a[l]` is discarded by the symmetric argument (`l++`).

Step	l	r	a[l] + a[r]	Action
1	0	5	1 + 11 = 12	too big, 11 cannot be in any pair: r--
2	0	4	1 + 8 = 9	too small, 1 cannot be in any pair: l++
3	1	4	3 + 8 = 11	too big: r--
4	1	3	3 + 6 = 9	too small: l++
5	2	3	4 + 6 = 10	equal: return [2, 3]

Dry run of pairSum on {1, 3, 4, 6, 8, 11}, target 10.

The pair of indexes [2, 3] is found in 5 steps for an array of 6 elements, with one pair examined per step. The brute force examines up to 15 pairs.

1.3.4 The edge-case checklist

Before an implementation is declared complete, it is checked against the following list. Most interview defects belong to one of these categories.

Category	Examples	What usually breaks
Size	empty array, one element, two elements	index -1 or out of range, loop never runs, returning the wrong default
Values	all equal, all negative, zero, duplicates, the extreme ints	max initialised to 0 when all values are negative; overflow
Order	already sorted, reverse sorted, one out of place	sorting assumptions, early exits
Position of the answer	first element, last element, not present at all	off-by-one at the boundaries, missing "not found" return
Structure (lists, trees)	null head, single node, cycle, skewed tree	NullPointerException, infinite loop, deep recursion
Strings	empty string, single character, repeated characters, spaces and upper case	`charAt` on empty string, case sensitivity

Edge cases to test.

Applied to the binary-search loop above: empty array (`hi = -1`, loop skipped, returns -1: correct); one element (`lo = hi = 0`, `mid = 0`, the key is found or the pointers cross after one step: correct); key smaller than all elements (`hi` becomes -1: correct); key larger than all elements (`lo` becomes `length` and is never used as an index: correct).

WATCH OUT
A dry run must follow the code, not the intention. Correcting a variable mentally during the trace conceals the defect. Each update is stated literally, for example: "`hi = mid - 1`, therefore hi is now 6."

INTERVIEW ANGLE
"Can you test your solution?" Visual inspection followed by the claim that the code appears correct is insufficient. A 4 to 6 element example is chosen, the variable columns are written down, and every iteration is executed in turn. The edge cases examined (empty, one element, duplicates, not found) are then stated. The interviewer observes whether the candidate follows the code or the expectation.

KEY POINTS
- A dry run is a table with one column per changing variable and one row per iteration, holding actual values.
- The trace follows the code literally; any gap between intention and code is the defect.
- The loop invariant is stated explicitly; it determines which updates are safe and explains the final state.
- The edge-case checklist is applied every time: empty, single element, duplicates, extreme values, answer at a boundary, absent answer.
- Each pattern chapter contains a dry-run table generated from a real execution, for comparison with the reader's own trace.

1.4 Java toolkit for DSA

Interview problems are language-independent, but implementation speed depends on fluency with the standard library. Looking up how to sort an array of intervals, or selecting the legacy `Stack` class, costs time and signals unfamiliarity. This chapter is a compact reference for the Java features used in the pattern chapters. Every listing was run on JDK 21.

1.4.1 Arrays, strings and characters

An array has a fixed length, resides on the heap, and its elements are initialised to 0, `false` or `null`. `int[]` stores primitives; `Integer[]` stores boxed objects. `arr.length` is a field (no parentheses), whereas `s.length()` on a `String` and `list.size()` on a collection are methods. The class `java.util.Arrays` provides `sort`, `fill`, `copyOf`, `copyOfRange`, `equals`, `binarySearch`, `toString` and `deepToString`.

Three properties account for most defects. First, `==` on arrays compares references, not contents; `Arrays.equals` (or `Arrays.deepEquals` for nested arrays) compares contents. Second, `System.out.println(arr)` prints a type descriptor and hash such as `[I@1b6d3586`; `Arrays.toString` produces the element list. Third, `Arrays.sort` on a primitive array does not accept a comparator; descending order or a custom ordering requires an object array (`Integer[]`, `int[][]`, `String[]`) and a `Comparator`.

Strings are immutable. Modification is performed with `StringBuilder` (`append`, `insert`, `reverse`, `deleteCharAt`, `setCharAt`, `length`) or by conversion with `toCharArray()` and `new String(chars)`. A `char` is a 16-bit unsigned integer, so arithmetic on characters is defined: `c - 'a'` is the zero-based letter index, `(char) (c + 1)` is the successor character, and `c - '0'` converts a digit character to its numeric value.

Main.java — JAVA

```java
import java.util.*;

public class Main {
    public static void main(String[] args) {
        int[] a = {5, 2, 9, 1, 7};
        int[] b = Arrays.copyOf(a, a.length);
        Arrays.sort(b);
        System.out.println(Arrays.toString(a) + " -> " + Arrays.toString(b));
        System.out.println("binarySearch 7 -> " + Arrays.binarySearch(b, 7));
        System.out.println("binarySearch 6 -> " + Arrays.binarySearch(b, 6));
        System.out.println(Arrays.toString(Arrays.copyOfRange(b, 1, 4)));
        int[][] grid = new int[2][3];
        for (int[] row : grid) Arrays.fill(row, -1);
        System.out.println(Arrays.deepToString(grid));
        System.out.println(a == b);
        System.out.println(Arrays.equals(a, Arrays.copyOf(a, 5)));
        Integer[] boxed = {5, 2, 9, 1, 7};
        Arrays.sort(boxed, Collections.reverseOrder());
```

Main.java (continued) JAVA

```java
        System.out.println(Arrays.toString(boxed));
        int[][] iv = {{5, 8}, {1, 3}, {2, 6}};
        Arrays.sort(iv, (x, y) -> Integer.compare(x[0], y[0]));
        System.out.println(Arrays.deepToString(iv));
        System.out.println(Arrays.stream(a).sum() + " " + Arrays.stream(a).max().getAsInt());

        String s = "hello";
        int[] freq = new int[26];
        for (char c : s.toCharArray()) freq[c - 'a']++;
        System.out.println("l count = " + freq['l' - 'a']);
        char c = 'a';
        c += 2;
        System.out.println(c + " " + (c + 1) + " " + (char) (c + 1));
        System.out.println(Character.isDigit('7') + " " + Character.isLetterOrDigit('_')
                + " " + Character.getNumericValue('7') + " " + ('7' - '0'));
        StringBuilder sb = new StringBuilder("abcd");
        sb.reverse().append('!').insert(0, "<").setCharAt(1, 'D');
        sb.deleteCharAt(sb.length() - 1);
        System.out.println(sb + " len=" + sb.length());
        char[] cs = s.toCharArray();
        Arrays.sort(cs);
        System.out.println(new String(cs) + " " + String.valueOf(cs, 1, 2));
        System.out.println("a,b,,c".split(",").length + " " + "  hi ".strip() + "|"
                + " " + "abc".substring(1) + " " + "abc".indexOf('c') + " " + "b".compareTo("d"));
        String t = "";
        for (int i = 0; i < 3; i++) t += i;
        System.out.println(t + " " + String.join("-", "x", "y", "z") + " " + "ab".repeat(2));
        System.out.println("abc".equals("ab" + "c") + " " + (new String("abc") == "abc"));
    }
}
```

Run it TERMINAL

```
$ java Main.java
[5, 2, 9, 1, 7] -> [1, 2, 5, 7, 9]
binarySearch 7 -> 3
binarySearch 6 -> -4
[2, 5, 7]
[[-1, -1, -1], [-1, -1, -1]]
false
true
[9, 7, 5, 2, 1]
[[1, 3], [2, 6], [5, 8]]
24 9
l count = 2
c 100 d
true false 7 7
<Dcba len=5
ehllo hl
4 hi| bc 2 -2
012 x-y-z abab
true false
```

Observations on the output follow.

- `binarySearch 6 -> -4`: for an absent key, `Arrays.binarySearch` returns `-(insertion point) - 1`. In `[1, 2, 5, 7, 9]` the key 6 would be inserted at index 3, hence -4. The result must be tested with `< 0`, not `== -1`.
- `Arrays.sort(iv, ...)` sorts the rows of a 2D array by their first element. The comparator uses `Integer.compare` rather than subtraction (see the overflow section).
- `c += 2` on a `char` compiles because compound assignment includes an implicit narrowing cast. The expression `c + 1` has type `int`, hence `(c + 1)` printed 100 whereas `(char) (c + 1)` printed `d`.
- `"a,b,,c".split(",").length` is 4: interior empty strings are retained, trailing empty strings are removed.
- `new String("abc") == "abc"` is `false`; strings are compared with `equals`.
- `t += i` in a loop is correct but copies the string on every iteration, giving O(n^2) total cost for long strings; `StringBuilder` is the appropriate alternative.

> **TIP**
> Frequency counts over lowercase letters are best implemented as `int[26]` with `freq[c - 'a']++`, which is faster and shorter than `HashMap<Character, Integer>`. For arbitrary ASCII characters, `int[128]` or `int[256]` is used.

1.4.2 Numbers: overflow, division and the Integer cache

`int` is a 32-bit signed type with range -2,147,483,648 to 2,147,483,647 (about plus or minus 2.1 x 10^9). `long` is 64-bit (about 9.2 x 10^18). Java does **not** signal overflow; the result silently wraps modulo 2^32 (or 2^64). If a sum or product may exceed about 2 x 10^9, `long` is required, and the conversion must precede the operation: `(long) a * b`, not `(long) (a * b)`.

Main.java JAVA

```java
import java.util.*;

public class Main {
    public static void main(String[] args) {
        System.out.println(Integer.MAX_VALUE + 1);
        int lo = 1_500_000_000, hi = 2_000_000_000;
        System.out.println("(lo+hi)/2 = " + (lo + hi) / 2 + "  lo+(hi-lo)/2 = " + (lo
                + (hi - lo) / 2));
        int big = 100_000;
        System.out.println(big * big + " " + (long) big * big + " " + 1L * big * big);
        try { Math.multiplyExact(big, big); } catch (ArithmeticException e)
                { System.out.println(e.getMessage()); }
        System.out.println(Math.abs(Integer.MIN_VALUE) + " " + (-Integer.MIN_VALUE));
        System.out.println(-7 / 2 + " " + -7 % 2 + " " + Math.floorMod(-7, 2) + " " + (7 >> 1)
                + " " + (-7 >>> 28));
        Integer[] arr = {Integer.MIN_VALUE, 1};
        Arrays.sort(arr, (x, y) -> x - y);
        System.out.println("a-b comparator: " + Arrays.toString(arr));
        Arrays.sort(arr, Integer::compare);
        System.out.println("Integer.compare: " + Arrays.toString(arr));
        Integer p = 127, q = 127, r = 128, u = 128;
        System.out.println((p == q) + " " + (r == u) + " " + r.equals(u));
        List<Integer> list = new ArrayList<>(List.of(10, 20, 30, 1));
        list.remove(1);
        System.out.println(list);
        list.remove(Integer.valueOf(1));
        System.out.println(list);
        try { List.of(1).add(2); } catch (UnsupportedOperationException e)
                { System.out.println("List.of is immutable"); }
        long sum = 0;
        for (int i = 0; i < 100_000; i++) sum += i;
        int wrong = 0;
        for (int i = 0; i < 100_000; i++) wrong += i;
        System.out.println(sum + " vs int " + wrong);
        System.out.println(Long.MAX_VALUE + " " + Integer.MAX_VALUE + " " + (char) ('a' + 1)
                + " " + ('a' + 1));
        System.out.println(Integer.bitCount(255) + " " + Integer.toBinaryString(10) + " "
                + Long.numberOfTrailingZeros(8));
        System.out.println((0.1 + 0.2) + " " + (0.1 + 0.2 == 0.3) + " " + (5 / 2) + " "
                + (5 / 2.0));
    }
}
```

Run it TERMINAL

```
$ java Main.java
-2147483648
(lo+hi)/2 = -397483648  lo+(hi-lo)/2 = 1750000000
1410065408 10000000000 10000000000
integer overflow
-2147483648 -2147483648
-3 -1 1 3 15
a-b comparator: [1, -2147483648]
Integer.compare: [-2147483648, 1]
true false true
[10, 30, 1]
[10, 30]
List.of is immutable
4999950000 vs int 704982704
9223372036854775807 2147483647 b 98
8 1010 3
0.30000000000000004 false 2 2.5
```

Each line of the output below illustrates a frequent source of error.

- `Integer.MAX_VALUE + 1` is `-2147483648`. For `lo = 1.5 x 10^9` and `hi = 2 x 10^9`, the midpoint `(lo + hi) / 2` evaluates to `-397483648`, a negative index, whereas `lo + (hi - lo) / 2` yields the correct `1750000000`. The second form is always used in binary search.
- `big * big` for `big = 100000` is `1410065408` (wrapped); `(long) big * big` is `10000000000`. `Math.multiplyExact` throws `ArithmeticException` instead of wrapping.
- `Math.abs(Integer.MIN_VALUE)` remains `-2147483648`, because +2147483648 is not representable. Negation of the minimum value has the same property.
- Integer division truncates toward zero: `-7 / 2` is `-3` and `-7 % 2` is `-1`. A non-negative remainder (cyclic indexing, modular arithmetic) is obtained with `Math.floorMod(-7, 2)`, which is `1`. The operator `>>` is an arithmetic shift (sign-preserving); `>>>` is a logical shift.
- The comparator `(x, y) -> x - y` ordered `Integer.MIN_VALUE` and `1` incorrectly because the subtraction overflowed. `Integer.compare(x, y)` is correct for all values.
- With `Integer p = 127, q = 127`, `p == q` is true, but for 128 it is false while `equals` remains true. Boxed values from -128 to 127 are cached, so reference comparison succeeds only for small values. Boxed values are compared with `equals` or unboxed first.
- `list.remove(1)` removes the element at **index** 1, whereas `list.remove(Integer.valueOf(1))` removes the first element **equal to** 1. For an `int` argument, overload resolution selects the index version.
- `List.of(...)` returns an immutable list; `new ArrayList<>(...)` is used when modification is required.
- `0.1 + 0.2 == 0.3` is false, since binary floating point cannot represent these decimals exactly. Integer formulations are preferred; otherwise doubles are compared within a tolerance.

GOTCHA
The Integer cache defect passes small tests and fails large ones. In `map.get(a) == map.get(b)` for a `Map<Integer, Integer>`, small counts compare equal by cache identity, while counts above 127 do not. The correct form is `.equals(...)`.

1.4.3 Collections you will use daily

The data structure is chosen according to the operation required most frequently. The listing below shows the standard idioms; their costs are given in the next chapter.

Need	Class	Key methods
Dynamic array	`ArrayList<Integer>`	add, get, set, remove(index), size, sort
Hash map, counting, lookup	`HashMap<K,V>`	put, get, getOrDefault, merge, computeIfAbsent, containsKey
Set of seen values	`HashSet<E>`	add (returns false if present), contains, remove
Ordered map or set, nearest key	`TreeMap`, `TreeSet`	floorKey, ceilingKey, lowerKey, higherKey, firstKey, pollFirst
Stack (LIFO) and queue (FIFO), deque	`ArrayDeque<E>`	push, pop, peek; offer, poll, peek; addFirst, addLast, pollFirst, pollLast
Min-heap or max-heap	`PriorityQueue<E>`	offer, poll, peek, size
Order-preserving map (LRU)	`LinkedHashMap`	access order plus `removeEldestEntry`

Which Java class for which job.

Main.java JAVA

```java
import java.util.*;

public class Main {
    record Pair(int a, int b) {}
    record Job(String name, int prio, int time) {}

    public static void main(String[] args) {
        Map<Character, Integer> cnt = new HashMap<>();
        for (char ch : "banana".toCharArray()) cnt.merge(ch, 1, Integer::sum);
        System.out.println(cnt);
        Map<Integer, List<String>> byLen = new TreeMap<>();
        for (String w : List.of("hi", "yo", "hey", "a")) byLen.computeIfAbsent(w.length(),
```

Main.java (continued) JAVA

```
                k -> new ArrayList<>()).add(w);
        System.out.println(byLen + " " + cnt.getOrDefault('z', 0));
        TreeMap<Integer, String> tm = new TreeMap<>(Map.of(10, "a", 20, "b", 30, "c"));
        System.out.println(tm.floorKey(25) + " " + tm.ceilingKey(25) + " " + tm.lowerKey(10) + " "
                + tm.firstKey() + " " + tm.lastEntry() + " " + tm.headMap(30));
        TreeSet<Integer> ts = new TreeSet<>(List.of(5, 1, 9));
        System.out.println(ts + " " + ts.higher(5) + " " + ts.pollFirst() + " "
                + ts.descendingSet());
        Deque<Integer> stack = new ArrayDeque<>();
        stack.push(1); stack.push(2); stack.push(3);
        System.out.println("stack " + stack + " peek=" + stack.peek() + " pop=" + stack.pop());
        Deque<Integer> queue = new ArrayDeque<>();
        queue.offer(1); queue.offer(2); queue.offer(3);
        System.out.println("queue " + queue + " poll=" + queue.poll() + " peekLast="
                + queue.peekLast());
        PriorityQueue<Integer> minH = new PriorityQueue<>(List.of(5, 1, 8, 3));
        PriorityQueue<Integer> maxH = new PriorityQueue<>(Comparator.reverseOrder());
        maxH.addAll(List.of(5, 1, 8, 3));
        System.out.println(minH.poll() + " " + maxH.poll() + " size=" + minH.size());
        PriorityQueue<int[]> pq = new PriorityQueue<>((x, y) -> Integer.compare(x[1], y[1]));
        pq.add(new int[]{0, 9}); pq.add(new int[]{1, 4}); pq.add(new int[]{2, 6});
        System.out.println(Arrays.toString(pq.poll()));
        List<Job> jobs = new ArrayList<>(List.of(new Job("a", 2, 30), new Job("b", 1, 50),
                new Job("c", 2, 10)));
        jobs.sort(Comparator.comparingInt(Job::prio).thenComparing(Job::time,
                Comparator.reverseOrder()));
        System.out.println(jobs);
        Set<Pair> seen = new HashSet<>();
        System.out.println(seen.add(new Pair(1, 2)) + " " + seen.add(new Pair(1, 2)) + " "
                + new Pair(3, 4));
        List<Integer> nums = new ArrayList<>(List.of(1, 2, 3, 4));
        Collections.reverse(nums); Collections.swap(nums, 0, 3);
        System.out.println(nums + " max=" + Collections.max(nums) + " "
                + Collections.frequency(nums, 2));
        @SuppressWarnings("unchecked")
        List<Integer>[] graph = new List[3];
        for (int i = 0; i < 3; i++) graph[i] = new ArrayList<>();
        graph[0].add(1); graph[0].add(2); graph[1].add(2);
        System.out.println(Arrays.toString(graph));
        int[][] jag = new int[3][];
        for (int i = 0; i < 3; i++) jag[i] = new int[i + 1];
        System.out.println(Arrays.deepToString(jag) + " rows=" + jag.length + " cols0="
                + jag[0].length);
        int[][] g = {{1, 2}, {3, 4}};
        int[][] shallow = g.clone();
        shallow[0][0] = 99;
        System.out.println(g[0][0]);
        int[][] dirs = {{1, 0}, {-1, 0}, {0, 1}, {0, -1}};
        int cnt2 = 0;
        for (int[] d : dirs) { int nr = 0 + d[0], nc = 0 + d[1]; if (nr >= 0 && nc >= 0) cnt2++; }
        System.out.println("neighbours of (0,0) in bounds: " + cnt2);
    }
}
```

Run it TERMINAL

```
$ java Main.java
{a=3, b=1, n=2}
{1=[a], 2=[hi, yo], 3=[hey]} 0
20 30 null 10 30=c {10=a, 20=b}
[1, 5, 9] 9 1 [9, 5]
stack [3, 2, 1] peek=3 pop=3
queue [1, 2, 3] poll=1 peekLast=3
1 8 size=3
[1, 4]
[Job[name=b, prio=1, time=50], Job[name=a, prio=2, time=30], Job[name=c, prio=2, time=10]]
true false Pair[a=3, b=4]
[1, 3, 2, 4] max=4 1
[[1, 2], [2], []]
[[0], [0, 0], [0, 0, 0]] rows=3 cols0=1
99
neighbours of (0,0) in bounds: 2
```

Observations on the listing follow.

- `map.merge(ch, 1, Integer::sum)` is the most concise correct counting idiom. `computeIfAbsent(k, x -> new ArrayList<>()).add(v)` is the most concise construction of a map of lists (adjacency lists, grouping of anagrams).
- `HashMap` iteration order is unspecified (here the keys happened to print in sorted order). `TreeMap` or `LinkedHashMap` is used when order matters.
- `ArrayDeque` implements both the stack and the queue. As a stack, `push`, `pop` and `peek` operate at the **front**, which is why the stack printed as `[3, 2, 1]`. As a queue, `offer` appends at the back and `poll` removes from the front. It is preferred to `java.util.Stack` (a legacy class with synchronised methods) and to `LinkedList` (higher memory use, slower). `ArrayDeque` does not permit `null` elements.
- `PriorityQueue` is a **min-heap**: the smallest element is removed first. A max-heap is obtained with `Comparator.reverseOrder()`. Iteration order and `toString` are not sorted; only successive `poll` calls yield sorted order.
- `TreeMap.floorKey(25)` returns the greatest key <= 25 (here 20); `ceilingKey(25)` the least key >= 25 (30); `lowerKey(10)` is `null` because no key is strictly smaller. Each operation is O(log n).
- A `record` such as `Pair` or `Job` supplies a constructor, accessors, `equals`, `hashCode` and `toString`, so it is usable as a `HashSet` element or `HashMap` key (`seen.add(new Pair(1, 2))` returned false on the second call). Records are preferred to `int[]` for coordinates, edges and (value, index) pairs, because `int[]` uses reference equality.
- `Comparator.comparingInt(Job::prio).thenComparing(...)` composes multi-key orderings: ascending priority, then descending time. Lambdas over `int[]` are equally valid: `(x, y) -> Integer.compare(x[1], y[1])`.
- Generic arrays cannot be created directly. `List<Integer>[] graph = new List[3]` produces an unchecked warning (suppressed above), and each slot must be initialised. The alternative `List<List<Integer>> graph = new ArrayList<>()` is common.
- A 2D array is an array of arrays. `new int[3][]` creates a jagged array whose rows are allocated separately. `grid.clone()` is **shallow**: row arrays are shared, so the modification of `shallow[0][0]` also changed `g[0][0]` to 99. A grid is copied by cloning each row. A direction table `int[][] dirs = {{1,0},{-1,0},{0,1},{0,-1}}` is the standard means of enumerating grid neighbours.

WATCH OUT

Removing an element from a collection during a for-each iteration usually throws `ConcurrentModificationException`, but not always: removal of the second-to-last element bypasses the check silently. Neither behaviour should be relied upon. The correct alternatives are `Iterator.remove()`, backward iteration by index, or collecting the elements to be removed first.

1.4.4 Fast input, output and recursion limits

Interview platforms supply the input as method parameters, whereas online judges and contests read standard input. `Scanner` is convenient but slow, since it parses with regular expressions. For 10^5 or more numbers, `BufferedReader` with `StringTokenizer` is used, and output is accumulated in a `StringBuilder` and printed once.

Main.java JAVA

```java
import java.io.*;
import java.util.*;

public class Main {
    static int depth = 0;
    static void down() { depth++; down(); }

    public static void main(String[] args) throws IOException {
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        int n = Integer.parseInt(br.readLine().trim());
        StringTokenizer st = new StringTokenizer(br.readLine());
        long total = 0;
        int[] a = new int[n];
        for (int i = 0; i < n; i++) { a[i] = Integer.parseInt(st.nextToken()); total += a[i]; }
        StringBuilder out = new StringBuilder();
        out.append(total).append('\n').append(Arrays.toString(a));
        System.out.println(out);
        try { down(); } catch (StackOverflowError e) { System.out.println("depth > 1000: "
                + (depth > 1000)); }
        System.out.println("depth reached (typical): " + depth);
    }
}
```

Feed it two lines of input — TERMINAL

```
$ printf '4\n3 1 4 1\n' | java Main.java
9
[3, 1, 4, 1]
depth > 1000: true
depth reached (typical): 23274
```

The last two output lines demonstrate the recursion limit. The method `down()` recurses until the JVM throws `StackOverflowError`; the program catches the error and prints the depth reached, which was 22,169 frames on the build machine (typical; the value depends on the machine, on JVM flags such as `-Xss`, and on frame size). A depth of about 10^4 may be regarded as safe and 10^5 as risky. If a tree or list may be that deep, the traversal is written with an explicit `Deque` as the stack. The limit can be raised by running the work in a `Thread` with a larger stack size, but conversion to iteration is the preferred answer in an interview.

TIP

Pre-sizing and reuse are inexpensive: `new ArrayList<>(n)` avoids resizes. When a sentinel "infinity" is later used in addition, `Integer.MAX_VALUE / 2` or `Long.MAX_VALUE / 2` is chosen, because `Integer.MAX_VALUE + 1` overflows.

INTERVIEW ANGLE

"How do you sort an array of intervals by start time in Java, and why not subtract?" `Arrays.sort(intervals, (a, b) -> Integer.compare(a[0], b[0]))` or `Comparator.comparingInt(a -> a[0])`. Subtraction (`a[0] - b[0]`) overflows when the operands are far apart (for example `Integer.MIN_VALUE` and 1) and produces an inconsistent ordering. In addition, `Arrays.sort` on objects is stable (TimSort), whereas on primitives stability is unobservable.

KEY POINTS

- Four habits remove most defects: `lo + (hi - lo) / 2`, `Integer.compare`, `equals` for boxed values and strings, and `Arrays.toString` for output.
- `ArrayDeque` serves as stack and queue, `PriorityQueue` is a min-heap, `TreeMap` supports nearest-key queries, and `int[26]` counts lowercase letters.
- Operands are cast to `long` before multiplication; `Integer.MAX_VALUE + 1` wraps silently.
- Arrays are objects with shallow `clone`; a 2D array is an array of arrays.
- Recursion deeper than about 10^4 may overflow the Java stack and is converted to iteration when necessary.

1.5 Core data structures you must know

A **data structure** organises data so that selected operations are inexpensive. The choice depends on which operation an algorithm performs most often: access by position, search by value, insertion at an end, removal of the minimum, or enumeration of the neighbours of a node. This chapter describes the eight structures that underlie nearly all interview problems, their costs, and the corresponding Java classes.

1.5.1 What each one is, in one paragraph

Array. A contiguous block of memory. Reading `a[i]` is O(1) because the address is `start + i * size`. Insertion or deletion in the interior shifts all subsequent elements: O(n). Search in an unsorted array is O(n); a sorted array permits binary search in O(log n). `ArrayList` is a resizable array with amortised O(1) append.

Linked list. A sequence of nodes, each holding a value and a reference to the next node (and, if doubly linked, to the previous node). Insertion or deletion after a node already held is O(1); access to the i-th element is O(n) because traversal is required. There is no random access and cache locality is poor, so `ArrayList` is usually faster in practice. Linked lists are prominent in interviews because they test pointer manipulation.

Stack. Last in, first out. Push and pop at one end are O(1). Applications: bracket matching, undo, depth-first search, expression evaluation, and nearest-greater-element problems. Java implementation: `ArrayDeque`.

Queue and deque. First in, first out. Enqueue at the back and dequeue from the front are O(1); a deque permits both operations at both ends. Applications: breadth-first search, level-order traversal, scheduling and sliding-window maxima.

Hash table. An array of buckets in which the key's `hashCode` selects the bucket. Insertion, lookup and deletion are O(1) on average, and no order is maintained. Java classes: `HashMap` and `HashSet`. The costs are additional memory and the requirement of a consistent `equals` and `hashCode` pair on keys.

Tree. A connected acyclic structure of nodes with children. A **binary tree** has at most two children per node. A **binary search tree** (BST) maintains the ordering left < node < right, so a lookup follows a single path and costs O(h), where h is the height: O(log n) if balanced, O(n) if skewed. Java's `TreeMap` and `TreeSet` are red-black trees with guaranteed O(log n). A **trie** is a tree keyed by characters and supports prefix queries.

Heap. A complete binary tree stored in an array in which every parent is not larger (min-heap) or not smaller (max-heap) than its children. The minimum is read in O(1); removal and insertion are O(log n). Java class: `PriorityQueue`. Applications: top-K selection, merging sorted lists, scheduling and Dijkstra's algorithm.

Graph. A set of vertices joined by edges, which may be directed, weighted or cyclic. Representations: an adjacency list (`List<List<Integer>>`, space O(V + E)) or an adjacency matrix (`boolean[V][V]`, space O(V^2)). Problems on grids, networks, dependencies and relationships are graph problems.

1.5.2 Operation costs

Structure	Access by index	Search by value	Insert	Delete	Notes
Array	O(1)	O(n); O(log n) if sorted	O(n)	O(n)	fixed size
ArrayList	O(1)	O(n)	O(1) amortised at end; O(n) elsewhere	O(n); O(1) at end	resizes by about 1.5 times
Singly linked list	O(n)	O(n)	O(1) at head or after a known node	O(1) given the previous node	no random access
Stack or queue (ArrayDeque)	top or front only: O(1)	O(n)	O(1)	O(1)	at the ends
Hash table	none	O(1) avg	O(1) avg	O(1) avg	no order; worst case higher
Balanced BST (TreeMap)	none	O(log n)	O(log n)	O(log n)	sorted, nearest-key queries
Binary heap (PriorityQueue)	min only: O(1)	O(n)	O(log n)	O(log n) for the min; O(n) arbitrary	not sorted
Graph, adjacency list	neighbours: O(degree)	edge test: O(degree)	O(1) add edge	O(degree)	space O(V + E)
Graph, adjacency matrix	edge test: O(1)	none	O(1)	O(1)	space O(V^2)

Typical costs. n = number of elements, h = tree height. "Avg" means average case.

Four of these costs were measured on the build machine with 100,000 elements (typical, JDK 21; run-to-run variation is large for the fast operations):

Task	Right choice	Wrong choice
Insert n items at the front	`ArrayDeque.addFirst`: 1 to 30 ms	`ArrayList.add(0, x)`: about 460 ms
2,000 membership tests on 100,000 items	`HashSet.contains`: about 0.3 ms	`ArrayList.contains`: 250 to 490 ms

Measured: the right structure versus the wrong one (n = 100,000).

The ratio is between two and three orders of magnitude, caused solely by the choice of class. This quantifies the guideline that the data structure should match the dominant operation.

sreeram

1.5.3 Choosing: the questions to ask

- Repeated lookup by value ("seen before?", "how many times?"): `HashSet` or `HashMap`.
- Repeated access to the minimum or maximum of changing data: `PriorityQueue`.
- Nearest smaller or larger value, or a sorted view under updates: `TreeMap` or `TreeSet`.
- Processing of the most recent item first (matching, undo, nesting): stack.
- Processing in arrival order or level by level: queue.
- Insertion and removal at both ends (sliding window): deque.
- Relationships between entities: graph, with BFS, DFS, union-find or Dijkstra as appropriate.

NOTE

A `HashMap` bucket with heavy collisions is converted to a balanced tree once it holds 8 or more entries (and the table has at least 64 buckets). For `Comparable` keys the worst-case lookup is therefore O(log n), which is better than the textbook O(n).

INTERVIEW ANGLE

"Array versus linked list: when is each better?" Array: random access, compact memory, cache-friendly iteration, amortised O(1) append; interior insertion and deletion are O(n). Linked list: O(1) insertion or deletion at a held node and no resizing; access is O(n) and each node carries overhead. In production Java, `ArrayList` or `ArrayDeque` is almost always preferable; the linked list is the answer when a problem supplies nodes and requires pointer rewiring (reverse, merge, delete a node).

KEY POINTS

- Eight structures cover nearly all problems: array, linked list, stack, queue, hash table, tree, heap, graph.
- Core costs: array index O(1), hash lookup O(1) expected, tree and heap operations O(log n), linked-list access O(n).
- The structure is chosen by the dominant operation; a wrong choice costs a factor of hundreds.
- Java classes: `ArrayList`, `ArrayDeque`, `HashMap`/`HashSet`, `TreeMap`/`TreeSet`, `PriorityQueue`; `Stack`, `Vector` and (usually) `LinkedList` are avoided.

1.6 Pattern map and how to choose

A **pattern** is a reusable strategy that reduces a brute force of O(n^2), O(2^n) or worse by exploiting structure in the problem: ordering, repeated sub-computation, monotonicity, a graph, or a decision tree. There are about two dozen patterns, and most interview problems are instances of one of them. The required skill is recognition: identifying the pattern from the statement before the implementation is begun.

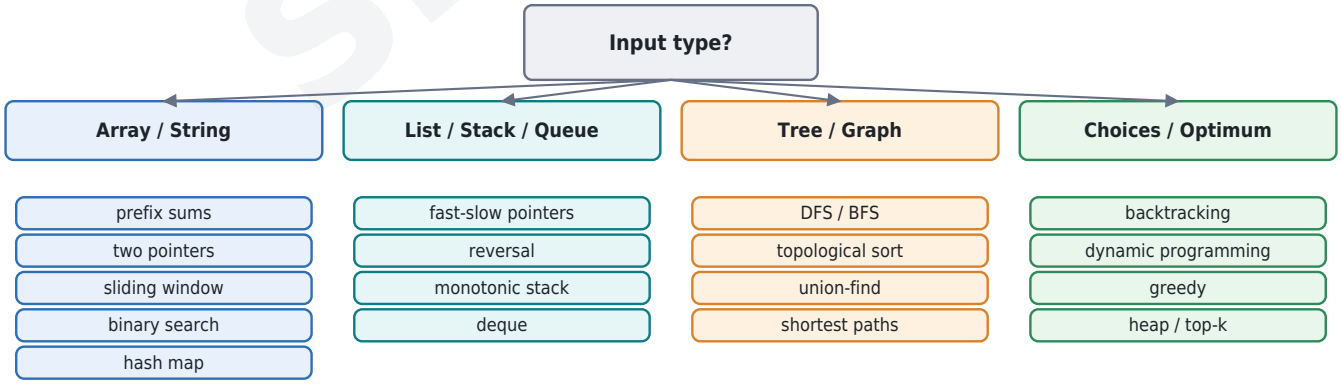


Figure 1.2. The map groups patterns by the structure of the data. The starting point is the input type (array, string, list, tree, graph, counting question) together with the clue words in the statement.

1.6.1 Clue to pattern

Clue in the statement	Pattern	Data structure	Cost
"sorted array", "find pair or triplet with sum"	Two pointers	indexes	O(n)
"longest/shortest subarray or substring with a property", "contiguous"	Sliding window	counts, map	O(n)
"sum of a range", "subarray sum equals k"	Prefix sum	array, hash map	O(n)
"have I seen", "count occurrences", "group by", "two numbers that"	Hash map or set	HashMap	O(n)
"sorted", "find minimum X such that", "maximise the minimum"	Binary search (on index or on the answer)	none	O(n log n) or O(log n)
"merge overlapping", "schedule", "meeting rooms"	Sort plus sweep	sorted array	O(n log n)
"in place", "O(1) space", "each value 1 to n"	In-place array tricks, cyclic sort	indexes	O(n)
"linked list", "cycle", "middle"	Fast and slow pointers, dummy node	pointers	O(n)
"matching brackets", "nested", "previous/next greater"	Stack, monotonic stack	ArrayDeque	O(n)
"max/min in every window of size k"	Monotonic deque	ArrayDeque	O(n)
"top K", "k-th largest", "median of a stream", "merge K sorted"	Heap	PriorityQueue	O(n log k)
"all subsets/permutations/combinations", "generate all"	Backtracking	recursion	O(2^n) or O(n!)
"tree", "depth", "path", "ancestor"	DFS on tree	recursion	O(n)
"level by level", "shortest path in unweighted graph or grid"	BFS	ArrayDeque	O(V + E)
"connected components", "islands", "reachable"	DFS, BFS, union-find	graph, DSU	O(V + E)
"prerequisites", "ordering with dependencies"	Topological sort	in-degree array	O(V + E)
"shortest path with weights"	Dijkstra (or Bellman-Ford)	PriorityQueue	O((V + E) log V)
"count the ways", "minimum cost", "maximum value", overlapping sub-problems	Dynamic programming	array or map	O(states x transitions)
"locally best choice is safe", "interval or activity selection"	Greedy	sort, heap	O(n log n)
"single number", "powers of two", "subset of at most 20 items"	Bit manipulation, bitmask	int	O(n)
"prefix", "dictionary of words"	Trie	tree of arrays	O(total length)

Clue in the statement, the pattern to try, and the target cost.

1.6.2 How to choose, step by step

1. **Examine the input.** Sorted array: two pointers or binary search. Unsorted array with a range question: prefix sums or sliding window. Tree: DFS or BFS. Grid or relationships: graph. Values 1 to n with a missing or duplicate element: in-place indexing or XOR.
2. **Examine the question word.** Count, maximum or minimum over choices: DP or greedy. Generate all: backtracking. Shortest: BFS (equal weights) or Dijkstra (weighted). K-th or top: heap or quickselect. Existence within a range: binary search.
3. **Read the constraint** (see the complexity chapter). n <= 20 permits exponential cost, n <= 5,000 permits O(n^2), and n up to 10^5 requires O(n log n) or better.

4. **Identify the repeated work in the brute force.** Recomputed sums: prefix sum. Rescanned windows: sliding window. Re-solved sub-problems: memoisation. Repeated minimum search: heap. Search for a partner value: hash map. The repeated work identifies the pattern.
5. **If two patterns apply, select the simpler**, mention the other as an alternative, and give both costs. For a sorted array and a pair-sum query, two pointers (O(1) space) is preferred, and a hash map (O(n) space) is the alternative.

TIP

Patterns may be combined: sliding window with hash map (longest substring with at most K distinct characters), binary search with greedy (ship packages within D days), BFS with bitmask, DP with binary search (longest increasing subsequence in O(n log n)). The outer pattern is identified first; the inner one is usually a helper.

INTERVIEW ANGLE

"How do you decide which approach to use for a problem you have never seen?" Restate the input and output, read the constraints, write the brute force, and locate its redundant work. Match the redundancy to a pattern: repeated range sums to prefix sums, repeated membership tests to hashing, monotone feasibility to binary search, overlapping sub-problems to DP. Confirm on a small example before coding. The answer demonstrates a method rather than a memorised list.

1.6.3 LeetCode starter list

The following ten easy problems form the warm-up for the book. Each uses one idea from this Part and each is frequently asked. They should be solved before the pattern chapters, which reuse their ideas. Every number, title and difficulty below was verified on the LeetCode site.

#	Title	Level	Key idea
1	Two Sum	Easy	One pass with a hash map from value to index
217	Contains Duplicate	Easy	`HashSet.add` returns false when the value was already there
242	Valid Anagram	Easy	Count letters with `int[26]`, increment for one string, decrement for the other
125	Valid Palindrome	Easy	Two pointers from both ends, skipping non-alphanumeric characters
121	Best Time to Buy and Sell Stock	Easy	Track the minimum price so far and the best difference
20	Valid Parentheses	Easy	Stack: push openers, pop and match on each closer
704	Binary Search	Easy	Closed-range loop with `lo + (hi - lo) / 2`
206	Reverse Linked List	Easy	Rewire `next` with prev, curr and a saved next pointer
21	Merge Two Sorted Lists	Easy	Dummy head node and a tail pointer, take the smaller front each time
104	Maximum Depth of Binary Tree	Easy	Recursion: 1 + max(depth of left, depth of right)
70	Climbing Stairs	Easy	Each answer is the sum of the previous two: the smallest DP

LeetCode warm-up problems.

KEY POINTS

- A pattern removes the repeated work of the brute force; recognition rests on clue words, input shape and constraints.
- Every problem begins with the input type, the question word, the constraint, the brute force and its repeated work.
- Many problems combine two patterns; the outer pattern is identified first.
- The ten warm-up problems precede the pattern chapters, and the mistakes log starts with the first of them.

PART 2

Arrays and Strings Patterns

Most interview problems start with an array or a string; the first five patterns eliminate redundant re-scanning of the input.

In this part

- **2.1** Prefix Sum and Difference Array
- **2.2** Two Pointers
- **2.3** Sliding Window: Fixed Size
- **2.4** Sliding Window: Variable Size
- **2.5** Hash Map and Frequency Counting
- **2.6** Binary search on sorted arrays
- **2.7** Binary search on the answer
- **2.8** Sorting-based problems and merge intervals
- **2.9** Kadane, greedy scans and in-place array tricks
- **2.10** Matrix and string patterns

2.1 Prefix Sum and Difference Array

Consider a sequence of queries, each requesting the sum of the elements from index l to index r. Direct evaluation by a loop costs O(n) per query. For an array of n elements and q queries the total is O(n × q); with n = q = 100,000 this requires ten billion additions, which exceeds a typical limit of 1 to 2 seconds.

The redundancy is that each query re-adds elements already added by earlier queries. A **prefix sum** array performs the addition once, during preprocessing, and answers each query with a single subtraction. The inverse construction, the **difference array**, applies range updates in O(1) each. Together they solve the class of range-sum and subarray-count problems.

2.1.1 The idea

Let `prefix[i]` be the sum of the first `i` elements, that is `nums[0] + ... + nums[i-1]`. Note the shift: `prefix` has `n + 1` entries and `prefix[0] = 0` (the sum of nothing). Then the sum of `nums[l..r]` (both ends included) is

```
prefix[r + 1] - prefix[l]
```

because `prefix[r+1]` accumulates all elements up to index `r` and `prefix[l]` accumulates all elements before index `l`. The leading zero removes the need for a special case when `l = 0`.

How to recognise it. The following clues in a problem statement indicate the pattern:

- "sum of elements between indices l and r", asked many times, with no updates in between (immutable array).
- "number of subarrays whose sum is exactly k" or "divisible by k" (prefix sums plus a hash map).
- "apply +x to every element from l to r", repeated many times, then read the final array (difference array).
- "sum of a rectangle in a matrix" (2D prefix sums).
- Any "balance" question, such as equal numbers of 0s and 1s: replace 0 by -1 and look for a subarray with sum 0.

Subarray sum equals k. For the counting variant, a subarray `nums[i..j]` has sum k exactly when `prefix[j+1] - prefix[i] == k`, which is the same as `prefix[i] == prefix[j+1] - k`. Hence a left-to-right scan maintains a hash map from each prefix value seen so far to its number of occurrences. At each position the count stored for `running - k` is added to the answer. This reduces an O(n²) pair search to O(n) and remains valid for negative elements, for which a sliding window is not applicable.

Template.java JAVA

```java
// range queries
long[] prefix = new long[n + 1];
for (int i = 0; i < n; i++) prefix[i + 1] = prefix[i] + nums[i];
long sum = prefix[r + 1] - prefix[l];

// count subarrays with sum k
Map<Integer, Integer> seen = new HashMap<>();
```

sroeram

Template.java (continued) JAVA

```java
seen.put(0, 1);                                  // empty prefix
int running = 0, count = 0;
for (int x : nums) {
    running += x;
    count += seen.getOrDefault(running - k, 0);  // look BEFORE inserting
    seen.merge(running, 1, Integer::sum);
}
```

The difference array. Store `diff[i] = a[i] - a[i-1]`. Adding `v` to every element in `[l, r]` requires `diff[l] += v` and `diff[r+1] -= v`: two writes, independent of the range length. After all updates, a running sum of `diff` reconstructs the final array. It is the exact inverse of the prefix sum and is the standard technique for batched range updates (for example flight bookings and car pooling).

The 2D version. For a matrix, `p[i][j]` is the sum of the rectangle from the top-left corner to cell `(i-1, j-1)`. It is built by inclusion and exclusion: `p[i+1][j+1] = m[i][j] + p[i][j+1] + p[i+1][j] - p[i][j]`. A query for the rectangle with corners `(r1, c1)` and `(r2, c2)` is `p[r2+1][c2+1] - p[r1][c2+1] - p[r2+1][c1] + p[r1][c1]`: the enclosing rectangle minus the strip above and the strip to the left, plus the corner region subtracted twice.

2.1.2 Dry run

First the range-sum part. Input: `nums = [3, 1, 4, 1, 5, 9, 2, 6]`. The program builds `prefix = [0, 3, 4, 8, 9, 14, 23, 25, 31]`. The query `sum[2..5]` is `prefix[6] - prefix[2] = 23 - 4 = 19`, which is `4 + 1 + 5 + 9`. The whole array is `prefix[8] - prefix[0] = 31`.

Now the counting variant. Input: `nums = [1, 2, 3, -1, 2]`, `k = 3`. The map starts as `{0=1}`. Each row below is real output of the traced program.

i	x	running	need	add	count	map after the step
0	1	1	-2	0	0	{0=1, 1=1}
1	2	3	0	1	1	{0=1, 1=1, 3=1}
2	3	6	3	1	2	{0=1, 1=1, 3=1, 6=1}
3	-1	5	2	0	2	{0=1, 1=1, 3=1, 5=1, 6=1}
4	2	7	4	0	2	{0=1, 1=1, 3=1, 5=1, 6=1, 7=1}

Subarray sum equals 3: one row per element.

At `i = 1` the running sum is 3 and `need = 0`. The map holds `0` once (the empty prefix), so the prefix `[1, 2]` has sum 3 and the count becomes 1. At `i = 2` the running sum is 6 and `need = 3`; the prefix value 3 occurred after `i = 1`, so the subarray following that point, `[3]`, has sum 3 and the count becomes 2. The remaining rows find no match. The two subarrays are `[1, 2]` and `[3]`.

For the difference array, take `n = 5` and three updates `(l, r, v)`: `(1, 3, +2)`, `(2, 4, +3)`, `(0, 1, +1)`. The program ends with `diff = [1, 2, 2, 0, -2, -3]`; running sums give `[1, 3, 5, 5, 3]`. Index 2 lies in the first and second ranges only, giving `2 + 3 = 5`, which agrees.

2.1.3 Java code

Main.java JAVA

```java
import java.util.Arrays;
import java.util.HashMap;
import java.util.Map;

public class Main {
    // Prefix sums: prefix[i] = sum of nums[0..i-1], so prefix has n+1 entries.
    static long[] buildPrefix(int[] nums) {
        long[] prefix = new long[nums.length + 1];
        for (int i = 0; i < nums.length; i++) {
            prefix[i + 1] = prefix[i] + nums[i];
        }
        return prefix;
    }

    // Sum of nums[l..r] inclusive in O(1).
    static long rangeSum(long[] prefix, int l, int r) {
        return prefix[r + 1] - prefix[l];
    }

    // Count subarrays whose sum equals k (works with negatives).
    static int subarraySum(int[] nums, int k, boolean trace) {
        Map<Integer, Integer> seen = new HashMap<>();
        seen.put(0, 1);                        // the empty prefix
        int running = 0, count = 0;
        for (int i = 0; i < nums.length; i++) {
            running += nums[i];
            int need = running - k;
            int add = seen.getOrDefault(need, 0);
            count += add;
            seen.merge(running, 1, Integer::sum);
            if (trace) {
                System.out.printf("i=%d x=%2d running=%2d need=%2d add=%d count=%d map=%s%n",
                        i, nums[i], running, need, add, count, seen);
            }
        }
        return count;
    }

    static int subarraySumBrute(int[] nums, int k) {
        int count = 0;
        for (int i = 0; i < nums.length; i++) {
            int sum = 0;
            for (int j = i; j < nums.length; j++) {
                sum += nums[j];
                if (sum == k) count++;
            }
        }
        return count;
    }

    // Difference array: add 'delta' to every element in [l, r] in O(1).
    static int[] applyUpdates(int n, int[][] updates) {
        int[] diff = new int[n + 1];
        for (int[] u : updates) {
            diff[u[0]] += u[2];
            diff[u[1] + 1] -= u[2];
        }
        int[] result = new int[n];
        int running = 0;
        for (int i = 0; i < n; i++) {
            running += diff[i];
            result[i] = running;
        }
        System.out.println("diff   = " + Arrays.toString(diff));
        return result;
    }

    // 2D prefix sums: p[i][j] = sum of matrix[0..i-1][0..j-1].
    static int[][] build2D(int[][] m) {
        int[][] p = new int[m.length + 1][m[0].length + 1];
        for (int i = 0; i < m.length; i++) {
            for (int j = 0; j < m[0].length; j++) {
                p[i + 1][j + 1] = m[i][j] + p[i][j + 1] + p[i + 1][j] - p[i][j];
```

sreeram

Main.java (continued) JAVA

```
            }
        }
        return p;
    }

    static int region(int[][] p, int r1, int c1, int r2, int c2) {
        return p[r2 + 1][c2 + 1] - p[r1][c2 + 1] - p[r2 + 1][c1] + p[r1][c1];
    }

    public static void main(String[] args) {
        int[] nums = {3, 1, 4, 1, 5, 9, 2, 6};
        long[] prefix = buildPrefix(nums);
        System.out.println("prefix = " + Arrays.toString(prefix));
        System.out.println("sum[2..5] = " + rangeSum(prefix, 2, 5));
        System.out.println("sum[0..7] = " + rangeSum(prefix, 0, 7));

        int[] a = {1, 2, 3, -1, 2};
        System.out.println("-- subarray sum equals 3, trace --");
        System.out.println("answer = " + subarraySum(a, 3, true));
        System.out.println("brute  = " + subarraySumBrute(a, 3));

        System.out.println("-- difference array --");
        int[][] updates = {{1, 3, 2}, {2, 4, 3}, {0, 1, 1}};
        System.out.println("result = " + Arrays.toString(applyUpdates(5, updates)));

        System.out.println("-- 2D prefix --");
        int[][] m = {{3, 0, 1}, {5, 6, 3}, {1, 2, 0}};
        int[][] p = build2D(m);
        for (int[] row : p) System.out.println(Arrays.toString(row));
        System.out.println("region(1,1)-(2,2) = " + region(p, 1, 1, 2, 2));
        System.out.println("region(0,0)-(1,2) = " + region(p, 0, 0, 1, 2));
    }
}
```

Run it TERMINAL

```
$ java Main.java
prefix = [0, 3, 4, 8, 9, 14, 23, 25, 31]
sum[2..5] = 19
sum[0..7] = 31
-- subarray sum equals 3, trace --
i=0 x= 1 running= 1 need=-2 add=0 count=0 map={0=1, 1=1}
i=1 x= 2 running= 3 need= 0 add=1 count=1 map={0=1, 1=1, 3=1}
i=2 x= 3 running= 6 need= 3 add=1 count=2 map={0=1, 1=1, 3=1, 6=1}
i=3 x=-1 running= 5 need= 2 add=0 count=2 map={0=1, 1=1, 3=1, 5=1, 6=1}
i=4 x= 2 running= 7 need= 4 add=0 count=2 map={0=1, 1=1, 3=1, 5=1, 6=1, 7=1}
answer = 2
brute  = 2
-- difference array --
diff   = [1, 2, 2, 0, -2, -3]
result = [1, 3, 5, 5, 3]
-- 2D prefix --
[0, 0, 0, 0]
[0, 3, 3, 4]
[0, 8, 14, 18]
[0, 9, 17, 21]
region(1,1)-(2,2) = 11
region(0,0)-(1,2) = 18
```

What to notice. `seen.put(0, 1)` is the empty prefix; omitting it loses every subarray that starts at index 0. The lookup precedes the insertion; otherwise, for `k = 0`, an empty subarray would be counted. `prefix` is `long[]` because a sum of many `int` values may overflow `int` without any exception. The 2D region result `11` is `6 + 3 + 2 + 0`, the bottom-right 2×2 block of the matrix.

2.1.4 Complexity and mistakes

Building a prefix array is O(n) time and O(n) space; each query is O(1). The hash-map counting version is O(n) time and O(n) space, against O(n²) for the brute force that tries every start and end. The difference array applies each update in O(1) and rebuilds in O(n), so m updates cost O(n + m) instead of O(n × m). The 2D prefix table costs O(rows × cols) once, then O(1) per rectangle.

Common mistakes:

- **Index-shift error.** The formula `prefix[r + 1] - prefix[l]` requires an array of size `n + 1`. Mixing conventions (size `n`, `prefix[r] - prefix[l-1]`) yields index -1 when `l = 0`.
- **Omitting the empty prefix** `seen.put(0, 1)`, which loses subarrays beginning at index 0.
- **Overflow.** Sums of `int` values can exceed 2.1 billion. `long` is required when n × (maximum value) can exceed that bound.
- **Inserting before the lookup** in the counting loop. The lookup must precede the insertion.
- **Difference array end index.** `diff` requires `n + 1` slots so that `diff[r + 1]` is a valid index when `r = n - 1`.
- **Divisible-by-k with negatives.** In Java `-7 % 5` is `-2`, not `3`. The remainder must be normalised with `((sum % k) + k) % k` before use as a map key.

2.1.5 Questions to expect

1. "Given an immutable array, answer many range-sum queries quickly." Clue: many queries, no updates. Build a prefix array; each query is O(1). Follow-up: if updates are interleaved with queries, prefix sums are insufficient and a Fenwick tree or segment tree is required.
2. "Count subarrays with sum exactly k; the array may contain negatives." Clue: an exact sum with negative elements excludes a sliding window. Prefix sum plus hash map in O(n).
3. "Longest subarray with equal numbers of 0s and 1s." Map 0 to -1, then find the longest subarray with sum 0 by storing the first index of each prefix value.
4. "Is there a subarray of length at least 2 whose sum is a multiple of k?" Store the first index of each `prefix % k`; a repeated remainder at distance at least 2 gives a positive answer.
5. "Apply a million range increments, then print the array." Difference array, O(n + m).
6. "Sum of a sub-rectangle of a matrix, many queries." 2D prefix sums, O(1) per query.
7. "Return the array where each element is the product of all the others, without division." A product analogue of prefix and suffix scans; each result is `prefixProduct * suffixProduct`.

INTERVIEW ANGLE

Why does "subarray sum equals k" not work with a sliding window? A sliding window shrinks when the sum is too large and grows when it is too small. This presupposes that adding an element never decreases the sum, that is, that all elements are non-negative. With negative elements the monotonicity precondition fails and the pointer logic is invalid. The prefix-sum approach does not depend on monotonicity: it tests only whether a prefix equal to `running - k` occurred earlier. It runs in O(n) time and O(n) space for arbitrary integers.

Preeram

2.1.6 LeetCode practice

#	Title	Level	Key idea
303	Range Sum Query - Immutable	Easy	Prefix array, answer each query by one subtraction
724	Find Pivot Index	Easy	Total sum minus left prefix minus current must equal left prefix
560	Subarray Sum Equals K	Medium	Prefix sum plus hash map of counts
304	Range Sum Query 2D - Immutable	Medium	2D prefix with inclusion and exclusion
525	Contiguous Array	Medium	Map 0 to -1, store first index of each prefix value
523	Continuous Subarray Sum	Medium	Store first index of each prefix remainder mod k
974	Subarray Sums Divisible by K	Medium	Count equal remainders; normalise negative remainders
1109	Corporate Flight Bookings	Medium	Difference array over the booking ranges
1094	Car Pooling	Medium	Difference array over stops, check capacity on the running sum
238	Product of Array Except Self	Medium	Prefix products times suffix products, no division

LeetCode problems for this pattern.

KEY POINTS

Prefix sums and difference arrays

- `prefix[i]` is the sum of the first `i` elements; any range sum is `prefix[r+1] - prefix[l]`, an O(1) query after an O(n) build.
- For "count subarrays with sum k", keep a map of prefix values seen so far and look up `running - k` before inserting; seed it with `{0=1}`.
- A difference array applies range updates in O(1) each; a running sum rebuilds the final array.
- Use `long` for sums, size the arrays `n + 1`, and normalise negative remainders before using them as keys.

2.2 Two Pointers

Consider the problem of finding two elements of a sorted array whose sum equals a target. Exhaustive pair enumeration costs $O(n^2)$, about five billion pair checks for n = 100,000. The enumeration ignores the ordering of the input: when a pair sum is too large, the ordering identifies which end of the pair must change.

The **two-pointers** pattern maintains two indices into one array (or two arrays) and advances them by a fixed rule so that each index moves in one direction only. Since no index moves backwards, the total number of moves is at most 2n, giving O(n) instead of $O(n^2)$.

2.2.1 The idea

There are three common shapes:

- **Opposite ends.** `lo` starts at the left, `hi` at the right, and they move toward each other. Used on sorted arrays (pair sums, 3Sum), for palindromes, and for "container" problems where moving the shorter side is the only move that can help.
- **Same direction (read and write).** A `read` pointer scans every element and a `write` pointer marks the position of the next retained element. Used for removing duplicates, moving zeroes, partitioning in place.
- **Two arrays.** One pointer per array, advanced by comparing the current elements (merging two sorted arrays, intersection of two sorted lists).

How to recognise it. Clue phrases: "sorted array" plus "pair/triplet with sum"; "in place with O(1) extra space"; "remove duplicates / zeroes in place"; "palindrome"; "maximise area/width between two positions"; "merge two sorted arrays".

The correctness of the opposite-ends strategy rests on an **invariant**, a condition that holds before and after every iteration. For the sorted pair sum the invariant is: if a valid pair exists, it lies within `[lo, hi]`. If `a[lo] + a[hi]` is less

than the target, then `a[lo]` combined with any element at or left of `hi` is also less than the target, so `lo` cannot belong to a solution and may be discarded. The symmetric argument discards `hi` when the sum exceeds the target.

Template.java JAVA

```java
// opposite ends (sorted input)
int lo = 0, hi = a.length - 1;
while (lo < hi) {
    int sum = a[lo] + a[hi];
    if (sum == target) return new int[]{lo, hi};
    if (sum < target) lo++; else hi--;
}

// read / write pointers (in-place filter)
int w = 0;
for (int r = 0; r < a.length; r++) {
    if (keep(a[r])) a[w++] = a[r];
}
// a[0..w-1] is the result
```

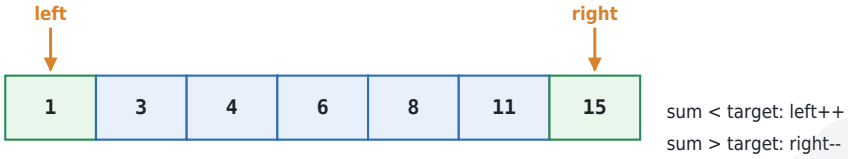


Figure 2.1. The two pointers approach each other on a sorted array; each step discards one end, so there are at most n steps in total.

2.2.2 Dry run

Problem: in the sorted array `[1, 3, 4, 6, 8, 11]` find two numbers that add up to 10. Each row is real output of the traced program.

step	lo	hi	a[lo], a[hi]	sum	action
1	0	5	1, 11	12	too big, hi--
2	0	4	1, 8	9	too small, lo++
3	1	4	3, 8	11	too big, hi--
4	1	3	3, 6	9	too small, lo++
5	2	3	4, 6	10	match, return [2, 3]

Two sum on a sorted array, target 10.

The search terminates after five steps for six elements; the pointers moved inward only. The container problem is analysed next. The heights are `[1, 8, 6, 2, 5, 4, 8, 3, 7]`; the water held between lines `lo` and `hi` is `min(h[lo], h[hi]) * (hi - lo)`. The **shorter** line is always advanced, because the area is bounded by it: retaining the shorter line while reducing the width cannot yield a larger area.

step	lo	hi	h[lo]	h[hi]	area	best
1	0	8	1	7	8	8
2	1	8	8	7	49	49
3	1	7	8	3	18	49
4	1	6	8	8	40	49
5	1	5	8	4	16	49
6	1	4	8	5	15	49
7	1	3	8	2	4	49
8	1	2	8	6	6	49

Container with most water.

sreram

Step 1 advances `lo` because 1 is shorter than 7. Step 2 attains the maximum area, 49 (width 7, height 7). The remaining steps reduce the width and none exceeds 49.

sreeram

2.2.3 Java code

Main.java JAVA

```java
import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

public class Main {
    // Opposite ends: sorted array, find a pair with the given sum. Returns indices or {-1,-1}.
    static int[] twoSumSorted(int[] a, int target, boolean trace) {
        int lo = 0, hi = a.length - 1, step = 1;
        while (lo < hi) {
            int sum = a[lo] + a[hi];
            if (trace) {
                System.out.printf("step %d lo=%d hi=%d a[lo]=%d a[hi]=%d sum=%d%n",
                        step++, lo, hi, a[lo], a[hi], sum);
            }
            if (sum == target) return new int[]{lo, hi};
            if (sum < target) lo++; else hi--;
        }
        return new int[]{-1, -1};
    }

    // Same direction: write pointer 'w' trails read pointer 'r'.
    static int removeDuplicates(int[] a) {
        if (a.length == 0) return 0;
        int w = 1;
        for (int r = 1; r < a.length; r++) {
            if (a[r] != a[w - 1]) a[w++] = a[r];
        }
        return w;
    }

    static void moveZeroes(int[] a) {
        int w = 0;
        for (int r = 0; r < a.length; r++) {
            if (a[r] != 0) {
                int t = a[w]; a[w] = a[r]; a[r] = t;
                w++;
            }
        }
    }

    // 3Sum: sort, fix i, two pointers on the rest, skip duplicates.
    static List<List<Integer>> threeSum(int[] nums) {
        int[] a = nums.clone();
        Arrays.sort(a);
        List<List<Integer>> out = new ArrayList<>();
        for (int i = 0; i < a.length - 2; i++) {
            if (a[i] > 0) break;
            if (i > 0 && a[i] == a[i - 1]) continue;
            int lo = i + 1, hi = a.length - 1;
            while (lo < hi) {
                int sum = a[i] + a[lo] + a[hi];
                if (sum < 0) lo++;
                else if (sum > 0) hi--;
                else {
                    out.add(List.of(a[i], a[lo], a[hi]));
                    lo++; hi--;
                    while (lo < hi && a[lo] == a[lo - 1]) lo++;
                    while (lo < hi && a[hi] == a[hi + 1]) hi--;
                }
            }
        }
        return out;
    }

    static int maxArea(int[] h, boolean trace) {
        int lo = 0, hi = h.length - 1, best = 0, step = 1;
        while (lo < hi) {
            int area = Math.min(h[lo], h[hi]) * (hi - lo);
            best = Math.max(best, area);
            if (trace) {
                System.out.printf("step %d lo=%d hi=%d h[lo]=%d h[hi]=%d area=%2d best=%2d%n",
                        step++, lo, hi, h[lo], h[hi], area, best);
            }
```

Sreeram

Main.java (continued) JAVA

```
            if (h[lo] < h[hi]) lo++; else hi--;
        }
        return best;
    }

    static int maxAreaBrute(int[] h) {
        int best = 0;
        for (int i = 0; i < h.length; i++)
            for (int j = i + 1; j < h.length; j++)
                best = Math.max(best, Math.min(h[i], h[j]) * (j - i));
        return best;
    }

    static boolean isPalindrome(String s) {
        int lo = 0, hi = s.length() - 1;
        while (lo < hi) {
            while (lo < hi && !Character.isLetterOrDigit(s.charAt(lo))) lo++;
            while (lo < hi && !Character.isLetterOrDigit(s.charAt(hi))) hi--;
            if (Character.toLowerCase(s.charAt(lo)) != Character.toLowerCase(s.charAt(hi))) {
                return false;
            }
            lo++; hi--;
        }
        return true;
    }

    public static void main(String[] args) {
        int[] sorted = {1, 3, 4, 6, 8, 11};
        System.out.println("-- two sum on sorted, target 10 --");
        System.out.println(Arrays.toString(twoSumSorted(sorted, 10, true)));

        int[] heights = {1, 8, 6, 2, 5, 4, 8, 3, 7};
        System.out.println("-- container with most water --");
        System.out.println("answer = " + maxArea(heights, true));
        System.out.println("brute  = " + maxAreaBrute(heights));

        int[] d = {0, 0, 1, 1, 1, 2, 2, 3};
        int len = removeDuplicates(d);
        System.out.println("unique length " + len + " -> "
                + Arrays.toString(Arrays.copyOf(d, len)));
        int[] z = {0, 1, 0, 3, 12};
        moveZeroes(z);
        System.out.println("moveZeroes -> " + Arrays.toString(z));
        System.out.println("3sum -> " + threeSum(new int[]{-1, 0, 1, 2, -1, -4}));
        System.out.println("3sum all zeros -> " + threeSum(new int[]{0, 0, 0, 0}));
        System.out.println(isPalindrome("A man, a plan, a canal: Panama"));
        System.out.println(isPalindrome("race a car"));
    }
}
```

Run it TERMINAL

```
$ java Main.java
-- two sum on sorted, target 10 --
step 1 lo=0 hi=5 a[lo]=1 a[hi]=11 sum=12
step 2 lo=0 hi=4 a[lo]=1 a[hi]=8 sum=9
step 3 lo=1 hi=4 a[lo]=3 a[hi]=8 sum=11
step 4 lo=1 hi=3 a[lo]=3 a[hi]=6 sum=9
step 5 lo=2 hi=3 a[lo]=4 a[hi]=6 sum=10
[2, 3]
-- container with most water --
step 1 lo=0 hi=8 h[lo]=1 h[hi]=7 area= 8 best= 8
step 2 lo=1 hi=8 h[lo]=8 h[hi]=7 area=49 best=49
step 3 lo=1 hi=7 h[lo]=8 h[hi]=3 area=18 best=49
step 4 lo=1 hi=6 h[lo]=8 h[hi]=8 area=40 best=49
step 5 lo=1 hi=5 h[lo]=8 h[hi]=4 area=16 best=49
step 6 lo=1 hi=4 h[lo]=8 h[hi]=5 area=15 best=49
step 7 lo=1 hi=3 h[lo]=8 h[hi]=2 area= 4 best=49
step 8 lo=1 hi=2 h[lo]=8 h[hi]=6 area= 6 best=49
answer = 49
brute  = 49
unique length 4 -> [0, 1, 2, 3]
moveZeroes -> [1, 3, 12, 0, 0]
```

Run it (continued) TERMINAL

```
3sum -> [[-1, -1, 2], [-1, 0, 1]]
3sum all zeros -> [[0, 0, 0]]
true
false
```

What to notice. In `removeDuplicates` the write pointer `w` addresses the next free slot, and a value is retained only if it differs from the last retained value `a[w - 1]`. In `threeSum` the outer index is skipped when it equals its predecessor, and after a triple is recorded both inner pointers skip repeated values; duplicates are thereby excluded without a `Set`. The early `break` when `a[i] > 0` is valid because the array is sorted and three positive values cannot sum to zero. `List.of` produces an immutable list, which suffices for results.

2.2.4 Complexity and mistakes

Opposite-ends and read/write scans run in O(n) time and O(1) extra space, because each pointer moves at most n times. 3Sum is $O(n^2)$: sorting costs O(n log n), after which each of the n choices of the first element is followed by a linear two-pointer scan. `Arrays.sort` on a primitive `int[]` is the standard choice.

Common mistakes:

- **Incorrect loop condition.** Pairs require `lo < hi`; `lo <= hi` would pair an element with itself.
- **Applying the pattern to unsorted data.** The inward-move argument holds only for sorted input; an unsorted pair sum requires a hash map.
- **Duplicate triples in 3Sum.** Failing to skip equal neighbours (for `i`, and for `lo` and `hi` after a match) yields repeated results.
- **Out-of-bounds access in skip loops.** The condition `lo < hi` must be retained inside every `while` loop that skips duplicates.
- **Overflow.** `a[lo] + a[hi]` can overflow `int` when values are near 2^31. A cast to `long` is required if the constraints permit large values.
- **Advancing the wrong side in the container problem.** The shorter wall must be advanced, never the taller.

2.2.5 Questions to expect

1. "Two sum, but the array is sorted." Clue: sorted input. Opposite ends give O(n) time and O(1) space (a hash map also works but uses O(n) space).
2. "Find all unique triplets that sum to zero." Clue: triplet plus uniqueness. Sort, fix one element, apply two pointers to the remainder, skip duplicates. Follow-up: 4Sum adds one loop ($O(n^3)$); k-Sum is recursion over the pair scan.
3. "Remove duplicates from a sorted array in place; return the new length." Clue: in place, sorted. Read/write pointers. Follow-up: allowing each value at most twice requires comparison with `a[w - 2]`.
4. "Move all zeroes to the end, keep the order of the others." Clue: stable partition in place. Swap each non-zero element to the write position.
5. "Is this string a palindrome, ignoring punctuation and case?" Clue: palindrome. Opposite ends with skipping. Follow-up: with one deletion permitted, test both skip options at the first mismatch.
6. "Container with the most water." Clue: maximise area between two positions. Opposite ends, move the shorter side.
7. "Sort an array of 0s, 1s and 2s in one pass." Clue: three-way partition. Three pointers (Dutch national flag partition).
8. "Squares of a sorted array (which may contain negatives), returned sorted." Clue: the largest square lies at one of the two ends. Opposite pointers fill the result from the back.

INTERVIEW ANGLE

Why is the container solution correct? Can the optimal pair be skipped? Assume the left wall is shorter. Any container retaining this wall with a closer right wall has a smaller width and is bounded by the same height, so its capacity does not exceed the one just measured. The left wall can therefore be discarded and `lo` advanced. Each step permanently discards one wall without excluding the optimum, and the loop terminates after n - 1 steps: O(n) time, O(1) space.

Sreeram

2.2.6 LeetCode practice

#	Title	Level	Key idea
125	Valid Palindrome	Easy	Opposite ends, skip non-alphanumeric characters
26	Remove Duplicates from Sorted Array	Easy	Read and write pointers, compare with last kept
283	Move Zeroes	Easy	Swap each non-zero into the write position
977	Squares of a Sorted Array	Easy	Compare absolute values at both ends, fill from the back
680	Valid Palindrome II	Easy	On first mismatch check both skip-one cases
167	Two Sum II - Input Array Is Sorted	Medium	Opposite ends on sorted input
75	Sort Colors	Medium	Three pointers partition in one pass
11	Container With Most Water	Medium	Move the shorter wall
15	3Sum	Medium	Sort, fix one, two pointers, skip duplicates
16	3Sum Closest	Medium	Same as 3Sum, track the smallest distance to target
18	4Sum	Medium	Two nested loops plus a two-pointer scan
42	Trapping Rain Water	Hard	Two pointers with running left and right maxima

LeetCode problems for this pattern.

KEY POINTS

Two pointers

- Pointers move in one direction each, so the work is O(n) after at most an O(n log n) sort.
- Opposite-ends movement requires a justification (sorted order or a monotone quantity) for discarding one side; the invariant should be stated explicitly.
- Read/write pointers filter or partition in place with O(1) extra space.
- For k-Sum problems: sort, fix elements with loops, finish with a pair scan, and skip duplicates explicitly.

2.3 Sliding Window: Fixed Size

Consider the problem of finding the maximum sum over all groups of k consecutive elements. Recomputing each group from scratch costs O(n × k), which approaches O(n²) when k is proportional to n. Adjacent windows, however, share all but two elements, and recomputing the shared part is redundant.

A **sliding window** exploits that overlap. For a window of fixed size k, a shift of one position adds the entering element on the right and removes the leaving element on the left. Each shift costs O(1), so the complete scan is O(n).

2.3.1 The idea

A running value (a sum, a count or a frequency table) is maintained for the current window. The first window is built in k steps. Thereafter, for each `right` from k to n-1, `a[right]` is added, `a[right - k]` is removed, and the answer is updated.

How to recognise it. The window length is fixed by the statement: "subarray of size k", "every k consecutive", "substring of length k", "k cards from the ends" (the complement is a window of size n - k), "all anagrams of p in s" (window length equals `p.length()`), "repeated 10-letter sequences".

Template.java — JAVA

```java
int window = 0;
for (int i = 0; i < k; i++) window += a[i];      // first window
int best = window;
for (int right = k; right < a.length; right++) {
    window += a[right] - a[right - k];           // enter right, leave right-k
    best = Math.max(best, window);
}
```

When the window state is a frequency table (anagram problems), "add" and "remove" become `count[c]++` and `count[c]--`, and the check is a comparison of two 26-slot arrays, which costs 26 steps, a constant.

2.3.2 Dry run

Input: `a = [2, 1, 5, 1, 3, 2]`, `k = 3`. Each row is real output.

step	action	window	sum	best	note
0	build first window	[0..2]	8	8	2 + 1 + 5
1	add a[3]=1, drop a[0]=2	[1..3]	7	8	8 + 1 - 2
2	add a[4]=3, drop a[1]=1	[2..4]	9	9	7 + 3 - 1
3	add a[5]=2, drop a[2]=5	[3..5]	6	9	9 + 2 - 5

Maximum sum of 3 consecutive elements.

Four window updates replace 4 × 3 = 12 element additions. The maximum is attained by `[5, 1, 3]` with sum 9.

sreeram

2.3.3 Java code

Main.java JAVA

```java
import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

public class Main {
    // Maximum sum of any k consecutive elements.
    static int maxSumWindow(int[] a, int k, boolean trace) {
        int window = 0;
        for (int i = 0; i < k; i++) window += a[i];
        int best = window;
        if (trace) System.out.printf("init window [0..%d] sum=%d best=%d%n", k - 1, window, best);
        for (int right = k; right < a.length; right++) {
            int left = right - k;                 // element leaving the window
            window += a[right] - a[left];
            best = Math.max(best, window);
            if (trace) {
                System.out.printf("add a[%d]=%d drop a[%d]=%d -> window [%d..%d] sum=%d best=%d%n",
                        right, a[right], left, a[left], left + 1, right, window, best);
            }
        }
        return best;
    }

    static int maxSumBrute(int[] a, int k) {
        int best = Integer.MIN_VALUE;
        for (int i = 0; i + k <= a.length; i++) {
            int sum = 0;
            for (int j = i; j < i + k; j++) sum += a[j];
            best = Math.max(best, sum);
        }
        return best;
    }

    static double maxAverage(int[] a, int k) {
        long window = 0;
        for (int i = 0; i < k; i++) window += a[i];
        long best = window;
        for (int i = k; i < a.length; i++) {
            window += a[i] - a[i - k];
            best = Math.max(best, window);
        }
        return (double) best / k;
    }

    // Fixed window over a string: start indices of anagrams of p inside s.
    static List<Integer> findAnagrams(String s, String p) {
        List<Integer> out = new ArrayList<>();
        if (p.length() > s.length()) return out;
        int[] need = new int[26], have = new int[26];
        for (char c : p.toCharArray()) need[c - 'a']++;
        for (int i = 0; i < s.length(); i++) {
            have[s.charAt(i) - 'a']++;
            if (i >= p.length()) have[s.charAt(i - p.length()) - 'a']--;
            if (i >= p.length() - 1 && Arrays.equals(need, have)) out.add(i - p.length() + 1);
        }
        return out;
    }

    // Max vowels in any substring of length k.
    static int maxVowels(String s, int k) {
        int cur = 0, best = 0;
        for (int i = 0; i < s.length(); i++) {
            if ("aeiou".indexOf(s.charAt(i)) >= 0) cur++;
            if (i >= k && "aeiou".indexOf(s.charAt(i - k)) >= 0) cur--;
            best = Math.max(best, cur);
        }
        return best;
    }

    public static void main(String[] args) {
        int[] a = {2, 1, 5, 1, 3, 2};
        System.out.println("-- max sum of 3 consecutive --");
        System.out.println("answer = " + maxSumWindow(a, 3, true));
```

Freeram

Main.java (continued) JAVA

```
        System.out.println("brute  = " + maxSumBrute(a, 3));
        System.out.println("max average k=4: " + maxAverage(new int[]{1, 12, -5, -6, 50, 3}, 4));
        System.out.println("anagrams: " + findAnagrams("cbaebabacd", "abc"));
        System.out.println("anagrams: " + findAnagrams("abab", "ab"));
        System.out.println("vowels: " + maxVowels("abciiidef", 3));
        int[] big = {Integer.MAX_VALUE, Integer.MAX_VALUE, Integer.MAX_VALUE};
        int bad = 0;
        for (int x : big) bad += x;
        long ok = 0;
        for (int x : big) ok += x;
        System.out.println("int sum = " + bad + ", long sum = " + ok);
    }
}
```

Run it TERMINAL

```
$ java Main.java
-- max sum of 3 consecutive --
init window [0..2] sum=8 best=8
add a[3]=1 drop a[0]=2 -> window [1..3] sum=7 best=8
add a[4]=3 drop a[1]=1 -> window [2..4] sum=9 best=9
add a[5]=2 drop a[2]=5 -> window [3..5] sum=6 best=9
answer = 9
brute  = 9
max average k=4: 12.75
anagrams: [0, 6]
anagrams: [0, 1, 2]
vowels: 3
int sum = 2147483645, long sum = 6442450941
```

What to notice. `findAnagrams` removes the leaving character only once the window is full (`i >= p.length()`) and compares only when the window holds `p.length()` characters. `Arrays.equals(int[], int[])` compares contents, whereas `==` on arrays compares references. In `maxAverage` the running sum is a `long`; the last output line shows the reason: three `Integer.MAX_VALUE` values sum to `2147483645` in `int` (after wrap-around) but to `6442450941` in `long`.

2.3.4 Complexity and mistakes

Time is O(n): one pass, constant work per step (O(n × 26) for the anagram check, still linear). Space is O(1) for sums and O(alphabet) for frequency tables.

- **Off-by-one on the leaving element.** The element leaving when `right` enters is `a[right - k]`, not `a[right - k + 1]`. Tracing the first two windows verifies the index.
- **k larger than n.** The precondition `k <= a.length` must be checked before the first window is built.
- **Overflow** of the running sum: `long` is required when values or k are large.
- **Initialising** `best` **to 0** when all elements may be negative; the first window value is the correct initial value.
- **Premature division.** For an average, the integer sum is tracked and divided once at the end; repeated division accumulates rounding error.

2.3.5 Questions to expect

1. "Maximum average of any subarray of length k." Clue: fixed length. Maximise the sum and divide once at the end.
2. "Find all start indices of anagrams of p in s." Clue: window length equals `p.length()`. Slide a 26-slot count array and compare after each shift.
3. "Does s2 contain a permutation of s1?" As above; return true at the first match.
4. "Pick exactly k cards from either end to maximise the score." Clue: the cards not taken form a contiguous block of n - k. Minimise that window, or slide the split point.
5. "Maximum number of vowels in a substring of length k." Update the vowel count for the entering and the leaving character.
6. "Is there a duplicate within distance k?" Window of size k as a `HashSet`: add the new element, remove `a[i - k]`.
7. "Maximum of every window of size k." The running sum is replaced by a monotonic deque (treated in the chapter on queues and deques); O(n) overall.

INTERVIEW ANGLE
The fixed-window loop is O(n). Can the asymptotic bound be improved? No, for a single query: every element must be inspected at least once, so Ω(n) is a lower bound. For many window-sum queries on the same array, prefix sums are preferable: O(n) preprocessing and O(1) per query for any window size.

2.3.6 LeetCode practice

#	Title	Level	Key idea
643	Maximum Average Subarray I	Easy	Running sum of size k, divide once
219	Contains Duplicate II	Easy	HashSet holding the last k elements
1343	Number of Sub-arrays of Size K and Average Greater than or Equal to Threshold	Medium	Compare window sum with k times threshold
1456	Maximum Number of Vowels in a Substring of Given Length	Medium	Add and remove a vowel count
1052	Grumpy Bookstore Owner	Medium	Best fixed window of extra satisfied customers
1423	Maximum Points You Can Obtain from Cards	Medium	Minimise the middle window of size n - k
2090	K Radius Subarray Averages	Medium	Window of size 2k + 1 with long sums
438	Find All Anagrams in a String	Medium	Slide a 26-slot count array
567	Permutation in String	Medium	Same count-array comparison, stop at first match
187	Repeated DNA Sequences	Medium	Slide a 10-character window, HashSet of seen
239	Sliding Window Maximum	Hard	Monotonic deque of candidate indices

LeetCode problems for this pattern.

KEY POINTS
Fixed-size sliding window

- Build the first window, then slide by adding `a[right]` and removing `a[right - k]`: O(n) total.
- Window state can be a sum, a count, or a frequency array compared with `Arrays.equals`.
- Watch the leaving index, k greater than n, and `int` overflow of the sum.
- If the window size is not fixed, the variable-size version of the pattern applies.

2.4 Sliding Window: Variable Size

Here the window length is not given, as in "longest substring without repeating characters" or "smallest subarray with sum at least s". Enumerating every start and end gives O(n²) windows, and validating each in O(n) gives O(n³), which is infeasible for n = 100,000.

The **variable-size sliding window** maintains a window `[left, right]` that grows and shrinks. The right edge advances to admit a new element; the left edge advances only to restore validity when the window rule is violated. Since both edges move monotonically to the right, there are at most 2n moves in total: O(n).

2.4.1 The idea

There are two variants, which differ in the position at which the answer is updated:

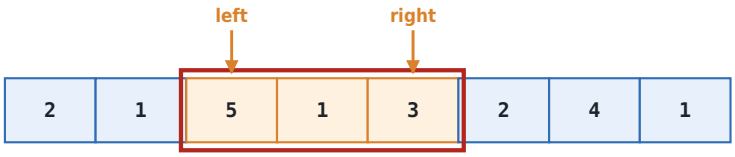
- **Longest valid window.** Expand `right`. While the window is invalid, advance `left` until validity is restored. Then record `right - left + 1`.
- **Shortest valid window.** Expand `right` until the window is valid. While it remains valid, record its length and advance `left`.

How to recognise it. The statement asks for the longest or shortest contiguous subarray or substring subject to a condition: "without repeating", "at most k distinct", "sum at least s", "contains all characters of t", "at most k replacements". The condition must be **monotone**: removing elements from a valid window preserves validity (longest variant), or adding elements to a valid window preserves validity (shortest variant). Hence non-negative elements are a precondition for "sum at least s"; with negative values validity cannot be restored by moving one edge.

Template.java JAVA

```java
// longest window satisfying a rule
int left = 0, best = 0;
for (int right = 0; right < n; right++) {
    add(a[right]);                      // include the new element
    while (!valid()) {                  // shrink until valid again
        remove(a[left]);
        left++;
    }
    best = Math.max(best, right - left + 1);
}
```

For "exactly k" conditions the standard reduction is `exactly(k) = atMost(k) - atMost(k - 1)`, because "at most" conditions are monotone while "exactly" conditions are not.



window [left, right]: expand right; shrink left to restore validity

Figure 2.2. The right edge only advances; the left edge advances only to restore validity, so no element is visited more than twice.

2.4.2 Dry run

Problem: longest substring without repeating characters in `"abcabcbb"`. The window holds a set of its characters. Each row is real output of the traced program (the window is shown after repairs).

step	right	char	left	window	length	best
1	0	a	0	a	1	1
2	1	b	0	ab	2	2
3	2	c	0	abc	3	3
4	3	a	1	bca	3	3
5	4	b	2	cab	3	3
6	5	c	3	abc	3	3
7	6	b	5	cb	2	3
8	7	b	7	b	1	3

Longest substring without repeating characters: abcabcbb.

At step 4 the incoming `a` is already in the set, so the left edge advances by one (removing the old `a`) and the window becomes `bca`. At step 7 the incoming `b` conflicts with the `b` at index 4: the left edge removes `a` and then `b` (one `while` iteration each), leaving `c`, and `cb` after the insertion. At step 8 the conflict is with the immediately preceding `b`, so the window reduces to the single `b`. The answer is 3.

sreram

2.4.3 Java code

Main.java JAVA

```java
import java.util.HashMap;
import java.util.HashSet;
import java.util.Map;
import java.util.Set;

public class Main {
    // Longest substring with all distinct characters (set version, with trace).
    static int lengthOfLongest(String s, boolean trace) {
        Set<Character> inWindow = new HashSet<>();
        int left = 0, best = 0, step = 1;
        for (int right = 0; right < s.length(); right++) {
            char c = s.charAt(right);
            while (inWindow.contains(c)) {
                inWindow.remove(s.charAt(left));
                left++;
            }
            inWindow.add(c);
            best = Math.max(best, right - left + 1);
            if (trace) {
                String win = s.substring(left, right + 1);
                System.out.printf("step %d right=%d char=%s left=%d window=%s len=%d best=%d%n",
                        step++, right, c, left, win, right - left + 1, best);
            }
        }
        return best;
    }

    // Same problem with a last-seen map: the left edge jumps.
    static int lengthOfLongestJump(String s) {
        Map<Character, Integer> last = new HashMap<>();
        int left = 0, best = 0;
        for (int right = 0; right < s.length(); right++) {
            char c = s.charAt(right);
            if (last.containsKey(c)) left = Math.max(left, last.get(c) + 1);
            last.put(c, right);
            best = Math.max(best, right - left + 1);
        }
        return best;
    }

    static int lengthOfLongestBrute(String s) {
        int best = 0;
        for (int i = 0; i < s.length(); i++) {
            Set<Character> seen = new HashSet<>();
            for (int j = i; j < s.length(); j++) {
                if (!seen.add(s.charAt(j))) break;
                best = Math.max(best, j - i + 1);
            }
        }
        return best;
    }

    // Longest substring with at most k distinct characters.
    static int longestKDistinct(String s, int k) {
        Map<Character, Integer> count = new HashMap<>();
        int left = 0, best = 0;
        for (int right = 0; right < s.length(); right++) {
            count.merge(s.charAt(right), 1, Integer::sum);
            while (count.size() > k) {
                char out = s.charAt(left++);
                if (count.merge(out, -1, Integer::sum) == 0) count.remove(out);
            }
            best = Math.max(best, right - left + 1);
        }
        return best;
    }

    // Longest repeating character replacement.
    static int characterReplacement(String s, int k) {
        int[] count = new int[26];
        int left = 0, maxFreq = 0, best = 0;
        for (int right = 0; right < s.length(); right++) {
            maxFreq = Math.max(maxFreq, ++count[s.charAt(right) - 'A']);
```

Sreeram

Main.java (continued) JAVA

```
            while (right - left + 1 - maxFreq > k) {
                count[s.charAt(left++) - 'A']--;
            }
            best = Math.max(best, right - left + 1);
        }
        return best;
    }

    // Minimum window substring: smallest window of s containing all chars of t.
    static String minWindow(String s, String t, boolean trace) {
        int[] need = new int[128];
        for (char c : t.toCharArray()) need[c]++;
        int missing = t.length();
        int left = 0, bestStart = 0, bestLen = Integer.MAX_VALUE;
        for (int right = 0; right < s.length(); right++) {
            if (need[s.charAt(right)]-- > 0) missing--;
            while (missing == 0) {
                if (right - left + 1 < bestLen) {
                    bestLen = right - left + 1;
                    bestStart = left;
                    if (trace) System.out.println("  valid window [" + left + ".." + right + "] = "
                            + s.substring(left, right + 1));
                }
                if (++need[s.charAt(left++)] > 0) missing++;
            }
        }
        return bestLen == Integer.MAX_VALUE ? "" : s.substring(bestStart, bestStart + bestLen);
    }

    // Subarrays with sum <= goal (non-negative array). Exactly g = atMost(g) - atMost(g-1).
    static int atMost(int[] a, int goal) {
        if (goal < 0) return 0;
        int left = 0, sum = 0, count = 0;
        for (int right = 0; right < a.length; right++) {
            sum += a[right];
            while (sum > goal) sum -= a[left++];
            count += right - left + 1;
        }
        return count;
    }

    public static void main(String[] args) {
        System.out.println("-- longest substring without repeats: abcabcbb --");
        System.out.println("answer = " + lengthOfLongest("abcabcbb", true));
        System.out.println("jump   = " + lengthOfLongestJump("abcabcbb"));
        System.out.println("brute  = " + lengthOfLongestBrute("abcabcbb"));
        System.out.println("abba -> " + lengthOfLongest("abba", false) + ", jump "
                + lengthOfLongestJump("abba"));
        System.out.println("at most 2 distinct in eceba: " + longestKDistinct("eceba", 2));
        System.out.println("replacement AABABBA k=1: " + characterReplacement("AABABBA", 1));
        System.out.println("-- minimum window ADOBECODEBANC / ABC --");
        System.out.println("answer = " + minWindow("ADOBECODEBANC", "ABC", true));
        int[] bin = {1, 0, 1, 0, 1};
        System.out.println("exactly 2: " + (atMost(bin, 2) - atMost(bin, 1)));
    }
}
```

Run it TERMINAL

```
$ java Main.java
-- longest substring without repeats: abcabcbb --
step 1 right=0 char=a left=0 window=a len=1 best=1
step 2 right=1 char=b left=0 window=ab len=2 best=2
step 3 right=2 char=c left=0 window=abc len=3 best=3
step 4 right=3 char=a left=1 window=bca len=3 best=3
step 5 right=4 char=b left=2 window=cab len=3 best=3
step 6 right=5 char=c left=3 window=abc len=3 best=3
step 7 right=6 char=b left=5 window=cb len=2 best=3
step 8 right=7 char=b left=7 window=b len=1 best=3
answer = 3
jump   = 3
brute  = 3
abba -> 2, jump 2
at most 2 distinct in eceba: 3
```

Run it (continued) TERMINAL

```
replacement AABABBA k=1: 4
-- minimum window ADOBECODEBANC / ABC --
  valid window [0..5] = ADOBEC
  valid window [8..12] = EBANC
  valid window [9..12] = BANC
answer = BANC
exactly 2: 4
```

What to notice. The `jump` version stores the last index of each character, so `left` advances directly past the previous occurrence. The `Math.max(left, ...)` is essential: in `"abba"`, when the final `a` arrives its last index is 0 while `left` is already 2, and moving `left` back to 1 would violate the window invariant. In `minWindow`, the `need` array goes negative for surplus characters, so only a decrement from a positive value reduces `missing`; the condition "all required characters present" is thereby tested in O(1). The three "valid window" lines illustrate the shortest-window variant: each valid window is recorded and then shrunk. `characterReplacement` never lowers `maxFreq` when the window shrinks; this is deliberate, because the answer can improve only when `maxFreq` actually increases.

2.4.4 Complexity and mistakes

Time is O(n): `right` advances n times and `left` at most n times in total, although a `while` loop is nested in a `for` loop (amortised analysis). Space is O(min(n, alphabet)) for the set or map. The brute force is $O(n^2)$ with an early break, or $O(n^3)$ without one.

- **Updating the answer at the wrong point.** For the longest window the update follows the repair; for the shortest it occurs while the window is valid and before shrinking.
- **Shrinking with `if` instead of `while`.** A single removal may not restore validity.
- **Omitting `Math.max(left, last + 1)`** in the jump version (the `"abba"` defect).
- **Retained map entries.** If `count.size()` is used as the distinct count, a key must be deleted when its count reaches 0.
- **Applying the pattern to negative numbers** under sum conditions. Prefix sums with a hash map are required instead.
- **Characters outside the table.** `need[128]` covers ASCII only; arbitrary Unicode requires a `HashMap<Character, Integer>`.

2.4.5 Questions to expect

1. "Longest substring without repeating characters." Clue: longest, no repeats. Set or last-seen map.
2. "Shortest subarray of positive integers with sum at least s." Clue: shortest, positive elements. Expand until valid, then shrink while valid.
3. "Longest substring with at most k distinct characters." Clue: at most k distinct. Frequency map; shrink while `map.size() > k`. Follow-up: fruit baskets is the case k = 2.
4. "Smallest window of s containing all characters of t." Clue: shortest, must contain a multiset. Need-array with a `missing` counter.
5. "Longest run of equal letters after at most k replacements." Clue: window length minus the maximum letter frequency must not exceed k.
6. "Longest run of 1s if at most k zeroes may be flipped." Same shape: window with at most k zeroes.
7. "Count subarrays with exactly k distinct values." Clue: exactly k. `atMost(k) - atMost(k - 1)`.
8. "Do this on a stream." The window state consists of counters, so elements are processed one at a time with O(window) memory.

INTERVIEW ANGLE

Why is the window algorithm O(n) when a while loop is nested in a for loop? The pointer moves are counted rather than the loops. `right` is incremented once per outer iteration, n times in total. `left` is incremented in the inner loop, but it never passes `right`, so it moves at most n times over the whole run. The total is at most 2n pointer moves, hence O(n) amortised.

2.4.6 LeetCode practice

#	Title	Level	Key idea
3	Longest Substring Without Repeating Characters	Medium	Set or last-seen map; shrink on duplicate
209	Minimum Size Subarray Sum	Medium	Shortest window with sum at least target (positives)
904	Fruit Into Baskets	Medium	Longest window with at most two distinct values
424	Longest Repeating Character Replacement	Medium	Window length minus max frequency at most k
1004	Max Consecutive Ones III	Medium	Longest window with at most k zeroes
930	Binary Subarrays With Sum	Medium	atMost(goal) minus atMost(goal - 1)
1838	Frequency of the Most Frequent Element	Medium	Sort, window where cost to raise all to max is within k
1658	Minimum Operations to Reduce X to Zero	Medium	Longest middle window with sum total - x
76	Minimum Window Substring	Hard	Need-array with a missing counter
992	Subarrays with K Different Integers	Hard	atMost(k) minus atMost(k - 1)

LeetCode problems for this pattern.

KEY POINTS

Variable-size sliding window

- Right edge expands every step; left edge moves forward only to repair the window. Total O(n).
- Longest window: record the answer after repairing. Shortest window: record while valid, then shrink.
- The rule must be monotone; for sums, this means non-negative values.
- "Exactly k" is `atMost(k) - atMost(k - 1)`.
- Use `while` (not `if`) for shrinking, and `long` for sums that can get large.

2.5 Hash Map and Frequency Counting

Many array and string problems require deciding whether an element occurred earlier or how many times it occurred. Scanning the preceding elements costs O(n) per query and O(n²) overall. A **hash table** answers both questions in O(1) on average, so a single pass suffices.

In Java the available structures are `HashMap<K, V>` (key to value), `HashSet<E>` (membership only), and `int[26]` or `int[128]` arrays when the keys are letters. The choice among them is part of the solution.

2.5.1 The idea

The pattern has four common forms:

- **Storing earlier elements to locate a partner.** Two sum: for each `x`, search for `target - x` among earlier elements.
- **Counting frequencies and comparing.** Anagrams, top k frequent, majority element. Two strings are anagrams exactly when their letter counts are equal.
- **Canonical form as key.** Group anagrams: the sorted letters of each word (or the 26 counts) serve as the map key. The same technique groups shifted strings and isomorphic patterns.
- **Set membership and run-start tests.** Longest consecutive sequence: counting begins only at a number whose predecessor is absent.

How to recognise it. "Find two elements that...", "first non-repeating", "duplicate", "anagram", "group by", "most frequent", "consecutive numbers in any order", and any statement that excludes sorting or requires better than O(n log n).

sreeram

Template.java JAVA

```java
Map<Integer, Integer> freq = new HashMap<>();
for (int x : nums) freq.merge(x, 1, Integer::sum);       // count

Map<String, List<String>> groups = new HashMap<>();
groups.computeIfAbsent(key, k -> new ArrayList<>()).add(word);   // group

Set<Integer> seen = new HashSet<>();
if (!seen.add(x)) { /* x was already present */ }        // duplicate test
```

`HashMap` provides O(1) average `get` and `put`. In the worst case a bucket with many collisions is converted (since Java 8) into a balanced tree, bounding the cost of adversarial keys by O(log n) per operation. A `TreeMap` costs O(log n) per operation and keeps keys sorted. For lowercase-letter keys an `int[26]` is faster and simpler than any map.

2.5.2 Dry run

Problem: longest consecutive sequence in `[100, 4, 200, 1, 3, 2]` in O(n) without sorting. All numbers are inserted into a set. For each number `x`: if `x - 1` is in the set, `x` is inside a run and is skipped; otherwise `x` starts a run, which is extended while `cur + 1` is present.

x	is x-1 in the set?	action	best
100	no	start; 101 absent; run length 1	1
4	yes (3)	not a start, skip	1
200	no	start; 201 absent; run length 1	1
1	no	start; walk 2, 3, 4; run 1..4 length 4	4
3	yes (2)	skip	4
2	yes (1)	skip	4

Longest consecutive sequence.

The inner `while` loop executes only from run starts, so each number is traversed at most once in total: O(n) despite the nested loop. For the two sum trace, with `[2, 11, 7, 15, 3]` and target 9: at `i = 0` the map is empty and `2 -> 0` is stored; at `i = 1`, `need = -2` is absent and `11 -> 1` is stored; at `i = 2`, `x = 7` and `need = 2` is present at index 0, so the result is `[0, 2]`.

sreeram

2.5.3 Java code

Main.java JAVA

```java
import java.util.ArrayList;
import java.util.Arrays;
import java.util.HashMap;
import java.util.HashSet;
import java.util.List;
import java.util.Map;
import java.util.Set;

public class Main {
    static int[] twoSum(int[] nums, int target, boolean trace) {
        Map<Integer, Integer> indexOf = new HashMap<>();   // value -> index
        for (int i = 0; i < nums.length; i++) {
            int need = target - nums[i];
            if (trace) System.out.printf("i=%d x=%d need=%d map=%s%n", i, nums[i], need, indexOf);
            Integer j = indexOf.get(need);
            if (j != null) return new int[]{j, i};
            indexOf.put(nums[i], i);
        }
        return new int[]{-1, -1};
    }

    static boolean isAnagram(String a, String b) {
        if (a.length() != b.length()) return false;
        int[] count = new int[26];
        for (int i = 0; i < a.length(); i++) {
            count[a.charAt(i) - 'a']++;
            count[b.charAt(i) - 'a']--;
        }
        for (int c : count) if (c != 0) return false;
        return true;
    }

    static List<List<String>> groupAnagrams(String[] words) {
        Map<String, List<String>> groups = new HashMap<>();
        for (String w : words) {
            char[] cs = w.toCharArray();
            Arrays.sort(cs);
            groups.computeIfAbsent(new String(cs), k -> new ArrayList<>()).add(w);
        }
        return new ArrayList<>(groups.values());
    }

    // Top k frequent by bucket sort: bucket[f] holds the values that occur f times.
    static List<Integer> topK(int[] nums, int k) {
        Map<Integer, Integer> freq = new HashMap<>();
        for (int x : nums) freq.merge(x, 1, Integer::sum);
        List<List<Integer>> bucket = new ArrayList<>();
        for (int i = 0; i <= nums.length; i++) bucket.add(new ArrayList<>());
        for (var e : freq.entrySet()) bucket.get(e.getValue()).add(e.getKey());
        List<Integer> out = new ArrayList<>();
        for (int f = nums.length; f >= 1 && out.size() < k; f--) {
            for (int v : bucket.get(f)) {
                if (out.size() < k) out.add(v);
            }
        }
        return out;
    }

    static int longestConsecutive(int[] nums, boolean trace) {
        Set<Integer> set = new HashSet<>();
        for (int x : nums) set.add(x);
        int best = 0;
        for (int x : nums) {
            if (set.contains(x - 1)) {
                if (trace) {
                    System.out.println("x=" + x + ": " + (x - 1) + " exists, not a start, skip");
                }
                continue;
            }
            int cur = x, len = 1;
            while (set.contains(cur + 1)) { cur++; len++; }
            best = Math.max(best, len);
            if (trace) {
```

Main.java (continued) JAVA

```
                System.out.println("x=" + x + ": start, run " + x + ".." + cur
                        + " length " + len + ", best=" + best);
            }
        }
        return best;
    }

    static boolean hasDuplicate(int[] nums) {
        Set<Integer> seen = new HashSet<>();
        for (int x : nums) if (!seen.add(x)) return true;
        return false;
    }

    public static void main(String[] args) {
        System.out.println("-- two sum: [2, 11, 7, 15, 3], target 9 --");
        System.out.println(Arrays.toString(twoSum(new int[]{2, 11, 7, 15, 3}, 9, true)));
        System.out.println(isAnagram("listen", "silent") + " " + isAnagram("rat", "car"));
        System.out.println(groupAnagrams(new String[]{"eat", "tea", "tan", "ate", "nat", "bat"}));
        System.out.println("top2: " + topK(new int[]{1, 1, 1, 2, 2, 3}, 2));
        System.out.println("-- longest consecutive: [100, 4, 200, 1, 3, 2] --");
        System.out.println("answer = " + longestConsecutive(new int[]{100, 4, 200, 1, 3, 2}, true));
        System.out.println("duplicate? " + hasDuplicate(new int[]{1, 2, 3, 1}));

        Integer a = 127, b = 127, c = 1000, d = 1000;
        System.out.println("127 == 127 : " + (a == b) + ", 1000 == 1000 : " + (c == d)
                + ", equals : " + c.equals(d));
        Map<Integer, Integer> m = new HashMap<>();
        m.put(1000, 5);
        m.put(1000, 7);
        System.out.println("map = " + m + " size " + m.size());
    }
}
```

Run it TERMINAL

```
$ java Main.java
-- two sum: [2, 11, 7, 15, 3], target 9 --
i=0 x=2 need=7 map={}
i=1 x=11 need=-2 map={2=0}
i=2 x=7 need=2 map={2=0, 11=1}
[0, 2]
true false
[[eat, tea, ate], [bat], [tan, nat]]
top2: [1, 2]
-- longest consecutive: [100, 4, 200, 1, 3, 2] --
x=100: start, run 100..100 length 1, best=1
x=4: 3 exists, not a start, skip
x=200: start, run 200..200 length 1, best=1
x=1: start, run 1..4 length 4, best=4
x=3: 2 exists, not a start, skip
x=2: 1 exists, not a start, skip
answer = 4
duplicate? true
127 == 127 : true, 1000 == 1000 : false, equals : true
map = {1000=7} size 1
```

The order of groups in `groupAnagrams` follows `HashMap` iteration order, so another JDK may print them in a different order; the groups themselves are the same.

What to notice. `twoSum` searches for the partner **before** inserting the current element, so an element is never paired with itself. `merge(x, 1, Integer::sum)` is the idiomatic counting operation. `topK` uses bucket sort: no value occurs more than `n` times, so `bucket[f]` lists the values with frequency `f`, and reading the buckets from high to low yields the top k in O(n) without a heap. In the last lines, `Integer` objects between -128 and 127 are cached, so `==` succeeds for 127 but fails for 1000; boxed values must be compared with `equals`. `HashMap` itself uses `equals`, so the second `put(1000, 7)` replaces the first.

2.5.4 Complexity and mistakes

Each pass is O(n) time on average with O(n) space for the table. Sorting-based alternatives cost O(n log n) time but may need only O(1) extra space. The anagram check is O(n) time and O(1) space (26 counters). Group anagrams with sorted keys costs O(n × L log L) for n words of length L; counting keys cost O(n × L).

- **Comparing `Integer` with `==`.** `equals` or unboxing to `int` is required.
- **Pairing an element with itself.** The lookup must precede the insertion (two sum with `target = 2x`).
- **Mutable or array keys.** An `int[]` used as a `HashMap` key is hashed by identity, so arrays with equal contents are distinct keys. A `String`, `List<Integer>` or `Arrays.toString` form must be used instead.
- `map.get(k) + 1` **on a missing key** throws `NullPointerException` (unboxing of null); `getOrDefault` or `merge` avoids this.
- **Omitting the anagram length check.** Strings of different lengths are never anagrams; testing this first avoids unnecessary work.
- **Invalid `char` arithmetic.** `count[c - 'a']` is valid only for lowercase `a` to `z`; mixed input requires an array of 128 or 256 slots.

2.5.5 Questions to expect

1. "Two sum on an unsorted array." Clue: a partner value is required. One-pass map from value to index.
2. "Are two strings anagrams?" Clue: same letters, same counts. Count array: increment for one string, decrement for the other.
3. "Group the words that are anagrams of each other." Clue: group by canonical form. The sorted word (or a 26-count string) as key.
4. "Return the k most frequent elements, better than O(n log n)." Clue: top k with a target better than sorting. Bucket sort by frequency (a min-heap of size k gives O(n log k) and is treated in the chapter on heaps).
5. "Longest consecutive sequence in O(n)." Clue: "consecutive in any order" with an O(n) bound. Set with a run-start test.
6. "First non-repeating character." Clue: count, then scan. Two passes over a count array.
7. "Does the array contain duplicates? Within distance k?" Set for the first, sliding window of a set for the second.
8. "Count pairs `(i, j, k, l)` from four arrays summing to zero." Clue: split into two halves. Map the pair sums of the first two arrays and look up the negated pair sums of the other two.

INTERVIEW ANGLE

Why is HashMap lookup O(1), and under what conditions does it cease to be O(1)? The key's `hashCode()` selects a bucket in an array, so a lookup addresses it directly; with a well-distributed hash and the default load factor of 0.75 the buckets remain short. Performance degrades when many keys share a bucket: a long chain is scanned (since Java 8 it may be held in a tree and searched in O(log n)). Hashing a long `String` costs time proportional to its length. Resizing makes an individual `put` occasionally O(n), but the amortised cost remains O(1).

2.5.6 LeetCode practice

#	Title	Level	Key idea
1	Two Sum	Easy	Map from value to index, look up target minus x
217	Contains Duplicate	Easy	HashSet add returns false on a repeat
242	Valid Anagram	Easy	Count array incremented and decremented
383	Ransom Note	Easy	Count magazine letters, spend them for the note
205	Isomorphic Strings	Easy	Two-way character mapping
387	First Unique Character in a String	Easy	Count pass, then first index with count 1
290	Word Pattern	Easy	Two-way map between pattern letters and words
49	Group Anagrams	Medium	Sorted word or count signature as the key
347	Top K Frequent Elements	Medium	Frequency map, then buckets indexed by count
451	Sort Characters By Frequency	Medium	Count, then buckets or a sorted list of entries
454	4Sum II	Medium	Map of pair sums from two arrays, look up the negation
36	Valid Sudoku	Medium	One set per row, column and box
128	Longest Consecutive Sequence	Medium	Set; count only from numbers with no predecessor

LeetCode problems for this pattern.

KEY POINTS

Hash map and frequency counting

- A hash table answers membership and count queries in O(1) average time, reducing many $O(n^2)$ scans to O(n).
- Use `int[26]` for lowercase letters, `HashSet` for membership, `HashMap` with `merge` or `computeIfAbsent` for counts and groups.
- The lookup precedes the insertion; canonical forms serve as keys; arrays and mutable objects must not be used as keys.
- Boxed integers are compared with `equals`; because of the `Integer` cache, `==` is reliable only for -128 to 127.

2.6 Binary search on sorted arrays

Binary search locates a key in a sorted sequence by repeatedly comparing it with the middle element and discarding the half that cannot contain it. Each comparison halves the search interval, so a sorted array of 10^9 elements requires at most about 30 comparisons, whereas a linear scan requires up to 10^9.

The principle is simple, but implementations are error-prone: off-by-one errors, non-terminating loops and integer overflow in the midpoint computation are well-documented defects, the last of which was present in the JDK's own `Arrays.binarySearch` for years. This chapter fixes a single loop template with a clear invariant and applies it to lower bound, upper bound, first and last position, rotated arrays and peak finding. The next chapter generalises the same template to problems without an array.

2.6.1 The idea

Binary search requires one precondition: a predicate whose value changes **exactly once** along the search range. In a sorted array, the predicate `a[i] >= target` evaluates to `false false false true true true`. The boundary between the last `false` and the first `true` is the result. Each iteration evaluates the predicate at the midpoint and discards the half that cannot contain the boundary.

Recognition clues in a problem statement:

- "sorted array" (or "sorted in ascending order", or "rotated sorted array")
- "find the first / last / leftmost / rightmost position of ..."
- a required complexity of O(log n), or "better than linear"
- "the array is large (10^5 to 10^9 elements)" and linear scan is too slow
- "find a peak" or "find the minimum" in an array that is only partially ordered

The template below is the reference form. The search range is the **half-open interval** `[lo, hi)`: `lo` is a candidate result and `hi` is one past the last candidate. The loop executes while the range is non-empty, and the range strictly decreases in every iteration.

Template: first index where the condition is true JAVA

```java
// returns the first index i in [0, n] with isTrue(i); n means "none"
int lo = 0, hi = n;
while (lo < hi) {
    int mid = lo + (hi - lo) / 2;      // never (lo + hi) / 2: it can overflow
    if (isTrue(mid)) hi = mid;          // mid might be the answer: keep it
    else lo = mid + 1;                  // mid is not the answer: drop it
}
return lo;                              // lo == hi: the boundary
```

Correctness argument. The invariant is: every index below `lo` has predicate value `false`, and every index at or above `hi` has predicate value `true`. When `isTrue(mid)` holds, `hi = mid` preserves the invariant, since `mid` itself may be the first true index. When it does not hold, `lo = mid + 1` preserves it, since `mid` is certainly not the result. Because `lo <= mid < hi`, both updates strictly reduce `hi - lo`, so the loop terminates with `lo == hi`, the boundary. If every value is `false`, the boundary is `n`; hence the result range is `[0, n]` rather than `[0, n-1]`.

Defining `isTrue(i)` as `a[i] >= target` yields the **lower bound**: the first position at which `target` can be inserted without violating the order. Defining it as `a[i] > target` yields the **upper bound**: the position immediately after the last occurrence of `target`. Every variant in this chapter reduces to one of these two with a different predicate.

GOTCHA

`(lo + hi) / 2` overflows once `lo + hi` exceeds `Integer.MAX_VALUE` (about 2.1 billion). With `lo = 2,000,000,000` and `hi = 2,100,000,000` the sum wraps to a negative number and `mid` becomes negative. The correct form is `lo + (hi - lo) / 2`. The defect was reported in the JDK's binary search implementation in 2006 and is a standard interview topic.

2.6.2 Dry run

Input: `a = [1, 2, 4, 4, 4, 7, 9]` (three occurrences of 4) and `target = 4`. The objective is the lower bound, i.e. the index of the **first** 4. Here `n = 7`, so the initial state is `lo = 0`, `hi = 7`. The trace was printed by an instrumented version of the program in the next section.

step	lo	hi	mid	a[mid]	action
1	0	7	3	4	4 >= 4 is true, so hi = mid = 3
2	0	3	1	2	2 >= 4 is false, so lo = mid + 1 = 2
3	2	3	2	4	4 >= 4 is true, so hi = mid = 2
end	2	2	-	-	lo == hi: answer is index 2

Lower bound of 4 in [1, 2, 4, 4, 4, 7, 9].

The table describes a shrinking interval. After step 1 the result lies in `[0, 3]`, because the 4 at index 3 may be the first one and is therefore retained. After step 2, index 1 holds a 2, so the result lies in `[2, 3]`. After step 3 the interval is `[2, 2]` and the loop terminates. Index 2 is the first 4. Terminating at the first match would have returned index 3, which is a valid match but not the first occurrence; this is the distinction between finding a match and finding a boundary.

A second trace covers the rotated-array search on `[4, 5, 6, 7, 0, 1, 2]` with `target = 0`. A **rotated sorted array** is a sorted array whose prefix has been moved to the end. At every step at least one of the two halves delimited by `mid` is sorted, and a single comparison identifies which one.

step	lo	hi	mid	a[mid]	reasoning
1	0	6	3	7	a[lo]=4 <= 7: left half [4..7] is sorted and 0 is not in it, so lo = 4
2	4	6	5	1	a[lo]=0 <= 1: left half [0..1] sorted and 0 is in it, so hi = mid - 1 = 4
3	4	4	4	0	a[mid] == target: return 4

Search for 0 in the rotated array [4, 5, 6, 7, 0, 1, 2] (closed range, lo <= hi).

2.6.3 Java code

All variants below share one skeleton. `firstLast` finds the first and last position, `searchRotated` and `findMin` handle rotation, `findPeak` requires no global order, and `searchMatrix` treats a row-major sorted matrix as a single sequence.

Main.java JAVA

```java
import java.util.Arrays;

public class Main {
    // first index i with a[i] >= target (a.length if none)
    static int lowerBound(int[] a, int target) {
        int lo = 0, hi = a.length;
        while (lo < hi) {
            int mid = lo + (hi - lo) / 2;
            if (a[mid] >= target) hi = mid;
            else lo = mid + 1;
        }
        return lo;
    }

    // first index i with a[i] > target
    static int upperBound(int[] a, int target) {
        int lo = 0, hi = a.length;
        while (lo < hi) {
            int mid = lo + (hi - lo) / 2;
            if (a[mid] > target) hi = mid;
            else lo = mid + 1;
        }
        return lo;
    }

    static int[] firstLast(int[] a, int target) {
        int first = lowerBound(a, target);
        if (first == a.length || a[first] != target) return new int[] {-1, -1};
        return new int[] {first, upperBound(a, target) - 1};
    }

    static int searchRotated(int[] a, int target) {
        int lo = 0, hi = a.length - 1;
        while (lo <= hi) {
            int mid = lo + (hi - lo) / 2;
            if (a[mid] == target) return mid;
            if (a[lo] <= a[mid]) {                  // left half sorted
                if (a[lo] <= target && target < a[mid]) hi = mid - 1;
                else lo = mid + 1;
            } else {                                // right half sorted
                if (a[mid] < target && target <= a[hi]) lo = mid + 1;
                else hi = mid - 1;
            }
        }
        return -1;
    }

    static int findMin(int[] a) {
        int lo = 0, hi = a.length - 1;
        while (lo < hi) {
            int mid = lo + (hi - lo) / 2;
            if (a[mid] > a[hi]) lo = mid + 1;
            else hi = mid;
        }
        return a[lo];
    }

    static int findPeak(int[] a) {
        int lo = 0, hi = a.length - 1;
        while (lo < hi) {
            int mid = lo + (hi - lo) / 2;
            if (a[mid] < a[mid + 1]) lo = mid + 1;
            else hi = mid;
        }
        return lo;
    }

    static boolean searchMatrix(int[][] m, int target) {
        int rows = m.length, cols = m[0].length;
```

sreeram

Main.java (continued) JAVA

```
        int lo = 0, hi = rows * cols - 1;
        while (lo <= hi) {
            int mid = lo + (hi - lo) / 2;
            int v = m[mid / cols][mid % cols];
            if (v == target) return true;
            if (v < target) lo = mid + 1;
            else hi = mid - 1;
        }
        return false;
    }

    public static void main(String[] args) {
        int[] a = {1, 2, 4, 4, 4, 7, 9};
        System.out.println("lowerBound(4) = " + lowerBound(a, 4));
        System.out.println("upperBound(4) = " + upperBound(a, 4));
        System.out.println("lowerBound(5) = " + lowerBound(a, 5));
        System.out.println("lowerBound(10) = " + lowerBound(a, 10));
        System.out.println("firstLast(4) = " + Arrays.toString(firstLast(a, 4)));
        System.out.println("firstLast(5) = " + Arrays.toString(firstLast(a, 5)));
        int[] r = {4, 5, 6, 7, 0, 1, 2};
        System.out.println("rotated search 0 -> " + searchRotated(r, 0));
        System.out.println("rotated search 3 -> " + searchRotated(r, 3));
        System.out.println("rotated min -> " + findMin(r));
        System.out.println("peak index -> " + findPeak(new int[] {1, 3, 5, 4, 2}));
        int[][] m = {{1, 3, 5, 7}, {10, 11, 16, 20}, {23, 30, 34, 60}};
        System.out.println("matrix has 16 -> " + searchMatrix(m, 16));
        System.out.println("matrix has 13 -> " + searchMatrix(m, 13));
        System.out.println("Arrays.binarySearch(a, 4) = " + Arrays.binarySearch(a, 4));
        System.out.println("Arrays.binarySearch(a, 5) = " + Arrays.binarySearch(a, 5));
    }
}
```

Run it TERMINAL

```
$ java Main.java
lowerBound(4) = 2
upperBound(4) = 5
lowerBound(5) = 5
lowerBound(10) = 7
firstLast(4) = [2, 4]
firstLast(5) = [-1, -1]
rotated search 0 -> 4
rotated search 3 -> -1
rotated min -> 0
peak index -> 2
matrix has 16 -> true
matrix has 13 -> false
Arrays.binarySearch(a, 4) = 3
Arrays.binarySearch(a, 5) = -6
```

What to notice:

- `lowerBound` and `upperBound` differ in one operator (`>=` versus `>`). `firstLast` uses both; the last position is `upperBound - 1`. The test `first == a.length || a[first] != target` is required, because the lower bound of an absent value designates an unrelated element.
- `searchRotated` uses the **closed** range `[lo, hi]` with `while (lo <= hi)` and the updates `mid - 1` and `mid + 1`. This second template is appropriate when a match can be recognised directly (`a[mid] == target`) and returned immediately; the half-open template is appropriate for boundary searches.
- `findMin` compares `a[mid]` with `a[hi]`, not with `a[lo]`. If `a[mid] > a[hi]` the minimum is strictly to the right of `mid`; otherwise `mid` may be the minimum, so `hi = mid`.
- `findPeak` compares `a[mid]` with its neighbour `a[mid + 1]`; the local slope determines which side must contain a peak. Because `lo < hi` inside the loop, `mid + 1` is always a valid index.
- `Arrays.binarySearch(a, 4)` returned 3: in the presence of duplicates it may return **any** matching index. For an absent key it returns `-(insertion point) - 1`, so `-6` denotes insertion point 5, the value that `lowerBound(a, 5)` printed.

2.6.4 Complexity and mistakes

Time: each iteration halves the range, so the loop executes at most about log2(n) + 1 times: O(log n); for n = 10^9 this is about 30 iterations. **Space:** O(1) for the iterative version; a recursive formulation uses O(log n) stack space. If the input is unsorted, sorting costs O(n log n), so binary search is justified only when several queries are performed.

Common defects:

- **Mixing templates.** Combining `while (lo < hi)` with `hi = n - 1`, or `while (lo <= hi)` with `hi = mid`, causes non-termination or skipped candidates. Initial `hi`, loop condition and update rules must be consistent with a single template.
- **Non-termination with `lo = mid`.** An update `lo = mid` requires the upper midpoint `lo + (hi - lo + 1) / 2`; otherwise `mid == lo` on a two-element range and the interval does not shrink.
- **Overflow** in `mid`, as shown above. Subtraction-based comparisons such as `target - a[mid]` can also overflow; `Integer.compare` or relational operators avoid this.
- **Unhandled absence.** After a lower bound, `lo` can equal `n`, and evaluating `a[lo]` then throws `ArrayIndexOutOfBoundsException`.
- **Duplicates in a rotated array.** With duplicates such as `[1, 0, 1, 1, 1]`, the test `a[lo] <= a[mid]` no longer establishes that the left half is sorted. The remedy is `lo++` (and `hi--`) when `a[lo] == a[mid] == a[hi]`, which degrades the worst case to O(n); this is the additional difficulty of the "II" variant.

2.6.5 Questions to expect

1. **"Find the first and last position of a target in a sorted array."** Clue: sorted, duplicates, O(log n). Two boundary searches: lower bound and upper bound minus one.
2. **"Search a target in a rotated sorted array."** Clue: sorted but rotated. Identify the sorted half, then test whether the target lies inside it.
3. **"Find the minimum in a rotated sorted array."** Compare `a[mid]` with `a[hi]`. Follow-up: what changes if there are duplicates? (Worst case O(n).)
4. **"Find a peak element; neighbours are never equal."** Clue: no sorted order but a local property. Follow the ascending side.
5. **"Search a matrix where each row is sorted and each row starts after the previous one ends."** Flatten the index: `row = mid / cols`, `col = mid % cols`. If rows and columns are sorted but rows are not chained, use the staircase walk from the top-right corner in O(rows + cols).
6. **"Integer square root without `Math.sqrt`."** Binary search on `x` for the largest `r` with `r * r <= x`; use `long` for the product.
7. **"Median of two sorted arrays in O(log(min(m, n)))."** Binary search on the partition point of the shorter array. This is the hardest classic of the pattern.

INTERVIEW ANGLE

Interviewer: "Can you search this sorted array faster than O(n)?" Candidate: "Binary search, O(log n) time and O(1) space. The search uses a half-open range `[lo, hi)`, computes `mid` as `lo + (hi - lo) / 2` to avoid overflow, and shrinks with `hi = mid` or `lo = mid + 1`." Interviewer: "How is the first occurrence found when duplicates exist?" Candidate: "The task becomes a boundary search rather than a match search: the lower bound with `a[mid] >= target`, followed by verification that `a[lo] == target`." Interviewer: "And for a rotated array?" Candidate: "At each step one half is sorted, identified by `a[lo] <= a[mid]`. If the target lies within that sorted half the other half is discarded, otherwise the sorted half is discarded. The complexity remains O(log n) without duplicates."

sreram

2.6.6 LeetCode practice

#	Title	Level	Key idea
704	Binary Search	Easy	Plain template; return the index or -1
35	Search Insert Position	Easy	Exactly the lower bound
69	Sqrt(x)	Easy	Largest r with r*r <= x; use long
34	Find First and Last Position of Element in Sorted Array	Medium	Lower bound and upper bound minus one
74	Search a 2D Matrix	Medium	Treat the matrix as one sorted array of rows*cols items
153	Find Minimum in Rotated Sorted Array	Medium	Compare a[mid] with a[hi]
33	Search in Rotated Sorted Array	Medium	Find the sorted half, then test the target against it
162	Find Peak Element	Medium	Move towards the larger neighbour
81	Search in Rotated Sorted Array II	Medium	As 33, but shrink both ends when a[lo] == a[mid] == a[hi]
4	Median of Two Sorted Arrays	Hard	Binary search the partition of the shorter array

LeetCode problems for this pattern.

KEY POINTS

- Binary search finds the boundary of a monotone predicate; sortedness of an array is the most common way to obtain one.
- Reference template: half-open `[lo, hi)`, `mid = lo + (hi - lo) / 2`, `hi = mid` when true, `lo = mid + 1` when false.
- Lower bound is `a[mid] >= target`, upper bound is `a[mid] > target`; first and last position use both.
- For rotated arrays decide which half is sorted; for peaks follow the slope; with duplicates the worst case degrades to O(n).
- `Arrays.binarySearch` returns any match, or `-(insertion point) - 1`, so it does not give the first occurrence.

2.7 Binary search on the answer

Some problems provide no sorted array. Consider: "What is the smallest eating speed that allows all banana piles to be finished within h hours?" The candidate answers form a numeric range (1 to the largest pile), and testing each speed from 1 upward costs O(max pile) feasibility checks of O(n) each. With piles up to 10^9 this is infeasible.

The technique is to apply binary search to the **answer space** instead of the input. The result is a value in a known range, and for each candidate a feasibility question is answered. If that question is monotone (true for a candidate implies true for every larger candidate), the template of the previous chapter finds the smallest feasible candidate with O(log R) evaluations, where R is the size of the range.

2.7.1 The idea

Define a **predicate** `ok(x)`: "the goal is achievable with parameter `x`". The technique requires `ok` to be **monotonic**: `false false false true true true` over the candidate range (or its mirror image for maximisation). The sequence is never materialised; `ok` is evaluated only at the probed midpoints, and each evaluation is typically a single greedy O(n) pass.

Recognition clues:

- "minimise the maximum" or "maximise the minimum" (largest sum, longest wait, smallest capacity, largest minimum distance)
- "find the smallest / largest X such that ...", where X is a speed, capacity, time, distance, divisor or number of days
- Verifying a given X is simple (a greedy pass), whereas computing the optimum directly is difficult
- The answer range is large (up to 10^9 or 10^18) while the input size is moderate
- Increasing capacity, time or speed never reduces feasibility (monotonicity)

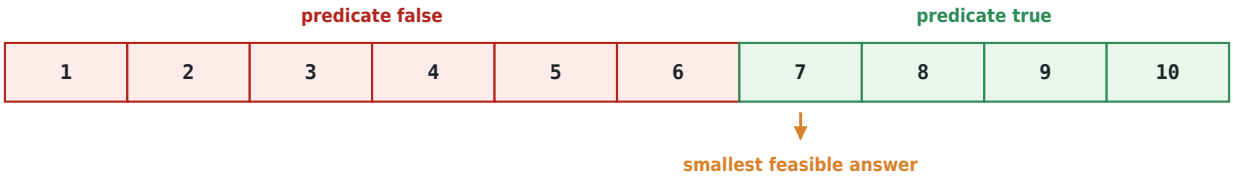


Figure 2.3. The search happens over the range of possible answers [lo, hi]; each probe tests one candidate and halves the range, exactly like searching an array.

The skeleton is the template of the previous chapter, with the array replaced by a numeric range and `isTrue(mid)` replaced by `ok(mid)`:

Template: smallest x in [lo, hi] with ok(x) JAVA

```java
static int firstTrue(int lo, int hi, IntPredicate ok) {
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (ok.test(mid)) hi = mid;      // mid works: try something smaller
        else lo = mid + 1;               // mid fails: the answer is larger
    }
    return lo;
}
```

Three design decisions determine correctness. **Bounds:** `lo` must be a value that can be the answer (for example the largest single item, since no smaller capacity can carry it), and `hi` a value that is guaranteed feasible (for example the total sum). **Direction:** minimisation searches for the first true; maximisation searches for the last true, implemented by inverting the predicate or by using the upper midpoint. **Check:** a greedy pass whose correctness must be argued and whose cost is O(n) or close to it.

2.7.2 Dry run

Koko Eating Bananas. Piles `[3, 6, 7, 11]`, `h = 8` hours. At speed `k`, a pile of `p` bananas requires `ceil(p / k)` hours (one pile is processed per hour). The answer lies in `[1, 11]`: speed 1 is the minimum meaningful speed and speed 11 finishes every pile in one hour. The objective is the smallest `k` whose total is at most 8 hours.

step	lo	hi	k	hours per pile	total	ok?
1	1	11	6	1 1 2 2	6	true
2	1	6	3	1 2 3 4	10	false
3	4	6	5	1 2 2 3	8	true
4	4	5	4	1 2 2 3	8	true
end	4	4	-	-	-	answer 4

Binary search over speeds. Hours per pile are ceil(3/k), ceil(6/k), ceil(7/k), ceil(11/k).

Speed 6 is feasible, so the answer is at most 6 (`hi = 6`). Speed 3 requires 10 hours, which is infeasible, so the answer exceeds 3 (`lo = 4`). Speeds 5 and 4 are feasible, and the interval collapses to `[4, 4]`. Since speed 3 failed, 4 is minimal. Four probes replace eleven; for a range of 10^9 about thirty probes suffice.

2.7.3 Java code

One generic `firstTrue` is combined with four predicates. Only the bounds and the O(n) check differ between problems.

Main.java JAVA

```java
import java.util.Arrays;
import java.util.function.IntPredicate;

public class Main {
    // smallest x in [lo, hi] with ok(x) true; ok must be false...false true...true
    static int firstTrue(int lo, int hi, IntPredicate ok) {
        while (lo < hi) {
            int mid = lo + (hi - lo) / 2;
            if (ok.test(mid)) hi = mid;
            else lo = mid + 1;
```

Main.java (continued) JAVA

```
        }
        return lo;
    }

    static int kokoSpeed(int[] piles, int h) {
        int max = Arrays.stream(piles).max().getAsInt();
        return firstTrue(1, max, k -> {
            long hours = 0;
            for (int p : piles) hours += (p + k - 1) / k;      // ceil(p / k)
            return hours <= h;
        });
    }

    static int shipCapacity(int[] w, int days) {
        int lo = Arrays.stream(w).max().getAsInt();
        int hi = Arrays.stream(w).sum();
        return firstTrue(lo, hi, cap -> {
            int used = 1, load = 0;
            for (int x : w) {
                if (load + x > cap) { used++; load = 0; }
                load += x;
            }
            return used <= days;
        });
    }

    static int splitArray(int[] nums, int k) {
        int lo = Arrays.stream(nums).max().getAsInt();
        int hi = Arrays.stream(nums).sum();
        return firstTrue(lo, hi, limit -> {
            int parts = 1, sum = 0;
            for (int x : nums) {
                if (sum + x > limit) { parts++; sum = 0; }
                sum += x;
            }
            return parts <= k;
        });
    }

    static int minDaysBouquets(int[] bloom, int m, int k) {
        if ((long) m * k > bloom.length) return -1;
        int lo = Arrays.stream(bloom).min().getAsInt();
        int hi = Arrays.stream(bloom).max().getAsInt();
        return firstTrue(lo, hi, day -> {
            int made = 0, run = 0;
            for (int b : bloom) {
                if (b <= day) { if (++run == k) { made++; run = 0; } }
                else run = 0;
            }
            return made >= m;
        });
    }

    static int isqrt(int x) {            // largest r with r*r <= x
        int lo = 0, hi = Math.min(x, 46340);
        while (lo < hi) {
            int mid = lo + (hi - lo + 1) / 2;          // upper mid: avoids infinite loop
            if ((long) mid * mid <= x) lo = mid;
            else hi = mid - 1;
        }
        return lo;
    }

    public static void main(String[] args) {
        System.out.println("koko [3,6,7,11] h=8 -> " + kokoSpeed(new int[] {3, 6, 7, 11}, 8));
        System.out.println("koko [30,11,23,4,20] h=5 -> "
            + kokoSpeed(new int[] {30, 11, 23, 4, 20}, 5));
        System.out.println("ship -> " + shipCapacity(new int[] {1, 2, 3, 4, 5, 6, 7, 8, 9, 10}, 5));
        System.out.println("split [7,2,5,10,8] k=2 -> "
            + splitArray(new int[] {7, 2, 5, 10, 8}, 2));
        System.out.println("bouquets -> " + minDaysBouquets(new int[] {1, 10, 3, 10, 2}, 3, 1));
        System.out.println("bouquets m=3,k=2 -> "
            + minDaysBouquets(new int[] {1, 10, 3, 10, 2}, 3, 2));
        System.out.println("isqrt(8) = " + isqrt(8) + ", isqrt(2147395600) = " + isqrt(2147395600));
```

Sriram

Main.java (continued) JAVA

```
        int lo = 2_000_000_000, hi = 2_100_000_000;
        System.out.println("(lo+hi)/2 overflow: " + (lo + hi) / 2 + " vs " + (lo + (hi - lo) / 2));
    }
}
```

Run it TERMINAL

```
$ java Main.java
koko [3,6,7,11] h=8 -> 4
koko [30,11,23,4,20] h=5 -> 30
ship -> 15
split [7,2,5,10,8] k=2 -> 18
bouquets -> 3
bouquets m=3,k=2 -> -1
isqrt(8) = 2, isqrt(2147395600) = 46340
(lo+hi)/2 overflow: -97483648 vs 2050000000
```

What to notice:

- **Koko.** `(p + k - 1) / k` is integer ceiling division. The accumulator `hours` is a `long` because many large piles at speed 1 can exceed the `int` range.
- **Ship packages.** The lower bound is the heaviest single package (a smaller capacity cannot carry it) and the upper bound is the total weight (one day). The check simulates loading in order and starts a new day when the next package does not fit.
- **Split array largest sum.** "Minimise the largest sum among `k` parts" is equivalent to "find the smallest limit such that the array can be cut into at most `k` parts, each with sum at most the limit". Dynamic programming solves it in O(k n^2); the binary search takes O(n log(sum)).
- **Bouquets.** The predicate scans for `k` adjacent flowers that have bloomed by the given day. The early test `m * k > n` uses a `long` cast because the product may overflow `int`.
- **Integer square root.** This variant seeks the **last** true (`mid * mid <= x`), hence the upper midpoint `lo + (hi - lo + 1) / 2` with the updates `lo = mid` and `hi = mid - 1`. The cap 46340 is the largest `r` for which `r * r` fits in `int`; the cast to `long` is an additional safeguard.
- The last output line demonstrates the overflow of `(lo + hi) / 2`.

2.7.4 Complexity and mistakes

Time: O(n log R), where `R = hi - lo` is the size of the answer range and `n` is the cost of one check; for Koko with piles up to 10^9 this is about 30 checks of O(n). **Space:** O(1) beyond the input.

Common defects:

- **Incorrect bounds.** A `lo` above the true answer excludes it; a `hi` for which `ok(hi)` is false makes the result meaningless. The weakest justifiable bounds should be chosen, with a proof that `ok(hi)` holds.
- **Non-monotonic predicate.** If `ok(5)` is true and `ok(6)` is false, binary search yields an arbitrary result. Monotonicity must be argued before coding.
- **Maximisation with the lower midpoint.** For "last true" with `lo = mid`, the upper midpoint `lo + (hi - lo + 1) / 2` is required; otherwise the loop does not terminate on a two-element range.
- **Overflow in the check.** Sums of weights, `hours`, `mid * mid` and time-times-trips products often need `long`.
- **Incorrect greedy check.** The check must be an exact feasibility test. In ship packages the order of packages must be preserved; in "magnetic force" the next ball is placed at the first position at least `d` from the previous one.

REAL WORLD

Binary search on the answer is used to determine a limit when only a pass/fail test exists: the largest batch size that fits in memory, the lowest timeout that passes a test suite, or `git bisect`, which searches commit history for the first failing commit using the test as the predicate.

2.7.5 Questions to expect

1. **"Minimum eating speed so all piles finish within h hours."** Clue: smallest speed, feasibility is easy. Search `[1, max]`, check with ceiling divisions.
2. **"Least ship capacity to deliver all packages in D days, order fixed."** Clue: minimise a maximum with a fixed order. Search `[max, sum]`.
3. **"Split an array into k non-empty parts to minimise the largest part sum."** Clue: "minimise the largest sum". Same as 2. Follow-up: how does it compare with DP? (DP is O(k n^2).)
4. **"Earliest day on which you can make m bouquets of k adjacent flowers."** Clue: earliest day, predicate counts runs. Return -1 when `m * k > n`.
5. **"Smallest divisor so that the sum of ceilings is at most a threshold."** Clue: smallest x with a monotone sum.
6. **"Place c balls into baskets so the minimum distance between any two is maximal."** Clue: "maximise the minimum". Sort, then greedily place a ball whenever the gap is at least `d`.
7. **"Minimum time for all buses to complete at least `totalTrips` trips."** Clue: time as the answer, trips = sum of `time / period`; the upper bound needs `long`.

INTERVIEW ANGLE

Interviewer: "How do you know binary search applies here? There is no sorted array." Candidate: "The answer is a number in a range, and feasibility is monotone: if speed k is feasible then any larger speed is feasible. Hence `ok(k)` has the form `false...false true...true`, and binary search finds the first true." Interviewer: "What is the complexity?" Candidate: "O(n log(max - min)): a logarithmic number of probes, each an O(n) greedy check. Space is O(1)." Interviewer: "Why is the greedy check correct?" Candidate: "For a fixed capacity, packing as much as possible into each day never uses more days than any other packing, so if the greedy needs more than D days, no packing exists."

2.7.6 LeetCode practice

#	Title	Level	Key idea
744	Find Smallest Letter Greater Than Target	Easy	Upper bound on a sorted letter array, wrap around to index 0
875	Koko Eating Bananas	Medium	Search speed in [1, max]; check with ceiling division
1011	Capacity To Ship Packages Within D Days	Medium	Search capacity in [max, sum]; greedy day counting
1283	Find the Smallest Divisor Given a Threshold	Medium	Smallest divisor whose ceiling-sum fits
1482	Minimum Number of Days to Make m Bouquets	Medium	Search the day; count runs of k bloomed flowers
1552	Magnetic Force Between Two Balls	Medium	Maximise the minimum gap; greedy placement on sorted positions
2187	Minimum Time to Complete Trips	Medium	Search time; trips = sum of time / period (use long)
410	Split Array Largest Sum	Hard	Minimise the largest part sum; greedy count of parts

LeetCode problems for this pattern.

KEY POINTS

- Binary search on the answer requires a monotone predicate `ok(x)` and a range `[lo, hi]` guaranteed to contain the answer.
- The code is the template of the previous chapter, `firstTrue(lo, hi, ok)`; only the bounds and the O(n) check vary.
- Phrases like "minimise the maximum" and "maximise the minimum" are the strongest clues.
- Time is O(n log R); the midpoint variant matters for "last true", and checks often require `long`.
- Monotonicity must be justified before coding; it is the part most often examined.

2.8 Sorting-based problems and merge intervals

Scheduling problems require detecting conflicts between time ranges, merging overlapping ranges into maximal blocks, and computing the peak number of simultaneously active ranges. Pairwise comparison of n intervals costs O(n^2). Sorting by start time places all relevant information in adjacent positions, so a single linear scan suffices and the total cost is O(n log n), dominated by the sort.

This chapter treats a family of problems with a common shape: items with a start and an end, an order imposed by sorting, and a linear sweep.

2.8.1 The idea

An interval `[start, end]` overlaps its successor in start order exactly when `next.start <= current.end`. After sorting by start, an interval can overlap only with its immediate predecessors in the merged block, so a single "current" interval is maintained and either extended or closed. The choice of sort key (start, end, or starts and ends separately) is the essential design decision.

Recognition clues:

- Input is a list of pairs, "intervals", "meetings", "ranges", "bookings", "segments"
- "merge overlapping", "insert into a sorted list of non-overlapping", "can a person attend all meetings"
- "minimum number of rooms / platforms / arrows / groups"
- "remove the minimum number of intervals so the rest do not overlap"
- The input order is arbitrary and the result is independent of it, which indicates that sorting is admissible

The sort key is selected according to the objective, as summarised below.

Goal	Sort by	Why
Merge overlapping intervals	start	Overlaps can then only be with the interval just before
Keep the most intervals without overlap	end	Finishing earliest leaves the most room (greedy)
Burst balloons with fewest arrows	end	Same as above: shoot at the earliest end
Count the busiest moment (rooms, platforms)	starts and ends in two separate arrays	Walk time forward, +1 on start, -1 on end
Compare two sorted lists of intervals	already sorted	Two pointers, advance the one that ends first

Choosing what to sort by.

The skeleton for merging, which most other problems adapt:

Template: sort by start, keep one current interval JAVA

```java
Arrays.sort(a, (x, y) -> Integer.compare(x[0], y[0]));
List<int[]> out = new ArrayList<>();
int[] cur = a[0].clone();
for (int i = 1; i < a.length; i++) {
    if (a[i][0] <= cur[1]) cur[1] = Math.max(cur[1], a[i][1]);   // overlap: extend
    else { out.add(cur); cur = a[i].clone(); }                   // gap: close and restart
}
out.add(cur);
```

GOTCHA

A comparator of the form `(x, y) -> x[0] - y[0]` is incorrect. If the operands have opposite signs and large magnitude (for example `Integer.MIN_VALUE` and a positive value), the subtraction overflows and returns the wrong sign, which makes the ordering inconsistent. `Integer.compare(x[0], y[0])` is the correct form. Note also that `Arrays.sort` with a comparator requires an object array; `int[][]` qualifies, because it is an array of `int[]` objects, whereas a primitive `int[]` cannot take a comparator.

2.8.2 Dry run

Input: `[[1,3], [8,10], [2,6], [15,18], [17,20]]`. The task is to merge all overlapping intervals. Step 0 sorts by start and produces `[[1,3], [2,6], [8,10], [15,18], [17,20]]`. A sweep then maintains `cur` as the interval under construction.

step	next interval	decision	cur afterwards	finished list
start	-	`cur = first interval`	`[1, 3]`	empty
1	`[2, 6]`	`2 <= 3: overlap, end = max(3, 6)`	`[1, 6]`	empty
2	`[8, 10]`	`8 > 6: gap, close cur`	`[8, 10]`	[1, 6]
3	`[15, 18]`	`15 > 10: gap, close cur`	`[15, 18]`	[1, 6] [8, 10]
4	`[17, 20]`	`17 <= 18: overlap, end = max(18, 20)`	`[15, 20]`	[1, 6] [8, 10]
end	-	`add the last cur`	-	[1, 6] [8, 10] [15, 20]

Merge sweep after sorting by start.

Two details determine correctness. First, the new end is `max(cur[1], next[1])`, not `next[1]`, because an interval may be contained in the current one (for example `[2, 3]` inside `[1, 10]`). Second, the test uses `<=`, so intervals that merely touch, such as `[1,4]` and `[4,5]`, are merged into `[1,5]`. Whether touching constitutes overlap is a specification question that should be clarified; the examples in the statement decide it.

2.8.3 Java code

The program below contains the family of solutions: merge, insert into a sorted list (three phases: before, overlapping, after), minimum number of meeting rooms by a two-array sweep, minimum removals by sorting on the end, intersection of two interval lists by two pointers, car pooling by a difference array, and a custom comparator (largest number).

Main.java JAVA

```java
import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

public class Main {
    static int[][] merge(int[][] intervals) {
        int[][] a = intervals.clone();
        Arrays.sort(a, (x, y) -> Integer.compare(x[0], y[0]));
        List<int[]> out = new ArrayList<>();
        int[] cur = a[0].clone();
        for (int i = 1; i < a.length; i++) {
            if (a[i][0] <= cur[1]) cur[1] = Math.max(cur[1], a[i][1]);   // overlap: extend
            else { out.add(cur); cur = a[i].clone(); }                   // gap: close current
        }
        out.add(cur);
        return out.toArray(new int[0][]);
    }

    static int[][] insert(int[][] sorted, int[] nw) {
        List<int[]> out = new ArrayList<>();
        int i = 0, n = sorted.length;
        while (i < n && sorted[i][1] < nw[0]) out.add(sorted[i++]);          // entirely before
        int[] cur = nw.clone();
        while (i < n && sorted[i][0] <= cur[1]) {                            // overlapping
            cur[0] = Math.min(cur[0], sorted[i][0]);
            cur[1] = Math.max(cur[1], sorted[i][1]);
            i++;
        }
        out.add(cur);
        while (i < n) out.add(sorted[i++]);                                  // entirely after
        return out.toArray(new int[0][]);
    }

    // minimum number of rooms = maximum number of overlapping meetings
    static int minRooms(int[][] meetings) {
        int n = meetings.length;
        int[] starts = new int[n], ends = new int[n];
```

Main.java (continued) JAVA

```
        for (int i = 0; i < n; i++) { starts[i] = meetings[i][0]; ends[i] = meetings[i][1]; }
        Arrays.sort(starts);
        Arrays.sort(ends);
        int rooms = 0, best = 0, j = 0;
        for (int i = 0; i < n; i++) {
            while (ends[j] <= starts[i]) { j++; rooms--; }     // a meeting finished: free a room
            rooms++;
            best = Math.max(best, rooms);
        }
        return best;
    }

    // fewest intervals to remove so the rest do not overlap: keep earliest finishing ones
    static int eraseOverlap(int[][] intervals) {
        int[][] a = intervals.clone();
        Arrays.sort(a, (x, y) -> Integer.compare(x[1], y[1]));
        int kept = 0, end = Integer.MIN_VALUE;
        for (int[] iv : a) {
            if (iv[0] >= end) { kept++; end = iv[1]; }
        }
        return a.length - kept;
    }

    static int[][] intersect(int[][] A, int[][] B) {
        List<int[]> out = new ArrayList<>();
        int i = 0, j = 0;
        while (i < A.length && j < B.length) {
            int lo = Math.max(A[i][0], B[j][0]);
            int hi = Math.min(A[i][1], B[j][1]);
            if (lo <= hi) out.add(new int[] {lo, hi});
            if (A[i][1] < B[j][1]) i++; else j++;                // drop the one that ends first
        }
        return out.toArray(new int[0][]);
    }

    static boolean carPooling(int[][] trips, int capacity) {      // trip = {people, from, to}
        int[] delta = new int[1001];
        for (int[] t : trips) { delta[t[1]] += t[0]; delta[t[2]] -= t[0]; }
        int load = 0;
        for (int d : delta) { load += d; if (load > capacity) return false; }
        return true;
    }

    public static void main(String[] args) {
        System.out.println(Arrays.deepToString(
            merge(new int[][] {{1,3},{8,10},{2,6},{15,18},{17,20}})));
        System.out.println(Arrays.deepToString(merge(new int[][] {{1,4},{4,5}})));
        System.out.println(Arrays.deepToString(
            insert(new int[][] {{1,2},{3,5},{6,7},{8,10},{12,16}}, new int[] {4, 8})));
        System.out.println("rooms = " + minRooms(new int[][] {{0,30},{5,10},{15,20}}));
        System.out.println("rooms = " + minRooms(new int[][] {{7,10},{2,4}}));
        System.out.println("remove = " + eraseOverlap(new int[][] {{1,2},{2,3},{3,4},{1,3}}));
        System.out.println(Arrays.deepToString(intersect(new int[][] {{0,2},{5,10},{13,23},{24,25}},
            new int[][] {{1,5},{8,12},{15,24},{25,26}})));
        System.out.println("carPooling = " + carPooling(new int[][] {{2,1,5},{3,3,7}}, 4)
            + ", " + carPooling(new int[][] {{2,1,5},{3,5,7}}, 3));
        Integer[] nums = {3, 30, 34, 5, 9};
        Arrays.sort(nums, (x, y) -> (y.toString() + x).compareTo(x.toString() + y));
        System.out.println("largest number: "
            + String.join("", Arrays.stream(nums).map(String::valueOf).toList()));
    }
}
```

Run it TERMINAL

```
$ java Main.java
[[1, 6], [8, 10], [15, 20]]
[[1, 5]]
[[1, 2], [3, 10], [12, 16]]
rooms = 2
rooms = 1
remove = 1
[[1, 2], [5, 5], [8, 10], [15, 23], [24, 24], [25, 25]]
carPooling = false, true
largest number: 9534330
```

What to notice:

- **Insert interval** needs no sort because the list is already sorted: it copies all intervals that end before the new one, merges those that overlap it, and copies the rest. Three loops, O(n).
- **Meeting rooms** sorts `starts` and `ends` separately. This discards the pairing but remains valid: the number of rooms in use at any instant depends only on how many meetings have started and how many have ended. The condition `ends[j] <= starts[i]` treats a meeting ending at time 10 as finished when another starts at time 10. (The original "Meeting Rooms II" problem is premium-only, so the practice table lists a free equivalent.)
- **Erase overlap** sorts by **end** and keeps every interval that starts at or after the end of the last kept one. This is the standard activity-selection greedy: the interval that finishes first leaves the largest remaining time.
- **Intersections** advance the pointer of the interval that ends first, because that interval cannot intersect anything further in the other list.
- **Car pooling** is a sweep line: passengers are added at the pick-up index and subtracted at the drop-off index, and a running sum is maintained. This is the difference-array technique of the prefix-sum chapter.
- **Largest number** uses a non-numeric comparator: `a` precedes `b` when the string `a + b` is lexicographically larger than `b + a`. Sorting `[3, 30, 34, 5, 9]` accordingly gives `9534330`.

2.8.4 Complexity and mistakes

Time: O(n log n) for the sort plus O(n) for the sweep, hence O(n log n). The sweep alone is linear; when the input is already sorted (insert interval, intersections) the whole solution is O(n). **Space:** O(n) for the output; `Arrays.sort` on an object array with a comparator is a stable merge-based sort (TimSort) that may use up to O(n) auxiliary memory.

Common defects:

- **Mutating the input.** The assignment `cur = a[i]` followed by `cur[1] = ...` modifies the original array. Cloning, or constructing new `int[]` objects, is required when the caller reuses the data.
- `<` **instead of** `<=` (or the reverse) in the overlap test. The treatment of touching intervals must be decided and verified on an example such as `[1,4]` and `[4,5]`.
- **Using** `next.end` **instead of** `max(cur.end, next.end)`, which can shrink the current interval.
- **Forgetting to add the last interval** after the loop.
- **Sorting by an unsuitable key** for greedy selection: sorting by start for "maximum number of non-overlapping intervals" fails on `[1,100], [2,3], [4,5]`, whereas sorting by end is correct.
- **Empty input.** `a[0]` throws on an empty array; a guard such as `if (a.length == 0) return ...` is required.

2.8.5 Questions to expect

1. **"Merge all overlapping intervals."** Clue: arbitrary order, "overlap". Sort by start, sweep with one current interval.
2. **"Insert a new interval into a sorted, non-overlapping list."** Clue: already sorted. Linear three-phase scan, no sort needed. Follow-up: use binary search to find the position, then merge (still O(n) to rebuild the list).
3. **"Can one person attend all meetings?"** Sort by start, check that every start is at least the previous end. (Premium problem; the idea is one line.)
4. **"How many conference rooms are needed?"** Clue: "at the same time". Sort starts and ends separately, or use a min-heap of end times. The heap version also tells you which room is reused.
5. **"Remove the minimum number of intervals so the rest do not overlap."** Clue: minimum removals = n minus maximum kept. Sort by end, keep greedily.
6. **"Minimum arrows to burst all balloons."** Same greedy as 5, but touching balloons share an arrow (`<=`).

7. **"Do these bookings ever exceed the capacity?"** Clue: running total over time. Difference array or sweep over sorted events. Follow-up: coordinates up to 10^9, so use a `TreeMap` of events instead of an array.
8. **"Arrange numbers to form the largest value."** Clue: custom order. Comparator on concatenations.

INTERVIEW ANGLE

Interviewer: "Why does sorting by start work for merging but sorting by end for scheduling?" Candidate: "For merging, every potential overlap partner must be adjacent. In start order, any interval that overlaps the current block starts no later than the block's end, so only the next interval needs inspection. For maximum non-overlapping selection the objective is to block as little future time as possible, which is achieved by the interval that ends first; hence the sort by end. The proof is an exchange argument: any optimal solution can replace its first interval by the earliest-ending one without losing an interval."

2.8.6 LeetCode practice

#	Title	Level	Key idea
56	Merge Intervals	Medium	Sort by start; extend or close the current interval
57	Insert Interval	Medium	Copy before, merge overlapping, copy after
729	My Calendar I	Medium	Reject a booking that overlaps any earlier one; sorted structure or brute force
986	Interval List Intersections	Medium	Two pointers; advance the list that ends first
435	Non-overlapping Intervals	Medium	Sort by end; keep what fits; answer is n minus kept
452	Minimum Number of Arrows to Burst Balloons	Medium	Sort by end; new arrow when the next start is beyond the last arrow
1094	Car Pooling	Medium	Difference array over stops and a running load
2406	Divide Intervals Into Minimum Number of Groups	Medium	Maximum overlap: sorted starts and ends, or a min-heap of ends
179	Largest Number	Medium	Comparator: a before b when a+b > b+a as strings

LeetCode problems for this pattern.

KEY POINTS

- Sorting reduces a pairwise problem to a single scan; the sort key follows the objective (start for merging, end for greedy selection).
- Two intervals overlap when `next.start <= cur.end`; merging uses `max` of the ends, and modified arrays are cloned.
- Counting simultaneous intervals is a sweep: +1 at starts, -1 at ends, track the maximum (two sorted arrays, a heap, or a difference array).
- Comparators use `Integer.compare`, never subtraction; the treatment of touching intervals must be clarified.
- Typical cost is O(n log n) time and O(n) space.

2.9 Kadane, greedy scans and in-place array tricks

Many array problems admit a direct solution with nested loops or auxiliary arrays, and are then restated with the requirement of O(n) time and O(1) extra space. The patterns in this chapter satisfy that requirement: a single scan that carries one or two running values (Kadane's maximum subarray, best time to buy and sell), reuse of the array itself as storage (cyclic sort, rotation by reversal, Dutch national flag), or a constant-size voting state (Boyer-Moore). Each is short to implement and sensitive to small errors, so each is derived from the invariant it maintains.

2.9.1 The idea

The common element is a **running summary**: after processing `a[0..i]`, a constant amount of information sufficient to answer for the prefix is retained, and it is updated in O(1) when `a[i+1]` is read. Each technique uses a different summary.

Trick	State carried while scanning	Typical problem
Kadane	Best sum of a subarray that ends here, and the best seen so far	Maximum subarray
Running minimum	Cheapest price so far	Best time to buy and sell stock
Prefix and suffix products	Product of everything to the left, then to the right	Product of array except self
Dutch national flag	Three regions: 0s, unknown, 2s, with three pointers	Sort an array of 0, 1, 2
Cyclic sort	Value v belongs at index v - 1; swap until it is there	First missing positive, duplicates
Reversal	Reverse whole, then both parts	Rotate array by k
Boyer-Moore voting	One candidate and a counter	Majority element

Tricks in this chapter and what each one keeps.

Recognition clues:

- "maximum / minimum sum of a contiguous subarray" (Kadane), or "maximum product" (track min and max)
- "one pass", "O(1) extra space", "in place", "do not use the library sort"
- Values in the range `1..n` stored in an array of length `n`: the indices serve as a hash table (cyclic sort)
- "element that appears more than n/2 times (or n/3)" (voting)
- Only three distinct values, "sort colours" (Dutch flag)

The skeleton of Kadane's algorithm:

Template: Kadane's algorithm JAVA

```java
int best = a[0], cur = a[0];            // cur = best sum of a subarray ENDING at i
for (int i = 1; i < a.length; i++) {
    cur = Math.max(a[i], cur + a[i]);   // extend the previous run, or start fresh at i
    best = Math.max(best, cur);
}
```

Justification. The best subarray ending at `i` is either `a[i]` alone or the best subarray ending at `i-1` extended by `a[i]`. A negative running sum `cur` can only decrease any extension, so `max(a[i], cur + a[i])` selects `a[i]` alone, which is equivalent to discarding the prefix. Initialising with `a[0]` rather than 0 makes all-negative arrays correct: the result is then the largest single element.

2.9.2 Dry run

Kadane on `[-2, 1, -3, 4, -1, 2, 1, -5, 4]`. The maximum subarray is `[4, -1, 2, 1]` with sum 6. The table is the measured trace; "restart" denotes `a[i] > cur + a[i]`.

i	a[i]	decision	cur	best
0	-2	initial	-2	-2
1	1	restart (1 > -2 + 1)	1	1
2	-3	extend (1 + -3 = -2 is better than -3)	-2	1
3	4	restart (4 > -2 + 4 = 2)	4	4
4	-1	extend	3	4
5	2	extend	5	5
6	1	extend	6	6
7	-5	extend	1	6
8	4	extend	5	6

Kadane trace.

sreram

The result is 6 at `i = 6`. The run restarted at index 3, the start of the subarray `[4, -1, 2, 1]`. To recover the subarray itself, the index of the last restart and the index at which `best` last improved are recorded.

A second trace covers the Dutch national flag (sorting `[2, 0, 2, 1, 1, 0]`). Three pointers partition the array into four regions: `[0, lo)` contains only 0s, `[lo, mid)` only 1s, `[mid, hi]` is unclassified, and `(hi, n)` contains only 2s. This partition is the loop invariant.

step	action	array	lo	mid	hi
1	a[mid]=2: swap(mid, hi), hi--	[0, 0, 2, 1, 1, 2]	0	0	4
2	a[mid]=0: swap(lo, mid), lo++, mid++	[0, 0, 2, 1, 1, 2]	1	1	4
3	a[mid]=0: swap(lo, mid), lo++, mid++	[0, 0, 2, 1, 1, 2]	2	2	4
4	a[mid]=2: swap(mid, hi), hi--	[0, 0, 1, 1, 2, 2]	2	2	3
5	a[mid]=1: mid++	[0, 0, 1, 1, 2, 2]	2	3	3
6	a[mid]=1: mid++	[0, 0, 1, 1, 2, 2]	2	4	3

Dutch national flag trace, array after each action.

Step 1 illustrates the essential rule: after a 2 is swapped to the back, `mid` is **not** advanced, because the element that arrived from the right end has not yet been classified. After a swap with `lo`, `mid` advances, because the element arriving from the left region is known to be a 1.

2.9.3 Java code

The program collects the family: Kadane, one-trade and unlimited-trade stock profit, product except self without division, Dutch flag, first missing positive by cyclic sort, rotation by three reversals, Boyer-Moore for n/2 and n/3, and maximum product (a variant of Kadane that tracks both the maximum and the minimum product, because multiplication by a negative number exchanges their roles).

Main.java JAVA

```java
import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

public class Main {
    static int maxSubarray(int[] a) {
        int best = a[0], cur = a[0];
        for (int i = 1; i < a.length; i++) {
            cur = Math.max(a[i], cur + a[i]);     // extend the run or restart here
            best = Math.max(best, cur);
        }
        return best;
    }

    static int maxProfit(int[] prices) {
        int minSoFar = Integer.MAX_VALUE, best = 0;
        for (int p : prices) {
            minSoFar = Math.min(minSoFar, p);
            best = Math.max(best, p - minSoFar);
        }
        return best;
    }

    static int maxProfitMany(int[] prices) {
        int total = 0;
        for (int i = 1; i < prices.length; i++)
            if (prices[i] > prices[i - 1]) total += prices[i] - prices[i - 1];
        return total;
    }

    static int[] productExceptSelf(int[] a) {
        int n = a.length;
        int[] out = new int[n];
        out[0] = 1;
        for (int i = 1; i < n; i++) out[i] = out[i - 1] * a[i - 1];   // product of all to the left
        int right = 1;
        for (int i = n - 1; i >= 0; i--) { out[i] *= right; right *= a[i]; }
```

Main.java (continued) JAVA

```
        return out;
    }

    static void sortColors(int[] a) {                // Dutch national flag
        int lo = 0, mid = 0, hi = a.length - 1;
        while (mid <= hi) {
            if (a[mid] == 0) swap(a, lo++, mid++);
            else if (a[mid] == 2) swap(a, mid, hi--);   // do not advance mid: new value unseen
            else mid++;
        }
    }

    static void swap(int[] a, int i, int j) { int t = a[i]; a[i] = a[j]; a[j] = t; }

    static int firstMissingPositive(int[] a) {       // cyclic sort: value v belongs at index v - 1
        int n = a.length;
        for (int i = 0; i < n; i++) {
            while (a[i] >= 1 && a[i] <= n && a[a[i] - 1] != a[i]) swap(a, i, a[i] - 1);
        }
        for (int i = 0; i < n; i++) if (a[i] != i + 1) return i + 1;
        return n + 1;
    }

    static void rotate(int[] a, int k) {
        k %= a.length;
        reverse(a, 0, a.length - 1);
        reverse(a, 0, k - 1);
        reverse(a, k, a.length - 1);
    }

    static void reverse(int[] a, int i, int j) { while (i < j) swap(a, i++, j--); }

    static int majority(int[] a) {                   // Boyer-Moore voting
        int cand = 0, count = 0;
        for (int x : a) {
            if (count == 0) cand = x;
            count += (x == cand) ? 1 : -1;
        }
        return cand;
    }

    static List<Integer> majorityThird(int[] a) {
        int c1 = 0, c2 = 1, n1 = 0, n2 = 0;
        for (int x : a) {
            if (x == c1) n1++;
            else if (x == c2) n2++;
            else if (n1 == 0) { c1 = x; n1 = 1; }
            else if (n2 == 0) { c2 = x; n2 = 1; }
            else { n1--; n2--; }
        }
        int k1 = 0, k2 = 0;
        for (int x : a) { if (x == c1) k1++; else if (x == c2) k2++; }
        List<Integer> res = new ArrayList<>();
        if (k1 > a.length / 3) res.add(c1);
        if (k2 > a.length / 3) res.add(c2);
        return res;
    }

    static int maxProduct(int[] a) {
        int mx = a[0], mn = a[0], best = a[0];
        for (int i = 1; i < a.length; i++) {
            int x = a[i];
            int hi = Math.max(x, Math.max(mx * x, mn * x));
            int lo = Math.min(x, Math.min(mx * x, mn * x));
            mx = hi; mn = lo;
            best = Math.max(best, mx);
        }
        return best;
    }

    public static void main(String[] args) {
        System.out.println("kadane = " + maxSubarray(new int[] {-2, 1, -3, 4, -1, 2, 1, -5, 4}));
        System.out.println("kadane all negative = " + maxSubarray(new int[] {-3, -1, -2}));
        System.out.println("one trade = " + maxProfit(new int[] {7, 1, 5, 3, 6, 4}));
        System.out.println("many trades = " + maxProfitMany(new int[] {7, 1, 5, 3, 6, 4}));
```

sreeram

Main.java (continued) JAVA

```
        System.out.println(Arrays.toString(productExceptSelf(new int[] {1, 2, 3, 4})));
        System.out.println(Arrays.toString(productExceptSelf(new int[] {-1, 1, 0, -3, 3})));
        int[] colors = {2, 0, 2, 1, 1, 0};
        sortColors(colors);
        System.out.println(Arrays.toString(colors));
        System.out.println("first missing = " + firstMissingPositive(new int[] {3, 4, -1, 1}));
        System.out.println("first missing = " + firstMissingPositive(new int[] {7, 8, 9, 11, 12}));
        int[] r = {1, 2, 3, 4, 5, 6, 7};
        rotate(r, 3);
        System.out.println(Arrays.toString(r));
        System.out.println("majority = " + majority(new int[] {2, 2, 1, 1, 1, 2, 2}));
        System.out.println("n/3 = " + majorityThird(new int[] {1, 1, 1, 3, 3, 2, 2, 2}));
        System.out.println("max product = " + maxProduct(new int[] {2, 3, -2, 4, -1}));
    }
}
```

Run it TERMINAL

```
$ java Main.java
kadane = 6
kadane all negative = -1
one trade = 5
many trades = 7
[24, 12, 8, 6]
[0, 0, 9, 0, 0]
[0, 0, 1, 1, 2, 2]
first missing = 2
first missing = 1
[5, 6, 7, 1, 2, 3, 4]
majority = 2
n/3 = [2, 1]
max product = 48
```

What to notice:

- **Stock, one trade.** The running minimum is the best purchase day so far; `p - minSoFar` is the best profit if the sale occurs today. Time O(n), space O(1). For unlimited trades the result is the sum of all positive day-to-day increases: 7 here (buy at 1, sell at 5; buy at 3, sell at 6).
- **Product except self.** The first loop stores in `out[i]` the product of all elements left of `i`; the second multiplies in the product of all elements to the right using the single variable `right`. Division is not used, so zeros need no special handling (the second example prints `[0, 0, 9, 0, 0]`).
- **Cyclic sort.** The `while` loop (not `if`) swaps until `a[i]` is at its home index, is out of range, or duplicates the value at its home index. Each swap places one value permanently, so the total work is O(n) despite the nesting. The first index with `a[i] != i + 1` identifies the missing value.
- **Rotate.** Reversing the whole array, then the first `k` and the last `n - k` elements, rotates right by `k` in place. The statement `k %= a.length` handles `k >= n`. If `k` becomes 0, `reverse(a, 0, -1)` is a no-op.
- **Boyer-Moore.** Pairs of distinct elements cancel; if a majority element exists, it survives the cancellation. The n/3 variant keeps two candidates. A second counting pass is **required** whenever the existence of a majority is not guaranteed.
- **Max product.** Keeping both `mx` and `mn` lets a negative factor exchange their roles. The product may overflow `int` for long arrays; the platform guarantees it does not, but the question may be raised in an interview.

2.9.4 Complexity and mistakes

All of these run in **O(n) time and O(1) extra space** (the output array of product-except-self is not counted as extra space in the usual statement). Cyclic sort contains a nested loop, but each swap fixes one element permanently, so the number of swaps is bounded by `n` (an amortised argument). Rotation by reversal performs about `2n` element moves; the direct method with a temporary array costs O(n) space.

Common defects:

- **Kadane initialised with 0.** An all-negative array then yields 0, which is not the sum of any non-empty subarray. The initial value must be `a[0]`.
- **Resetting `cur` to 0 on a negative value** without first updating `best`. The form `max(a[i], cur + a[i])` avoids the defect.

- **Stock problems: selling before buying.** The minimum must be updated first, then the profit computed; the initial minimum is `Integer.MAX_VALUE`, not 0.
- **Dutch flag: advancing `mid` after a swap with `hi`.** This loses an unclassified element, as shown above.
- **Cyclic sort with `if` instead of `while`**, or non-termination on duplicates; the test `a[a[i]-1] != a[i]` guarantees termination.
- **Boyer-Moore without verification** when a majority is not guaranteed.
- **Wrong rotation direction.** Rotation to the right by `k` moves the last `k` elements to the front. Test: `[1,2,3,4,5,6,7]`, `k = 3`, expected `[5,6,7,1,2,3,4]`.

2.9.5 Questions to expect

1. **"Largest sum of a contiguous subarray."** Clue: contiguous, maximum sum. Kadane in O(n). Follow-up: return the indices; or the circular version (max of Kadane and total minus minimum subarray, with the all-negative edge case).
2. **"Best day to buy and sell once, and and unlimited trades?"** Running minimum; sum of positive differences.
3. **"Product of all elements except the current one, no division, O(1) extra space."** Left product pass plus right product pass.
4. **"Sort an array of 0s, 1s, 2s in one pass without a library sort."** Dutch national flag with three pointers. Follow-up: counting sort is two passes, flag is one.
5. **"Find the smallest missing positive integer in O(n) time and O(1) space."** Cyclic sort; ignore values outside `1..n`. Alternative: use sign marking.
6. **"Find all numbers missing from, or duplicated in, an array of 1..n."** Cyclic sort, or negate `a[|x|-1]` as a marker. Another route: XOR or the sum formula `n(n+1)/2` for exactly one missing value.
7. **"Rotate the array right by k in place."** Three reversals. Follow-up: a juggling algorithm using gcd also works.
8. **"Majority element (more than n/2), then more than n/3."** Boyer-Moore with one or two candidates and a verification pass.

INTERVIEW ANGLE

Interviewer: "Why does Kadane's algorithm work? What is the correctness argument?" Candidate: "Define `cur` as the best sum of a subarray ending exactly at index i. It is either `a[i]` alone or the best subarray ending at i-1 extended by `a[i]`, so `cur = max(a[i], cur + a[i])`. The global optimum is the maximum of `cur` over all i, since the optimal subarray ends somewhere. This is a one-dimensional dynamic program; as each step reads only the previous value, a single variable suffices: O(n) time, O(1) space." Interviewer: "And for the maximum product?" Candidate: "A negative factor can turn the smallest product into the largest, so both the maximum and the minimum product ending at i are maintained."

sreram

2.9.6 LeetCode practice

#	Title	Level	Key idea
121	Best Time to Buy and Sell Stock	Easy	Running minimum price; best profit so far
169	Majority Element	Easy	Boyer-Moore voting
268	Missing Number	Easy	Sum formula, XOR, or cyclic sort
448	Find All Numbers Disappeared in an Array	Easy	Place each value at its index, or negate as a marker
53	Maximum Subarray	Medium	Kadane: extend or restart
122	Best Time to Buy and Sell Stock II	Medium	Sum of all positive day-to-day rises
238	Product of Array Except Self	Medium	Left products pass, then right products pass
75	Sort Colors	Medium	Dutch national flag with lo, mid, hi
189	Rotate Array	Medium	Reverse all, reverse first k, reverse the rest
229	Majority Element II	Medium	Boyer-Moore with two candidates plus a count check
152	Maximum Product Subarray	Medium	Track current max and min product
287	Find the Duplicate Number	Medium	Cyclic placement, or fast and slow pointers on the index graph
41	First Missing Positive	Hard	Cyclic sort; answer is the first index with a wrong value

LeetCode problems for this pattern.

KEY POINTS

- Kadane: `cur = max(a[i], cur + a[i])`, `best = max(best, cur)`; the initial value is `a[0]` so that negative arrays are handled.
- Most one-pass techniques carry a constant-size summary (minimum so far, left product, candidate and count) instead of auxiliary arrays.
- Cyclic sort treats the array as a hash table in which value `v` belongs at index `v - 1`; `while` and the duplicate check are required.
- The Dutch flag does not advance `mid` after a swap with `hi`; rotation is three reversals; Boyer-Moore needs a verification pass unless a majority is guaranteed.
- All are O(n) time, O(1) extra space.

2.10 Matrix and string patterns

Two-dimensional arrays and strings are characterised by algorithms that are simple to state but sensitive to index bookkeeping. Spiral traversal, in-place image rotation, longest palindromic substring and substring search all require careful treatment of boundaries. This chapter presents reliable procedures: shrinking boundaries for the spiral, transposition followed by row reversal for rotation, marker rows for zeroing, a staircase walk for sorted matrices, expansion around a centre for palindromes, and the principle of KMP for linear-time pattern matching.

2.10.1 The idea

Matrices. An `m x n` matrix is an array of `m` rows with `n` entries each, accessed as `grid[row][col]`. The standard procedures are:

- *Spiral*: maintain four boundaries `top, bottom, left, right`, walk one side, then move that boundary inward.
- *Rotate 90 degrees clockwise in place*: transpose (swap `[i][j]` with `[j][i]`), then reverse every row. Counter-clockwise: transpose, then reverse every column (or reverse the rows first).
- *Set zeroes with O(1) space*: use the first row and first column as flag storage for "this column/row must be zeroed", after remembering whether they themselves held a zero.
- *Sorted matrix (rows and columns ascending)*: start at the top-right corner; if the value is too big go left, if too small go down. Every step discards a whole row or column.

Palindromes. A palindrome is symmetric about its centre. A string of length `n` has `2n - 1` centres (n characters and n-1 gaps). Expanding outward from each centre while the end characters match enumerates every palindromic substring in O(n^2) time and O(1) space, without a dynamic-programming table.

Pattern matching. Naive search compares the pattern at every text position and costs O(n m) in the worst case. KMP (Knuth-Morris-Pratt) never moves backwards in the text: it precomputes, for each prefix of the pattern, the length of its longest proper prefix that is also a suffix (a *border*), and uses it to determine the shift after a mismatch.

Recognition clues:

- "in place", "O(1) extra space", "rotate", "spiral", "layer by layer" for matrices
- "each row sorted and each column sorted" (staircase) versus "row-major sorted" (flatten and binary search, last chapter's matrix problem)
- "palindromic substring", "longest palindrome", "count palindromes" (expand around centre)
- "find the first occurrence of a needle in a haystack", "repeated substring" (KMP / prefix function)

Template: expand around a centre JAVA

```java
for (int center = 0; center < 2 * s.length() - 1; center++) {
    int l = center / 2, r = l + center % 2;      // even center: r = l + 1 (a gap between letters)
    while (l >= 0 && r < s.length() && s.charAt(l) == s.charAt(r)) {
        // s[l..r] is a palindrome here
        l--; r++;
    }
}
```

The single loop over `2n - 1` centres handles odd and even lengths uniformly: for `center = 2k` the expansion starts at `l = r = k` (odd length), and for `center = 2k + 1` at `l = k, r = k + 1` (even length).

2.10.2 Dry run

First, the image `[[1,2,3],[4,5,6],[7,8,9]]` is rotated by 90 degrees clockwise. The rule is transposition followed by reversal of each row. The trace shows the matrix after each swap.

step	matrix after the step
start	[[1,2,3],[4,5,6],[7,8,9]]
swap (0,1) with (1,0)	[[1,4,3],[2,5,6],[7,8,9]]
swap (0,2) with (2,0)	[[1,4,7],[2,5,6],[3,8,9]]
swap (1,2) with (2,1)	[[1,4,7],[2,5,8],[3,6,9]]
reverse row 0	[[7,4,1],[2,5,8],[3,6,9]]
reverse row 1	[[7,4,1],[8,5,2],[3,6,9]]
reverse row 2	[[7,4,1],[8,5,2],[9,6,3]]

Rotate image in place (transpose, then reverse each row).

The transposition visits only cells above the diagonal (`j > i`); visiting all cells would swap each pair twice and restore the original. The left column `1, 4, 7`, read bottom to top as `7, 4, 1`, becomes the top row, which is the effect of a clockwise rotation.

Second, expansion around the centre is traced on `"babad"` (length 5, hence 9 centres). Each row gives the starting pair, the stopping pair (first mismatch or boundary) and the palindrome found, `s.substring(l + 1, r)`.

centre	start (l, r)	stops at (l, r)	palindrome
0	0, 0	-1, 1	b
1	0, 1	0, 1	empty (b != a)
2	1, 1	-1, 3	bab
3	1, 2	1, 2	empty (a != b)
4	2, 2	0, 4	aba
5	2, 3	2, 3	empty (b != a)
6	3, 3	2, 4	a

sreram

centre	start (l, r)	stops at (l, r)	palindrome
7	3, 4	3, 4	empty (a != d)
8	4, 4	3, 5	d

Palindromic expansion on "babad".

The longest palindromes are `bab` and `aba` (length 3 each; either is accepted). The length of a palindrome found is `r - l - 1`, because the loop exits one step beyond both ends.

2.10.3 Java code

The listing contains spiral order (with the two `if` guards that prevent a single remaining row or column from being traversed twice), rotation, set zeroes, the staircase search, counting and longest palindromic substring, and the KMP prefix function with the search loop.

Main.java JAVA

```java
import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

public class Main {
    static List<Integer> spiral(int[][] m) {
        List<Integer> out = new ArrayList<>();
        int top = 0, bottom = m.length - 1, left = 0, right = m[0].length - 1;
        while (top <= bottom && left <= right) {
            for (int c = left; c <= right; c++) out.add(m[top][c]);
            top++;
            for (int r = top; r <= bottom; r++) out.add(m[r][right]);
            right--;
            if (top <= bottom) {
                for (int c = right; c >= left; c--) out.add(m[bottom][c]);
                bottom--;
            }
            if (left <= right) {
                for (int r = bottom; r >= top; r--) out.add(m[r][left]);
                left++;
            }
        }
        return out;
    }

    static void rotate(int[][] m) {                     // clockwise, in place
        int n = m.length;
        for (int i = 0; i < n; i++)
            for (int j = i + 1; j < n; j++) {           // transpose
                int t = m[i][j]; m[i][j] = m[j][i]; m[j][i] = t;
            }
        for (int[] row : m)                             // reverse each row
            for (int i = 0, j = n - 1; i < j; i++, j--) {
                int t = row[i]; row[i] = row[j]; row[j] = t;
            }
    }

    static void setZeroes(int[][] m) {                  // O(1) extra: first row/col are markers
        int rows = m.length, cols = m[0].length;
        boolean firstRow = false, firstCol = false;
        for (int c = 0; c < cols; c++) if (m[0][c] == 0) firstRow = true;
        for (int r = 0; r < rows; r++) if (m[r][0] == 0) firstCol = true;
        for (int r = 1; r < rows; r++)
            for (int c = 1; c < cols; c++)
                if (m[r][c] == 0) { m[r][0] = 0; m[0][c] = 0; }
        for (int r = 1; r < rows; r++)
            for (int c = 1; c < cols; c++)
                if (m[r][0] == 0 || m[0][c] == 0) m[r][c] = 0;
        if (firstRow) Arrays.fill(m[0], 0);
        if (firstCol) for (int r = 0; r < rows; r++) m[r][0] = 0;
    }

    static boolean searchMatrix2(int[][] m, int target) {   // staircase from top-right
        int r = 0, c = m[0].length - 1;
        while (r < m.length && c >= 0) {
            if (m[r][c] == target) return true;
```

sreeram

Main.java (continued) JAVA

```
            if (m[r][c] > target) c--;
            else r++;
        }
        return false;
    }

    static int countPalindromes(String s) {
        int count = 0;
        for (int center = 0; center < 2 * s.length() - 1; center++) {
            int l = center / 2, r = l + center % 2;          // odd centre: l == r; even: r = l + 1
            while (l >= 0 && r < s.length() && s.charAt(l) == s.charAt(r)) { count++; l--; r++; }
        }
        return count;
    }

    static String longestPalindrome(String s) {
        int bestL = 0, bestR = 0;
        for (int center = 0; center < 2 * s.length() - 1; center++) {
            int l = center / 2, r = l + center % 2;
            while (l >= 0 && r < s.length() && s.charAt(l) == s.charAt(r)) { l--; r++; }
            if (r - l - 1 > bestR - bestL + 1) { bestL = l + 1; bestR = r - 1; }
        }
        return s.substring(bestL, bestR + 1);
    }

    // pi[i] = length of the longest proper border of p[0..i]
    static int[] prefixFunction(String p) {
        int[] pi = new int[p.length()];
        for (int i = 1, k = 0; i < p.length(); i++) {
            while (k > 0 && p.charAt(i) != p.charAt(k)) k = pi[k - 1];
            if (p.charAt(i) == p.charAt(k)) k++;
            pi[i] = k;
        }
        return pi;
    }

    static int kmpSearch(String text, String p) {
        if (p.isEmpty()) return 0;
        int[] pi = prefixFunction(p);
        for (int i = 0, k = 0; i < text.length(); i++) {
            while (k > 0 && text.charAt(i) != p.charAt(k)) k = pi[k - 1];
            if (text.charAt(i) == p.charAt(k)) k++;
            if (k == p.length()) return i - k + 1;
        }
        return -1;
    }

    public static void main(String[] args) {
        System.out.println(spiral(new int[][] {{1,2,3},{4,5,6},{7,8,9}}));
        System.out.println(spiral(new int[][] {{1,2,3,4},{5,6,7,8},{9,10,11,12}}));
        int[][] img = {{1,2,3},{4,5,6},{7,8,9}};
        rotate(img);
        System.out.println(Arrays.deepToString(img));
        int[][] z = {{1,1,1},{1,0,1},{1,1,1}};
        setZeroes(z);
        System.out.println(Arrays.deepToString(z));
        int[][] m = {{1,4,7,11},{2,5,8,12},{3,6,9,16},{10,13,14,17}};
        System.out.println(searchMatrix2(m, 14) + " " + searchMatrix2(m, 15));
        System.out.println(countPalindromes("aaa") + " " + countPalindromes("abc"));
        System.out.println(longestPalindrome("babad") + " " + longestPalindrome("cbbd"));
        System.out.println(Arrays.toString(prefixFunction("abababaca")));
        System.out.println(kmpSearch("sadbutsad", "sad") + " " + kmpSearch("leetcode", "leeto")
            + " " + kmpSearch("abababaca", "ababca"));
        String t = "abcabc";
        boolean rep = (t + t).substring(1, 2 * t.length() - 1).contains(t);
        System.out.println("repeated pattern: " + rep);
    }
}
```

Sreeram

Run it TERMINAL

```
$ java Main.java
[1, 2, 3, 6, 9, 8, 7, 4, 5]
[1, 2, 3, 4, 8, 12, 11, 10, 9, 5, 6, 7]
[[7, 4, 1], [8, 5, 2], [9, 6, 3]]
[[1, 0, 1], [0, 0, 0], [1, 0, 1]]
true false
6 3
bab bb
[0, 0, 1, 2, 3, 0, 1]
0 -1 2
repeated pattern: true
```

What to notice:

- **Spiral.** After the top row and right column are traversed, the bottom row is valid only if `top <= bottom` and the left column only if `left <= right`. Without these guards a one-row or one-column remainder is emitted twice. The 3x4 example (`[1, 2, 3, 4, 8, 12, 11, 10, 9, 5, 6, 7]`) exercises them.
- **Set zeroes.** The order of operations is significant: first record whether row 0 and column 0 contain a zero, then use them as markers for the inner cells, then apply the markers to the inner cells, and **finally** zero row 0 and column 0. Any earlier zeroing destroys the markers.
- **Staircase search.** From the top-right corner each comparison eliminates one row or one column, so at most `m + n` steps are performed. Starting at the top-left corner fails, because both neighbours are larger.
- **Palindromes.** `countPalindromes("aaa")` is 6: `a, a, a, aa, aa, aaa`. The counting version increments inside the `while` loop; the longest-substring version records the best window after the loop.
- **Prefix function.** For `ababaca` the output is `[0, 0, 1, 2, 3, 0, 1]`: at index 4 (`ababa`) the longest border is `aba` (length 3). The search loop applies the same fallback `k = pi[k - 1]` after a mismatch, so the text index `i` never decreases.
- The last line shows a one-line trick for "repeated substring pattern": a string is a repetition of a smaller block exactly when it occurs inside `(t + t)` with its first and last character removed.

2.10.4 Complexity and mistakes

Spiral, rotate, set zeroes: O(m n) time, O(1) extra space (excluding the output list of the spiral). **Staircase search:** O(m + n). **Expansion around centres:** `2n - 1` centres, each expanding up to `n/2` steps, hence O(n^2) time and O(1) space; Manacher's algorithm reduces the longest-palindrome problem to O(n) but is rarely required. **KMP:** O(n + m) time (the fallback iterations of the inner `while` are bounded by the earlier increments of `k`, an amortised argument) and O(m) space for the prefix table. Naive search is O(n m) in the worst case, for example pattern `aaa...ab` in text `aaa...a`. In production code `String.indexOf` is used.

Common defects:

- **Rotation with a copy** when the requirement is in place. The transposition method uses no second matrix; it applies only to square matrices, since a non-square matrix cannot be rotated in place.
- **Transposing the entire matrix** (`j` starting at 0 instead of `i + 1`), which swaps each pair twice.
- **Spiral without the guards**, which duplicates elements on thin matrices; or `<` where `<=` is required, which skips the centre cell of an odd-sized square.
- **Confusing `m` and `n`:** `grid.length` is the number of rows and `grid[0].length` the number of columns. On a non-square matrix, indexing `grid[i][j]` with `i` bounded by the column count throws `ArrayIndexOutOfBoundsException`.
- **Zeroing in the same pass as marking** in set zeroes, which erases the whole matrix.
- **Building strings with `+=` in a loop** (O(n^2)); `StringBuilder` is the correct tool. Note that `s.charAt(i) - 'a'` yields an index in 0-25 only for lower-case letters.
- **Off-by-one in `substring`:** `s.substring(l + 1, r)` has an inclusive start and an exclusive end.

2.10.5 Questions to expect

1. **"Return the elements of a matrix in spiral order."** Clue: "spiral". Four shrinking boundaries; mind the guards. Follow-up: generate an `n x n` spiral filled with `1..n^2` (same walk, writing instead of reading).
2. **"Rotate an `n x n` image by 90 degrees in place."** Clue: "in place". Transpose then reverse rows. Follow-up: counter-clockwise? Anti-transpose? Rotate four cells at a time layer by layer.
3. **"If a cell is zero, set its entire row and column to zero, in O(1) space."** Marker row and column; remember their own state first.
4. **"Search a matrix whose rows and columns are sorted."** Staircase from top-right in O(m + n). Compare with the "row-major sorted" version, where binary search on the flattened index gives O(log(m n)).
5. **"Count the palindromic substrings / find the longest palindromic substring."** Expand around each of the `2n - 1` centres. Follow-up: can you do better than O(n^2)? (Manacher; DP is also O(n^2) but needs O(n^2) space.)
6. **"Implement `strStr` / `indexOf`."** The naive double loop is acceptable to start; mention KMP (or Rabin-Karp with a rolling hash) when asked to avoid the worst case.
7. **"Is this string built by repeating a substring?"** Doubling trick `(s + s)` without ends, or the prefix function: with `p = n - pi[n-1]`, the string repeats exactly when `p` divides `n` and `pi[n-1] > 0`.
8. **"Simulate one generation of Conway's Game of Life in place."** Encode old and new state in the same cell (for example bit 1 = next state), then shift.

INTERVIEW ANGLE

Interviewer: "How would you rotate the image without a second matrix?" Candidate: "A clockwise rotation maps `(i, j)` to `(j, n - 1 - i)`. This decomposes into two in-place steps: transposition, which sends `(i, j)` to `(j, i)`, then reversal of each row, which sends `(j, i)` to `(j, n - 1 - i)`. Time is O(n^2) and extra space O(1). The anticlockwise rotation transposes and then reverses each column." Interviewer: "Does this apply to a 3x5 matrix?" Candidate: "Not in place: the result is 5x3, a different shape, so a new matrix is required."

2.10.6 LeetCode practice

#	Title	Level	Key idea
28	Find the Index of the First Occurrence in a String	Easy	Naive double loop, or KMP prefix function
459	Repeated Substring Pattern	Easy	Find s inside (s+s) without first and last char, or use the border
54	Spiral Matrix	Medium	Four shrinking boundaries with guards for thin remainders
59	Spiral Matrix II	Medium	Same walk, writing 1..n^2
48	Rotate Image	Medium	Transpose, then reverse each row
73	Set Matrix Zeroes	Medium	Use first row and column as markers; zero them last
240	Search a 2D Matrix II	Medium	Staircase from the top-right corner
289	Game of Life	Medium	Encode old and new states together to update in place
6	Zigzag Conversion	Medium	Bounce a row index up and down, append to one StringBuilder per row
647	Palindromic Substrings	Medium	Expand around 2n-1 centres and count
5	Longest Palindromic Substring	Medium	Expand around centre, remember the best window
214	Shortest Palindrome	Hard	Longest palindromic prefix via KMP prefix function on s + # + reverse(s)

LeetCode problems for this pattern.

sreeram

KEY POINTS

- Matrix procedures: spiral = four shrinking boundaries with guards; rotation = transposition then row reversal; set zeroes = markers in the first row and column; sorted rows and columns = staircase from the top-right corner.
- Palindromes: expansion around `2n - 1` centres in O(n^2) time and O(1) space; the length found is `r - l - 1` after the loop.
- The KMP prefix function stores the longest border of each prefix, so the text index never decreases: O(n + m).
- Most defects are index defects: rows versus columns, inclusive versus exclusive ends, and the order of operations in in-place updates.
- `StringBuilder` is used for string construction; `String.indexOf` for practical substring search.

sreeram

PART 3

Linked Lists, Stacks and Queues

Pointer surgery, last-in-first-out and first-in-first-out thinking: the structures behind half of all interview questions.

In this part

3.1 Linked list basics, dummy node and reversal
3.2 Fast and slow pointers
3.3 Stack patterns
3.4 Monotonic stack
3.5 Queue, deque and monotonic deque

3.1 Linked list basics, dummy node and reversal

A singly linked list is a sequence of nodes. Each node holds a value and one reference, `next`, to its successor; the list itself is represented by a reference to the first node, the **head**. Positional access is not available, so reaching the k-th node requires k traversal steps. Conversely, once a node has been reached, insertion or deletion adjacent to it requires O(1) time and no element shifting.

Linked list problems contain little algorithmic content; they test pointer discipline. If a single reference is overwritten prematurely, part of the list becomes unreachable. The brute-force approach (copying all values into an `ArrayList`, modifying the array and rebuilding the list) is correct but requires O(n) auxiliary space and does not exercise the intended skill. The pattern in this chapter rewires a list in place, in O(n) time and O(1) auxiliary space, without losing nodes.

3.1.1 The idea

Three tools cover most of the problems in this family.

Tool 1: the three-pointer walk. A list is reversed by a single traversal that redirects each `next` reference to the predecessor. Three references are maintained: `prev` (the already reversed prefix), `curr` (the node being redirected) and `next` (saved before `curr.next` is overwritten). The order of the four statements in the loop body is essential: save, redirect, advance `prev`, advance `curr`.

Tool 2: the dummy node. A dummy (sentinel) node is an artificial node placed before the real head. It eliminates the special case in which the head itself changes. Without it, insertion, deletion or merging at the head requires a separate conditional branch. With it, every node, including the first, has a predecessor, and `dummy.next` is returned as the result.

Tool 3: the tail pointer. When building a new list (merging, partitioning), keep a `tail` reference to the last node built so far and append with `tail.next = node; tail = tail.next`.

How to recognise it

- "Reverse the list", "reverse a part of it", "reverse every k nodes".
- "Merge", "splice", "reorder", "partition" or "rotate" linked lists, especially with "in place" or "O(1) extra space".
- "Remove all nodes with value x" or "delete duplicates" where the head might be removed.
- A function that returns a list whose head may differ from the input head.

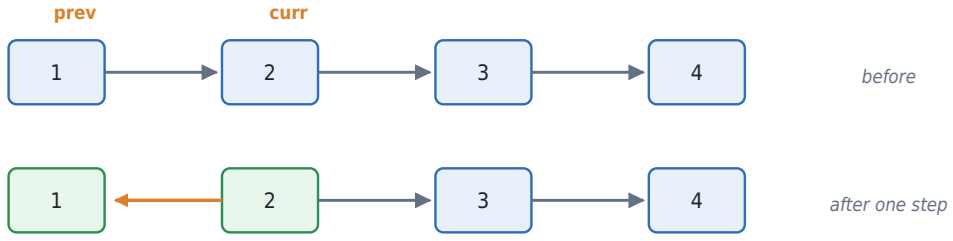


Figure 3.1. Reversal flips one arrow per step: prev trails curr, and next is saved before the flip so the rest of the list is never lost.

Template: reverse a list

Reverse.java JAVA

```java
static ListNode reverse(ListNode head) {
    ListNode prev = null, curr = head;
    while (curr != null) {
        ListNode next = curr.next;   // 1. save the rest of the list
        curr.next = prev;            // 2. flip the arrow
        prev = curr;                 // 3. grow the reversed part
        curr = next;                 // 4. move on
    }
    return prev;                     // prev is the new head
}
```

Template: build with a dummy node

Dummy.java JAVA

```java
ListNode dummy = new ListNode(0), tail = dummy;
// for each node you decide to keep: tail.next = node; tail = tail.next;
tail.next = null;                    // important if the last node still points somewhere
return dummy.next;
```

3.1.2 Dry run

The list `1 -> 2 -> 3 -> 4 -> 5` is reversed. Before the loop, `prev = null` and `curr` refers to node 1. The table gives the state at the end of each loop iteration, taken from a real trace of the program in the next section.

Step	prev	curr	Reversed part (starting at prev)
1	1	2	1
2	2	3	2 -> 1
3	3	4	3 -> 2 -> 1
4	4	5	4 -> 3 -> 2 -> 1
5	5	null	5 -> 4 -> 3 -> 2 -> 1

Reversing 1 -> 2 -> 3 -> 4 -> 5, state after each iteration.

In step 1, node 1 is `curr`; `next = 2` is saved, `1.next` is set to `null` (the previous value of `prev`), and `prev` and `curr` advance to nodes 1 and 2 respectively. The loop invariant, which holds after every step, is: `prev` **is the head of a correctly reversed list of the nodes already visited, and** `curr` **is the head of the unvisited remainder.** When `curr` becomes `null`, the postcondition holds and `prev` is the result. An empty input is handled implicitly: the loop body never executes and `prev` remains `null`.

The merge of `1 -> 3 -> 5` with `2 -> 4 -> 6 -> 8` is traced analogously: at each step the smaller front node is appended to `tail`, and only the pointer of the list from which the node was taken advances.

Step	a (front of A)	b (front of B)	Action	Output so far
1	3	2	take A	1
2	3	4	take B	1, 2
3	5	4	take A	1, 2, 3
4	5	6	take B	1, 2, 3, 4
5	null	6	take A	1, 2, 3, 4, 5
end	null	6	append rest of B	1, 2, 3, 4, 5, 6, 8

Merging two sorted lists, state after each iteration.

3.1.3 Java code

The program below defines a minimal `ListNode`, helpers to build and print lists, and five solutions: iterative and recursive reversal, reversing positions `left..right` (LeetCode 92), reversing in groups of k (LeetCode 25) and merging two sorted lists (LeetCode 21). The trace prints are the ones used in the dry run.

Main.java JAVA

```
import java.util.*;

public class Main {
    static class ListNode {
        int val;
        ListNode next;
        ListNode(int val) { this.val = val; }
    }

    static ListNode build(int... vals) {
        ListNode dummy = new ListNode(0), tail = dummy;
        for (int v : vals) { tail.next = new ListNode(v); tail = tail.next; }
        return dummy.next;
    }

    static String show(ListNode head) {
        StringBuilder sb = new StringBuilder();
        for (ListNode n = head; n != null; n = n.next) {
            if (sb.length() > 0) sb.append(" -> ");
            sb.append(n.val);
        }
        return sb.length() == 0 ? "(empty)" : sb.toString();
    }

    // Iterative reversal with a trace of each step
    static ListNode reverse(ListNode head, boolean trace) {
        ListNode prev = null, curr = head;
        int step = 0;
        while (curr != null) {
            ListNode next = curr.next;   // 1. save
            curr.next = prev;            // 2. flip
            prev = curr;                 // 3. advance prev
            curr = next;                 //    advance curr
            if (trace) {
                step++;
                System.out.println(step + " | prev=" + prev.val + " curr="
                    + (curr == null ? "null" : curr.val) + " | reversed part: " + show(prev));
            }
        }
        return prev;
    }

    static ListNode reverseRecursive(ListNode head) {
        if (head == null || head.next == null) return head;
        ListNode newHead = reverseRecursive(head.next);
        head.next.next = head;
        head.next = null;
        return newHead;
    }

    // LeetCode 92: reverse positions left..right (1-indexed)
    static ListNode reverseBetween(ListNode head, int left, int right) {
        ListNode dummy = new ListNode(0);
        dummy.next = head;
        ListNode before = dummy;
        for (int i = 1; i < left; i++) before = before.next;
        ListNode curr = before.next;
        for (int i = 0; i < right - left; i++) {
            ListNode moving = curr.next;     // node to move to the front of the segment
            curr.next = moving.next;
            moving.next = before.next;
            before.next = moving;
        }
        return dummy.next;
    }

    // LeetCode 25
    static ListNode reverseKGroup(ListNode head, int k) {
```

Sreeram

Main.java (continued) JAVA

```
        ListNode dummy = new ListNode(0);
        dummy.next = head;
        ListNode groupPrev = dummy;
        while (true) {
            ListNode kth = groupPrev;
            for (int i = 0; i < k && kth != null; i++) kth = kth.next;
            if (kth == null) break;                  // fewer than k nodes left
            ListNode groupNext = kth.next;
            ListNode prev = groupNext, curr = groupPrev.next;
            while (curr != groupNext) {
                ListNode nxt = curr.next;
                curr.next = prev;
                prev = curr;
                curr = nxt;
            }
            ListNode oldFirst = groupPrev.next;      // becomes the tail of the group
            groupPrev.next = kth;
            groupPrev = oldFirst;
        }
        return dummy.next;
    }

    // LeetCode 21
    static ListNode mergeTwo(ListNode a, ListNode b, boolean trace) {
        ListNode dummy = new ListNode(0), tail = dummy;
        int step = 0;
        List<Integer> out = new ArrayList<>();
        while (a != null && b != null) {
            String pick;
            if (a.val <= b.val) { out.add(a.val); tail.next = a; a = a.next; pick = "take A"; }
            else { out.add(b.val); tail.next = b; b = b.next; pick = "take B"; }
            tail = tail.next;
            if (trace) System.out.println(++step + " | a=" + (a == null ? "null" : a.val)
                + " b=" + (b == null ? "null" : b.val) + " | " + pick + " | out so far: " + out);
        }
        tail.next = (a != null) ? a : b;
        if (trace) System.out.println("end | append the rest of B | out: " + show(dummy.next));
        return dummy.next;
    }

    // LeetCode 83 using a dummy-free walk, and 203-style removal with dummy
    static ListNode removeValue(ListNode head, int target) {
        ListNode dummy = new ListNode(0);
        dummy.next = head;
        ListNode curr = dummy;
        while (curr.next != null) {
            if (curr.next.val == target) curr.next = curr.next.next;
            else curr = curr.next;
        }
        return dummy.next;
    }

    public static void main(String[] args) {
        System.out.println("Input: " + show(build(1, 2, 3, 4, 5)));
        System.out.println("-- trace of iterative reverse --");
        System.out.println("reversed: " + show(reverse(build(1, 2, 3, 4, 5), true)));
        System.out.println("recursive: " + show(reverseRecursive(build(1, 2, 3, 4, 5))));
        System.out.println("empty: " + show(reverse(null, false)));
        System.out.println("between 2..4: " + show(reverseBetween(build(1, 2, 3, 4, 5), 2, 4)));
        System.out.println("k=2: " + show(reverseKGroup(build(1, 2, 3, 4, 5), 2)));
        System.out.println("k=3: " + show(reverseKGroup(build(1, 2, 3, 4, 5), 3)));
        System.out.println("-- trace of merge --");
        System.out.println("A: 1 -> 3 -> 5   B: 2 -> 4 -> 6 -> 8");
        System.out.println("result: " + show(mergeTwo(build(1, 3, 5), build(2, 4, 6, 8), true)));
        System.out.println("remove 6: " + show(removeValue(build(6, 6, 1, 6, 2), 6)));
    }
}
```

sreeram

Compile and run TERMINAL

```
$ java Main.java
Input: 1 -> 2 -> 3 -> 4 -> 5
-- trace of iterative reverse --
1 | prev=1 curr=2 | reversed part: 1
2 | prev=2 curr=3 | reversed part: 2 -> 1
3 | prev=3 curr=4 | reversed part: 3 -> 2 -> 1
4 | prev=4 curr=5 | reversed part: 4 -> 3 -> 2 -> 1
5 | prev=5 curr=null | reversed part: 5 -> 4 -> 3 -> 2 -> 1
reversed: 5 -> 4 -> 3 -> 2 -> 1
recursive: 5 -> 4 -> 3 -> 2 -> 1
empty: (empty)
between 2..4: 1 -> 4 -> 3 -> 2 -> 5
k=2: 2 -> 1 -> 4 -> 3 -> 5
k=3: 3 -> 2 -> 1 -> 4 -> 5
-- trace of merge --
A: 1 -> 3 -> 5   B: 2 -> 4 -> 6 -> 8
1 | a=3 b=2 | take A | out so far: [1]
2 | a=3 b=4 | take B | out so far: [1, 2]
3 | a=5 b=4 | take A | out so far: [1, 2, 3]
4 | a=5 b=6 | take B | out so far: [1, 2, 3, 4]
5 | a=null b=6 | take A | out so far: [1, 2, 3, 4, 5]
end | append the rest of B | out: 1 -> 2 -> 3 -> 4 -> 5 -> 6 -> 8
result: 1 -> 2 -> 3 -> 4 -> 5 -> 6 -> 8
remove 6: 1 -> 2
```

What to notice

- In `reverseBetween`, `before` is advanced to the node preceding the segment; then the node following `curr` is repeatedly moved to the front of the segment. `curr` does not move and ends as the segment tail. This head-insertion form requires a single pass and, because of the dummy node, no special case for `left` equal to 1.
- In `reverseKGroup`, the existence of a full group is first verified by advancing `k` steps. If fewer than `k` nodes remain, the traversal terminates and the remainder is left unchanged, as the problem requires. The reversal is initialised with `prev = groupNext`, so the reversed group is already linked to the remainder of the list.
- In the recursive reversal, `head.next.next = head` flips the arrow after the recursive call returns, and `head.next = null` stops the old head from creating a cycle. Recursion uses O(n) stack space; on a list of 100,000 nodes it can throw `StackOverflowError`, so prefer the iterative version in interviews unless asked.
- `mergeTwo` uses `<=` so that equal values keep the order of A before B; this makes the merge **stable**.

3.1.4 Complexity and mistakes

Reversal, merge and the k-group version each visit every node a constant number of times: **O(n) time, O(1) extra space** (the recursive reversal is O(n) space for the call stack). `reverseBetween` is O(right) time. Merging lists of lengths m and n is O(m + n), and it reuses the existing nodes, so no new nodes are allocated.

Mistakes people make

- **Overwriting `curr.next` before saving it.** The rest of the list becomes unreachable. Always save `next` first.
- **Returning `curr` instead of `prev`.** After the loop `curr` is `null`; the new head is `prev`.
- **Forgetting the dummy node when the head can change**, then patching the head with extra `if` statements that miss a case (for example `left == 1` in LeetCode 92).
- **Leaving a stale `next` on the last node.** When a new list is built from existing nodes, the last node may still reference the old list, producing a cycle or duplicated nodes. `tail.next = null` must be set, or the remainder appended explicitly.
- **Not handling `null` and single-node lists.** Test with an empty list, one node and two nodes before anything else.

GOTCHA

Comparing node values with `==` on `Integer` fields. If the node stores `Integer val` instead of `int val`, the expression `a.val == b.val` compares references and fails for values outside -128..127. Use `int`, or `.equals`. The operator `==` on two `ListNode` references is the correct test for node identity (cycles, intersections) but never for equality of stored data.

3.1.5 Questions to expect

1. **Reverse a singly linked list, iteratively and recursively.** Clue: "in place". Three pointers; recursion reverses the tail first, then hooks the head onto the end.
2. **Reverse the nodes between positions left and right.** Clue: "a part of the list". Dummy node, walk to the node before the segment, head-insert `right - left` times.
3. **Reverse nodes in groups of k; leftover nodes stay as they are.** Clue: "every k". Count k ahead, reverse a group, reconnect, repeat. Follow-up: "do it in O(1) space" (no recursion).
4. **Merge two sorted lists. Merge k sorted lists.** Clue: "sorted", "merge". Dummy plus tail for two; a `PriorityQueue` of list heads (or pairwise merging) for k, costing O(N log k).
5. **Swap every two adjacent nodes without changing values.** Clue: "nodes, not values". It is k-group with k = 2; the dummy node handles the head.
6. **Add two numbers stored as reversed digit lists.** Clue: "digits in a list". Walk both lists with a carry; the dummy node builds the answer.
7. **Reorder `L0 -> L1 -> ... -> Ln` into `L0 -> Ln -> L1 -> Ln-1 ...`.** Clue: "reorder in place". Find the middle (next chapter), reverse the second half, interleave.
8. **Deep-copy a list where each node also has a random pointer.** Clue: "copy". A hash map from old node to new node, or interleave copies in the original list for O(1) space.

INTERVIEW ANGLE

Reversal is the standard opening question; the follow-ups determine the outcome. **Interviewer:** Reverse iteratively. **Candidate:** Three pointers, one pass, O(n) time and O(1) space; `next` is saved first so that the remainder is never lost. **Interviewer:** Reverse recursively. **Candidate:** Reverse the remainder, then set `head.next.next = head` and `head.next = null`. The stack space is O(n), so the iterative form is preferable for long lists. **Interviewer:** What changes for a doubly linked list? **Candidate:** Each node additionally has `prev`; reversal swaps `prev` and `next` in every node and returns the former tail.

3.1.6 LeetCode practice

#	Title	Level	Key idea
206	Reverse Linked List	Easy	The three-pointer loop; also try the recursive form.
21	Merge Two Sorted Lists	Easy	Dummy node plus tail; append the non-empty remainder.
83	Remove Duplicates from Sorted List	Easy	Skip `curr.next` while it equals `curr`; head never changes.
92	Reverse Linked List II	Medium	Walk to the node before the segment, head-insert repeatedly.
24	Swap Nodes in Pairs	Medium	Reverse in groups of two with a dummy node.
2	Add Two Numbers	Medium	Digit-by-digit addition with a carry and a dummy head.
143	Reorder List	Medium	Middle, reverse second half, interleave.
138	Copy List with Random Pointer	Medium	Map old nodes to copies, then wire `next` and `random`.
25	Reverse Nodes in k-Group	Hard	Check k nodes exist, reverse the group, reconnect.
23	Merge k Sorted Lists	Hard	Min-heap of the k current heads, O(N log k).

LeetCode problems for this pattern.

sreeram

KEY POINTS

- Reversal is a loop of four statements: save `next`, flip `curr.next`, advance `prev`, advance `curr`. Return `prev`.
- A dummy node removes the "head changes" special case; return `dummy.next`.
- Build new lists with a `tail` pointer and attach the leftover tail explicitly.
- Always test empty, one-node and two-node lists; draw the arrows before coding.
- Linked list problems reduce to O(n) time and O(1) auxiliary space when nodes are rewired instead of values being copied.

3.2 Fast and slow pointers

Several linked list questions appear to require extra memory or a second pass: whether the list contains a cycle, where the middle is, which node is the third from the end. The direct solutions use a `HashSet` of visited nodes (O(n) space) or a first pass to determine the length followed by a second pass to the answer. Both are correct, but the usual follow-up requires one pass and O(1) space.

Fast and slow pointers (also called the tortoise and the hare, or Floyd's algorithm for the cycle case) answer that follow-up. Two references move through the structure at different speeds, and their relative positions reveal the answer.

3.2.1 The idea

Start both references at the head. In every step `slow` moves one node and `fast` moves two. Three facts do all the work.

1. **Middle.** When `fast` reaches the end, `slow` has covered half the distance, so `slow` is at the middle.
2. **Cycle.** If the list has a cycle, `fast` enters it and eventually laps `slow`. Inside a cycle, the gap between them shrinks by exactly one each step, so they must land on the same node. If there is no cycle, `fast` hits `null`.
3. **Fixed gap.** If `fast` starts n nodes ahead and both then move one node at a time, when `fast` reaches the end `slow` is exactly n nodes from the end.

How to recognise it

- "Detect a cycle", "find where the cycle begins", "is there a loop?"
- "Find the middle" or "split the list in half" (merge sort on lists, palindrome check, reorder).
- "The n-th node from the end" in one pass.
- "Two lists that may share a tail" (the intersection problem).
- A value sequence defined by `x -> f(x)` that might repeat (happy number), or an array where `i -> nums[i]` behaves like a list (find the duplicate).

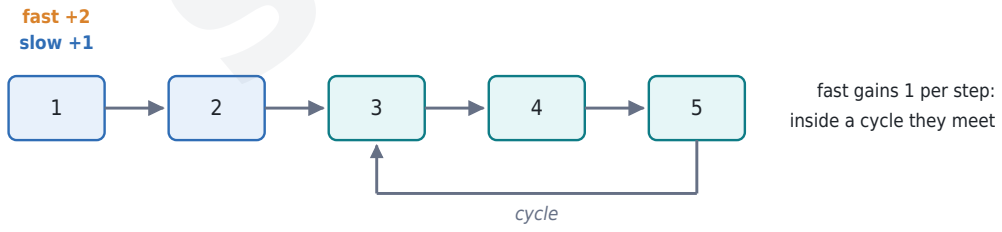


Figure 3.2. Two pointers at different speeds: fast gains one node per step on slow, so inside a cycle they must meet.

Template: find the middle

Middle.java — JAVA

```java
ListNode slow = head, fast = head;
while (fast != null && fast.next != null) {
    slow = slow.next;
    fast = fast.next.next;
}
return slow;   // second middle if the length is even
```

The loop condition tests `fast` first and then `fast.next`, because `fast.next.next` is well defined only when both are non-null (the precondition of the loop body). Reversing the two tests causes a `NullPointerException`.

3.2.2 Dry run

Cycle detection. The list is `3 -> 2 -> 0 -> -4`, and the last node points back to node 2. All values are distinct, so a value names a node. The cycle begins at node 2. Let F be the number of nodes before the cycle (here 1, the node 3) and C the cycle length (here 3).

Step	slow	fast	Note
1	2	0	both moved; not equal
2	0	2	fast wrapped around the cycle
3	-4	-4	slow == fast: a cycle exists

Phase 1: slow moves 1, fast moves 2.

Phase 2 finds the entry point. Keep `slow` where it met `fast` (node -4), put a new pointer `p` back at the head (node 3), and move both one node at a time until they meet.

Step	p	slow	Note
start	3	-4	p at head, slow at the meeting node
1	2	2	p == slow: this node is where the cycle starts

Phase 2: p restarts at the head, slow stays at the meeting point.

Correctness of phase 2. At the meeting point, slow has traversed F + a steps (a is the distance from the cycle start to the meeting point) and fast has traversed twice as many. Fast has also traversed F + a plus a whole number of cycle laps, hence F + a = k times C for some integer k. Therefore F = k times C minus a: F steps from the meeting point complete k - 1 laps and arrive at the cycle start. A pointer starting at the head reaches the cycle start after F steps, so both pointers arrive simultaneously. In the example, F = 1, and one step from node -4 along the cycle reaches node 2.

3.2.3 Java code

The program implements cycle detection and entry, middle, palindrome check (find the first middle, reverse the second half, compare, then restore), happy number, removing the n-th node from the end, the intersection of two lists, and the "find the duplicate" array trick. The trace in `detectCycle` produced the tables above.

Main.java JAVA

```java
import java.util.*;

public class Main {
    static class ListNode {
        int val;
        ListNode next;
        ListNode(int val) { this.val = val; }
    }

    static ListNode build(int... vals) {
        ListNode dummy = new ListNode(0), tail = dummy;
        for (int v : vals) { tail.next = new ListNode(v); tail = tail.next; }
        return dummy.next;
    }

    static ListNode nodeAt(ListNode head, int index) {
        while (index-- > 0) head = head.next;
        return head;
    }

    static String show(ListNode head) {
        StringBuilder sb = new StringBuilder();
        for (ListNode n = head; n != null; n = n.next) {
            if (sb.length() > 0) sb.append(" -> ");
            sb.append(n.val);
        }
        return sb.length() == 0 ? "(empty)" : sb.toString();
    }

    static boolean hasCycle(ListNode head) {
```

Main.java (continued) JAVA

```
        ListNode slow = head, fast = head;
        while (fast != null && fast.next != null) {
            slow = slow.next;
            fast = fast.next.next;
            if (slow == fast) return true;
        }
        return false;
    }

    // Returns the node where the cycle starts, or null. Prints a trace if asked.
    static ListNode detectCycle(ListNode head, boolean trace) {
        ListNode slow = head, fast = head;
        int step = 0;
        while (fast != null && fast.next != null) {
            slow = slow.next;
            fast = fast.next.next;
            if (trace) System.out.println(++step + " | slow=" + slow.val + " fast=" + fast.val);
            if (slow == fast) {
                ListNode p = head;                // phase 2: restart one pointer from head
                int walk = 0;
                while (p != slow) {
                    p = p.next; slow = slow.next;
                    walk++;
                    if (trace)
                        System.out.println("p" + walk + " | p=" + p.val + " slow=" + slow.val);
                }
                return p;
            }
        }
        return null;
    }

    static ListNode middle(ListNode head) {        // second middle when length is even
        ListNode slow = head, fast = head;
        while (fast != null && fast.next != null) {
            slow = slow.next;
            fast = fast.next.next;
        }
        return slow;
    }

    static ListNode reverse(ListNode head) {
        ListNode prev = null;
        while (head != null) {
            ListNode nxt = head.next;
            head.next = prev;
            prev = head;
            head = nxt;
        }
        return prev;
    }

    static boolean isPalindrome(ListNode head) {
        if (head == null || head.next == null) return true;
        ListNode slow = head, fast = head;
        while (fast.next != null && fast.next.next != null) {   // first middle
            slow = slow.next;
            fast = fast.next.next;
        }
        ListNode second = reverse(slow.next);
        ListNode a = head, b = second;
        boolean ok = true;
        while (b != null) {
            if (a.val != b.val) { ok = false; break; }
            a = a.next; b = b.next;
        }
        slow.next = reverse(second);                            // restore the list
        return ok;
    }

    static int digitSquareSum(int n) {
        int sum = 0;
        while (n > 0) { int d = n % 10; sum += d * d; n /= 10; }
        return sum;
    }
```

sreeram

Main.java (continued) JAVA

```
    static boolean isHappy(int n) {
        int slow = n, fast = digitSquareSum(n);
        while (fast != 1 && slow != fast) {
            slow = digitSquareSum(slow);
            fast = digitSquareSum(digitSquareSum(fast));
        }
        return fast == 1;
    }

    static ListNode removeNthFromEnd(ListNode head, int n) {
        ListNode dummy = new ListNode(0);
        dummy.next = head;
        ListNode lead = dummy, lag = dummy;
        for (int i = 0; i < n; i++) lead = lead.next;           // open a gap of n nodes
        while (lead.next != null) { lead = lead.next; lag = lag.next; }
        lag.next = lag.next.next;                               // lag sits just before the target
        return dummy.next;
    }

    static ListNode getIntersection(ListNode a, ListNode b) {
        ListNode p = a, q = b;
        while (p != q) {
            p = (p == null) ? b : p.next;
            q = (q == null) ? a : q.next;
        }
        return p;                                               // meeting node, or null
    }

    // LeetCode 287: array as a linked list i -> nums[i]
    static int findDuplicate(int[] nums) {
        int slow = nums[0], fast = nums[0];
        do { slow = nums[slow]; fast = nums[nums[fast]]; } while (slow != fast);
        slow = nums[0];
        while (slow != fast) { slow = nums[slow]; fast = nums[fast]; }
        return slow;
    }

    public static void main(String[] args) {
        // 3 -> 2 -> 0 -> -4 -> back to node with value 2
        ListNode head = build(3, 2, 0, -4);
        nodeAt(head, 3).next = nodeAt(head, 1);
        System.out.println("List: 3 -> 2 -> 0 -> -4 -> (back to 2)");
        System.out.println("hasCycle: " + hasCycle(head));
        ListNode start = detectCycle(head, true);
        System.out.println("cycle starts at value " + start.val);
        System.out.println("no cycle: " + hasCycle(build(1, 2, 3)));

        System.out.println("middle of 1..5: " + middle(build(1, 2, 3, 4, 5)).val);
        System.out.println("middle of 1..6: " + middle(build(1, 2, 3, 4, 5, 6)).val);
        System.out.println("palindrome 1,2,2,1: " + isPalindrome(build(1, 2, 2, 1)));
        System.out.println("palindrome 1,2,3,2,1: " + isPalindrome(build(1, 2, 3, 2, 1)));
        System.out.println("palindrome 1,2,3: " + isPalindrome(build(1, 2, 3)));
        System.out.println("happy 19: " + isHappy(19) + ", happy 2: " + isHappy(2));
        System.out.println("remove 2nd from end of 1..5: "
            + show(removeNthFromEnd(build(1, 2, 3, 4, 5), 2)));
        System.out.println("remove 5th from end of 1..5: "
            + show(removeNthFromEnd(build(1, 2, 3, 4, 5), 5)));

        ListNode shared = build(8, 4, 5);
        ListNode a = build(4, 1), b = build(5, 6, 1);
        a.next.next = shared;
        b.next.next.next = shared;
        System.out.println("intersection value: " + getIntersection(a, b).val);
        System.out.println("disjoint: " + getIntersection(build(1, 2), build(3)));
        System.out.println("duplicate in [1,3,4,2,2]: " + findDuplicate(new int[]{1, 3, 4, 2, 2}));
        System.out.println("duplicate in [3,1,3,4,2]: " + findDuplicate(new int[]{3, 1, 3, 4, 2}));
    }
}
```

Compile and run TERMINAL

```
$ java Main.java
List: 3 -> 2 -> 0 -> -4 -> (back to 2)
hasCycle: true
1 | slow=2 fast=0
2 | slow=0 fast=2
3 | slow=-4 fast=-4
p1 | p=2 slow=2
cycle starts at value 2
no cycle: false
middle of 1..5: 3
middle of 1..6: 4
palindrome 1,2,2,1: true
palindrome 1,2,3,2,1: true
palindrome 1,2,3: false
happy 19: true, happy 2: false
remove 2nd from end of 1..5: 1 -> 2 -> 3 -> 5
remove 5th from end of 1..5: 2 -> 3 -> 4 -> 5
intersection value: 8
disjoint: null
duplicate in [1,3,4,2,2]: 2
duplicate in [3,1,3,4,2]: 3
```

What to notice

- `isPalindrome` uses `fast.next != null && fast.next.next != null` so that `slow` stops at the **first** middle in even-length lists; `slow.next` then begins the second half. The final `reverse` restores the list, preserving the input for the caller.
- `isHappy` has no list at all. The sequence "sum of squares of digits" either reaches 1 or enters a cycle, and a function `f(x)` plays the role of `node.next`. Fast applies `f` twice per step.
- `removeNthFromEnd` opens a gap of `n` nodes between `lead` and `lag`. The dummy node makes removing the head (n equal to the length) work with no special case, as the second test line shows.
- `getIntersection` makes each pointer walk its own list and then the other one. Both walk lengths `m + n`, so they meet at the intersection node, or both reach `null` together when the lists are disjoint.
- `findDuplicate` treats `i -> nums[i]` as a linked list. For an array of n + 1 values in `1..n`, index 0 is never a target, so it serves as the head, and a duplicate value corresponds to two indices referencing the same node: the cycle entry.

3.2.4 Complexity and mistakes

Every algorithm here is **O(n) time and O(1) space**. For the cycle phase 1, slow enters the cycle after F steps and fast closes the gap of at most C nodes at one node per step, so the total is at most F + C steps, which is O(n). Compare with the hash-set solution: O(n) time but O(n) space. The two-pass count-then-walk solution to "n-th from the end" is also O(n) time; the one-pass version is better mainly because it works on a stream and shows control of pointers.

Mistakes people make

- **Wrong loop condition.** Use `while (fast != null && fast.next != null)`. Checking only `fast.next` crashes on empty lists; checking `fast.next.next` first crashes on odd lengths.
- **Comparing values instead of nodes.** In a list with repeated values `slow.val == fast.val` is not a cycle. Compare references: `slow == fast`.
- **Starting `fast` one ahead of `slow` and then checking equality before moving.** Both start at the head; check equality after moving.
- **Off-by-one in the gap.** For "n-th from the end", the gap is n nodes with a dummy start. Starting both at `head` instead of `dummy` makes removing the head impossible.
- **Wrong middle for even lengths.** The first or the second middle must be chosen explicitly; the two loop conditions above yield different results. The choice should be stated.

> **TIP**
> When a pointer problem is unclear, draw five nodes and execute the loop manually for one odd and one even length. If the trace and the code disagree, the defect lies in the loop condition rather than in the idea.

3.2.5 Questions to expect

1. **Does the linked list have a cycle? Do it in O(1) space.** Clue: "loop", "cycle". Floyd: slow one step, fast two; they meet iff there is a cycle.
2. **Return the node where the cycle begins.** Clue: "start of the cycle". After the meeting, restart one pointer at the head and advance both by one until they meet.
3. **How long is the cycle?** Follow-up. After meeting, walk one pointer around until it returns to the meeting node and count steps.
4. **Find the middle node.** Clue: "middle", "split". State which middle is returned for even lengths.
5. **Is the list a palindrome in O(1) space?** Clue: "palindrome" plus "O(1) space". Middle, reverse the second half, compare, optionally restore.
6. **Remove the n-th node from the end in one pass.** Clue: "from the end". Gap of n between two pointers plus a dummy node.
7. **Where do two singly linked lists intersect?** Clue: "share nodes". Switch heads at the end of each list, or compute the length difference.
8. **Find the repeated number in an array of n + 1 numbers in `1..n` without modifying the array and with O(1) space.** Clue: "no modification" excludes sorting. Treat the array as a function and use cycle detection.

INTERVIEW ANGLE

"Why must the fast and slow pointers meet inside a cycle rather than pass each other?" Inside the cycle, each step advances `fast` by two nodes and `slow` by one, so the distance from `slow` to `fast` (along the direction of travel) changes by exactly one per step. A change of one cannot skip the value zero, so the pointers meet. With speeds one and three the gap changes by two per step and, in a cycle of even length, may never become zero. For this reason the speeds one and two are used.

3.2.6 LeetCode practice

#	Title	Level	Key idea
141	Linked List Cycle	Easy	Floyd: slow one step, fast two; meeting means a cycle.
876	Middle of the Linked List	Easy	Return `slow` when `fast` runs out; second middle for even length.
202	Happy Number	Easy	The digit-square function is the "next" pointer; detect a cycle.
160	Intersection of Two Linked Lists	Easy	Each pointer walks A then B; they meet at the intersection or at null.
234	Palindrome Linked List	Easy	First middle, reverse second half, compare.
19	Remove Nth Node From End of List	Medium	Dummy node and a gap of n between two pointers.
142	Linked List Cycle II	Medium	After meeting, restart at the head and move both one step.
287	Find the Duplicate Number	Medium	Array index to value is a list; the duplicate is the cycle entry.
143	Reorder List	Medium	Middle, reverse the second half, then interleave the halves.

LeetCode problems for this pattern.

KEY POINTS

- `slow` moves 1, `fast` moves 2: at the end of the list `slow` is at the middle; inside a cycle they meet.
- To find the cycle entry, restart one pointer at the head and move both one step at a time.
- A fixed gap between two pointers gives "n-th from the end" in one pass; add a dummy node for head removal.
- Any sequence defined by `x -> f(x)` (digits, array indices) is a linked list in disguise.
- Loop condition: `fast != null && fast.next != null`, in that order. Compare node references, not values.

3.3 Stack patterns

A **stack** is a last-in, first-out (LIFO) container: the most recently added element is the first to be removed. Stacks arise whenever a problem has nested structure (brackets, function calls, undo history, directory paths containing `..`) or requires the most recent unfinished context to be restored.

The brute-force approach to such problems is repeated rescanning: for brackets, the pairs `()`, `[]` and `{}` are deleted repeatedly until no change occurs, which requires O(n^2) time. A stack decides each character in O(1) by inspecting only the most recent unresolved element.

3.3.1 The idea

Push when something opens or is still undecided; pop when something closes or gets decided. The top of the stack is always "the innermost thing that is still open".

How to recognise it

- Matching or nesting: brackets, tags, quotes, `[` and `]` in encoded strings.
- "Undo" semantics: backspace characters, `..` in a path, cancelling adjacent pairs.
- Postfix or prefix expressions, calculators, anything where operands wait for their operator.
- "Most recent" or "previous unresolved" element: min stack, the previous bigger element (next chapter).
- A recursive solution that must be executed without recursion (iterative DFS, expression parsing).

Which class to use in Java

Use `Deque<T> stack = new ArrayDeque<>()` with `push`, `pop` and `peek`. Do not use `java.util.Stack`: it extends `Vector`, so every method is synchronised (slower), it allows index access that breaks the stack abstraction, and the Javadoc itself recommends `Deque`. `ArrayDeque` does not allow `null` elements; `pop()` on an empty deque throws `NoSuchElementException`, while `peek()` returns `null`. In an `ArrayDeque` used as a stack, iteration goes from the top down.

Template: match and cancel

StackTemplate.java — JAVA

```java
Deque<Character> stack = new ArrayDeque<>();
for (char c : s.toCharArray()) {
    if (isOpener(c)) {
        stack.push(c);                      // remember what must be closed
    } else {
        if (stack.isEmpty() || !matches(stack.pop(), c)) return false;
    }
}
return stack.isEmpty();                     // anything left over is unclosed
```

3.3.2 Dry run

The string `3[a2[c]]` is decoded (an encoded string: `k[text]` denotes text repeated k times, and brackets may nest). The expected result is `accaccacc`. Two stacks hold the work in progress: `counts` holds the multiplier pending for each open bracket and `prefixes` holds the text built before that bracket. `cur` is the text currently being built and `k` the number being read. Each stack is shown with its **top first**; `""` is an empty string (the program prints it as nothing, so a stack holding one empty string appears as `[]` in the raw output).

Step	Char	cur and k	counts	prefixes (top first)
1	3	cur="" k=3	[]	[]
2	[	cur="" k=0	[3]	[""]
3	a	cur="a" k=0	[3]	[""]
4	2	cur="a" k=2	[3]	[""]
5	[	cur="" k=0	[2, 3]	["a", ""]
6	c	cur="c" k=0	[2, 3]	["a", ""]
7	]	cur="acc" k=0	[3]	[""]

Step	Char	cur and k	counts	prefixes (top first)
8	]	cur="accaccacc" k=0	[]	[]

Decoding 3[a2[c]] character by character (state after each step).

At step 2 the first `[` stores the multiplier 3 and the (empty) text built so far, then starts a fresh `cur`. At step 5 the second `[` stores the multiplier 2 and the text `a`. At step 7 the first `]` takes `cur="c"`, repeats it twice and appends the result to the stored prefix `a`, giving `acc`. At step 8 the last `]` repeats `acc` three times onto the empty prefix. The invariant is: **the stacks contain exactly the unclosed brackets, innermost on top.**

3.3.3 Java code

The program solves valid parentheses, a min stack, evaluation of reverse Polish notation, the decode-string problem above (with its trace), backspace string compare, simplification of a Unix path, and an iterative tree preorder as an instance of replacing recursion by an explicit stack. Reverse Polish notation (RPN) writes `2 1 + 3 *` for `(2 + 1) * 3`: operands precede the operator.

Main.java JAVA

```java
import java.util.*;

public class Main {
    static boolean isValid(String s) {
        Deque<Character> stack = new ArrayDeque<>();
        for (char c : s.toCharArray()) {
            switch (c) {
                case '(' -> stack.push(')');
                case '[' -> stack.push(']');
                case '{' -> stack.push('}');
                default -> {
                    if (stack.isEmpty() || stack.pop() != c) return false;
                }
            }
        }
        return stack.isEmpty();
    }

    static class MinStack {
        private final Deque<int[]> stack = new ArrayDeque<>();   // {value, min so far}
        void push(int x) {
            int min = stack.isEmpty() ? x : Math.min(x, stack.peek()[1]);
            stack.push(new int[]{x, min});
        }
        void pop() { stack.pop(); }
        int top() { return stack.peek()[0]; }
        int getMin() { return stack.peek()[1]; }
    }

    static int evalRPN(String[] tokens) {
        Deque<Integer> st = new ArrayDeque<>();
        for (String t : tokens) {
            switch (t) {
                case "+" -> st.push(st.pop() + st.pop());
                case "*" -> st.push(st.pop() * st.pop());
                case "-" -> { int b = st.pop(), a = st.pop(); st.push(a - b); }
                case "/" -> { int b = st.pop(), a = st.pop(); st.push(a / b); }
                default -> st.push(Integer.parseInt(t));
            }
        }
        return st.pop();
    }

    static String decodeString(String s, boolean trace) {
        Deque<Integer> counts = new ArrayDeque<>();
        Deque<StringBuilder> prefixes = new ArrayDeque<>();
        StringBuilder cur = new StringBuilder();
        int k = 0, step = 0;
        for (char c : s.toCharArray()) {
            String action;
            if (Character.isDigit(c)) {
                k = k * 10 + (c - '0');
                action = "build number";
            } else if (c == '[') {
```

sreeram

Main.java (continued) JAVA

```
                counts.push(k); prefixes.push(cur);
                cur = new StringBuilder(); k = 0;
                action = "push count and text, reset";
            } else if (c == ']') {
                int times = counts.pop();
                StringBuilder prev = prefixes.pop();
                prev.append(String.valueOf(cur).repeat(times));
                cur = prev;
                action = "pop, repeat " + times + " times";
            } else {
                cur.append(c);
                action = "append letter";
            }
            if (trace) System.out.println(++step + " | '" + c + "' | cur=\"" + cur + "\" k=" + k
                + " counts=" + counts + " prefixes=" + prefixes + " | " + action);
        }
        return cur.toString();
    }

    static boolean backspaceCompare(String s, String t) {
        return build(s).equals(build(t));
    }
    private static String build(String s) {
        StringBuilder sb = new StringBuilder();
        for (char c : s.toCharArray()) {
            if (c != '#') sb.append(c);
            else if (sb.length() > 0) sb.setLength(sb.length() - 1);
        }
        return sb.toString();
    }

    static String simplifyPath(String path) {
        Deque<String> stack = new ArrayDeque<>();
        for (String part : path.split("/")) {
            if (part.isEmpty() || part.equals(".")) continue;
            if (part.equals("..")) { if (!stack.isEmpty()) stack.pop(); }
            else stack.push(part);
        }
        if (stack.isEmpty()) return "/";
        StringBuilder sb = new StringBuilder();
        Iterator<String> it = stack.descendingIterator();   // bottom of the stack first
        while (it.hasNext()) sb.append('/').append(it.next());
        return sb.toString();
    }

    // Explicit stack imitating recursion: iterative DFS preorder on a tiny tree
    record Node(int val, Node left, Node right) {}
    static List<Integer> preorder(Node root) {
        List<Integer> out = new ArrayList<>();
        Deque<Node> stack = new ArrayDeque<>();
        if (root != null) stack.push(root);
        while (!stack.isEmpty()) {
            Node n = stack.pop();
            out.add(n.val());
            if (n.right() != null) stack.push(n.right());
            if (n.left() != null) stack.push(n.left());
        }
        return out;
    }

    public static void main(String[] args) {
        for (String s : new String[]{"()[]{}", "([{}])", "(]", "((", "]"})
            System.out.println("isValid(\"" + s + "\") = " + isValid(s));

        MinStack ms = new MinStack();
        ms.push(5); ms.push(3); ms.push(7); ms.push(3);
        System.out.println("min=" + ms.getMin() + " top=" + ms.top());
        ms.pop();
        System.out.println("after pop: min=" + ms.getMin() + " top=" + ms.top());
        ms.pop(); ms.pop();
        System.out.println("after two more pops: min=" + ms.getMin() + " top=" + ms.top());

        System.out.println("RPN 2 1 + 3 * = " + evalRPN(new String[]{"2", "1", "+", "3", "*"}));
        System.out.println("RPN 4 13 5 / + = " + evalRPN(new String[]{"4", "13", "5", "/", "+"}));
        System.out.println("RPN 3 4 - = " + evalRPN(new String[]{"3", "4", "-"}));
```

freeram

Main.java (continued) JAVA

```
        System.out.println("-- decode 3[a2[c]] --");
        System.out.println("result: " + decodeString("3[a2[c]]", true));
        System.out.println("decode 2[ab]3[c]: " + decodeString("2[ab]3[c]", false));
        System.out.println("decode 10[z]: " + decodeString("10[z]", false));

        System.out.println("backspace ab#c vs ad#c: " + backspaceCompare("ab#c", "ad#c"));
        System.out.println("backspace a## vs c#d#: " + backspaceCompare("a##", "c#d#"));
        System.out.println("backspace a#c vs b: " + backspaceCompare("a#c", "b"));

        System.out.println(simplifyPath("/home//foo/"));
        System.out.println(simplifyPath("/a/./b/../../c/"));
        System.out.println(simplifyPath("/../"));

        Node tree = new Node(1,
            new Node(2, new Node(4, null, null), null),
            new Node(3, null, null));
        System.out.println("iterative preorder: " + preorder(tree));
    }
}
```

Compile and run TERMINAL

```
$ java Main.java
isValid("()[]{}") = true
isValid("([{}])") = true
isValid("(]") = false
isValid("((") = false
isValid("]") = false
min=3 top=3
after pop: min=3 top=7
after two more pops: min=5 top=5
RPN 2 1 + 3 * = 9
RPN 4 13 5 / + = 6
RPN 3 4 - = -1
-- decode 3[a2[c]] --
1 | '3' | cur="" k=3 counts=[] prefixes=[] | build number
2 | '[' | cur="" k=0 counts=[3] prefixes=[] | push count and text, reset
3 | 'a' | cur="a" k=0 counts=[3] prefixes=[] | append letter
4 | '2' | cur="a" k=2 counts=[3] prefixes=[] | build number
5 | '[' | cur="" k=0 counts=[2, 3] prefixes=[a, ] | push count and text, reset
6 | 'c' | cur="c" k=0 counts=[2, 3] prefixes=[a, ] | append letter
7 | ']' | cur="acc" k=0 counts=[3] prefixes=[] | pop, repeat 2 times
8 | ']' | cur="accaccacc" k=0 counts=[] prefixes=[] | pop, repeat 3 times
result: accaccacc
decode 2[ab]3[c]: ababccc
decode 10[z]: zzzzzzzzzz
backspace ab#c vs ad#c: true
backspace a## vs c#d#: true
backspace a#c vs b: false
/home/foo
/c
/
iterative preorder: [1, 2, 4, 3]
```

What to notice

- `isValid` pushes the **expected closing** character instead of the opener, so the check on a closer is just `stack.pop() != c`. It removes the need for a lookup map.
- `MinStack` stores `{value, minimum so far}` in each entry. Popping removes both, so the minimum always reflects the current contents. The alternative, a second stack of minimums, is the same idea.
- In `evalRPN`, for `-` and `/` the **second** popped value is the left operand, so `b` is popped first, then `a`. Getting that backwards turns `3 4 -` into `1` instead of `-1`. Division uses `int` division, which truncates toward zero as the problem requires.
- `decodeString` uses `String.valueOf(cur).repeat(times)` (`String.repeat`, Java 11) and builds numbers digit by digit, so `10[z]` works.
- `simplifyPath` splits on `/`. Empty parts come from `//` or a trailing slash, and `.` is ignored. Because `ArrayDeque.push` adds at the front, `descendingIterator()` is used to print from the bottom.

- `backspaceCompare` here uses `StringBuilder` as a stack (`setLength` to pop). An O(1)-space version walks both strings from the end and counts pending `#` characters; it is a typical follow-up.

3.3.4 Complexity and mistakes

Each character is pushed at most once and popped at most once, so every solution is **O(n) time**. Space is **O(n)** in the worst case (all openers, or deeply nested brackets); the min stack doubles the memory but keeps every operation at O(1).

Mistakes people make

- **Popping an empty stack.** Check `isEmpty()` before `pop()` on a closer (the input `]` crashes otherwise).
- **Forgetting the final emptiness check.** `((` never fails inside the loop; the leftovers are the error.
- **Reversed operand order** for subtraction and division in RPN.
- **Using `Stack` or `peek()` results carelessly.** `peek()` returns `null` on an empty `ArrayDeque`; auto-unboxing that `null` into an `int` throws `NullPointerException`.
- **Comparing boxed values with `==`.** `stack.peek() == stack.peek()` on `Integer` works for small values and silently fails for large ones. Compare with `.equals` or unbox into an `int` first.

GOTCHA

`ArrayDeque` is both a stack and a queue, so `push`/`pop`/`peek` work at the **front** (head). Mixing `add`/`offer` (which insert at the tail) with `pop` (which removes from the head) uses the structure as a queue unintentionally. One end must be used consistently: `push`, `pop`, `peek` for a stack; `offer`, `poll`, `peek` for a queue.

3.3.5 Questions to expect

1. **Check whether a string of brackets is balanced.** Clue: "valid", "matching". Push expected closers; pop on each closer; stack must end empty. Follow-up: return the length of the longest valid substring (a stack of indices).
2. **Design a stack that returns the minimum in O(1).** Clue: "design", "constant time". Store the running minimum with each element. Follow-up: O(1) extra space using an encoded difference (subtle; it is mentioned here only, not implemented).
3. **Evaluate an arithmetic expression in postfix form.** Clue: "reverse Polish". Push numbers, on an operator pop two and push the result. Follow-up: convert infix to postfix with the shunting-yard algorithm.
4. **Decode `3[a2[c]]`.** Clue: "nested repetition". Two stacks (or recursion). Follow-up: what is the output length? It can be exponential in the input length.
5. **Compare two strings after typing with backspaces.** Clue: "#". Stack, or two pointers from the end for O(1) space.
6. **Simplify a Unix path.** Clue: "..", ".", "//". Split on `/`, stack of directory names.
7. **Remove all adjacent duplicates from a string.** Clue: "adjacent equal pair cancels". Stack of characters (or a `StringBuilder`), compare the top with the new character.
8. **Do an inorder or preorder traversal of a tree without recursion.** Clue: "iteratively". The explicit stack takes the place of the call stack.

INTERVIEW ANGLE

"Why does a stack solve bracket matching?" A closing bracket must match the **most recent** opening bracket that is still unclosed, and "most recent unclosed" is exactly the top of a stack. Every opener is pushed once and popped once, so the algorithm is O(n) time with O(n) worst-case space. If the interviewer adds several bracket types, the stack handles them without change; only the match check differs. If they add "return the position of the first error", store indices instead of characters.

3.3.6 LeetCode practice

#	Title	Level	Key idea
20	Valid Parentheses	Easy	Push the expected closer; pop and compare on each closer.
1047	Remove All Adjacent Duplicates In String	Easy	Stack of characters; pop when the new character equals the top.
844	Backspace String Compare	Easy	Build both strings with a stack, or compare from the end.
155	Min Stack	Medium	Store the minimum so far together with each value.
150	Evaluate Reverse Polish Notation	Medium	Push numbers; on an operator pop right then left operand.
71	Simplify Path	Medium	Split on `/`; push names, pop on `..`, skip `.` and empty parts.
394	Decode String	Medium	Stack of counts and previous text, or recursion.
735	Asteroid Collision	Medium	Stack of survivors; resolve collisions while the top moves right and the new one moves left.
402	Remove K Digits	Medium	Pop larger previous digits while you still have removals left.
32	Longest Valid Parentheses	Hard	Stack of indices with a base index, or two counting passes.

LeetCode problems for this pattern.

KEY POINTS

- Use a stack for nesting and "most recent unresolved" problems; each element is pushed once and popped once, so the cost is O(n).
- In Java use `Deque<T> stack = new ArrayDeque<>()` with `push`, `pop`, `peek`; avoid `java.util.Stack`.
- Always guard `pop` with `isEmpty()` and check the stack is empty at the end for matching problems.
- Storing extra information next to each element (running minimum, previous text, index) is the standard way to extend a stack.
- Anything done with recursion can be done with an explicit stack, which avoids `StackOverflowError`.

3.4 Monotonic stack

Many array problems ask, for every element, about its nearest neighbour that is bigger or smaller: the next warmer day, the next greater element, how far back the price was lower. The brute force looks to the right (or left) of every index until it finds the answer, which is O(n^2) when the array is sorted the wrong way. The **monotonic stack** answers all n questions in a single pass of O(n) by keeping candidates in order and discarding anything that can never be an answer again.

3.4.1 The idea

A monotonic stack is an ordinary stack with one additional invariant: the values it contains are sorted (strictly or non-strictly) from bottom to top. Before a new element is pushed, every element that would violate the order is popped. Each popped element has thereby found its answer: the element that forced its removal is its next greater (or smaller) neighbour.

For "next greater to the right" the stack holds **indices** whose values are decreasing from bottom to top. When a new value is larger than the value at the top, the top has found its answer; pop it, record the answer, and compare with the new top. Then push the new index.

Indices are stored rather than values because the answers usually require distances (`i - j`) or widths, and the value can always be recovered from the index.

How to recognise it

- "Next greater / smaller element", "previous greater / smaller element".
- "How many days until ...", "how far back can the range extend ...", "span".
- Rectangles or water bounded by taller bars on both sides (histogram, trapping rain water).
- "Remove digits to make the number smallest": a larger digit preceding a smaller one is removed.
- Circular arrays: walk the array twice.

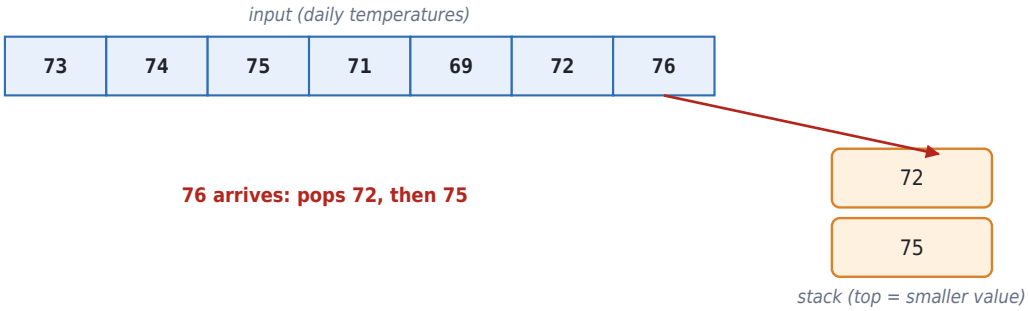


Figure 3.3. The stack keeps indices of values in decreasing order; a larger arrival pops smaller values, and each popped index has found its next greater element.

Template: next greater element to the right

NextGreater.java JAVA

```java
int[] res = new int[n];                     // distance, value or -1: depends on the problem
Deque<Integer> stack = new ArrayDeque<>();  // indices, values decreasing bottom to top
for (int i = 0; i < n; i++) {
    while (!stack.isEmpty() && a[stack.peek()] < a[i]) {
        int j = stack.pop();                // a[i] is the next greater of a[j]
        res[j] = i - j;
    }
    stack.push(i);
}
// indices still on the stack have no greater element to their right
```

For "next smaller" flip the comparison. For "previous greater" read the stack top **before** pushing (it is the nearest earlier larger element, if one survived).

3.4.2 Dry run

Input temperatures: `[73, 74, 75, 71, 69, 72, 76, 73]`. For each day, the number of days until a warmer temperature is required, or 0 if none exists. The stack holds day indices, shown top first.

Day i	t[i]	Pops (index and answer set)	Stack after push
0	73	none	[0]
1	74	pop 0, ans 1	[1]
2	75	pop 1, ans 1	[2]
3	71	none	[3, 2]
4	69	none	[4, 3, 2]
5	72	pop 4, ans 1; pop 3, ans 2	[5, 2]
6	76	pop 5, ans 1; pop 2, ans 4	[6]
7	73	none	[7, 6]

Daily temperatures, state after each day.

Final answer: `[1, 1, 4, 2, 1, 1, 0, 0]`. The indices 6 and 7 never got popped, so their answer stays 0. Notice day 5: the temperature 72 pops 69 and 71 but stops at 75 because 75 is not smaller. Because each index enters once and leaves once, the whole table costs at most 2n stack operations.

A second trace shows the same machinery computing areas. Largest rectangle in the histogram `[2, 1, 5, 6, 2, 3]`: when a bar is popped, its rectangle extends left to the new stack top (exclusive) and right to the current index (exclusive).

At i	Current bar	Popped height	Width	Area
1	1	2	1	2
4	2	6	1	6
4	2	5	2	10
6 (sentinel)	0	3	1	3
6 (sentinel)	0	2	4	8
6 (sentinel)	0	1	6	6

Largest rectangle in a histogram: every pop computes one candidate area.

The maximum area is 10, the two bars of heights 5 and 6 with height 5. At index 6 (past the last bar) the code pretends there is a bar of height 0 so that the stack is emptied and all remaining rectangles are measured.

3.4.3 Java code

The program contains daily temperatures with the trace and a brute-force version to compare, next greater element on a circular array, largest rectangle in a histogram, trapping rain water by stack and by two pointers, and the stock span class.

Main.java JAVA

```java
import java.util.*;

public class Main {
    // Days until a warmer temperature. Stack holds indices with decreasing temperatures.
    static int[] dailyTemperatures(int[] t, boolean trace) {
        int n = t.length;
        int[] ans = new int[n];
        Deque<Integer> stack = new ArrayDeque<>();
        for (int i = 0; i < n; i++) {
            StringBuilder popped = new StringBuilder();
            while (!stack.isEmpty() && t[stack.peek()] < t[i]) {
                int j = stack.pop();
                ans[j] = i - j;
                popped.append("pop ").append(j).append(" (ans=").append(i - j).append(") ");
            }
            stack.push(i);
            if (trace) System.out.println(i + " | t=" + t[i] + " | " + (popped.length() == 0
                ? "no pop " : popped) + "| stack (top first)=" + stack);
        }
        return ans;
    }

    // Next greater element in a circular array: walk the array twice
    static int[] nextGreaterCircular(int[] a) {
        int n = a.length;
        int[] res = new int[n];
        Arrays.fill(res, -1);
        Deque<Integer> st = new ArrayDeque<>();
        for (int i = 0; i < 2 * n; i++) {
            int x = a[i % n];
            while (!st.isEmpty() && a[st.peek()] < x) res[st.pop()] = x;
            if (i < n) st.push(i);
        }
        return res;
    }

    // Largest rectangle in a histogram; a sentinel height 0 at the end flushes the stack
    static int largestRectangle(int[] h, boolean trace) {
        Deque<Integer> st = new ArrayDeque<>();
        int best = 0;
        for (int i = 0; i <= h.length; i++) {
            int cur = (i == h.length) ? 0 : h[i];
            while (!st.isEmpty() && h[st.peek()] > cur) {
                int height = h[st.pop()];
                int left = st.isEmpty() ? -1 : st.peek();
                int width = i - left - 1;
```

Main.java (continued) JAVA

```
                int area = height * width;
                if (trace) System.out.println("i=" + i + " cur=" + cur + " | pop height " + height
                    + " width " + width + " area " + area);
                best = Math.max(best, area);
            }
            st.push(i);
        }
        return best;
    }

    static int trapStack(int[] h) {
        Deque<Integer> st = new ArrayDeque<>();
        int water = 0;
        for (int i = 0; i < h.length; i++) {
            while (!st.isEmpty() && h[st.peek()] < h[i]) {
                int bottom = h[st.pop()];
                if (st.isEmpty()) break;
                int left = st.peek();
                int width = i - left - 1;
                int bounded = Math.min(h[left], h[i]) - bottom;
                water += width * bounded;
            }
            st.push(i);
        }
        return water;
    }

    static int trapTwoPointers(int[] h) {
        int l = 0, r = h.length - 1, lmax = 0, rmax = 0, water = 0;
        while (l < r) {
            if (h[l] < h[r]) {
                lmax = Math.max(lmax, h[l]);
                water += lmax - h[l++];
            } else {
                rmax = Math.max(rmax, h[r]);
                water += rmax - h[r--];
            }
        }
        return water;
    }

    // LeetCode 901
    static class StockSpanner {
        private final Deque<int[]> st = new ArrayDeque<>();   // {price, span}
        int next(int price) {
            int span = 1;
            while (!st.isEmpty() && st.peek()[0] <= price) span += st.pop()[1];
            st.push(new int[]{price, span});
            return span;
        }
    }

    // Brute force for comparison: O(n^2)
    static int[] dailyBrute(int[] t) {
        int[] ans = new int[t.length];
        for (int i = 0; i < t.length; i++)
            for (int j = i + 1; j < t.length; j++)
                if (t[j] > t[i]) { ans[i] = j - i; break; }
        return ans;
    }

    public static void main(String[] args) {
        int[] temps = {73, 74, 75, 71, 69, 72, 76, 73};
        System.out.println("temps: " + Arrays.toString(temps));
        int[] res = dailyTemperatures(temps, true);
        System.out.println("answer: " + Arrays.toString(res));
        System.out.println("brute : " + Arrays.toString(dailyBrute(temps)));

        System.out.println("circular [1,2,1]: "
            + Arrays.toString(nextGreaterCircular(new int[]{1, 2, 1})));
        System.out.println("circular [5,4,3,2,1]: "
            + Arrays.toString(nextGreaterCircular(new int[]{5, 4, 3, 2, 1})));

        int[] hist = {2, 1, 5, 6, 2, 3};
        System.out.println("histogram: " + Arrays.toString(hist));
```

Sreeram

Main.java (continued) JAVA

```
        System.out.println("largest rectangle: " + largestRectangle(hist, true));

        int[] rain = {0, 1, 0, 2, 1, 0, 1, 3, 2, 1, 2, 1};
        System.out.println("rain stack: " + trapStack(rain)
            + ", two pointers: " + trapTwoPointers(rain));

        StockSpanner sp = new StockSpanner();
        int[] prices = {100, 80, 60, 70, 60, 75, 85};
        StringBuilder sb = new StringBuilder();
        for (int p : prices) sb.append(sp.next(p)).append(' ');
        System.out.println("spans for " + Arrays.toString(prices) + ": " + sb.toString().trim());
    }
}
```

Compile and run TERMINAL

```
$ java Main.java
temps: [73, 74, 75, 71, 69, 72, 76, 73]
0 | t=73 | no pop | stack (top first)=[0]
1 | t=74 | pop 0 (ans=1) | stack (top first)=[1]
2 | t=75 | pop 1 (ans=1) | stack (top first)=[2]
3 | t=71 | no pop | stack (top first)=[3, 2]
4 | t=69 | no pop | stack (top first)=[4, 3, 2]
5 | t=72 | pop 4 (ans=1) pop 3 (ans=2) | stack (top first)=[5, 2]
6 | t=76 | pop 5 (ans=1) pop 2 (ans=4) | stack (top first)=[6]
7 | t=73 | no pop | stack (top first)=[7, 6]
answer: [1, 1, 4, 2, 1, 1, 0, 0]
brute : [1, 1, 4, 2, 1, 1, 0, 0]
circular [1,2,1]: [2, -1, 2]
circular [5,4,3,2,1]: [-1, 5, 5, 5, 5]
histogram: [2, 1, 5, 6, 2, 3]
i=1 cur=1 | pop height 2 width 1 area 2
i=4 cur=2 | pop height 6 width 1 area 6
i=4 cur=2 | pop height 5 width 2 area 10
i=6 cur=0 | pop height 3 width 1 area 3
i=6 cur=0 | pop height 2 width 4 area 8
i=6 cur=0 | pop height 1 width 6 area 6
largest rectangle: 10
rain stack: 6, two pointers: 6
spans for [100, 80, 60, 70, 60, 75, 85]: 1 1 1 2 1 4 6
```

What to notice

- The loop condition `a[stack.peek()] < a[i]` is **strict**. With `<=`, equal temperatures would pop each other, changing the meaning from "strictly warmer" to "warmer or equal". In the stock span (price less than or equal) the comparison is `<=`, because a span includes equal prices. Strictness must be derived from the problem statement.
- In `nextGreaterCircular`, indices are pushed only during the first pass (`i < n`), while both passes may pop. That is how elements near the end find greater values at the start.
- In `largestRectangle`, after popping, the left boundary is the new top of the stack (or -1 when empty), and the width is `i - left - 1`. The sentinel at `i == h.length` flushes the stack.
- `StockSpanner` stores `{price, span}` and merges spans while popping, so every call costs O(1) amortised and no history array is needed.
- The trapping rain water stack solution fills water **layer by layer** between a bottom bar and the nearer walls. The two-pointer solution (see the chapter on two pointers in the arrays Part) reaches the same total with O(1) space; show both if asked.

3.4.4 Complexity and mistakes

Time O(n) for all of them, even though there is a `while` inside a `for`: each index is pushed once and popped at most once, so the total work over the whole loop is at most 2n operations (this is amortised analysis, see the Foundations part). **Space O(n)** for the stack.

Mistakes people make

- **Storing values when indices are needed** (or the reverse). Distances and widths require indices.
- **Wrong strictness** (`<` versus `<=`) causing off-by-one answers with duplicates. Test an array of equal numbers.

- **Not handling the leftovers.** Elements left on the stack have no answer; initialise the result with the default (0 or -1) and do not forget the sentinel in the histogram problem.
- **Wrong width.** Width is `i - left - 1` where `left` is the new top after popping, not `i - j`.
- **Multiplying in `int` when areas are large.** A histogram with 100,000 bars each of height 10,000 gives area 10^9, which still fits in an `int` but is at its limit; use `long` for anything larger.
- **Circular arrays: pushing twice.** Indices must be pushed only in the first pass; otherwise answers are derived from stale copies.

TIP
The stack order follows from the query. Next **greater** requires decreasing values on the stack (a larger arrival pops smaller values). Next **smaller** requires increasing values on the stack.

3.4.5 Questions to expect

1. **For each day, how many days until a warmer temperature?** Clue: "until a greater". Stack of indices, answer `i - j` on pop.
2. **Next greater element of each number of `nums1` inside `nums2`.** Clue: "greater element" plus a subset relation. Run the stack over `nums2`, store results in a `HashMap` from value to answer.
3. **Next greater element in a circular array.** Clue: "circular". Iterate `2n` indices, use `i % n`.
4. **Largest rectangle in a histogram.** Clue: "rectangle" bounded by smaller bars. Pop when a smaller bar arrives; width from the new top. Follow-up: maximal rectangle of 1s in a binary matrix, by applying this to each row's histogram.
5. **How much rain water does an elevation map hold?** Clue: "trapped". Stack by layers, two pointers, or prefix/suffix maxima arrays. Be ready to compare their space use.
6. **Online stock span.** Clue: "consecutive days up to today". Stack of price and span, merge spans on pop.
7. **Remove k digits to get the smallest number.** Clue: "smallest number after deletions". Increasing stack of digits, pop while the top is larger than the new digit and removals remain; strip leading zeros.
8. **Car fleet.** Clue: "cars catching up, cannot pass". Sort by position, compute arrival times from the front, count how often a car is slower than the fleet ahead.

INTERVIEW ANGLE
"The solution has a while loop inside a for loop. Why is it not O(n^2)?" Each index is pushed exactly once and popped at most once. The inner loop does not restart; it only pops items that an earlier iteration pushed. Summed over the whole run, the pops cannot exceed the pushes, so the total number of operations is at most 2n. That is an amortised argument: one iteration can be expensive, but the average per element is constant.

3.4.6 LeetCode practice

#	Title	Level	Key idea
496	Next Greater Element I	Easy	Monotonic stack over nums2 plus a map from value to answer.
739	Daily Temperatures	Medium	Stack of indices; answer is the index difference at pop time.
503	Next Greater Element II	Medium	Circular: iterate 2n steps with `i % n`.
901	Online Stock Span	Medium	Stack of price and span; merge spans while popping.
853	Car Fleet	Medium	Sort by position, track arrival times from the front.
84	Largest Rectangle in Histogram	Hard	Pop taller bars; width from the new top; sentinel zero at the end.
42	Trapping Rain Water	Hard	Layer-by-layer with a stack, or two pointers with running maxima.
85	Maximal Rectangle	Hard	Build a histogram per row, then reuse the histogram solution.

LeetCode problems for this pattern.

KEY POINTS

- A monotonic stack keeps indices ordered; a new element pops everything it beats, and each popped element has just found its answer.
- Decreasing stack for "next greater", increasing stack for "next smaller"; decide strict versus non-strict from the statement.
- The `while` inside the `for` is still O(n): each index is pushed once and popped at most once.
- For widths and distances store indices; for circular arrays loop to `2n` and push only in the first pass.
- Use a sentinel (a final bar of height 0) to flush the stack at the end.

3.5 Queue, deque and monotonic deque

A **queue** is first-in, first-out (FIFO): elements are removed in the order of insertion. A **deque** (double-ended queue, pronounced "deck") permits insertion and removal at both ends. Queues model waiting lines and schedulers and, most importantly for this text, underlie breadth-first search (BFS), which visits vertices in order of increasing distance.

This chapter covers three things: implementing and designing queues (queue from stacks, circular buffer), the **monotonic deque** for sliding-window maximum, and cache design (LRU), where a queue-like ordering is combined with a hash map.

3.5.1 The idea

Queues in Java

Use `Queue<T> q = new ArrayDeque<>()` or `Deque<T>`. Each operation has two forms: one that throws and one that returns a special value.

Purpose	Throws on failure	Returns special value	Note
add at tail	add / addLast	offer / offerLast	queue enqueue
remove from head	remove / removeFirst	poll / pollFirst	queue dequeue; poll gives null when empty
look at head	element / getFirst	peek / peekFirst	peek gives null when empty
add at head	addFirst	offerFirst	also what push does
remove from tail	removeLast	pollLast	needed for the monotonic deque

Queue and deque methods (ArrayDeque).

The methods `offer` and `poll` are preferred in solutions because they do not throw. `LinkedList` is needed only for `null` elements or list methods; `ArrayDeque` is faster and uses less memory.

The monotonic deque

Sliding window maximum asks for the maximum of every window of k consecutive elements. The brute force scans k elements per window: O(nk). A heap gives O(n log n). A monotonic deque gives O(n).

Keep a deque of **indices** whose values are decreasing from front to back. The front is always the maximum of the current window. For each new index i:

1. If the front index has left the window (`front <= i - k`), remove it from the front.
2. While the back value is less than or equal to `a[i]`, remove it from the back: it is older and smaller, so it can never be a maximum again.
3. Add i at the back. Once `i >= k - 1`, the answer for this window is `a[front]`.

This is the monotonic stack from the previous chapter, with one extra operation: removing expired elements from the other end.

How to recognise it

- "Maximum (or minimum) of every window of size k", or the longest window where max minus min is bounded.
- "Process in arrival order", "level by level", "fewest steps": BFS with a queue.
- "Implement a queue with two stacks", "design a circular queue", "design a cache that evicts the least recently used item".

sreeram

- Moving averages, "number of requests in the last 3000 ms": a queue of timestamps from which the head is polled while it lies outside the window.

Template: sliding window with a monotonic deque

WindowTemplate.java JAVA

```
Deque<Integer> dq = new ArrayDeque<>();            // indices, values decreasing
for (int i = 0; i < n; i++) {
    if (!dq.isEmpty() && dq.peekFirst() <= i - k) dq.pollFirst();      // left the window
    while (!dq.isEmpty() && a[dq.peekLast()] <= a[i]) dq.pollLast();   // dominated
    dq.offerLast(i);
    if (i >= k - 1) result[i - k + 1] = a[dq.peekFirst()];
}
```

3.5.2 Dry run

Input: `nums = [1, 3, -1, -3, 5, 3, 6, 7]`, `k = 3`. Expected output `[3, 3, 5, 5, 6, 7]` (6 windows). The deque is shown as indices and as the values they point to.

i	Add	Deque indices	Deque values	Window max
0	1	[0]	[1]	not full yet
1	3	[1]	[3]	not full yet
2	-1	[1, 2]	[3, -1]	3
3	-3	[1, 2, 3]	[3, -1, -3]	3
4	5	[4]	[5]	5
5	3	[4, 5]	[5, 3]	5
6	6	[6]	[6]	6
7	7	[7]	[7]	7

Sliding window maximum, state after each index.

Three moments teach everything. At i = 1 the value 3 removes index 0 from the back (1 is smaller and older). At i = 4 the front index 1 has expired (window is now indices 2 to 4), and then 5 removes all remaining smaller values, leaving only index 4. At i = 6 and 7 the new value dominates the whole deque. The deque is never longer than k, and the front is always the window maximum.

3.5.3 Java code

The program has a queue built from two stacks, a circular queue on a fixed array, sliding window maximum (with trace and brute force), an LRU cache two ways (`LinkedHashMap` in access order, and a hand-written hash map plus doubly linked list), and a BFS preview that finds the shortest path in a small grid.

Main.java JAVA

```
import java.util.*;

public class Main {
    // Queue using two stacks: amortised O(1) per operation
    static class MyQueue {
        private final Deque<Integer> in = new ArrayDeque<>(), out = new ArrayDeque<>();
        void push(int x) { in.push(x); }
        int pop() { move(); return out.pop(); }
        int peek() { move(); return out.peek(); }
        boolean empty() { return in.isEmpty() && out.isEmpty(); }
        private void move() {
            if (out.isEmpty()) while (!in.isEmpty()) out.push(in.pop());
        }
    }

    // Circular queue on a fixed array: head index + size
    static class MyCircularQueue {
        private final int[] data;
        private int head = 0, size = 0;
        MyCircularQueue(int k) { data = new int[k]; }
        boolean enQueue(int v) {
```

Main.java (continued) JAVA

```
            if (isFull()) return false;
            data[(head + size) % data.length] = v;
            size++;
            return true;
        }
        boolean deQueue() {
            if (isEmpty()) return false;
            head = (head + 1) % data.length;
            size--;
            return true;
        }
        int Front() { return isEmpty() ? -1 : data[head]; }
        int Rear() { return isEmpty() ? -1 : data[(head + size - 1) % data.length]; }
        boolean isEmpty() { return size == 0; }
        boolean isFull() { return size == data.length; }
    }

    // Sliding window maximum: deque of indices, values decreasing front to back
    static int[] maxSlidingWindow(int[] nums, int k, boolean trace) {
        int n = nums.length;
        int[] res = new int[n - k + 1];
        Deque<Integer> dq = new ArrayDeque<>();
        for (int i = 0; i < n; i++) {
            if (!dq.isEmpty() && dq.peekFirst() <= i - k) dq.pollFirst();       // expired
            while (!dq.isEmpty() && nums[dq.peekLast()] <= nums[i]) dq.pollLast(); // dominated
            dq.offerLast(i);
            if (i >= k - 1) res[i - k + 1] = nums[dq.peekFirst()];
            if (trace) {
                List<Integer> vals = new ArrayList<>();
                for (int idx : dq) vals.add(nums[idx]);
                System.out.println(i + " | add " + nums[i] + " | deque indices=" + dq
                    + " values=" + vals + " | " + (i >= k - 1
                    ? "max=" + nums[dq.peekFirst()] : "window not full"));
            }
        }
        return res;
    }

    static int[] maxSlidingBrute(int[] nums, int k) {
        int[] res = new int[nums.length - k + 1];
        for (int i = 0; i < res.length; i++) {
            int m = Integer.MIN_VALUE;
            for (int j = i; j < i + k; j++) m = Math.max(m, nums[j]);
            res[i] = m;
        }
        return res;
    }

    // LRU cache, the short way: LinkedHashMap in access order
    static class LRUCacheShort<K, V> extends LinkedHashMap<K, V> {
        private final int capacity;
        LRUCacheShort(int capacity) {
            super(16, 0.75f, true);          // true = access order
            this.capacity = capacity;
        }
        @Override
        protected boolean removeEldestEntry(Map.Entry<K, V> eldest) {
            return size() > capacity;
        }
    }

    // LRU cache by hand: HashMap + doubly linked list with sentinels
    static class LRUCache {
        private static class Node {
            int key, val;
            Node prev, next;
            Node(int k, int v) { key = k; val = v; }
        }
        private final int capacity;
        private final Map<Integer, Node> map = new HashMap<>();
        private final Node head = new Node(0, 0), tail = new Node(0, 0);   // sentinels

        LRUCache(int capacity) {
            this.capacity = capacity;
            head.next = tail;
```

Main.java (continued) JAVA

```
            tail.prev = head;
        }
        int get(int key) {
            Node n = map.get(key);
            if (n == null) return -1;
            unlink(n);
            addFront(n);                        // most recently used sits next to head
            return n.val;
        }
        void put(int key, int value) {
            Node n = map.get(key);
            if (n != null) {
                n.val = value;
                unlink(n);
                addFront(n);
                return;
            }
            if (map.size() == capacity) {
                Node lru = tail.prev;           // least recently used sits next to tail
                unlink(lru);
                map.remove(lru.key);
            }
            n = new Node(key, value);
            map.put(key, n);
            addFront(n);
        }
        private void unlink(Node n) { n.prev.next = n.next; n.next.prev = n.prev; }
        private void addFront(Node n) {
            n.next = head.next; n.prev = head;
            head.next.prev = n; head.next = n;
        }
        String order() {                        // most recent first
            StringBuilder sb = new StringBuilder();
            for (Node n = head.next; n != tail; n = n.next) sb.append(n.key).append(' ');
            return sb.toString().trim();
        }
    }

    // BFS preview: fewest moves on a tiny grid ('#' is a wall)
    static int shortestPath(String[] grid) {
        int rows = grid.length, cols = grid[0].length();
        boolean[][] seen = new boolean[rows][cols];
        Deque<int[]> q = new ArrayDeque<>();
        q.offer(new int[]{0, 0});
        seen[0][0] = true;
        int[][] dirs = {{1, 0}, {-1, 0}, {0, 1}, {0, -1}};
        int dist = 0;
        while (!q.isEmpty()) {
            for (int sz = q.size(); sz > 0; sz--) {          // one level = one distance
                int[] cur = q.poll();
                if (cur[0] == rows - 1 && cur[1] == cols - 1) return dist;
                for (int[] d : dirs) {
                    int r = cur[0] + d[0], c = cur[1] + d[1];
                    if (r < 0 || c < 0 || r >= rows || c >= cols) continue;
                    if (seen[r][c] || grid[r].charAt(c) == '#') continue;
                    seen[r][c] = true;
                    q.offer(new int[]{r, c});
                }
            }
            dist++;
        }
        return -1;
    }

    public static void main(String[] args) {
        MyQueue q = new MyQueue();
        q.push(1); q.push(2); q.push(3);
        System.out.println("queue via stacks: peek=" + q.peek() + " pop=" + q.pop());
        q.push(4);
        System.out.println("pop=" + q.pop() + " pop=" + q.pop() + " pop=" + q.pop()
            + " empty=" + q.empty());

        MyCircularQueue cq = new MyCircularQueue(3);
        System.out.println("enQueue 1,2,3,4: " + cq.enQueue(1) + " " + cq.enQueue(2) + " "
            + cq.enQueue(3) + " " + cq.enQueue(4));
```

Main.java (continued) JAVA

```
        System.out.println("Rear=" + cq.Rear() + " isFull=" + cq.isFull());
        System.out.println("deQueue=" + cq.deQueue() + " enQueue 4=" + cq.enQueue(4)
            + " Rear=" + cq.Rear() + " Front=" + cq.Front());

        int[] nums = {1, 3, -1, -3, 5, 3, 6, 7};
        System.out.println("nums=" + Arrays.toString(nums) + " k=3");
        System.out.println("answer: " + Arrays.toString(maxSlidingWindow(nums, 3, true)));
        System.out.println("brute : " + Arrays.toString(maxSlidingBrute(nums, 3)));

        LRUCacheShort<Integer, Integer> lru = new LRUCacheShort<>(2);
        lru.put(1, 1); lru.put(2, 2);
        System.out.println("short get(1)=" + lru.get(1));
        lru.put(3, 3);
        System.out.println("short get(2)=" + lru.get(2) + " keys=" + lru.keySet());

        LRUCache c = new LRUCache(2);
        c.put(1, 1); c.put(2, 2);
        System.out.println("hand get(1)=" + c.get(1) + " order=" + c.order());
        c.put(3, 3);
        System.out.println("hand get(2)=" + c.get(2) + " order=" + c.order());
        c.put(4, 4);
        System.out.println("hand get(1)=" + c.get(1) + " get(3)=" + c.get(3) + " get(4)=" + c.get(4)
            + " order=" + c.order());

        String[] grid = {
            "..#.",
            ".##.",
            "...."};
        System.out.println("shortest path: " + shortestPath(grid));
    }
}
```

Compile and run TERMINAL

```
$ java Main.java
queue via stacks: peek=1 pop=1
pop=2 pop=3 pop=4 empty=true
enQueue 1,2,3,4: true true true false
Rear=3 isFull=true
deQueue=true enQueue 4=true Rear=4 Front=2
nums=[1, 3, -1, -3, 5, 3, 6, 7] k=3
0 | add 1 | deque indices=[0] values=[1] | window not full
1 | add 3 | deque indices=[1] values=[3] | window not full
2 | add -1 | deque indices=[1, 2] values=[3, -1] | max=3
3 | add -3 | deque indices=[1, 2, 3] values=[3, -1, -3] | max=3
4 | add 5 | deque indices=[4] values=[5] | max=5
5 | add 3 | deque indices=[4, 5] values=[5, 3] | max=5
6 | add 6 | deque indices=[6] values=[6] | max=6
7 | add 7 | deque indices=[7] values=[7] | max=7
answer: [3, 3, 5, 5, 6, 7]
brute : [3, 3, 5, 5, 6, 7]
short get(1)=1
short get(2)=null keys=[1, 3]
hand get(1)=1 order=1 2
hand get(2)=-1 order=3 1
hand get(1)=-1 get(3)=3 get(4)=4 order=4 3
shortest path: 5
```

What to notice

- **Queue from two stacks.** Items go onto `in`. When `out` is empty and a pop or peek arrives, everything moves from `in` to `out`, which reverses the order so the oldest item is on top. Each element moves at most once, so `pop` is O(1) amortised.
- **Circular queue.** The tail position is `(head + size) % capacity`, so no separate tail variable can get out of sync. Tracking `size` makes "full" and "empty" unambiguous; with only `head` and `tail` indices the two states look identical.
- `LinkedHashMap` **as an LRU.** The constructor argument `true` switches to access order, so every `get` and `put` moves the entry to the end. Overriding `removeEldestEntry` evicts the head, the least recently used entry, after an insertion. In the output, `get(2)` returns `null` after key 3 was added, and `keySet()` shows `[1, 3]`. The map class is not thread-safe.

- **Hand-written LRU.** The `HashMap` finds a node in O(1); the doubly linked list orders nodes from most to least recently used and lets a node unlink itself in O(1). Two sentinel nodes (`head` and `tail`) remove every null check. This is the solution interviewers expect.
- **BFS levels.** Taking `q.size()` nodes per outer iteration processes one distance level at a time, so `dist` is the number of moves. The `seen` array is set when a cell is **enqueued**, not when it is dequeued, otherwise a cell can be queued many times.

3.5.4 Complexity and mistakes

- Sliding window maximum: **O(n) time**, since each index enters and leaves the deque once, and **O(k) space** for the deque (plus the O(n) result).
- Queue from stacks: O(1) amortised per operation; a single `pop` can cost O(n). Circular queue: O(1) per operation, O(k) space.
- LRU cache: `get` and `put` are O(1) average with a hash map plus doubly linked list; space O(capacity).
- BFS on a grid with R rows and C columns: O(R times C) time and space.

Mistakes people make

- **Storing values instead of indices in the window deque**, then being unable to tell when the front has expired.
- **Expiring after reading the answer.** The expired front must be removed before `peekFirst()` is read.
- **Trimming with `<` where `<=` is intended.** With `<`, equal values remain in the deque. The maximum is still correct, but the deque holds redundant entries. Trimming with `<=` keeps it strictly decreasing.
- **Circular queue modulo mistakes.** The next insertion slot is `(head + size) % cap`, but the last stored element, which `Rear()` returns, is at `(head + size - 1) % cap`.
- **Marking BFS nodes visited on dequeue**, which creates duplicates in the queue and can turn an O(V + E) search into a much slower one.
- **LRU details**: updating a key that already exists must also refresh its recency, and eviction must remove the key from the map as well as the list.

> **WATCH OUT**
> Do not use `LinkedList` or `ArrayList.remove(0)` as a queue for large inputs in a time-limited setting. `ArrayList.remove(0)` shifts every element, so n dequeues cost O(n^2). `ArrayDeque.poll` is O(1). `ArrayDeque` should be used unless there is a specific reason not to.

3.5.5 Questions to expect

1. **Implement a queue using two stacks (and a stack using two queues).** Clue: "implement X using Y". Lazy transfer from `in` to `out` gives amortised O(1). For the stack using queues, rotate after each push.
2. **Design a circular queue of fixed size.** Clue: "fixed buffer", "no shifting". Array, `head`, `size`, modulo arithmetic. Follow-up: a circular deque.
3. **Sliding window maximum.** Clue: "max of every window of size k". Monotonic deque of indices. Follow-ups: sliding window minimum (flip the comparison), or the longest subarray where max minus min is at most a limit (two deques, one for max and one for min).
4. **Number of recent requests in the last 3000 ms.** Clue: "calls in a time window". Queue of timestamps; poll from the head while older than the window.
5. **Design an LRU cache with O(1) get and put.** Clue: "evict the least recently used". Hash map plus doubly linked list; or `LinkedHashMap` with access order if the interviewer allows the library.
6. **Design an LFU cache.** Clue: "least frequently used", ties broken by recency. Hash map from key to node, plus a map from frequency to a linked list, and a tracked minimum frequency. Mention it as the harder cousin of LRU.
7. **Shortest path in an unweighted grid or graph.** Clue: "fewest moves/steps". BFS with a queue; mark visited when enqueuing. This is developed fully in the graph chapters.
8. **Shortest subarray with sum at least K when there are negative numbers.** Clue: "shortest", "negative values break the sliding window". Prefix sums with a monotonic deque of indices.

sreeram

INTERVIEW ANGLE

"Design an LRU cache. What data structures and why?" **Candidate:** O(1) lookup and O(1) reordering are required. A hash map from key to node provides lookup. A doubly linked list keeps nodes in recency order: on every `get` or `put` the node is moved to the front, and when the capacity is exceeded the node at the back is removed and its key deleted from the map. A singly linked list is insufficient because removing a node from the middle requires its predecessor. Sentinel head and tail nodes avoid null checks. **Interviewer:** Make it thread-safe. **Candidate:** Guard `get` and `put` with a lock; a single lock is simplest, and lock striping or a library such as `Caffeine` would be discussed for production use.

3.5.6 LeetCode practice

#	Title	Level	Key idea
933	Number of Recent Calls	Easy	Queue of timestamps; poll while older than 3000.
232	Implement Queue using Stacks	Easy	Input and output stacks, transfer only when output is empty.
225	Implement Stack using Queues	Easy	After each push, rotate the queue so the new item is at the front.
994	Rotting Oranges	Medium	Multi-source BFS, one level per minute.
622	Design Circular Queue	Medium	Array with `head`, `size` and modulo arithmetic.
641	Design Circular Deque	Medium	Same buffer, with moves in both directions.
146	LRU Cache	Medium	Hash map plus doubly linked list with sentinels.
1438	Longest Continuous Subarray With Absolute Diff Less Than or Equal to Limit	Medium	Window with two monotonic deques (max and min).
239	Sliding Window Maximum	Hard	Monotonic deque of indices, expire from the front.
862	Shortest Subarray with Sum at Least K	Hard	Prefix sums with a deque of increasing prefix values.
460	LFU Cache	Hard	Key map plus frequency buckets of linked lists, track the minimum frequency.

LeetCode problems for this pattern.

KEY POINTS

- Queue: `offer` at the tail, `poll` at the head; use `ArrayDeque`, not `LinkedList`, and not `ArrayList.remove(0)`.
- A monotonic deque of indices gives the window maximum in O(n): expire from the front, trim dominated values from the back.
- A queue from two stacks has amortised O(1) operations; a circular queue uses `(head + size) % capacity`.
- LRU = hash map plus doubly linked list with sentinels; `LinkedHashMap` with access order is the short form.
- In BFS, nodes are marked visited when enqueued, and one level is processed per outer iteration to count distance.

PART 4

Recursion, Backtracking and Trees

Functions that call themselves, search trees of choices, and the data structure built out of recursion itself.

In this part

4.1 Recursion and Divide and Conquer
4.2 Backtracking: Subsets, Permutations, Combinations
4.3 Backtracking on Grids and Constraints
4.4 Binary Tree DFS
4.5 Binary Tree BFS and Level Order
4.6 BST and Tree Construction
4.7 Tries

4.1 Recursion and Divide and Conquer

Recursion means a function solves a problem by calling itself on a smaller version of the same problem. It underlies most of the techniques in this Part: tree traversals, backtracking, memoised dynamic programming, merge sort and quickselect.

Many interview problems have a natural recursive shape ("the answer for a tree is built from the answers for its two subtrees"), and the brute-force solution is often a recursion that recomputes the same sub-problems over and over. Inspection of the call tree permits the work to be counted, redundant computation to be identified, and the algorithm to be corrected. Divide and conquer is the disciplined version: split the input, solve the pieces recursively, combine the results.

4.1.1 The idea

Every correct recursive function has three parts.

- **Base case**: an input small enough to be answered directly, without recursion. Without it the recursion never stops.
- **Recursive case**: reduce the input to one or more smaller inputs and invoke the function itself.
- **Combine step**: turn the answers of the smaller calls into the answer for this call (sometimes there is nothing to combine).

Correctness follows by induction. The recursive case may *assume that the call on the smaller input returns the correct answer* (the inductive hypothesis). It is then sufficient to show that, given those answers, the function produces the correct result for its own input (the postcondition) and that the input strictly decreases on every call, so that the base case is reached.

Each call gets its own **stack frame**, a block of memory holding its parameters, local variables and the return address. Frames accumulate on the thread's call stack and are popped as calls return. A recursion of depth d therefore uses O(d) stack space, even if it allocates nothing else. The JVM stack is small (often about 1 MB per thread by default), so a depth of some ten thousand frames or more can throw `StackOverflowError`; the exact limit depends on frame size and JVM settings.

How to recognise it

- "Split the array in half" or "solve for the left and right part": divide and conquer.
- The problem definition itself is recursive: trees, nested structures, "the n-th term depends on earlier terms".
- "Compute x to the power n", "sort", "k-th smallest", "all ways to ...": a smaller instance of the same question hides inside.
- The brute force is exponential because it re-solves the same sub-problems: add a memo (a cache) to the recursion.

Sreeram

Template

Template.java JAVA

```java
// Recursion template (fragment)
static Result solve(Input in) {
    if (isSmallEnough(in)) return directAnswer(in);      // base case
    Input[] parts = split(in);                           // strictly smaller inputs
    Result[] sub = new Result[parts.length];
    for (int i = 0; i < parts.length; i++) sub[i] = solve(parts[i]);   // trust the calls
    return combine(sub);                                 // build this answer
}
```

Divide and conquer is that template where the split is balanced (usually halves) and the combine step is cheap. The cost follows a *recurrence*, an equation for T(n) in terms of T of smaller inputs:

Recurrence	Total cost	Typical example	Why
`T(n) = T(n/2) + O(1)`	O(log n)	binary search, fast power	one half is discarded, constant work
`T(n) = T(n/2) + O(n)`	O(n)	quickselect on average	work shrinks geometrically: n + n/2 + n/4 + ...
`T(n) = 2 T(n/2) + O(n)`	O(n log n)	merge sort	log n levels, each doing n total work
`T(n) = 2 T(n/2) + O(1)`	O(n)	tree height on a balanced tree	n nodes, constant work each
`T(n) = T(n-1) + O(1)`	O(n)	factorial, list recursion	one chain of n calls
`T(n) = T(n-1) + T(n-2)`	O(2^n)	naive Fibonacci	each call spawns two big calls

Common recurrences and what they solve.

4.1.2 Dry run

Fast exponentiation computes x to the power n with far fewer multiplications than the obvious loop. The idea: x^n = (x^(n/2))^2 when n is even, and x^n = (x^(n/2))^2 * x when n is odd (integer division). Each call halves n, so about log2(n) calls are needed instead of n.

The trace below follows `power(2, 10)`. The program prints one line when a call starts and one when it returns, indented by call depth. Input: x = 2, n = 10.

Step	Call	Depth	What happens	Returns
1	`power(2, 10)`	0	n is not 0, ask for power(2, 5)	waits
2	`power(2, 5)`	1	ask for power(2, 2)	waits
3	`power(2, 2)`	2	ask for power(2, 1)	waits
4	`power(2, 1)`	3	ask for power(2, 0)	waits
5	`power(2, 0)`	4	base case, n == 0	1
6	`power(2, 1)`	3	n odd: 1 * 1 * 2	2
7	`power(2, 2)`	2	n even: 2 * 2	4
8	`power(2, 5)`	1	n odd: 4 * 4 * 2	32
9	`power(2, 10)`	0	n even: 32 * 32	1024

Call stack for power(2, 10). Calls go down the table, returns come back up.

Five frames were alive at the deepest point (steps 1 to 5), and only four combine steps (steps 6 to 9) were needed, compared with nine multiplications for the obvious loop. The stack grows to its maximum, hits the base case, then unwinds: every "waits" row becomes a "returns" row in reverse order. The invariant is that the stack grows monotonically until the base case is reached and then unwinds in the reverse order of the calls.

sreeram

4.1.3 Java code

The first program contains everything this chapter discusses: factorial, iterative fast power (with the `Integer.MIN_VALUE` overflow case handled), merge sort, Tower of Hanoi, randomised quickselect, naive versus memoised Fibonacci, and a deliberate stack overflow.

Main.java JAVA

```
import java.util.*;

public class Main {
    static long fact(int n) { return n <= 1 ? 1 : n * fact(n - 1); }

    static double myPow(double x, int n) {
        long e = n;                     // widen first: -Integer.MIN_VALUE overflows int
        if (e < 0) { x = 1 / x; e = -e; }
        double result = 1;
        while (e > 0) {
            if ((e & 1) == 1) result *= x;
            x *= x;
            e >>= 1;
        }
        return result;
    }

    static void mergeSort(int[] a, int lo, int hi, int[] tmp) {   // sorts a[lo, hi)
        if (hi - lo < 2) return;
        int mid = lo + (hi - lo) / 2;
        mergeSort(a, lo, mid, tmp);
        mergeSort(a, mid, hi, tmp);
        int i = lo, j = mid, k = lo;
        while (i < mid && j < hi) tmp[k++] = a[i] <= a[j] ? a[i++] : a[j++];
        while (i < mid) tmp[k++] = a[i++];
        while (j < hi) tmp[k++] = a[j++];
        System.arraycopy(tmp, lo, a, lo, hi - lo);
    }

    static int moves = 0;
    static void hanoi(int n, char from, char to, char via, boolean print) {
        if (n == 0) return;
        hanoi(n - 1, from, via, to, print);      // park n-1 disks on the spare peg
        moves++;
        if (print) System.out.println("move disk " + n + " " + from + " -> " + to);
        hanoi(n - 1, via, to, from, print);      // bring them on top of disk n
    }

    static final Random RNG = new Random(7);
    static int quickSelect(int[] a, int k) {      // k-th smallest, 0-based
        int lo = 0, hi = a.length - 1;
        while (lo < hi) {
            int p = partition(a, lo, hi);
            if (p == k) return a[p];
            if (p < k) lo = p + 1; else hi = p - 1;
        }
        return a[lo];
    }
    static int partition(int[] a, int lo, int hi) {
        swap(a, lo + RNG.nextInt(hi - lo + 1), hi);   // random pivot goes to the end
        int pivot = a[hi], i = lo;
        for (int j = lo; j < hi; j++) if (a[j] < pivot) swap(a, i++, j);
        swap(a, i, hi);                           // pivot lands in its final position
        return i;
    }
    static void swap(int[] a, int i, int j) { int t = a[i]; a[i] = a[j]; a[j] = t; }

    static int fibCalls = 0;
    static long fibSlow(int n) { fibCalls++; return n < 2 ? n : fibSlow(n - 1) + fibSlow(n - 2); }
    static long fibMemo(int n, long[] memo) {
        if (n < 2) return n;
        if (memo[n] != 0) return memo[n];
        return memo[n] = fibMemo(n - 1, memo) + fibMemo(n - 2, memo);
    }
    static int deep(int n) { return deep(n + 1) + 1; }

    public static void main(String[] args) {
        System.out.println("fact(5) = " + fact(5));
```

Oreram

Main.java (continued) JAVA

```
        System.out.println("myPow(2, 10) = " + myPow(2, 10));
        System.out.println("myPow(2, -2) = " + myPow(2, -2));
        System.out.println("myPow(2, MIN) = " + myPow(2, Integer.MIN_VALUE));
        int[] arr = {38, 27, 43, 3, 9, 82, 10};
        mergeSort(arr, 0, arr.length, new int[arr.length]);
        System.out.println("sorted = " + Arrays.toString(arr));
        hanoi(3, 'A', 'C', 'B', true);
        moves = 0; hanoi(20, 'A', 'C', 'B', false);
        System.out.println("hanoi(20) moves = " + moves);
        int[] b = {7, 10, 4, 3, 20, 15};
        System.out.println("3rd smallest = " + quickSelect(b.clone(), 2));
        System.out.println("2nd largest = " + quickSelect(b.clone(), b.length - 2));
        System.out.println("fibSlow(30) = " + fibSlow(30) + " calls=" + fibCalls);
        System.out.println("fibMemo(30) = " + fibMemo(30, new long[31]));
        try { deep(1); }
        catch (StackOverflowError e) { System.out.println("StackOverflowError caught"); }
    }
}
```

Output (JDK 21) TERMINAL

```
$ java Main.java
fact(5) = 120
myPow(2, 10) = 1024.0
myPow(2, -2) = 0.25
myPow(2, MIN) = 0.0
sorted = [3, 9, 10, 27, 38, 43, 82]
move disk 1 A -> C
move disk 2 A -> B
move disk 1 C -> B
move disk 3 A -> C
move disk 1 B -> A
move disk 2 B -> C
move disk 1 A -> C
hanoi(20) moves = 1048575
3rd smallest = 7
2nd largest = 15
fibSlow(30) = 832040 calls=2692537
fibMemo(30) = 832040
StackOverflowError caught
```

What to notice

- `mid = lo + (hi - lo) / 2` avoids overflow of `(lo + hi) / 2` on huge arrays.
- Merge sort is *stable* because of `<=` when the two heads are equal: the left element goes first.
- Hanoi: moving n disks takes T(n) = 2 T(n-1) + 1 moves, which solves to 2^n - 1. For n = 20 that is 1,048,575, matching the output. No algorithm can do better; this is an exponential *problem*, not a slow solution.
- `fibSlow(30)` makes 2,692,537 calls to compute a number that needs only 30 distinct sub-problems. `fibMemo` stores each result once, making it O(n). This is the teaser for the dynamic programming Part: memoisation is just "recursion plus a cache".
- `quickSelect` is a loop, not recursion, because after partitioning only one side matters. The expected cost is O(n) because the pivot is random; it is the engine behind "k-th largest" questions (an alternative to heaps).
- `myPow` uses the binary expansion of the exponent iteratively: no stack, O(log n) time, O(1) space. The recursive version in the dry run is the same idea.

4.1.4 Complexity and mistakes

Time equals the number of calls times the work per call. Analysis uses the call tree: for fast power it is a single chain of log2(n) calls; for merge sort it is a balanced tree with log2(n) levels and O(n) total work per level, giving O(n log n); for naive Fibonacci the tree has about 1.6^n nodes. **Space** equals the maximum number of live frames (the depth of the tree) plus any additionally allocated memory. Merge sort uses O(n) for the temp array plus O(log n) stack. Quickselect with random pivots uses O(1) extra here (it is a loop) and expected O(n) time but O(n^2) worst case.

Mistakes to avoid:

- **Missing or unreachable base case.** `power(x, n)` with a base case `n == 1` loops forever for `n == 0`. Ask: "which inputs can reach the base case, and do all of them?"

- **The input does not shrink.** Calling `solve(n)` from `solve(n)`, or `solve(mid)` when `mid == hi` in a range split, recurses forever. Check the smallest ranges (length 0, 1, 2) by hand.
- **Overflow in the combine step.** `fact(21)` overflows `long`; `(lo + hi) / 2` overflows `int`. Also `-Integer.MIN_VALUE == Integer.MIN_VALUE`, which is why `myPow` widens to `long` before negating.
- **Recomputing sub-problems.** If the call tree has the same call appearing many times (Fibonacci, grid paths), add a memo. If the sub-problems never repeat (merge sort, Hanoi), a memo only wastes memory.
- **Deep recursion in production code.** A linked list or skewed tree with 100,000 nodes can overflow the stack. An explicit `Deque` or a loop should be used instead.

GOTCHA
A recursive method that never returns does not hang quietly: the JVM throws `StackOverflowError` once the thread stack is exhausted. It is an `Error`, not an `Exception`; catching it (as the demo does) is fine for experiments but never a fix for a missing base case.

4.1.5 Questions to expect

1. "Compute x to the power n in O(log n)." Clue: halve the exponent. Handle negative n and `Integer.MIN_VALUE`; mention the iterative bit version.
2. "Sort an array without using the library; guarantee O(n log n)." Merge sort. Follow-up: "What if memory is tight?" Heap sort is O(1) extra; merge sort needs O(n) for arrays but O(1) for linked lists.
3. "Find the k-th largest element." Clue: a full sort is unnecessary. Quickselect gives expected O(n); a size-k min-heap gives O(n log k). Follow-up: "What is quickselect's worst case, and how is it avoided?" O(n^2); random pivot or median-of-medians.
4. "Move n disks between pegs." Pure recursion: solve n-1, move one, solve n-1 again. Know that 2^n - 1 moves are required.
5. "What is the time complexity of this recursive function?" Write the recurrence, then solve it with the tree method (levels times work per level).
6. "A recursive solution is too slow; improve it." Spot repeated calls, add a memo (top-down DP), or convert to a loop (bottom-up).
7. "Find the k-th symbol in the n-th row of a grammar that doubles each row." Clue: row n is built from row n-1, so recurse on the parent index `(k + 1) / 2`.

INTERVIEW ANGLE
"Can you convert this recursion to iteration?" Yes: recursion is equivalent to an explicit stack of frames. Maintain an explicit `Deque` holding the state of each frame (arguments plus "where to resume"). For tail-like recursion (the last action is the call, as in quickselect) a plain loop is enough. Java does not eliminate tail calls, so an iterative form is preferable for deep inputs, and this should be stated before coding.

Sreeram

4.1.6 LeetCode practice

#	Title	Level	Key idea
509	Fibonacci Number	Easy	Two-term recurrence; compare naive, memo, and two-variable loop
70	Climbing Stairs	Easy	Same recurrence as Fibonacci in disguise: ways(n) = ways(n-1) + ways(n-2)
169	Majority Element	Easy	Divide and conquer: majority of the whole is the majority of one half (or use Boyer-Moore)
50	Pow(x, n)	Medium	Halve the exponent; handle negative n and Integer.MIN_VALUE
912	Sort an Array	Medium	Implement merge sort or heap sort; avoid O(n^2) pitfalls
215	Kth Largest Element in an Array	Medium	Quickselect with a random pivot, or a min-heap of size k
779	K-th Symbol in Grammar	Medium	Row n comes from row n-1: recurse on the parent and flip if k is even
241	Different Ways to Add Parentheses	Medium	Split at each operator, solve both sides, combine all pairs
23	Merge k Sorted Lists	Hard	Divide and conquer: merge lists in pairs, halving the count each round

LeetCode problems for this pattern.

KEY POINTS

Recursion = base case + smaller sub-problem + combine; trust the call on the smaller input.

- Draw the call tree. Time is (number of calls) x (work per call); space is the depth of the tree plus allocations.
- Divide and conquer with halves and an O(n) combine gives O(n log n); one half discarded gives O(log n).
- Repeated identical calls indicate the need for a memo; Fibonacci with and without it shows the exponential-to-linear jump.
- The JVM stack is limited and Java has no tail-call elimination: very deep recursion needs an explicit stack.
- Guard integer overflow: `lo + (hi - lo) / 2`, `long` for factorials and for negating `Integer.MIN_VALUE`.

4.2 Backtracking: Subsets, Permutations, Combinations

Some problems require producing *every* arrangement that satisfies a rule: all subsets of a set, all orderings of a list, all ways to pick k items, all strings of balanced brackets. The brute-force idea is to generate every possible candidate and filter. For a set of n elements that is 2^n subsets or n! orderings, which is unavoidable if the answer itself has that many items. The inefficiency lies in generating candidates that could have been rejected early, and in expressing the generator as n nested loops, which only works for a fixed n.

Backtracking solves both problems. It builds a candidate one decision at a time, recursing after every decision, and *undoes* the decision on the way back so the same data structure can be reused for the next branch. Because it builds partial candidates, it can stop a branch the moment it becomes hopeless (pruning).

4.2.1 The idea

The search is modelled as a tree. The root is the empty candidate. Each edge is one decision ("take element 2", "put a (here"). Each node is a partial candidate. A leaf, or a node that satisfies the goal, is an answer. Backtracking is a depth-first walk over this tree.

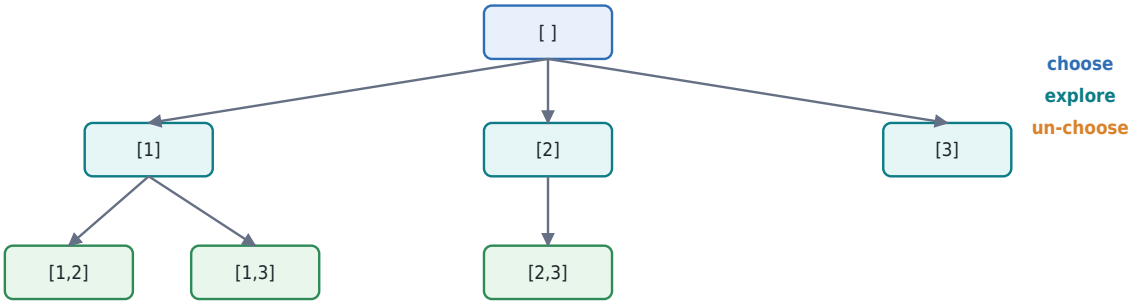


Figure 4.1. The decision tree of backtracking. A step down the tree is "choose"; the recursive call is "explore"; the step back up is "unchoose", which restores the state so the next sibling starts clean.

Every backtracking solution has the same three beats, usually written as a loop over the available choices:

1. **Choose**: add one option to the current path (and update any helper state such as `used[]`).
2. **Explore**: recurse to make the next decision.
3. **Unchoose**: remove the option again, restoring the state exactly as it was.

How to recognise it

- "Return all ...", "list every ...", "generate all ...": the output is a collection of combinatorial objects.
- "Find all combinations that sum to ...", "all valid arrangements".
- Input size is small (n up to about 10 to 20), the hint that exponential time is expected.
- Decisions are independent steps: take or skip, which letter next, which unused element goes in this slot.
- "Count the ways" with a tiny n also fits, but if n is large and sub-problems repeat, think dynamic programming instead.

Template

Template.java JAVA

```java
// Backtracking template (fragment)
static void backtrack(State s, List<Integer> path, List<List<Integer>> out) {
    if (isGoal(s)) {                          // 1. record an answer (copy the path!)
        out.add(new ArrayList<>(path));
        return;                               //    or keep going, e.g. subsets record at every node
    }
    for (int choice : candidates(s)) {        // 2. loop over the options at this step
        if (!isValid(s, choice)) continue;    //    prune early
        path.add(choice);                     // 3. choose
        backtrack(next(s, choice), path, out);    // 4. explore
        path.remove(path.size() - 1);         // 5. unchoose
    }
}
```

The variant depends on two questions. First: *can order matter?* Subsets and combinations treat `[1, 2]` and `[2, 1]` as the same, so a `start` index is passed and only later elements are considered. Permutations treat them as different, so the loop ranges over all elements and tracks which are `used`. Second: *can elements repeat?* Combination Sum allows reusing an element (recurse with `i`, not `i + 1`); duplicates in the input need sorting plus a "skip equal sibling" rule.

Problem family	What the loop ranges over	Recursion argument	Duplicates and reuse
Subsets	indices from `start` to end	`i + 1`	sort, skip `nums[i] == nums[i-1]` when `i > start`
Combinations	numbers from `start` to n	`i + 1`	same; prune with how many are still needed
Combination sum	candidates from `start`	`i` (reuse allowed) or `i + 1`	sort, break when candidate exceeds remainder
Permutations	all indices not yet `used`	none (loop restarts at 0)	sort, skip if equal to previous and previous unused
Strings (letters, parentheses)	the legal characters at this position	next position	none, but the legality rules prune

Choosing the variant.

4.2.2 Dry run

Take `subsets([1, 2, 3])`. The program records the current path at every node of the tree, then tries to extend it with each later element. Input: `nums = [1, 2, 3]`. The trace below is real output from the program (each line is one call; `start` is the first index the loop may use).

Call	start	path when called	What the call does
1	0	`[]`	record [], then loop i = 0, 1, 2 (choose 1 first)
2	1	`[1]`	record [1]; loop i = 1, 2 (choose 2 first)
3	2	`[1, 2]`	record [1, 2]; loop i = 2 (choose 3)
4	3	`[1, 2, 3]`	record [1, 2, 3]; loop is empty (start == length), return
5	3	`[1, 3]`	back in call 2, unchoose 2 then choose 3: record [1, 3]; return
6	2	`[2]`	back in call 1, unchoose 1, choose 2: record [2]; loop i = 2 (choose 3)
7	3	`[2, 3]`	record [2, 3]; return
8	3	`[3]`	back in call 1, unchoose 2, choose 3: record [3]; return

Subsets of [1, 2, 3], one row per call.

Eight calls, eight subsets, which is 2^3. The order of results is depth-first: a set is recorded the moment it is built, before extensions of it. Notice the `start` argument at work: after choosing 2, the next call starts at index 2, so the path `[2, 1]` is never generated and each subset appears exactly once. Between rows 4 and 5 the code removed 3, then 2, then added 3 - the *unchoose* steps that do not print a line but are what make backtracking correct.

4.2.3 Java code

The program below holds eight classic problems. They share the same skeleton; compare how only the loop range, the recursion argument and the pruning line change.

Main.java JAVA

```java
import java.util.*;

public class Main {
    // 1. Subsets (distinct values)
    static List<List<Integer>> subsets(int[] nums) {
        List<List<Integer>> out = new ArrayList<>();
        subsetsRec(nums, 0, new ArrayList<>(), out);
        return out;
    }
    static void subsetsRec(int[] nums, int start, List<Integer> path, List<List<Integer>> out) {
        out.add(new ArrayList<>(path));
        for (int i = start; i < nums.length; i++) {
            path.add(nums[i]);
            subsetsRec(nums, i + 1, path, out);
            path.remove(path.size() - 1);
        }
    }

    // 2. Subsets II (duplicates in input)
    static List<List<Integer>> subsetsWithDup(int[] nums) {
        Arrays.sort(nums);
        List<List<Integer>> out = new ArrayList<>();
        dupRec(nums, 0, new ArrayList<>(), out);
        return out;
    }
    static void dupRec(int[] nums, int start, List<Integer> path, List<List<Integer>> out) {
        out.add(new ArrayList<>(path));
        for (int i = start; i < nums.length; i++) {
            if (i > start && nums[i] == nums[i - 1]) continue;   // skip same value at this level
            path.add(nums[i]);
            dupRec(nums, i + 1, path, out);
            path.remove(path.size() - 1);
        }
    }

    // 3. Permutations (distinct values) with a used[] array
    static List<List<Integer>> permute(int[] nums) {
        List<List<Integer>> out = new ArrayList<>();
```

Main.java (continued) JAVA

```
        permRec(nums, new boolean[nums.length], new ArrayList<>(), out);
        return out;
    }
    static void permRec(int[] nums, boolean[] used, List<Integer> path, List<List<Integer>> out) {
        if (path.size() == nums.length) { out.add(new ArrayList<>(path)); return; }
        for (int i = 0; i < nums.length; i++) {
            if (used[i]) continue;
            used[i] = true;
            path.add(nums[i]);
            permRec(nums, used, path, out);
            path.remove(path.size() - 1);
            used[i] = false;
        }
    }

    // 4. Permutations II (duplicates)
    static List<List<Integer>> permuteUnique(int[] nums) {
        Arrays.sort(nums);
        List<List<Integer>> out = new ArrayList<>();
        uniqRec(nums, new boolean[nums.length], new ArrayList<>(), out);
        return out;
    }
    static void uniqRec(int[] nums, boolean[] used, List<Integer> path, List<List<Integer>> out) {
        if (path.size() == nums.length) { out.add(new ArrayList<>(path)); return; }
        for (int i = 0; i < nums.length; i++) {
            if (used[i]) continue;
            if (i > 0 && nums[i] == nums[i - 1] && !used[i - 1]) continue;
            used[i] = true;
            path.add(nums[i]);
            uniqRec(nums, used, path, out);
            path.remove(path.size() - 1);
            used[i] = false;
        }
    }

    // 5. Combinations C(n, k)
    static List<List<Integer>> combine(int n, int k) {
        List<List<Integer>> out = new ArrayList<>();
        combRec(n, k, 1, new ArrayList<>(), out);
        return out;
    }
    static void combRec(int n, int k, int start, List<Integer> path, List<List<Integer>> out) {
        if (path.size() == k) { out.add(new ArrayList<>(path)); return; }
        // prune: need (k - path.size()) more numbers, so i can be at most n - need + 1
        for (int i = start; i <= n - (k - path.size()) + 1; i++) {
            path.add(i);
            combRec(n, k, i + 1, path, out);
            path.remove(path.size() - 1);
        }
    }

    // 6. Combination Sum (reuse allowed)
    static List<List<Integer>> combinationSum(int[] cand, int target) {
        Arrays.sort(cand);
        List<List<Integer>> out = new ArrayList<>();
        csRec(cand, target, 0, new ArrayList<>(), out);
        return out;
    }
    static void csRec(int[] c, int remain, int start, List<Integer> path, List<List<Integer>> out) {
        if (remain == 0) { out.add(new ArrayList<>(path)); return; }
        for (int i = start; i < c.length; i++) {
            if (c[i] > remain) break;               // sorted, so everything after is too big
            path.add(c[i]);
            csRec(c, remain - c[i], i, path, out);  // i, not i + 1: same number may be reused
            path.remove(path.size() - 1);
        }
    }

    // 7. Letter combinations
    static final String[] KEYS = {"", "", "abc", "def", "ghi", "jkl", "mno", "pqrs", "tuv", "wxyz"};
    static List<String> letterCombinations(String digits) {
        List<String> out = new ArrayList<>();
        if (digits.isEmpty()) return out;
        lcRec(digits, 0, new StringBuilder(), out);
        return out;
```

Sheram

Main.java (continued) JAVA

```
    }
    static void lcRec(String d, int idx, StringBuilder sb, List<String> out) {
        if (idx == d.length()) { out.add(sb.toString()); return; }
        for (char ch : KEYS[d.charAt(idx) - '0'].toCharArray()) {
            sb.append(ch);
            lcRec(d, idx + 1, sb, out);
            sb.setLength(sb.length() - 1);
        }
    }

    // 8. Generate parentheses
    static List<String> generateParenthesis(int n) {
        List<String> out = new ArrayList<>();
        gpRec(n, 0, 0, new StringBuilder(), out);
        return out;
    }
    static void gpRec(int n, int open, int close, StringBuilder sb, List<String> out) {
        if (sb.length() == 2 * n) { out.add(sb.toString()); return; }
        if (open < n) {
            sb.append('(');
            gpRec(n, open + 1, close, sb, out);
            sb.setLength(sb.length() - 1);
        }
        if (close < open) {
            sb.append(')');
            gpRec(n, open, close + 1, sb, out);
            sb.setLength(sb.length() - 1);
        }
    }

    public static void main(String[] args) {
        System.out.println("subsets [1,2,3]        = " + subsets(new int[]{1, 2, 3}));
        System.out.println("subsets II [1,2,2]     = " + subsetsWithDup(new int[]{2, 1, 2}));
        System.out.println("permute [1,2,3]        = " + permute(new int[]{1, 2, 3}));
        System.out.println("permuteUnique [1,1,2]  = " + permuteUnique(new int[]{1, 1, 2}));
        System.out.println("combine(4, 2)          = " + combine(4, 2));
        System.out.println("combinationSum [2,3,5] = " + combinationSum(new int[]{5, 3, 2}, 8));
        System.out.println("letters \"23\"           = " + letterCombinations("23"));
        System.out.println("parens(3)              = " + generateParenthesis(3));
        System.out.println("count parens(8)        = " + generateParenthesis(8).size());
        int[] eight = {1, 2, 3, 4, 5, 6, 7, 8};
        System.out.println("count permute(8)       = " + permute(eight).size());
    }
}
```

Output (JDK 21) TERMINAL

```
$ java Main.java
subsets [1,2,3]        = [[], [1], [1, 2], [1, 2, 3], [1, 3], [2], [2, 3], [3]]
subsets II [1,2,2]     = [[], [1], [1, 2], [1, 2, 2], [2], [2, 2]]
permute [1,2,3]        = [[1, 2, 3], [1, 3, 2], [2, 1, 3], [2, 3, 1], [3, 1, 2], [3, 2, 1]]
permuteUnique [1,1,2]  = [[1, 1, 2], [1, 2, 1], [2, 1, 1]]
combine(4, 2)          = [[1, 2], [1, 3], [1, 4], [2, 3], [2, 4], [3, 4]]
combinationSum [2,3,5] = [[2, 2, 2, 2], [2, 3, 3], [3, 5]]
letters "23"           = [ad, ae, af, bd, be, bf, cd, ce, cf]
parens(3)              = [((())), (()()), (())(), ()(()), ()()()]
count parens(8)        = 1430
count permute(8)       = 40320
```

What to notice

- **Copy the path when recording it**: `out.add(new ArrayList<>(path))`. If `path` itself is added, every stored answer is the same list object, and after the recursion finishes they all print as empty. This is the number one backtracking bug in Java.
- **Skipping duplicates**: in `dupRec`, `i > start && nums[i] == nums[i - 1]` skips an equal value only among *siblings* (same loop). Using `i > 0` would wrongly forbid `[2, 2]`, because the second 2 is a child of the first, not a sibling. The input `{2, 1, 2}` was sorted inside the method; sorting is what makes equal values adjacent.
- **Permutations with duplicates**: `!used[i - 1]` means "the previous equal value is not currently in the path, so using this one would repeat a branch already explored".
- **Pruning in `combine`**: the loop bound `n - (k - path.size()) + 1` stops when there are not enough numbers left to fill the combination. It cuts the tree without changing the answers.

- **Combination sum** sorts the candidates so that `break` (not `continue`) is correct once a candidate exceeds the remainder.
- **Strings**: `sb.setLength(sb.length() - 1)` is the cheap unchoose for a `StringBuilder`. Generate Parentheses only recurses on moves that keep the string valid (`open < n`, `close < open`), so it never builds an invalid string: no filtering step is needed. The count 1430 for n = 8 is the 8th Catalan number.

4.2.4 Complexity and mistakes

The cost is (number of nodes in the decision tree) x (work per node). Subsets: 2^n nodes, and copying each answer costs O(n), so O(n x 2^n). Permutations: n! leaves, each costing O(n) to copy, so O(n x n!). Combinations C(n, k): O(k x C(n, k)). Letter combinations: up to 4^n for n digits. Generate Parentheses: Catalan(n), about 4^n / (n x sqrt(n)). Space: O(n) recursion depth plus the output, which dominates. When an interviewer asks about complexity, state it as "output size times the cost of building one answer".

Mistakes to avoid:

- Forgetting to copy the path (see above) or forgetting the unchoose step, so state leaks into sibling branches.
- Using `i` where `i + 1` belongs: `[1, 1]` appears in subsets, or a permutation reuses an element.
- Using `i > 0` instead of `i > start` in duplicate skipping, which silently drops valid answers like `[2, 2]`.
- Removing from a `List<Integer>` with `path.remove(x)` where `x` is an `int`: that removes by *index*. Use `path.remove(path.size() - 1)` (or `removeLast()` on a `Deque`). Avoid `path.remove(Integer.valueOf(x))` too, because it removes the first match.
- Pruning too late: checking the sum only at the end of combination sum explores huge useless subtrees. Check before recursing, or `break` on sorted input.

GOTCHA

`List<List<Integer>> out; out.add(path);` compiles and runs, and returns a list of N references to the same list, which has been emptied by the unchoose steps. A result consisting of empty lists indicates this defect.

4.2.5 Questions to expect

1. "Return all subsets of the array." Clue: "all". Take-or-skip via `start` index. Follow-up: "What if there are duplicates?" Sort and skip equal siblings.
2. "Return all permutations." `used[]` array, or swap in place (`swap(i, start)`, recurse, swap back). Follow-up: "next permutation instead of all of them" is a separate in-place algorithm.
3. "Choose k numbers from 1..n." Start index plus pruning on remaining count. Follow-up: "only those summing to a target" (Combination Sum III).
4. "Find combinations that sum to target; may reuse numbers." Recurse with `i`, sort and break. Follow-up: "each number once, with duplicates in the input" (Combination Sum II): recurse with `i + 1` and skip equal siblings.
5. "Phone keypad letter combinations." One level per digit, loop over its letters. The tree has no pruning, the count is the product of letters per digit.
6. "Generate all valid bracket strings of n pairs." Track `open` and `close`; add `)` only if `close < open`. Follow-up: "Why is the count Catalan?" Be ready to say it grows roughly as 4^n.
7. "Can this be done iteratively?" For subsets, build incrementally: start with `[[]]` and for each number append it to a copy of every existing subset. Or enumerate bitmasks from 0 to 2^n - 1.

INTERVIEW ANGLE

"How do you avoid duplicate results?" Sort the input so equal values are adjacent. Then, at each level of the recursion, skip a candidate if it equals the previous candidate *at the same level* (`i > start && nums[i] == nums[i-1]`), because choosing it would reproduce a branch already explored. For permutations the same idea uses `used[i - 1]`. Follow-up: "Could a `Set<List<Integer>>` be used instead?" It is correct and simpler, but the duplicate branches are still explored, so it is slower and uses more memory; offer it as a fallback, then show the pruned version.

4.2.6 LeetCode practice

#	Title	Level	Key idea
78	Subsets	Medium	Record at every node; loop from `start` so each set appears once
46	Permutations	Medium	Loop over all indices, track `used[]` for the current path
77	Combinations	Medium	Choose k of n; prune when too few numbers remain
17	Letter Combinations of a Phone Number	Medium	One recursion level per digit, loop over its letters
22	Generate Parentheses	Medium	Add `(` while open < n and `)` while close < open
39	Combination Sum	Medium	Reuse allowed: recurse with `i`; sort and break on overshoot
216	Combination Sum III	Medium	Fixed size k and numbers 1 to 9; stop when sum or size is exceeded
90	Subsets II	Medium	Sort, skip equal siblings (`i > start`)
47	Permutations II	Medium	Sort, skip if equal to previous value that is not in use
40	Combination Sum II	Medium	Each number once with duplicates: `i + 1` plus sibling skip

LeetCode problems for this pattern.

KEY POINTS

Backtracking = depth-first search over a decision tree: choose, explore, unchoose.

- Copy the path when recording an answer; always restore state on the way back.
- `start` index for unordered selections (subsets, combinations); `used[]` for permutations.
- Sort first, then skip equal siblings (`i > start`) to remove duplicate answers.
- Prune as early as possible (sorted break, remaining-count bound, validity checks).
- Complexity is output size times cost per answer: O(n x 2^n) for subsets, O(n x n!) for permutations.

4.3 Backtracking on Grids and Constraints

The previous chapter generated objects. This one searches for a configuration that obeys *constraints*: a word traced through a letter grid, queens placed so none attack each other, digits placed in a sudoku, a string cut into palindromes. The brute-force approach (try every arrangement, then check) is infeasible: 8 queens on a 64-square board have about 4.4 billion placements. Constraint backtracking checks the rules after *every single move*, abandons a branch the moment a rule breaks, and so visits only a tiny fraction of the space.

4.3.1 The idea

The skeleton is the same as before (choose, explore, unchoose), with two additions. First, a fast **validity test** before each choice, which is the pruning. Second, the recursion often **returns a boolean**: "did this branch find a solution?" For "does a solution exist" problems (word search, sudoku) the search stops at the first success; for "find all" or "count" problems (N-Queens) it continues.

How to recognise it

- A grid and a path: "does the word exist in the grid", "paths that visit every cell once", "maximum gold that can be collected".
- Placement under rules: "place n queens", "fill the sudoku", "no two in the same row, column or diagonal".
- "Partition/cut the string so that every piece is ...": each cut point is a decision.
- Small input again (board up to 9 x 9, grid up to about 6 x 6 for hard variants, strings up to 16 characters).
- Words like "valid", "without conflicts", "all configurations".

Template

Template.java JAVA

```java
// Constraint search (fragment): returns true as soon as one complete solution exists
static boolean solve(State s) {
    if (complete(s)) return true;
    for (Move m : movesFrom(s)) {
        if (!legal(s, m)) continue;       // prune: never enter an illegal state
        apply(s, m);                      // choose, mutating shared state
        if (solve(s)) return true;        // explore (stop at the first success)
        undo(s, m);                       // unchoose
    }
    return false;                         // every move failed: tell the caller to backtrack
}
```

Grid search adds one trick: **mark the cell visited by overwriting it** (for example with `'#'`), recurse in four directions, then restore the original letter. This avoids a separate `visited` array and makes unchoose a single assignment. For queens, scanning the board for attacks is unnecessary. Three boolean arrays: occupied columns, occupied "/" diagonals (indexed by `r + c`, constant along a diagonal), and occupied "\" diagonals (indexed by `r - c + n`). Then every legality test is O(1).

4.3.2 Dry run

The trace covers N-Queens for n = 4: place one queen per row, row 0 first, and try columns left to right. A column is skipped when the column or either diagonal is already taken. Input: n = 4. The program prints one line per decision; there are 77 lines in total and the key ones are shown, with their real step numbers. `queens` lists the column chosen in each row so far (so `[1, 3, 0, 2]` means row 0 has its queen in column 1, row 1 in column 3, and so on).

Step	Position tried	queens	Result
1	row 0, col 0	`[0]`	place
2-3	row 1, cols 0 and 1	`[0]`	attacked (same column, then diagonal), skip
4	row 1, col 2	`[0, 2]`	place
5-8	row 2, cols 0 to 3	`[0, 2]`	all four attacked; row 2 has no move, back up
10	row 1, col 3	`[0, 3]`	place (the only other choice for row 1)
12	row 2, col 1	`[0, 3, 1]`	place
13-16	row 3, cols 0 to 3	`[0, 3, 1]`	all attacked, back up
18-21	rows 2 and 1 exhausted	`[0]`	no solution starts with a queen at (0, 0)
22	row 0, col 1	`[1]`	place
26	row 1, col 3	`[1, 3]`	place (cols 0 to 2 were attacked)
27	row 2, col 0	`[1, 3, 0]`	place
30	row 3, col 2	`[1, 3, 0, 2]`	place
31	row 4 reached	`[1, 3, 0, 2]`	SOLUTION

N-Queens, n = 4. Selected steps of the real trace (the rest are repeats of the same moves).

After step 31 the search continues (it must, since all solutions are required) and finds the mirror image `[2, 0, 3, 1]` at step 47, then exhausts the rest of the tree. Out of 4^4 = 256 raw placements, the whole search produced only 77 trace lines, and two solutions exist for n = 4. A dead end at step 5 cut off every arrangement of rows 3 and below that begin with `[0, 2]`: that is what pruning buys.

4.3.3 Java code

Four problems: Word Search on a small grid, N-Queens (printing boards for n = 4 and counting solutions up to n = 9), Palindrome Partitioning, and a complete Sudoku solver with a plain "first empty cell" strategy.

Main.java JAVA

```java
import java.util.*;

public class Main {
    // ---------- Word Search ----------
    static boolean exist(char[][] board, String word) {
        for (int r = 0; r < board.length; r++)
            for (int c = 0; c < board[0].length; c++)
                if (dfs(board, word, 0, r, c)) return true;
        return false;
    }
    static boolean dfs(char[][] b, String w, int k, int r, int c) {
        if (k == w.length()) return true;
        if (r < 0 || c < 0 || r >= b.length || c >= b[0].length) return false;
        if (b[r][c] != w.charAt(k)) return false;
        char saved = b[r][c];
        b[r][c] = '#';                                  // choose: mark visited
        boolean found = dfs(b, w, k + 1, r + 1, c) || dfs(b, w, k + 1, r - 1, c)
                     || dfs(b, w, k + 1, r, c + 1) || dfs(b, w, k + 1, r, c - 1);
        b[r][c] = saved;                                // unchoose
        return found;
    }

    // ---------- N-Queens ----------
    static List<List<String>> solveNQueens(int n) {
        List<List<String>> out = new ArrayList<>();
        place(n, 0, new int[n], new boolean[n], new boolean[2 * n], new boolean[2 * n], out);
        return out;
    }
    static void place(int n, int r, int[] q, boolean[] col, boolean[] d1, boolean[] d2,
                      List<List<String>> out) {
        if (r == n) {
            List<String> board = new ArrayList<>();
            for (int i = 0; i < n; i++) {
                char[] row = new char[n];
                Arrays.fill(row, '.');
                row[q[i]] = 'Q';
                board.add(new String(row));
            }
            out.add(board);
            return;
        }
        for (int c = 0; c < n; c++) {
            int a = r + c, b = r - c + n;
            if (col[c] || d1[a] || d2[b]) continue;
            col[c] = d1[a] = d2[b] = true;
            q[r] = c;
            place(n, r + 1, q, col, d1, d2, out);
            col[c] = d1[a] = d2[b] = false;
        }
    }

    // ---------- Palindrome Partitioning ----------
    static List<List<String>> partition(String s) {
        List<List<String>> out = new ArrayList<>();
        part(s, 0, new ArrayList<>(), out);
        return out;
    }
    static void part(String s, int start, List<String> path, List<List<String>> out) {
        if (start == s.length()) { out.add(new ArrayList<>(path)); return; }
        for (int end = start + 1; end <= s.length(); end++) {
            if (!isPal(s, start, end - 1)) continue;     // prune: only palindromic pieces
            path.add(s.substring(start, end));
            part(s, end, path, out);
            path.remove(path.size() - 1);
        }
    }
    static boolean isPal(String s, int i, int j) {
        while (i < j) if (s.charAt(i++) != s.charAt(j--)) return false;
        return true;
    }

    // ---------- Sudoku ----------
    static int nodes = 0;
    static boolean solveSudoku(char[][] g) {
        for (int r = 0; r < 9; r++)
```

sieram

Main.java (continued) JAVA

```java
            for (int c = 0; c < 9; c++) {
                if (g[r][c] != '.') continue;
                for (char d = '1'; d <= '9'; d++) {
                    if (!ok(g, r, c, d)) continue;
                    g[r][c] = d; nodes++;
                    if (solveSudoku(g)) return true;
                    g[r][c] = '.';
                }
                return false;                           // no digit fits this cell: dead end
            }
        return true;                                    // no empty cell left
    }
    static boolean ok(char[][] g, int r, int c, char d) {
        for (int i = 0; i < 9; i++) {
            if (g[r][i] == d || g[i][c] == d) return false;
            if (g[3 * (r / 3) + i / 3][3 * (c / 3) + i % 3] == d) return false;
        }
        return true;
    }

    public static void main(String[] args) {
        char[][] board = {
            {'A', 'B', 'C', 'E'},
            {'S', 'F', 'C', 'S'},
            {'A', 'D', 'E', 'E'}};
        System.out.println("ABCCED: " + exist(board, "ABCCED"));
        System.out.println("SEE:    " + exist(board, "SEE"));
        System.out.println("ABCB:   " + exist(board, "ABCB"));

        for (List<String> sol : solveNQueens(4)) {
            System.out.println(String.join("\n", sol));
            System.out.println();
        }
        for (int n = 1; n <= 9; n++) System.out.print(solveNQueens(n).size() + " ");
        System.out.println("<- solutions for n = 1..9");

        System.out.println(partition("aab"));

        String[] puzzle = {
            "53..7....", "6..195...", ".98....6.", "8...6...3", "4..8.3..1",
            "7...2...6", ".6....28.", "...419..5", "....8..79"};
        char[][] g = new char[9][];
        for (int i = 0; i < 9; i++) g[i] = puzzle[i].toCharArray();
        boolean solved = solveSudoku(g);
        System.out.println("solved=" + solved + " placements=" + nodes);
        for (char[] row : g) System.out.println(new String(row));
    }
}
```

Output (JDK 21) TERMINAL

```
$ java Main.java
ABCCED: true
SEE:    true
ABCB:   false
.Q..
...Q
Q...
..Q.

..Q.
Q...
...Q
.Q..

1 0 0 2 10 4 40 92 352 <- solutions for n = 1..9
[[a, a, b], [aa, b]]
solved=true placements=4208
534678912
672195348
198342567
859761423
426853791
```

```
Output (JDK 21)  (continued)                                    TERMINAL

713924856
961537284
287419635
345286179
```

What to notice

- **Word Search** (`dfs`): the bounds check precedes the letter check, and the base case `k == w.length()` comes first. The `||` chain short-circuits, so the first direction that succeeds stops the search. The restore `b[r][c] = saved` runs on both success and failure, which keeps the board intact for the next starting cell.
- **N-Queens**: the two diagonal arrays are sized `2 * n` because `r + c` ranges over 0 to 2n-2 and `r - c + n` over 1 to 2n-1. The solution counts 1, 0, 0, 2, 10, 4, 40, 92, 352 are the well-known sequence for n = 1 to 9.
- **Palindrome Partitioning**: the loop picks the *end* of the next piece; a piece is explored only if it is a palindrome (prune). For `"aab"` the two partitions are `a|a|b` and `aa|b`. For longer strings precompute `isPal` in an O(n^2) table to avoid rechecking.
- **Sudoku**: `return false` right after the digit loop is the dead-end signal, and the `g[r][c] = '.'` line is the unchoose. The `ok` test uses `3 * (r / 3) + i / 3` and `3 * (c / 3) + i % 3` to walk the 3 x 3 box that contains the cell. This puzzle required 4,208 placements. Harder puzzles need a smarter order: choose the empty cell with the fewest legal digits first (the "most constrained variable" idea), and keep row, column and box digit sets as bitmasks.

4.3.4 Complexity and mistakes

These searches are exponential in the worst case. Word Search is O(m x n x 3^L) for an L-letter word (4 directions at the first step, then at most 3 because the cell just visited is excluded), with O(L) stack. N-Queens is at worst O(n!) because each row has at most n minus (rows already placed) choices; real behaviour is far better thanks to diagonal pruning. Sudoku is bounded by 9^(empty cells) but with constraints is usually fast. Palindrome Partitioning is O(n x 2^n): up to 2^(n-1) ways to cut the string, and O(n) to copy each. Space for all of them is the recursion depth, O(L), O(n), O(81) and O(n) respectively, plus output.

Mistakes to avoid:

- **Not restoring the cell** in word search (or restoring only on the success path). The board is then corrupted for the next start.
- **Checking the bounds after reading the cell**: `b[r][c]` before `r >= 0` gives `ArrayIndexOutOfBoundsException`. Bounds first, content second.
- **Diagonal index mistakes**: using `r - c` directly gives negative indices. Offset by `n - 1` or `n`.
- **Returning too early** in an "all solutions" problem: `return true` after the first solution means N-Queens counts only one.
- **Pruning with a wrong rule**: a prune must never cut a branch that could still contain a solution. The prune should be validated on small inputs against the unpruned version.
- **Using `String +=` in the loop**: it makes a new string on every move, so prefer `StringBuilder` with `setLength` or a `List<String>` path.

WATCH OUT

In word search, if the target word is longer than the number of cells, or needs letters the grid does not have (count letters first), the search can consume exponential time before returning false. A quick frequency check up front, or searching from the rarer end of the word, are standard speed-ups for the hard variants (Word Search II).

4.3.5 Questions to expect

1. "Does the word exist in the grid by moving to adjacent cells, using each cell once?" Clue: path + no reuse. DFS with in-place marking. Follow-up: "Many words at once" is Word Search II, which uses a trie (see the chapter on tries).
2. "Place n queens so none attack." Row by row, three boolean arrays. Follow-up: "Count the solutions only" (N-Queens II): same code, `count++` at the base. Bitmask version for speed.
3. "Solve the sudoku." Fill the first empty cell with each legal digit. Follow-up: "Validate a partially filled board" is a one-pass check with three sets of seen digits.
4. "Split the string into palindromes in all ways." Choose the next piece's end, prune non-palindromes. Follow-up: "minimum cuts" is dynamic programming, not backtracking.

5. "Restore IP addresses from a digit string." Four segments of one to three digits in 0 to 255 with no leading zero (unless the segment is exactly 0): a depth-4 tree, pruning on segment validity and remaining length.
6. "Collect the maximum gold from a grid, start and stop anywhere." Backtrack from every non-zero cell, marking cells along the current path; take the best total. Same marking trick as word search.
7. "Count paths that visit every empty cell exactly once." Backtracking on a grid with a visited count; stop when the end cell is reached having used all cells.

INTERVIEW ANGLE

"How would you make the sudoku solver faster?" Three upgrades, in order of payoff: keep row, column and box constraints as bitmasks so the legality test is a couple of bit operations; always branch on the empty cell with the fewest candidates (fail early, small branching factor); and propagate forced moves (a cell with a single candidate is filled immediately). Mention that this is exactly how constraint solvers work, and that exact cover (Algorithm X, "dancing links") is the heavy-duty alternative.

4.3.6 LeetCode practice

#	Title	Level	Key idea
36	Valid Sudoku	Medium	Not backtracking, but the same row, column and box bookkeeping with sets
79	Word Search	Medium	DFS from each cell, mark visited in place, restore on return
131	Palindrome Partitioning	Medium	Choose the end of each piece; recurse only on palindromic prefixes
93	Restore IP Addresses	Medium	Four segments, each 1 to 3 digits, 0 to 255, no leading zeros
1219	Path with Maximum Gold	Medium	Backtrack from every gold cell, collect, mark, restore
51	N-Queens	Hard	Row by row with column and two diagonal arrays
52	N-Queens II	Hard	Same search, count instead of building boards
37	Sudoku Solver	Hard	First empty cell, try 1 to 9 with legality check, undo on failure
980	Unique Paths III	Hard	Grid backtracking that must cover every empty cell exactly once
212	Word Search II	Hard	Backtracking on the grid guided by a trie of all the words

LeetCode problems for this pattern.

KEY POINTS

Constraint backtracking = choose, check the rules immediately, explore, undo.

- Return a boolean for "does a solution exist"; keep going for "find all" or "count".
- Mark visited by overwriting the cell and restore it on return; always check bounds before reading the cell.
- N-Queens needs only three boolean arrays: column, `r + c`, and `r - c + n`.
- Pruning is what makes exponential search practical; choose the most constrained decision first.
- Complexity is exponential (for example O(m x n x 3^L) for word search), so the bound should be stated precisely together with the pruning rules.

sreeram

4.4 Binary Tree DFS

A binary tree is a recursive structure: a node whose left and right children are themselves binary trees (or empty). That makes recursion the natural tool, and most tree problems fall into one of a few shapes. The brute-force approach treats a tree as an array: collecting nodes into lists and then searching them, or recomputing a property (such as height) at every node from scratch, which turns an O(n) job into O(n^2) on a skewed tree.

Depth-first search (DFS) goes as deep as possible along one branch before backing up. On a tree it needs no visited set, because there are no cycles, and it uses stack space proportional to the height h of the tree: O(log n) for a balanced tree, O(n) for a chain.

4.4.1 The idea

Define a `TreeNode` with `val`, `left`, `right`. Every tree problem reduces to one question: *what does my function return for a subtree, and how are the two children's answers combined?* Three styles cover almost everything.

- **Bottom-up (postorder)**: recurse into both children first, then compute this node's answer from theirs. Height, diameter, balanced check, max path sum, invert.
- **Top-down (preorder)**: pass information *down* as parameters, such as the remaining target sum, the allowed value range, or the current depth. Path Sum, Validate BST bounds, Count Good Nodes.
- **Both**: a helper returns one thing to the parent while updating a separate global answer. Diameter and max path sum use this "return one value, record another" trick.

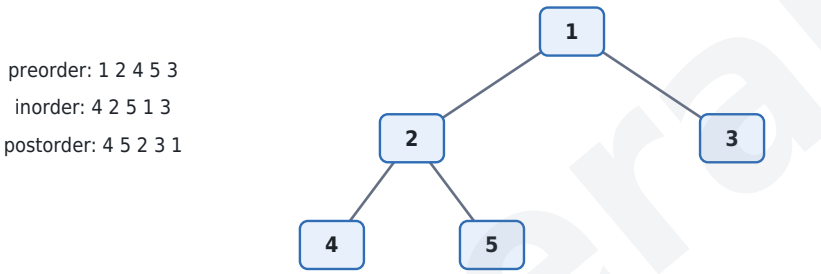


Figure 4.2. The three depth-first orders on the same tree. Preorder visits a node before its children, inorder between them, postorder after. Breadth-first (level) order is the subject of the next chapter.

How to recognise it

- The input is a `TreeNode root` and the question asks about paths, depth, symmetry, equality, or "every node satisfies ...".
- "Longest/maximum/minimum path", "height", "diameter", "balanced", "same", "subtree".
- A property of the whole tree defined by a property of its two subtrees.
- "Root to leaf" paths (top-down with an accumulator) or "any node to any node" paths (bottom-up with a global maximum).

Template

Template.java — JAVA

```java
// Bottom-up: answer for a subtree is built from the answers of its children (fragment)
static int solve(TreeNode n) {
    if (n == null) return BASE;                 // empty tree: 0 for height, true for balanced
    int l = solve(n.left);
    int r = solve(n.right);
    best = Math.max(best, combineForAnswer(n, l, r));   // optional global answer
    return combineForParent(n, l, r);                    // what the parent needs
}

// Top-down: information flows in through parameters (fragment)
static boolean solve(TreeNode n, int carried) {
    if (n == null) return BASE;
    int next = update(carried, n);
    return solve(n.left, next) && solve(n.right, next);
}
```

sreeram

The three traversal orders differ only in *when* the node is visited relative to the two recursive calls. Preorder (node, left, right) is used to copy a tree and to serialise it. Inorder (left, node, right) visits a binary search tree in sorted order. Postorder (left, right, node) is bottom-up: children are finished before their parent, which is how you delete a tree or compute heights.

4.4.2 Dry run

The example computes the **diameter** (the number of edges on the longest path between any two nodes) of the tree with root 1, children 2 and 3, and node 2 having children 4 and 5. Input: `[1, 2, 3, 4, 5]`. The longest path is 4 - 2 - 1 - 3 (or 5 - 2 - 1 - 3), which has 3 edges. The helper returns the *height* of a subtree (counted in nodes, so a leaf has height 1 and an empty subtree 0) and updates `best` with `leftHeight + rightHeight` at each node, the longest path that bends at that node. Postorder visits 4, 5, 2, 3, 1.

Step	Node	Left height	Right height	Path through node (l + r)	Returns to parent	best
1	4	0	0	0	1	0
2	5	0	0	0	1	0
3	2	1	1	2	2	2
4	3	0	0	0	1	2
5	1	2	1	3	3	3

Postorder trace of the diameter computation (heights counted in nodes, diameter in edges).

At step 3 node 2 combines the two leaves: the path 4 - 2 - 5 has 2 edges, so `best` becomes 2, but the parent can only extend *one* of the two branches, so it receives height 2 (the taller side plus itself). At step 5 the root joins the height-2 subtree on the left with the height-1 subtree on the right: 2 + 1 = 3 edges, the answer. The invariant: **the value returned upward (a straight path) differs from the value recorded (a bent path)**. Almost every "path" question on trees is this trick.

4.4.3 Java code

The listing builds trees from a LeetCode-style level-order array (`null` marks a missing node), then implements recursive and iterative traversals and the standard DFS problems. Each iterative traversal uses an explicit `ArrayDeque` as a stack, so it does not depend on JVM stack size.

Main.java JAVA

```java
import java.util.*;

public class Main {
    static class TreeNode {
        int val;
        TreeNode left, right;
        TreeNode(int val) { this.val = val; }
    }

    // Build from a level-order array where null means "no node" (LeetCode style).
    static TreeNode build(Integer... vals) {
        if (vals.length == 0 || vals[0] == null) return null;
        TreeNode root = new TreeNode(vals[0]);
        Deque<TreeNode> q = new ArrayDeque<>();
        q.add(root);
        int i = 1;
        while (!q.isEmpty() && i < vals.length) {
            TreeNode cur = q.poll();
            if (vals[i] != null) { cur.left = new TreeNode(vals[i]); q.add(cur.left); }
            i++;
            if (i < vals.length && vals[i] != null) {
                cur.right = new TreeNode(vals[i]);
                q.add(cur.right);
            }
            i++;
        }
        return root;
    }

    // ---- recursive traversals ----
```

Main.java (continued) JAVA

```
    static void pre(TreeNode n, List<Integer> out) {
        if (n == null) return;
        out.add(n.val);                 // node first
        pre(n.left, out);
        pre(n.right, out);
    }
    static void in(TreeNode n, List<Integer> out) {
        if (n == null) return;
        in(n.left, out);
        out.add(n.val);                 // node in the middle
        in(n.right, out);
    }
    static void post(TreeNode n, List<Integer> out) {
        if (n == null) return;
        post(n.left, out);
        post(n.right, out);
        out.add(n.val);                 // node last
    }

    // ---- iterative traversals ----
    static List<Integer> inorderIter(TreeNode root) {
        List<Integer> out = new ArrayList<>();
        Deque<TreeNode> stack = new ArrayDeque<>();
        TreeNode cur = root;
        while (cur != null || !stack.isEmpty()) {
            while (cur != null) { stack.push(cur); cur = cur.left; }   // go as far left as possible
            cur = stack.pop();
            out.add(cur.val);                                          // visit
            cur = cur.right;                                           // then the right subtree
        }
        return out;
    }
    static List<Integer> preorderIter(TreeNode root) {
        List<Integer> out = new ArrayList<>();
        if (root == null) return out;
        Deque<TreeNode> stack = new ArrayDeque<>();
        stack.push(root);
        while (!stack.isEmpty()) {
            TreeNode n = stack.pop();
            out.add(n.val);
            if (n.right != null) stack.push(n.right);   // right first so left is popped first
            if (n.left != null) stack.push(n.left);
        }
        return out;
    }
    static List<Integer> postorderIter(TreeNode root) {
        LinkedList<Integer> out = new LinkedList<>();
        if (root == null) return out;
        Deque<TreeNode> stack = new ArrayDeque<>();
        stack.push(root);
        while (!stack.isEmpty()) {                      // reversed "root, right, left"
            TreeNode n = stack.pop();
            out.addFirst(n.val);
            if (n.left != null) stack.push(n.left);
            if (n.right != null) stack.push(n.right);
        }
        return out;
    }

    // ---- classic DFS problems ----
    static int height(TreeNode n) {
        return n == null ? 0 : 1 + Math.max(height(n.left), height(n.right));
    }

    static int diameter;                        // longest path measured in edges
    static int diameterOf(TreeNode root) { diameter = 0; depth(root); return diameter; }
    static int depth(TreeNode n) {
        if (n == null) return 0;
        int l = depth(n.left), r = depth(n.right);
        diameter = Math.max(diameter, l + r);  // path that bends at this node
        return 1 + Math.max(l, r);             // path that continues up to the parent
    }

    static boolean sameTree(TreeNode a, TreeNode b) {
        if (a == null || b == null) return a == b;
```

Sreeram

Main.java (continued) JAVA

```java
        return a.val == b.val && sameTree(a.left, b.left) && sameTree(a.right, b.right);
    }

    static boolean hasPathSum(TreeNode n, int target) {
        if (n == null) return false;
        if (n.left == null && n.right == null) return n.val == target;   // must end at a leaf
        return hasPathSum(n.left, target - n.val) || hasPathSum(n.right, target - n.val);
    }

    static int bestPath;                      // max path sum (values may be negative)
    static int maxPathSum(TreeNode root) {
        bestPath = Integer.MIN_VALUE;
        gain(root);
        return bestPath;
    }
    static int gain(TreeNode n) {
        if (n == null) return 0;
        int l = Math.max(0, gain(n.left));     // ignore a negative branch
        int r = Math.max(0, gain(n.right));
        bestPath = Math.max(bestPath, n.val + l + r);   // bend here
        return n.val + Math.max(l, r);                  // extend upward
    }

    static TreeNode invert(TreeNode n) {
        if (n == null) return null;
        TreeNode t = n.left; n.left = invert(n.right); n.right = invert(t);
        return n;
    }

    // Balanced: returns height, or -1 if unbalanced anywhere (one pass, O(n))
    static boolean isBalanced(TreeNode root) { return check(root) != -1; }
    static int check(TreeNode n) {
        if (n == null) return 0;
        int l = check(n.left); if (l == -1) return -1;
        int r = check(n.right); if (r == -1) return -1;
        if (Math.abs(l - r) > 1) return -1;
        return 1 + Math.max(l, r);
    }

    public static void main(String[] args) {
        TreeNode t = build(1, 2, 3, 4, 5, null, 6);
        List<Integer> a = new ArrayList<>(), b = new ArrayList<>(), c = new ArrayList<>();
        pre(t, a); in(t, b); post(t, c);
        System.out.println("preorder  " + a + " iter " + preorderIter(t));
        System.out.println("inorder   " + b + " iter " + inorderIter(t));
        System.out.println("postorder " + c + " iter " + postorderIter(t));
        System.out.println("height = " + height(t));
        System.out.println("diameter = " + diameterOf(build(1, 2, 3, 4, 5)));
        System.out.println("same tree = " + sameTree(build(1, 2, 3), build(1, 2, 3))
                + ", " + sameTree(build(1, 2), build(1, null, 2)));
        TreeNode ps = build(5, 4, 8, 11, null, 13, 4, 7, 2, null, null, null, 1);
        System.out.println("hasPathSum 22 = " + hasPathSum(ps, 22) + ", 9 = " + hasPathSum(ps, 9));
        System.out.println("maxPathSum = " + maxPathSum(build(-10, 9, 20, null, null, 15, 7)));
        List<Integer> inv = new ArrayList<>();
        pre(invert(build(4, 2, 7, 1, 3, 6, 9)), inv);
        System.out.println("inverted preorder = " + inv);
        System.out.println("balanced: " + isBalanced(build(3, 9, 20, null, null, 15, 7))
                + ", " + isBalanced(build(1, 2, 2, 3, 3, null, null, 4, 4)));
        // skewed tree: recursion depth equals n
        TreeNode skew = new TreeNode(0), cur = skew;
        for (int i = 1; i < 100_000; i++) { cur.right = new TreeNode(i); cur = cur.right; }
        System.out.println("iterative inorder on 100000-node chain: " + inorderIter(skew).size());
        try { height(skew); }
        catch (StackOverflowError e) { System.out.println("recursive height overflowed"); }
    }
}
```

Output (JDK 21) TERMINAL

```
$ java Main.java
preorder  [1, 2, 4, 5, 3, 6] iter [1, 2, 4, 5, 3, 6]
inorder   [4, 2, 5, 1, 3, 6] iter [4, 2, 5, 1, 3, 6]
postorder [4, 5, 2, 6, 3, 1] iter [4, 5, 2, 6, 3, 1]
height = 3
```

Output (JDK 21) (continued) TERMINAL

```
diameter = 3
same tree = true, false
hasPathSum 22 = true, 9 = false
maxPathSum = 42
inverted preorder = [4, 7, 9, 6, 2, 3, 1]
balanced: true, false
iterative inorder on 100000-node chain: 100000
recursive height overflowed
```

What to notice

- **Iterative inorder**: push the whole left spine, pop and visit, then move to the right child. The stack always holds the ancestors whose right subtrees are still pending.
- **Iterative preorder** pushes the right child first so the left is popped first. **Iterative postorder** is the trick of producing "node, right, left" and reversing it (here with `addFirst`).
- `hasPathSum` tests `n.left == null && n.right == null` before comparing. Without the leaf test, a tree `[1, 2]` with target 1 would wrongly return true, because node 1 is not a leaf.
- `maxPathSum` clamps child gains at 0 (`Math.max(0, gain(...))`): a branch with a negative sum is better dropped. `bestPath` starts at `Integer.MIN_VALUE` because all values may be negative. The result 42 is the path 15 - 20 - 7.
- `isBalanced` returns the height or a sentinel -1 so the check is a single O(n) pass. The naive version calls `height` at every node and is O(n^2) on a skewed tree.
- `sameTree`: `a == b` after the null test is true only if both are null. One null and one non-null is false.
- The chain of 100,000 nodes is handled by the iterative traversal, while recursive `height` overflowed the stack on the build machine (the exact limit depends on JVM settings and frame size).

4.4.4 Complexity and mistakes

Each node is visited once and does O(1) work, so time is **O(n)** for all these problems. Space is the recursion depth: **O(h)**, where h = log n for a balanced tree and n for a chain. The iterative versions use O(h) for the explicit stack. Morris traversal achieves O(1) extra space by temporarily rewiring pointers; mention it, but do not volunteer to code it unless asked.

Mistakes to avoid:

- **Forgetting the null base case**, or putting the leaf test where only null is handled, producing `NullPointerException` on `n.left.val`.
- **Confusing height and depth, nodes and edges.** Fix the convention first: LeetCode's max depth counts nodes; diameter counts edges. The helper must state which one it returns.
- **Returning the bent path to the parent** in diameter or max path sum. A parent can extend only one child's branch.
- **Using `int` global state across calls**: static fields such as `best` must be reset per query (see `diameterOf`), or LeetCode test cases will leak into each other.
- **Comparing `Integer` objects with `==`**, for example `a.val == b.val` when `val` is declared `Integer`. Use `int`, or `.equals`.
- **Validating a BST by comparing only parent and child.** Each node must respect bounds inherited from all ancestors (see the next-but-one chapter).

GOTCHA

A "path" in these problems is a sequence of connected nodes that does not have to start at the root or end at a leaf (Diameter, Max Path Sum), but in Path Sum I and II it must run from the root to a leaf. The definition determines the code.

Sreeram

4.4.5 Questions to expect

1. "Print the tree inorder/preorder/postorder, then do it without recursion." Know all six versions. Follow-up: "Morris traversal for O(1) space."
2. "Find the maximum depth / minimum depth." Max: `1 + max(l, r)`. Minimum depth has a subtle precondition: a node with one child is not a leaf, so use `1 + min(l, r)` only when both children exist (or use BFS).
3. "Is this tree height-balanced?" Bottom-up with a -1 sentinel; mention the O(n^2) top-down version and why it is worse.
4. "Diameter of a binary tree." Return height, record `l + r`. Follow-up: "weighted edges" - same code with weights.
5. "Maximum path sum where nodes can be negative." Clamp child gains at 0, record `val + l + r`, return `val + max(l, r)`.
6. "Do two trees match? Is one a subtree of the other? Is a tree symmetric?" `sameTree` as a building block; symmetric compares `a.left` with `b.right` and `a.right` with `b.left`.
7. "Invert a binary tree." Swap children recursively (or with a queue). Mention the anecdote only if asked; the code is four lines.
8. "Root-to-leaf path sums, return all paths." Backtracking on a path list: add, recurse, remove (same choose/explore/unchoose rhythm as the earlier chapters).

INTERVIEW ANGLE

"Walk me through recursion on a tree and tell me its space complexity." Each call handles one node and delegates the subtrees, so the maximum stack depth equals the tree height h. A balanced tree gives O(log n), a degenerate chain gives O(n). For inputs that may form a long chain, an iterative traversal with an explicit stack avoids `StackOverflowError`.

4.4.6 LeetCode practice

#	Title	Level	Key idea
104	Maximum Depth of Binary Tree	Easy	`1 + max(left, right)` bottom-up
100	Same Tree	Easy	Compare values and recurse on both pairs of children
226	Invert Binary Tree	Easy	Swap the children at every node
94	Binary Tree Inorder Traversal	Easy	Recursive, then iterative with a stack (left spine)
144	Binary Tree Preorder Traversal	Easy	Stack with the right child pushed first
101	Symmetric Tree	Easy	Mirror comparison: left with right, right with left
110	Balanced Binary Tree	Easy	Return height or -1 in one pass
543	Diameter of Binary Tree	Easy	Return height, record left + right
572	Subtree of Another Tree	Easy	For each node, run Same Tree
112	Path Sum	Easy	Subtract down the path, test only at leaves
113	Path Sum II	Medium	Backtracking path list with copy at the leaf
437	Path Sum III	Medium	Prefix sums in a hash map along the root path
124	Binary Tree Maximum Path Sum	Hard	Clamp gains at 0, bend at every node

LeetCode problems for this pattern.

KEY POINTS

Tree problems are recursion on subtrees: decide what each call returns and how to combine two children.

- Bottom-up (postorder) for properties built from children; top-down (preorder) for information passed down as parameters.
- For path problems return the best *straight* path and record the best *bent* path in a global.
- O(n) time, O(h) stack space; use an explicit stack when the tree may be a long chain.
- Check the leaf definition (`left == null && right == null`) and nodes-versus-edges counting before coding.

sreeram

4.5 Binary Tree BFS and Level Order

Some questions are about *levels*: "the nodes at each depth", "the rightmost visible node", "the shallowest leaf", "connect each node to its right neighbour". Depth-first search can answer them with extra bookkeeping (a depth parameter and lists indexed by depth), but breadth-first search (BFS) fits the question exactly: it visits every node of depth d before any node of depth d + 1. It uses a queue, a first-in-first-out line, to remember the nodes that are waiting.

4.5.1 The idea

Put the root in a queue. Repeat: take the node at the front, process it, and add its non-null children at the back. Because children go to the back, all nodes of one level are in front of all nodes of the next. To work *level by level*, freeze the queue's size at the start of each round: that many nodes belong to the current level, and anything added during the round belongs to the next.

How to recognise it

- "Level order", "by level", "row by row", "zigzag".
- "Left view", "right side view", "the last/first node on each level".
- "Minimum depth", "shortest distance to a leaf": BFS stops at the first leaf it meets, so it never explores the deeper parts of the tree.
- "Connect nodes at the same level", "average of each level", "largest value in each row".
- The same technique on a grid or graph (shortest path in an unweighted graph) is covered in the graphs Part; the tree version is the gentle introduction.

Template

Template.java — JAVA

```java
// Level-order BFS skeleton (fragment)
Deque<TreeNode> q = new ArrayDeque<>();
if (root != null) q.offer(root);
while (!q.isEmpty()) {
    int size = q.size();                // number of nodes on this level - freeze it
    for (int i = 0; i < size; i++) {
        TreeNode n = q.poll();          // front of the queue
        // process n (add to this level's list, check i == size - 1, ...)
        if (n.left != null)  q.offer(n.left);
        if (n.right != null) q.offer(n.right);
    }
    // finish the level
}
```

4.5.2 Dry run

Input tree `[3, 9, 20, null, null, 15, 7]`: root 3, with children 9 and 20, and 20 has children 15 and 7. The goal is the level order `[[3], [9, 20], [15, 7]]`. The table shows the queue before each round, the nodes taken, and the queue after (real output of `levelOrderTrace` in the listing below).

Round	size	Queue before	Nodes polled (level)	Queue after
1	1	[3]	[3]	[9, 20]
2	2	[9, 20]	[9, 20]	[15, 7]
3	2	[15, 7]	[15, 7]	[]

Level-order trace. The queue is shown front to back.

Round 1 takes 3 and appends its two children. Round 2 freezes `size = 2` and takes both 9 and 20: node 9 has no children, node 20 appends 15 and 7. Round 3 takes 15 and 7, which have no children, so the queue empties and the loop ends. The frozen `size` is what separates the levels: without it the result is one flat list `[3, 9, 20, 15, 7]`, which is correct as a plain BFS order but not as a list of levels.

4.5.3 Java code

The program implements level order (with the trace), zigzag, right side view, minimum depth, connecting next pointers with O(1) extra space, and serialise/deserialise using BFS with `null` markers.

Main.java JAVA

```java
import java.util.*;

public class Main {
    static class TreeNode {
        int val;
        TreeNode left, right, next;
        TreeNode(int val) { this.val = val; }
    }

    static TreeNode build(Integer... vals) {
        if (vals.length == 0 || vals[0] == null) return null;
        TreeNode root = new TreeNode(vals[0]);
        Deque<TreeNode> q = new ArrayDeque<>();
        q.add(root);
        int i = 1;
        while (!q.isEmpty() && i < vals.length) {
            TreeNode cur = q.poll();
            if (vals[i] != null) { cur.left = new TreeNode(vals[i]); q.add(cur.left); }
            i++;
            if (i < vals.length && vals[i] != null) {
                cur.right = new TreeNode(vals[i]);
                q.add(cur.right);
            }
            i++;
        }
        return root;
    }

    static String show(Deque<TreeNode> q) {
        StringBuilder sb = new StringBuilder("[");
        for (TreeNode n : q) sb.append(sb.length() > 1 ? ", " : "").append(n.val);
        return sb.append("]").toString();
    }

    // Level order with a trace of the queue (for the dry run)
    static List<List<Integer>> levelOrderTrace(TreeNode root) {
        List<List<Integer>> res = new ArrayList<>();
        if (root == null) return res;
        Deque<TreeNode> q = new ArrayDeque<>();
        q.offer(root);
        int round = 0;
        while (!q.isEmpty()) {
            int size = q.size();                        // freeze the number of nodes on this level
            String before = show(q);
            List<Integer> level = new ArrayList<>();
            for (int i = 0; i < size; i++) {
                TreeNode n = q.poll();
                level.add(n.val);
                if (n.left != null) q.offer(n.left);
                if (n.right != null) q.offer(n.right);
            }
            System.out.println(++round + " | size=" + size + " | " + before + " | " + level
                    + " | " + show(q));
            res.add(level);
        }
        return res;
    }

    static List<List<Integer>> levelOrder(TreeNode root) {
        List<List<Integer>> res = new ArrayList<>();
        if (root == null) return res;
        Deque<TreeNode> q = new ArrayDeque<>();
        q.offer(root);
        while (!q.isEmpty()) {
            int size = q.size();
            List<Integer> level = new ArrayList<>();
            for (int i = 0; i < size; i++) {
                TreeNode n = q.poll();
                level.add(n.val);
```

Main.java (continued) JAVA

```
                if (n.left != null) q.offer(n.left);
                if (n.right != null) q.offer(n.right);
            }
            res.add(level);
        }
        return res;
    }

    static List<List<Integer>> zigzag(TreeNode root) {
        List<List<Integer>> res = new ArrayList<>();
        if (root == null) return res;
        Deque<TreeNode> q = new ArrayDeque<>();
        q.offer(root);
        boolean leftToRight = true;
        while (!q.isEmpty()) {
            int size = q.size();
            LinkedList<Integer> level = new LinkedList<>();
            for (int i = 0; i < size; i++) {
                TreeNode n = q.poll();
                if (leftToRight) level.addLast(n.val); else level.addFirst(n.val);
                if (n.left != null) q.offer(n.left);
                if (n.right != null) q.offer(n.right);
            }
            res.add(level);
            leftToRight = !leftToRight;
        }
        return res;
    }

    static List<Integer> rightSideView(TreeNode root) {
        List<Integer> view = new ArrayList<>();
        if (root == null) return view;
        Deque<TreeNode> q = new ArrayDeque<>();
        q.offer(root);
        while (!q.isEmpty()) {
            int size = q.size();
            for (int i = 0; i < size; i++) {
                TreeNode n = q.poll();
                if (i == size - 1) view.add(n.val);      // last node of the level
                if (n.left != null) q.offer(n.left);
                if (n.right != null) q.offer(n.right);
            }
        }
        return view;
    }

    static int minDepth(TreeNode root) {
        if (root == null) return 0;
        Deque<TreeNode> q = new ArrayDeque<>();
        q.offer(root);
        int depth = 1;
        while (!q.isEmpty()) {
            for (int i = q.size(); i > 0; i--) {
                TreeNode n = q.poll();
                if (n.left == null && n.right == null) return depth;   // first leaf = shallowest
                if (n.left != null) q.offer(n.left);
                if (n.right != null) q.offer(n.right);
            }
            depth++;
        }
        return depth;
    }

    // Connect next pointers on ANY binary tree, O(1) extra space (uses the next links themselves)
    static void connect(TreeNode root) {
        TreeNode levelStart = root;
        while (levelStart != null) {
            TreeNode dummy = new TreeNode(0), tail = dummy;
            for (TreeNode n = levelStart; n != null; n = n.next) {
                if (n.left != null)  { tail.next = n.left;  tail = tail.next; }
                if (n.right != null) { tail.next = n.right; tail = tail.next; }
            }
            levelStart = dummy.next;          // the first node of the next level
        }
    }
```

sreeram

Main.java (continued) JAVA

```java
    // Serialize / deserialize with BFS and "null" markers (LinkedList allows null elements)
    static String serialize(TreeNode root) {
        List<String> out = new ArrayList<>();
        Deque<TreeNode> q = new LinkedList<>();
        q.add(root);
        while (!q.isEmpty()) {
            TreeNode n = q.poll();
            if (n == null) { out.add("null"); continue; }
            out.add(String.valueOf(n.val));
            q.add(n.left);
            q.add(n.right);
        }
        int end = out.size();
        while (end > 0 && out.get(end - 1).equals("null")) end--;   // trim trailing nulls
        return String.join(",", out.subList(0, end));
    }

    static TreeNode deserialize(String data) {
        if (data.isEmpty()) return null;
        String[] p = data.split(",");
        TreeNode root = new TreeNode(Integer.parseInt(p[0]));
        Deque<TreeNode> q = new ArrayDeque<>();
        q.add(root);
        int i = 1;
        while (!q.isEmpty() && i < p.length) {
            TreeNode n = q.poll();
            if (!p[i].equals("null")) {
                n.left = new TreeNode(Integer.parseInt(p[i]));
                q.add(n.left);
            }
            i++;
            if (i < p.length && !p[i].equals("null")) {
                n.right = new TreeNode(Integer.parseInt(p[i]));
                q.add(n.right);
            }
            i++;
        }
        return root;
    }

    public static void main(String[] args) {
        TreeNode t = build(3, 9, 20, null, null, 15, 7);
        System.out.println(levelOrderTrace(t));
        System.out.println("zigzag  = " + zigzag(build(1, 2, 3, 4, 5, 6, 7)));
        System.out.println("right   = " + rightSideView(build(1, 2, 3, null, 5, null, 4)));
        System.out.println("minDepth = " + minDepth(build(2, null, 3, null, 4, null, 5, null, 6)));
        TreeNode c = build(1, 2, 3, 4, 5, null, 7);
        connect(c);
        System.out.println("next of 4 = " + c.left.left.next.val + ", next of 5 = "
                + c.left.right.next.val + ", next of 7 = " + c.right.right.next);
        String data = serialize(build(1, 2, 3, null, null, 4, 5));
        System.out.println("serialized = " + data);
        System.out.println("round trip = " + serialize(deserialize(data)));
        System.out.println("empty tree = '" + serialize(null) + "'");
    }
}
```

Output (JDK 21) TERMINAL

```
$ java Main.java
1 | size=1 | [3] | [3] | [9, 20]
2 | size=2 | [9, 20] | [9, 20] | [15, 7]
3 | size=2 | [15, 7] | [15, 7] | []
[[3], [9, 20], [15, 7]]
zigzag  = [[1], [3, 2], [4, 5, 6, 7]]
right   = [1, 3, 4]
minDepth = 5
next of 4 = 5, next of 5 = 7, next of 7 = null
serialized = 1,2,3,null,null,4,5
round trip = 1,2,3,null,null,4,5
empty tree = ''
```

What to notice

- **Use `ArrayDeque` as the queue**, not `Stack` and not a hand-rolled array. `offer` adds at the tail, `poll` removes from the head. `ArrayDeque` does not accept `null` elements, which is why `serialize` uses a `LinkedList` (it allows `null`) to carry the "missing child" markers.
- **Zigzag** is level order plus a boolean that flips each level; `LinkedList.addFirst` writes right-to-left levels without a reverse step. Only the storage order of each level's values changes, not the traversal order.
- **Right side view**: `i == size - 1` picks the last node of each level. The left-view variant tests `i == 0`.
- **Min depth** returns as soon as it polls a node with no children. That is the shallowest leaf, and BFS finds it without visiting deeper levels. In the skewed example `[2, null, 3, null, 4, null, 5, null, 6]` the answer is 5, because a node with only one child is not a leaf.
- `connect` builds the `next` links of the *next* level while walking the current level through its own `next` links, so it needs no queue (O(1) space). A dummy head makes appending uniform. It works for any binary tree, not only perfect ones.
- **Serialise** the tree to `1,2,3,null,null,4,5`: this is the same format LeetCode uses, trailing `null`s trimmed. **Deserialise** reads the string in the same order, giving each dequeued parent the next two tokens as its children. The empty tree becomes the empty string.

4.5.4 Complexity and mistakes

Time is O(n): every node enters and leaves the queue once. Space is O(w), where w is the maximum width of the tree (the largest number of nodes on one level). For a perfect binary tree w is about n/2, so the queue is O(n); for a chain it is O(1). DFS needs O(h) stack, so the better choice depends on the shape: wide and shallow trees favour DFS for space, narrow and deep trees favour BFS.

Mistakes to avoid:

- **Not freezing the size.** Writing `for (int i = 0; i < q.size(); i++)` re-evaluates `q.size()` while children are being added, mixing levels.
- **Offering `null` into an `ArrayDeque`** causes `NullPointerException`. Test children before offering, or use `LinkedList` deliberately.
- **Forgetting the empty-tree guard**: `q.offer(null)` crashes; check `root == null` first.
- **Using `q.remove()` or `q.element()` on an empty queue** throws; `poll()` and `peek()` return `null` instead. Pick one family and know the difference.
- **Min depth by recursion with `1 + min(l, r)`** gives the wrong answer when one child is missing (the missing side counts as depth 0). BFS avoids this entirely.
- **Reversing the wrong thing in zigzag.** Alternating the direction of insertion into the level list is correct; alternating the order of enqueueing breaks the next level.

GOTCHA

Inserting at the front of an `ArrayList` (`level.add(0, x)`) shifts every existing element, so zigzag built that way costs O(k^2) per level of k nodes. `LinkedList.addFirst` is O(1). Alternatively, collect left to right and call `Collections.reverse(level)` on the odd levels, which is O(k).

4.5.5 Questions to expect

1. "Return the level order traversal." The template above. Follow-up: "bottom-up order" (Level Order II): reverse the list of levels at the end, or `addFirst` each level.
2. "Zigzag level order." Alternate the direction of storing values per level.
3. "What does the tree look like from the right/left side?" Last/first node of each level. DFS alternative: visit right first and record the first node seen at each new depth.
4. "Find the minimum depth." BFS returns on the first leaf; mention why DFS needs care.
5. "Connect each node to its next right node." Perfect tree: use the parent's `next`. General tree: the dummy-head level walk. Follow-up: "constant extra space" - this is the point of the question.
6. "Average / maximum / sum of each level." Compute inside the inner loop; use `long` for sums to avoid overflow with large values.

Sriram

7. "Serialise and deserialise a binary tree." Select a format (BFS with markers, or preorder with markers) and justify how the end of each subtree is determined. Follow-up: "For a BST, can the markers be omitted?" Yes, preorder values alone are enough because the order property fixes the shape.

INTERVIEW ANGLE
"BFS or DFS for this tree problem?" The choice follows from the question: level-based answers (views, width, minimum depth, level aggregates) point to BFS; path-based or subtree-based answers (diameter, path sum, validity of a property) point to DFS. Then state the space: BFS O(width), DFS O(height). The preferred traversal is the one whose auxiliary memory is smaller for the expected tree shape.

4.5.6 LeetCode practice

#	Title	Level	Key idea
102	Binary Tree Level Order Traversal	Medium	Queue with the frozen level size
107	Binary Tree Level Order Traversal II	Medium	Level order, then reverse the list of levels
637	Average of Levels in Binary Tree	Easy	Sum each level in a `long`, divide by its size
111	Minimum Depth of Binary Tree	Easy	BFS stops at the first leaf
199	Binary Tree Right Side View	Medium	Take the last node of every level
103	Binary Tree Zigzag Level Order Traversal	Medium	Flip the insertion direction each level
515	Find Largest Value in Each Tree Row	Medium	Track the maximum within each level
116	Populating Next Right Pointers in Each Node	Medium	Perfect tree: link using the parent's `next`
117	Populating Next Right Pointers in Each Node II	Medium	Any tree: build the next level's links with a dummy head
297	Serialize and Deserialize Binary Tree	Hard	BFS or preorder with null markers; parse in the same order

LeetCode problems for this pattern.

KEY POINTS
Level-by-level questions are BFS: queue plus a frozen `size` per round.

- `ArrayDeque` with `offer` and `poll`; guard `root == null`; never offer `null` to an `ArrayDeque`.
- Right/left view = last/first of each level; zigzag = alternate insertion direction; min depth = first leaf.
- Time O(n), space O(maximum width); DFS is O(height), so pick by tree shape.
- Next-pointer connection can be done in O(1) space by walking the previous level's links.

4.6 BST and Tree Construction

A **binary search tree (BST)** is a binary tree with an ordering rule: for every node, all values in its left subtree are smaller and all values in its right subtree are larger (the strict version used by LeetCode has no duplicates). That one rule makes search, insert and delete O(h): at every node roughly half of the remaining tree can be discarded, as in binary search on a sorted array. The brute-force view, scanning all n nodes, discards that structure.

This chapter covers three connected skills: using the ordering (validate, k-th smallest, lowest common ancestor, iterator), rebuilding a tree from its traversals, and building a balanced BST from sorted data.

4.6.1 The idea

The central fact: **the inorder traversal of a BST is the sorted sequence.** Most BST questions reduce to that sentence. "k-th smallest" is the k-th element of an inorder walk. "Validate" means the inorder sequence is strictly increasing. A BST iterator is an inorder walk paused after each element.

A BST and where the lowest common ancestor (LCA) sits

```
        5
      /   \
     3     7        inorder: 2 3 4 5 6 7 8   (sorted)
    / \   / \
   2   4 6   8      LCA(2, 4) = 3    LCA(2, 8) = 5    LCA(3, 4) = 3
```

How to recognise it

- "Binary search tree" in the statement: use the ordering instead of visiting everything.
- "k-th smallest/largest", "sorted order", "next smaller element": inorder (reverse inorder for largest).
- "Validate a BST": propagate bounds, or check that inorder is increasing.
- "Lowest common ancestor": in a BST walk down; in a general tree recurse on both sides.
- "Construct a tree from preorder and inorder (or postorder and inorder)": the first preorder element is the root, and its position in inorder splits left from right.
- "Convert a sorted array/list to a balanced BST": pick the middle as the root.

Template

Template.java JAVA

```java
// BST search: discard half the tree at every step (fragment)
TreeNode cur = root;
while (cur != null && cur.val != target) cur = target < cur.val ? cur.left : cur.right;
```

Validate.java JAVA

```java
// Validate with bounds: each node must lie strictly inside (lo, hi) (fragment)
static boolean valid(TreeNode n, long lo, long hi) {
    if (n == null) return true;
    if (n.val <= lo || n.val >= hi) return false;
    return valid(n.left, lo, n.val) && valid(n.right, n.val, hi);
}
```

For construction the template is a sentence: take `root = preorder[next++]`, find `mid`, the root's index in the inorder array, then build the left subtree from inorder `[lo, mid-1]` and the right from `[mid+1, hi]`, in that order.

Why not just compare each node with its children? Because the rule applies to the whole subtree. The tree `5 -> (1, 6 -> (3, 7))` has every parent-child pair in order, yet 3 sits in the right subtree of 5 and is smaller than 5, so it is not a BST. Passing down the allowed range `(lo, hi)` catches this.

4.6.2 Dry run

Build the tree from `preorder = [3, 9, 20, 15, 7]` and `inorder = [9, 3, 15, 20, 7]`. Rule: the next unused preorder value is the root of the current inorder range. Find it in the inorder array (a hash map gives the index in O(1)); everything to its left in that range is the left subtree, everything to its right is the right subtree. Recurse on the left first, because preorder lists the left subtree before the right. The table is the real trace of `construct`.

Step	Root (next preorder)	Inorder range	Split index	Meaning
1	3	[0, 4]	1	left part [0, 0] (size 1), right part [2, 4] (size 3)
2	9	[0, 0]	0	one element, both sides empty: leaf 9 (left child of 3)
3	20	[2, 4]	3	left part [2, 2], right part [4, 4]: 20 is the right child of 3
4	15	[2, 2]	2	leaf 15 (left child of 20)
5	7	[4, 4]	4	leaf 7 (right child of 20)

Constructing the tree from preorder and inorder.

The tree is 3 with children 9 and 20, and 20 has children 15 and 7. The shared counter `pre` advances through the preorder array exactly once, which is why the left recursion must run *before* the right one: the preorder array lists the whole left subtree before any right-subtree value. Swap the two calls and you build garbage.

4.6.3 Java code

The listing contains: BST validation with `long` bounds, k-th smallest by early-exit iterative inorder, LCA for a BST (iterative) and for a general tree (recursive), construction from preorder and inorder, a BST iterator, a sorted array turned into a height-balanced BST, and BST insert and delete.

Main.java JAVA

```
import java.util.*;

public class Main {
    static class TreeNode {
        int val;
        TreeNode left, right;
        TreeNode(int val) { this.val = val; }
    }

    static TreeNode build(Integer... vals) {
        if (vals.length == 0 || vals[0] == null) return null;
        TreeNode root = new TreeNode(vals[0]);
        Deque<TreeNode> q = new ArrayDeque<>();
        q.add(root);
        int i = 1;
        while (!q.isEmpty() && i < vals.length) {
            TreeNode cur = q.poll();
            if (vals[i] != null) { cur.left = new TreeNode(vals[i]); q.add(cur.left); }
            i++;
            if (i < vals.length && vals[i] != null) {
                cur.right = new TreeNode(vals[i]);
                q.add(cur.right);
            }
            i++;
        }
        return root;
    }

    static void inorder(TreeNode n, List<Integer> out) {
        if (n == null) return;
        inorder(n.left, out);
        out.add(n.val);
        inorder(n.right, out);
    }

    // ---- validate BST: every node must lie inside the open interval (lo, hi) ----
    static boolean isValidBST(TreeNode n, long lo, long hi) {
        if (n == null) return true;
        if (n.val <= lo || n.val >= hi) return false;
        return isValidBST(n.left, lo, n.val) && isValidBST(n.right, n.val, hi);
    }

    // ---- kth smallest: stop the iterative inorder after k pops ----
    static int kthSmallest(TreeNode root, int k) {
        Deque<TreeNode> stack = new ArrayDeque<>();
        TreeNode cur = root;
        while (cur != null || !stack.isEmpty()) {
            while (cur != null) { stack.push(cur); cur = cur.left; }
            cur = stack.pop();
            if (--k == 0) return cur.val;
            cur = cur.right;
        }
        throw new IllegalArgumentException("k out of range");
    }

    // ---- LCA in a BST: walk down using the ordering ----
    static TreeNode lcaBST(TreeNode root, int p, int q) {
        TreeNode cur = root;
        while (cur != null) {
            if (p < cur.val && q < cur.val) cur = cur.left;
            else if (p > cur.val && q > cur.val) cur = cur.right;
            else return cur;                        // paths diverge here (or one equals cur)
        }
        return null;
    }

    // ---- LCA in a general binary tree ----
    static TreeNode lca(TreeNode n, TreeNode p, TreeNode q) {
```

Main.java (continued) JAVA

```
        if (n == null || n == p || n == q) return n;
        TreeNode l = lca(n.left, p, q), r = lca(n.right, p, q);
        return (l != null && r != null) ? n : (l != null ? l : r);
    }
    static TreeNode find(TreeNode n, int v) {
        if (n == null || n.val == v) return n;
        TreeNode l = find(n.left, v);
        return l != null ? l : find(n.right, v);
    }

    // ---- build from preorder + inorder ----
    static int pre;                                     // next index in preorder
    static Map<Integer, Integer> pos;                   // value -> index in inorder
    static TreeNode buildTree(int[] preorder, int[] inorder, boolean trace) {
        pos = new HashMap<>();
        for (int i = 0; i < inorder.length; i++) pos.put(inorder[i], i);
        pre = 0;
        return construct(preorder, 0, inorder.length - 1, trace);
    }
    static TreeNode construct(int[] preorder, int lo, int hi, boolean trace) {
        if (lo > hi) return null;
        int rootVal = preorder[pre++];
        int mid = pos.get(rootVal);
        if (trace) System.out.println("root=" + rootVal + " | inorder range [" + lo + ", " + hi
                + "] | split at " + mid + " | left size " + (mid - lo)
                + ", right size " + (hi - mid));
        TreeNode node = new TreeNode(rootVal);
        node.left = construct(preorder, lo, mid - 1, trace);     // left BEFORE right
        node.right = construct(preorder, mid + 1, hi, trace);
        return node;
    }

    // ---- BST iterator (amortised O(1) next, O(h) memory) ----
    static class BSTIterator {
        private final Deque<TreeNode> stack = new ArrayDeque<>();
        BSTIterator(TreeNode root) { pushLeft(root); }
        private void pushLeft(TreeNode n) { for (; n != null; n = n.left) stack.push(n); }
        boolean hasNext() { return !stack.isEmpty(); }
        int next() {
            TreeNode n = stack.pop();
            pushLeft(n.right);
            return n.val;
        }
    }

    // ---- sorted array -> height-balanced BST ----
    static TreeNode sortedArrayToBST(int[] a, int lo, int hi) {
        if (lo > hi) return null;
        int mid = lo + (hi - lo) / 2;
        TreeNode n = new TreeNode(a[mid]);
        n.left = sortedArrayToBST(a, lo, mid - 1);
        n.right = sortedArrayToBST(a, mid + 1, hi);
        return n;
    }

    // ---- BST search / insert / delete ----
    static TreeNode insert(TreeNode n, int v) {
        if (n == null) return new TreeNode(v);
        if (v < n.val) n.left = insert(n.left, v); else n.right = insert(n.right, v);
        return n;
    }
    static TreeNode delete(TreeNode n, int key) {
        if (n == null) return null;
        if (key < n.val) n.left = delete(n.left, key);
        else if (key > n.val) n.right = delete(n.right, key);
        else {
            if (n.left == null) return n.right;          // zero or one child: splice out
            if (n.right == null) return n.left;
            TreeNode succ = n.right;                     // two children: smallest of right subtree
            while (succ.left != null) succ = succ.left;
            n.val = succ.val;
            n.right = delete(n.right, succ.val);
        }
        return n;
    }
```

Main.java (continued) JAVA

```
    static List<Integer> preorder(TreeNode n) {
        List<Integer> out = new ArrayList<>();
        Deque<TreeNode> st = new ArrayDeque<>();
        if (n != null) st.push(n);
        while (!st.isEmpty()) {
            TreeNode x = st.pop();
            out.add(x.val);
            if (x.right != null) st.push(x.right);
            if (x.left != null) st.push(x.left);
        }
        return out;
    }

    public static void main(String[] args) {
        TreeNode bst = build(5, 3, 7, 2, 4, 6, 8);
        System.out.println("valid BST? " + isValidBST(bst, Long.MIN_VALUE, Long.MAX_VALUE));
        TreeNode trap = build(5, 1, 6, null, null, 3, 7);        // 3 is in the right subtree of 5
        System.out.println("trap tree valid? " + isValidBST(trap, Long.MIN_VALUE, Long.MAX_VALUE));
        TreeNode edge = build(Integer.MAX_VALUE);
        System.out.println("MAX_VALUE alone valid? "
                + isValidBST(edge, Long.MIN_VALUE, Long.MAX_VALUE));
        System.out.println("kth smallest k=3: " + kthSmallest(bst, 3));
        System.out.println("LCA BST (2, 4) = " + lcaBST(bst, 2, 4).val
                + ", (2, 8) = " + lcaBST(bst, 2, 8).val);
        System.out.println("LCA tree (4, 8) = " + lca(bst, find(bst, 4), find(bst, 8)).val);

        int[] preorder = {3, 9, 20, 15, 7}, inorderArr = {9, 3, 15, 20, 7};
        TreeNode built = buildTree(preorder, inorderArr, true);
        List<Integer> chk = new ArrayList<>();
        inorder(built, chk);
        System.out.println("rebuilt inorder = " + chk + ", preorder = " + preorder(built));

        BSTIterator it = new BSTIterator(bst);
        StringBuilder sb = new StringBuilder();
        while (it.hasNext()) sb.append(it.next()).append(' ');
        System.out.println("iterator: " + sb.toString().trim());

        TreeNode fromArr = sortedArrayToBST(new int[]{-10, -3, 0, 5, 9}, 0, 4);
        System.out.println("sorted array -> BST preorder = " + preorder(fromArr));

        TreeNode t = null;
        for (int v : new int[]{50, 30, 70, 20, 40, 60, 80}) t = insert(t, v);
        t = delete(t, 50);
        System.out.println("after delete 50, preorder = " + preorder(t));
    }
}
```

Output (JDK 21) TERMINAL

```
$ java Main.java
valid BST? true
trap tree valid? false
MAX_VALUE alone valid? true
kth smallest k=3: 4
LCA BST (2, 4) = 3, (2, 8) = 5
LCA tree (4, 8) = 5
root=3 | inorder range [0, 4] | split at 1 | left size 1, right size 3
root=9 | inorder range [0, 0] | split at 0 | left size 0, right size 0
root=20 | inorder range [2, 4] | split at 3 | left size 1, right size 1
root=15 | inorder range [2, 2] | split at 2 | left size 0, right size 0
root=7 | inorder range [4, 4] | split at 4 | left size 0, right size 0
rebuilt inorder = [9, 3, 15, 20, 7], preorder = [3, 9, 20, 15, 7]
iterator: 2 3 4 5 6 7 8
sorted array -> BST preorder = [0, -10, -3, 5, 9]
after delete 50, preorder = [60, 30, 20, 40, 70, 80]
```

What to notice

- **Bounds as `long`**: the test `MAX_VALUE alone valid? true` is the edge case. If bounds were `int` with `Integer.MIN_VALUE`/`MAX_VALUE` as sentinels, a node holding exactly `Integer.MAX_VALUE` would be rejected. Using `long` or `Integer` objects with `null` for "no bound" avoids it.
- `kthSmallest` stops after k pops, so it costs O(h + k), not O(n). For repeated queries on a tree that changes, store subtree sizes in the nodes (an "order-statistic tree") and answer in O(h).

- `lcaBST` needs no recursion and no stack: while both values are on the same side, descend; the first node where they split (or equal one of them) is the LCA.
- `lca` **for a general tree**: if the node is `p` or `q`, return it. If both sides return non-null, this node is the LCA. Otherwise pass up whichever side found something. It assumes both nodes exist in the tree.
- `buildTree`: the `HashMap` from value to inorder index replaces an O(n) search per node, making the whole construction O(n). It assumes distinct values.
- `BSTIterator` keeps only the left spine on its stack: O(h) memory, and `next()` is amortised O(1) because each node is pushed and popped once over a full run.
- `sortedArrayToBST` picks the middle element, so the two halves differ in size by at most one: height is about log2(n). Always taking the first element would produce a chain.
- `delete` shows the three cases: no child (return the other side, which is null), one child (splice out), two children (copy the inorder successor, the smallest value in the right subtree, then delete that successor).

4.6.4 Complexity and mistakes

Search, insert and delete cost **O(h)**: O(log n) on a balanced tree, O(n) on a chain, which results from inserting sorted data into an unbalanced BST. Self-balancing trees (AVL, red-black) keep h at O(log n); in Java, `TreeMap` and `TreeSet` are red-black trees. Validate, construct, and the full traversals cost O(n). Space is O(h) for recursion, plus O(n) for the hash map in construction.

Mistakes to avoid:

- **Local-only validation** (parent versus child). Always carry bounds from all ancestors.
- **Bounds with** `int` **sentinels** or `<=` versus `<` confusion with duplicates. Ask whether duplicates are allowed and on which side they go.
- **Wrong build order**: recursing right before left when consuming preorder. For postorder + inorder the order flips: build right first, consuming postorder from the end.
- **Recomputing the inorder index by scanning** for each node. That makes construction O(n^2) on a skewed tree; use the map.
- **LCA when a node is the ancestor of the other.** Returning the node itself is correct (a node counts as its own descendant in LeetCode's definition).
- **Deleting a node and forgetting to reassign**: write `n.left = delete(n.left, key)` so the parent's link is updated. Without the assignment, a deleted leaf stays in the tree.

> **GOTCHA**
> Repeatedly locating the middle of a linked list is avoidable: it costs O(n log n). Convert the list to an array first (O(n) extra space), or simulate the inorder construction with a moving list pointer for O(n) time and O(log n) space.

4.6.5 Questions to expect

1. "Validate that a tree is a BST." Bounds recursion or inorder strictly increasing. Follow-up: "what if duplicates are allowed on the right?" Change one comparison; say which.
2. "Find the k-th smallest element, and then handle frequent updates." Iterative inorder with early stop, then augmented nodes with subtree sizes.
3. "Lowest common ancestor in a BST, then in a general tree." Iterative split-point walk, then the two-sided recursion. Follow-up: "with parent pointers" becomes the intersection of two linked lists.
4. "Rebuild the tree from preorder and inorder." Root from preorder, split by inorder index. Follow-up: "from inorder and postorder" (root from the end, build right first); "from preorder alone" works only for a BST (use bounds).
5. "Implement an iterator with `next()` and `hasNext()` in O(h) memory." Stack of the left spine.
6. "Convert a sorted array (or list) into a height-balanced BST." Middle as root. Follow-up: "is the result unique?" No, even length gives two valid choices of middle.
7. "Insert into and delete from a BST." Recursion that returns the (possibly new) subtree root; three cases for delete.
8. "Why use a BST over a sorted array or hash map?" Ordered operations with dynamic updates: floor/ceiling, range queries, in-order iteration in O(log n) per update.

sreeram

INTERVIEW ANGLE

"What is the worst case of a BST and how is it avoided?" Inserting already sorted data produces a chain, so search degrades to O(n). Avoid it with a self-balancing tree (AVL or red-black; `TreeMap` is red-black in Java), by shuffling input before inserting, or by building from sorted data with the middle-element method. A strong follow-up answer: "Rotations restore balance in O(1) per fix, so the cost stays O(log n) per operation."

4.6.6 LeetCode practice

#	Title	Level	Key idea
700	Search in a Binary Search Tree	Easy	Walk left or right by comparison
108	Convert Sorted Array to Binary Search Tree	Easy	Middle element as root, recurse on halves
701	Insert into a Binary Search Tree	Medium	Descend to a null slot and attach the new node
98	Validate Binary Search Tree	Medium	Pass `(lo, hi)` bounds down; use `long` bounds
230	Kth Smallest Element in a BST	Medium	Inorder with early exit after k nodes
235	Lowest Common Ancestor of a Binary Search Tree	Medium	First node where the two values split sides
236	Lowest Common Ancestor of a Binary Tree	Medium	Both sides non-null means this node is the LCA
173	Binary Search Tree Iterator	Medium	Stack holding the left spine
105	Construct Binary Tree from Preorder and Inorder Traversal	Medium	Preorder root plus inorder index map
106	Construct Binary Tree from Inorder and Postorder Traversal	Medium	Root from the end of postorder; build right subtree first
450	Delete Node in a BST	Medium	Splice out, or copy the inorder successor
109	Convert Sorted List to Binary Search Tree	Medium	Middle of the list, or inorder construction with a moving pointer

LeetCode problems for this pattern.

KEY POINTS

A BST's inorder traversal is sorted; use the ordering to discard half the tree at each step.

- Validate with bounds from all ancestors (as `long`), never parent versus child only.
- Preorder gives roots, inorder gives the left/right split: build left before right and use a hash map for indices.
- Search, insert, delete are O(h); only a balanced tree guarantees O(log n), and `TreeMap` is Java's balanced BST.
- LCA: split-point walk in a BST, two-sided recursion in a general tree.

4.7 Tries

Consider a collection of 100,000 words that must support "is this word present?", "which words start with `ca`?", and "give me five suggestions for the prefix `car`". A hash set answers the first question in O(L) (L is the word length, since hashing reads every character) but cannot answer the second without scanning every word. A sorted array can find a prefix by binary search, but inserting a new word costs O(n). A **trie** (pronounced "try", taken from the word retrieval; also called a prefix tree) answers all three in time proportional to the length of the query, independent of how many words are stored.

4.7.1 The idea

A trie is a tree where each edge is labelled with one character and each path from the root spells a prefix. Words that share a prefix share the nodes of that prefix, so the common beginning is stored only once. A node has up to 26 children for lowercase letters (an array of 26 references, or a `HashMap<Character, Node>` for large alphabets) and a flag `end` marking that a word finishes there. The flag is needed because `car` is a word and also a prefix of `card`.

The trie after inserting cat, car, dog, cap, cab, card, care (end-of-word nodes marked with *)

```
(root)
├─ c
│  └─ a
│     ├─ b*
│     ├─ p*
│     ├─ r*
│     │  ├─ d*
│     │  └─ e*
│     └─ t*
└─ d
   └─ o
      └─ g*
```

How to recognise it

- Many strings and many *prefix* queries: "starts with", "autocomplete", "words with this prefix", "longest common prefix of many words".
- "Dictionary of words" combined with a grid or a stream: the trie prunes search paths that cannot lead to any word (Word Search II).
- "Replace words by their shortest root", "longest word built one character at a time".
- Wildcard search such as `b.d`: the trie restricts branching to existing children.
- Maximum XOR of two numbers: a trie over the *bits* of the numbers (an XOR trie).

Template

Template.java JAVA

```java
// Trie node and the two loops every trie method is built from (fragment)
static class Node {
    Node[] next = new Node[26];
    boolean end;
}

static void insert(Node root, String word) {
    Node cur = root;
    for (char ch : word.toCharArray()) {
        if (cur.next[ch - 'a'] == null) cur.next[ch - 'a'] = new Node();
        cur = cur.next[ch - 'a'];
    }
    cur.end = true;
}

static Node walk(Node root, String s) {       // follow s; null if the path breaks
    Node cur = root;
    for (char ch : s.toCharArray()) {
        cur = cur.next[ch - 'a'];
        if (cur == null) return null;
    }
    return cur;
}
```

`search(word)` is `walk` followed by "node exists and `end` is true". `startsWith(prefix)` is equivalent to "the walk succeeded". Everything else (autocomplete, counting, wildcard) walks to the prefix node and then runs a DFS below it.

4.7.2 Dry run

`cat`, `car` and `dog` are inserted into an empty trie. For each character, the algorithm either follows an existing child (reuse) or create one. `pass` counts how many inserted words go through the node. This is the real trace from the program (note: the root is node 0 and is not printed).

Step	Word	Char	Action	pass after
1	cat	c	create node under root	1
2	cat	a	create node under c	1
3	cat	t	create node under a, then mark end	1
4	car	c	reuse the existing c node	2

Step	Word	Char	Action	pass after
5	car	a	reuse the existing a node	2
6	car	r	create a sibling of t under a, then mark end	1
7	dog	d	create node under root (no shared prefix)	1
8	dog	o	create node under d	1
9	dog	g	create node under o, then mark end	1

Inserting cat, car and dog.

Three words of three letters need 9 character steps but only 7 new nodes plus the root: 8 nodes total, because `ca` is shared. Now search: `search("car")` follows c, a, r and finds `end = true`; `search("ca")` reaches a node but its `end` is false, so it returns false; `startsWith("ca")` returns true; `startsWith("cz")` falls off the trie at `z`. Each query touches at most as many nodes as the query has characters.

4.7.3 Java code

One program with: a `Trie` class (insert, search, startsWith, prefix counts via `pass`, autocomplete with a limit, and wildcard search where `.` matches any letter), Word Search II (trie plus grid DFS), and a binary XOR trie for the maximum XOR pair.

Main.java JAVA

```java
import java.util.*;

public class Main {
    static class Trie {
        private static class Node {
            Node[] next = new Node[26];
            boolean end;                        // a word finishes at this node
            int pass;                           // how many words pass through (for counts)
        }
        private final Node root = new Node();
        int nodes = 1;

        void insert(String word, boolean trace) {
            Node cur = root;
            for (char ch : word.toCharArray()) {
                int i = ch - 'a';
                boolean created = cur.next[i] == null;
                if (created) { cur.next[i] = new Node(); nodes++; }
                cur = cur.next[i];
                cur.pass++;
                if (trace) System.out.println("insert " + word + " | char " + ch + " | "
                        + (created ? "create node" : "reuse node") + " | pass=" + cur.pass);
            }
            cur.end = true;
        }
        void insert(String word) { insert(word, false); }

        private Node walk(String s) {           // follow s from the root, or null if it breaks
            Node cur = root;
            for (char ch : s.toCharArray()) {
                cur = cur.next[ch - 'a'];
                if (cur == null) return null;
            }
            return cur;
        }
        boolean search(String w)     { Node n = walk(w); return n != null && n.end; }
        boolean startsWith(String p) { return walk(p) != null; }
        int countPrefix(String p)    { Node n = walk(p); return n == null ? 0 : n.pass; }

        // autocomplete: up to k words with the prefix, in alphabetical order
        List<String> suggest(String prefix, int k) {
            List<String> out = new ArrayList<>();
            Node n = walk(prefix);
            if (n != null) collect(n, new StringBuilder(prefix), out, k);
            return out;
        }
        private void collect(Node n, StringBuilder sb, List<String> out, int k) {
            if (out.size() == k) return;
            if (n.end) out.add(sb.toString());
```

Main.java (continued) JAVA

```
            for (int i = 0; i < 26 && out.size() < k; i++) {
                if (n.next[i] == null) continue;
                sb.append((char) ('a' + i));
                collect(n.next[i], sb, out, k);
                sb.setLength(sb.length() - 1);
            }
        }

        // wildcard search: '.' matches any one letter
        boolean searchWild(String w) { return wild(root, w, 0); }
        private boolean wild(Node n, String w, int idx) {
            if (n == null) return false;
            if (idx == w.length()) return n.end;
            char ch = w.charAt(idx);
            if (ch != '.') return wild(n.next[ch - 'a'], w, idx + 1);
            for (Node child : n.next) if (child != null && wild(child, w, idx + 1)) return true;
            return false;
        }
    }

    // ---- Word Search II: walk the grid and the trie together ----
    static class TNode {
        TNode[] next = new TNode[26];
        String word;                            // non-null at the end of a dictionary word
    }
    static List<String> findWords(char[][] board, String[] words) {
        TNode root = new TNode();
        for (String w : words) {
            TNode cur = root;
            for (char ch : w.toCharArray()) {
                if (cur.next[ch - 'a'] == null) cur.next[ch - 'a'] = new TNode();
                cur = cur.next[ch - 'a'];
            }
            cur.word = w;
        }
        List<String> found = new ArrayList<>();
        for (int r = 0; r < board.length; r++)
            for (int c = 0; c < board[0].length; c++) dfs(board, r, c, root, found);
        return found;
    }
    static void dfs(char[][] b, int r, int c, TNode parent, List<String> found) {
        if (r < 0 || c < 0 || r >= b.length || c >= b[0].length) return;
        char ch = b[r][c];
        if (ch == '#' || parent.next[ch - 'a'] == null) return;     // visited, or no word continues
        TNode node = parent.next[ch - 'a'];
        if (node.word != null) { found.add(node.word); node.word = null; }   // report once
        b[r][c] = '#';
        dfs(b, r + 1, c, node, found);
        dfs(b, r - 1, c, node, found);
        dfs(b, r, c + 1, node, found);
        dfs(b, r, c - 1, node, found);
        b[r][c] = ch;
    }

    // ---- XOR trie: maximum XOR of two numbers (bitwise trie of 31 bits) ----
    static int maxXor(int[] nums) {
        int[][] child = new int[nums.length * 31 + 2][2];     // child[node][bit], 0 = none
        int size = 1;
        for (int x : nums) {
            int cur = 0;
            for (int b = 30; b >= 0; b--) {
                int bit = (x >> b) & 1;
                if (child[cur][bit] == 0) child[cur][bit] = size++;
                cur = child[cur][bit];
            }
        }
        int best = 0;
        for (int x : nums) {
            int cur = 0, val = 0;
            for (int b = 30; b >= 0; b--) {
                int want = 1 - ((x >> b) & 1);                // opposite bit makes the XOR bit 1
                if (child[cur][want] != 0) { val |= 1 << b; cur = child[cur][want]; }
                else cur = child[cur][1 - want];
            }
            best = Math.max(best, val);
```

Main.java (continued) JAVA

```
        }
        return best;
    }

    public static void main(String[] args) {
        Trie t = new Trie();
        for (String w : new String[]{"cat", "car", "dog"}) t.insert(w, true);
        System.out.println("nodes = " + t.nodes + " (including the root)");
        for (String w : new String[]{"cap", "card", "care", "cab", "bat"}) t.insert(w);
        System.out.println("search(car)=" + t.search("car") + " search(ca)=" + t.search("ca")
                + " startsWith(ca)=" + t.startsWith("ca")
                + " startsWith(cz)=" + t.startsWith("cz"));
        System.out.println("countPrefix(ca)=" + t.countPrefix("ca")
                + " countPrefix(car)=" + t.countPrefix("car"));
        System.out.println("suggest(ca, 3) = " + t.suggest("ca", 3));
        System.out.println("suggest(car, 5) = " + t.suggest("car", 5));
        System.out.println("searchWild(c.t)=" + t.searchWild("c.t") + " searchWild(..g)="
                + t.searchWild("..g") + " searchWild(c..d)=" + t.searchWild("c..d")
                + " searchWild(.a)=" + t.searchWild(".a"));

        char[][] board = {
            {'o', 'a', 'a', 'n'},
            {'e', 't', 'a', 'e'},
            {'i', 'h', 'k', 'r'},
            {'i', 'f', 'l', 'v'}};
        String[] dict = {"oath", "pea", "eat", "rain"};
        System.out.println("grid words = " + findWords(board, dict));
        System.out.println("max XOR = " + maxXor(new int[]{3, 10, 5, 25, 2, 8}));
    }
}
```

Output (JDK 21) TERMINAL

```
$ java Main.java
insert cat | char c | create node | pass=1
insert cat | char a | create node | pass=1
insert cat | char t | create node | pass=1
insert car | char c | reuse node | pass=2
insert car | char a | reuse node | pass=2
insert car | char r | create node | pass=1
insert dog | char d | create node | pass=1
insert dog | char o | create node | pass=1
insert dog | char g | create node | pass=1
nodes = 8 (including the root)
search(car)=true search(ca)=false startsWith(ca)=true startsWith(cz)=false
countPrefix(ca)=6 countPrefix(car)=3
suggest(ca, 3) = [cab, cap, car]
suggest(car, 5) = [car, card, care]
searchWild(c.t)=true searchWild(..g)=true searchWild(c..d)=true searchWild(.a)=false
grid words = [oath, eat]
max XOR = 28
```

What to notice

- **Autocomplete** is "walk to the prefix node, then DFS its children in order a to z". Because children are visited alphabetically, results come out sorted, and the `k` limit stops the search early. The cost is O(L + size of the explored part), not the size of the whole trie.
- `countPrefix` shows why storing counts is cheap and useful: `ca` is on the path of six words, `car` of three. This is the idea behind Map Sum Pairs and "how many words start with ...".
- **Wildcard** `.`: only the dot branches into all children; ordinary letters follow a single edge. Worst case it explores the entire trie below, so cost is O(26^dots x L) in the worst case, but real dictionaries are sparse.
- **Word Search II**: instead of running a separate word search per word, one DFS over the grid carries the current trie node. The moment no word continues (`parent.next[...] == null`), the branch is abandoned. Setting `node.word = null` after reporting avoids duplicates. A further optimisation, pruning empty trie branches after a word is found, is mentioned in the questions below.
- **XOR trie**: each number becomes a path of 31 bits, from the most significant. To maximise `x XOR y`, at every bit prefer the child with the *opposite* bit (it contributes a 1 to the result). The answer for `{3, 10, 5, 25, 2, 8}` is 28 (5 XOR 25). The arrays-of-ints representation avoids millions of small objects.

4.7.4 Complexity and mistakes

Insert, search and startsWith are **O(L)**, where L is the length of the word, regardless of how many words are stored. Memory is O(total characters) in the worst case, times 26 references per node for the array version: that is the principal cost. 100,000 words of average length 8 could create up to 800,000 nodes, each holding an array of 26 references plus object headers, which is far more than the raw strings. Use a `HashMap<Character, Node>` (or a compressed trie) when the alphabet is large or the trie is sparse. A hash set uses less memory when you never need prefix queries.

Mistakes to avoid:

- **Treating a prefix as a word**: `startsWith` and `search` differ only by the `end` check. Mixing them up fails `search("ca")`.
- **Index errors**: `ch - 'a'` assumes lowercase letters only; an uppercase letter or a digit gives a negative or out-of-range index and `ArrayIndexOutOfBoundsException`. Check the constraints, or use a map.
- **Missing the empty string or a single-character word.** The loops handle them, but `end` on the root must be thought about if the empty word is allowed.
- **Moving to a child before creating it in insert** (`cur = cur.next[i]` while it is null) gives `NullPointerException`.
- **Reporting the same word twice** in Word Search II when it can be formed along several paths; clear the stored word after the first hit.
- **Recursing with a shared** `StringBuilder` without undoing the append in autocomplete.

WATCH OUT
A trie of 26-element arrays is memory hungry. In an interview mention the trade-off in one sentence ("array children are fastest, map children are smaller for sparse data"), and pick the array unless the alphabet is large or unspecified.

4.7.5 Questions to expect

1. "Implement a trie with insert, search and startsWith." The template above. Be ready to explain the role of the `end` flag.
2. "Add and search words where `.` matches any letter." Trie plus DFS that branches on the dot.
3. "Design autocomplete: top 3 suggestions for each prefix typed." Walk the prefix node by node; either DFS to collect sorted words, or store top-k at each node. The same effect is possible with a sorted array and binary search (the Search Suggestions problem).
4. "Find all dictionary words in a grid." Build a trie of the words, DFS from each cell. Follow-up: prune trie branches once their words are found so repeated visits get cheaper.
5. "Replace each word in a sentence with its shortest dictionary root." Insert roots, then walk each word and stop at the first `end` node.
6. "Maximum XOR of two numbers in an array." Binary trie of the bits; greedy opposite-bit walk. Time O(32 n), space O(32 n). Follow-up: "XOR with a limit or in a subarray" extends the idea with counts or persistence.
7. "Longest word that can be built one character at a time from other words." Trie (or sorted set); every prefix along the path must itself be a word.
8. "Trie versus hash map?" The hash map is smaller and faster for exact lookups. The trie wins for prefix queries, sorted enumeration, wildcard matching and longest-prefix matching (for example, IP routing).

INTERVIEW ANGLE
"How would you store autocomplete suggestions at scale?" A trie gives prefix lookup in O(L). For ranking, store at each node the top-k completions with their frequencies, updated offline in batches, so a query is O(L + k). Mention memory: compress single-child chains (a radix tree), or shard by first letters. Keep the answer to the data structure unless the interviewer asks about distributed design.

4.7.6 LeetCode practice

#	Title	Level	Key idea
208	Implement Trie (Prefix Tree)	Medium	The classic node with 26 children and an end flag
648	Replace Words	Medium	Stop at the first end-of-word node on the walk
720	Longest Word in Dictionary	Medium	Only extend through nodes that are words themselves
211	Design Add and Search Words Data Structure	Medium	DFS that branches over all children on a dot
677	Map Sum Pairs	Medium	Store a value per node, sum below the prefix node
1268	Search Suggestions System	Medium	Walk the prefix and collect up to three sorted words
421	Maximum XOR of Two Numbers in an Array	Medium	Binary trie, take the opposite bit when it exists
212	Word Search II	Hard	One grid DFS guided by a trie of all words

LeetCode problems for this pattern.

KEY POINTS

A trie stores strings as paths from the root, so shared prefixes are stored once.

- Insert, search and startsWith cost O(L); the `end` flag distinguishes words from prefixes.
- Prefix questions (autocomplete, counts, wildcard, longest prefix) walk to the prefix node then DFS below it.
- Memory is the cost: 26 references per node; use maps or radix compression for sparse data.
- Tries over bits (XOR trie) solve maximum-XOR problems in O(32 n); tries guide grid DFS in Word Search II.

sreeram

PART 5

Heaps and Graphs

Priority queues for "best so far" problems, then graphs: traversal, ordering, connectivity and shortest paths.

In this part

- **5.1** Heaps and Top-K
- **5.2** Graph Representation, DFS and BFS
- **5.3** Topological Sort
- **5.4** Union-Find (Disjoint Set Union)
- **5.5** Shortest Paths

5.1 Heaps and Top-K

A recurring class of problems requires repeated access to the extremal element of a changing collection: the k largest values, the next task to execute, the running median. In a sorted array each insertion costs O(n) because later elements must be shifted; in an unsorted array each extraction of the extremum costs O(n) because a scan is required. A **heap** (priority queue) supports insertion and extraction in O(log n) and inspection of the extremum in O(1).

The brute-force solution to "k largest of n values" sorts the entire input in O(n log n) time. This performs more work than the specification requires: only k candidates need to be retained. A heap of size k solves the problem in O(n log k) time, which is asymptotically better for small k, and it is the only feasible approach when the values arrive as an unbounded stream.

5.1.1 The idea

A binary heap is a complete binary tree stored in a plain array. In a **min-heap** every parent is less than or equal to its children, so the smallest element is always at index 0. For the node at index `i`, the parent is at `(i - 1) / 2` and the children are at `2 * i + 1` and `2 * i + 2`. There are no node objects and no pointers, which is why a heap is fast and compact.

Two operations keep the shape valid. **Sift up** (used by `offer`) puts the new element at the end and swaps it with its parent while it is smaller. **Sift down** (used by `poll`) moves the last element to the root and swaps it with its smaller child while it is larger. Both walk one root-to-leaf path, so both are O(log n).



Figure 5.1. A min-heap of size k keeps the k largest values seen so far; the root is the weakest of them, so it is the one to throw out when a better value arrives.

Java's `PriorityQueue` is a **min-heap** by default. A max-heap requires a comparator: `new PriorityQueue<>(Collections.reverseOrder())` or `(a, b) -> Integer.compare(b, a)`.

Method	Cost	Notes
`offer(x) / add(x)`	O(log n)	sift up. Null is not allowed.
`peek()`	O(1)	the smallest element, or null if empty.
`poll()`	O(log n)	removes and returns the smallest, or null if empty.
`remove(Object o)`	O(n)	linear search, then a repair. Avoid it in loops.
`new PriorityQueue<>(collection)`	O(n)	builds the heap in linear time (heapify).

sreram

Method	Cost	Notes
`size(), isEmpty()`	O(1)	constant time.
`contains(o)`	O(n)	there is no index by value.

PriorityQueue operations in Java.

GOTCHA
Printing a `PriorityQueue` or iterating over it does NOT give sorted order. It shows the internal array, which only guarantees that each parent is not larger than its children. The only way to get sorted order is to call `poll()` repeatedly. The program below prints this difference.

How to recognise it

- "the k largest / smallest / most frequent / closest ..."
- "kth largest element in a stream", where data keeps arriving.
- "repeatedly take the smallest/largest and then update", for example merging sorted lists or scheduling tasks.
- "median of a running collection".
- The best item is required, but a full sort is not.

Template: top-K with a bounded heap

To retain the K LARGEST values, a MIN-heap of size K is used. Its root is the smallest of the K candidates, and a new value that exceeds the root replaces it. To retain the K SMALLEST values, a MAX-heap of size K is used. The heap direction is the opposite of the intuitive one because the heap must expose the element that is to be discarded.

Template.java JAVA

```java
// keep the k largest values seen so far
PriorityQueue<Integer> heap = new PriorityQueue<>();   // min-heap
for (int x : nums) {
    heap.offer(x);
    if (heap.size() > k) heap.poll();                  // drop the smallest candidate
}
// heap.peek() is now the kth largest; the heap holds the k largest
```

5.1.2 Dry run

Problem: determine the 2nd largest element of `[3, 2, 1, 5, 6, 4]` (LeetCode 215) with a min-heap of size k = 2. The "heap" column shows the internal array printed by the program in the next section.

Step	x	Action	Heap after
1	3	size 0 < 2, push	[3]
2	2	size 1 < 2, push	[2, 3]
3	1	heap full; 1 is not bigger than root 2, skip	[2, 3]
4	5	5 > root 2: poll 2, push 5	[3, 5]
5	6	6 > root 3: poll 3, push 6	[5, 6]
6	4	4 is not bigger than root 5, skip	[5, 6]

Kth largest with a min-heap, k = 2.

The heap ends as `[5, 6]`: the two largest values. The root, 5, is the second largest, which is the answer. The heap never holds more than two numbers, so memory is O(k), and each step costs at most one `poll` and one `offer`.

The second dry run is the harder idea of this chapter: the running median. Keep two heaps. `low` is a max-heap holding the smaller half, `high` is a min-heap holding the larger half. The invariant is: every value in `low` is at most every value in `high`, and `low` has the same size as `high` or one more. Then the median is the root of `low` (odd count) or the average of the two roots (even count). To add `x`: push it into `low`, move the largest of `low` over to `high` (so ordering stays correct), and if `high` became bigger than `low`, move its smallest back.

Step	add	low (max-heap)	high (min-heap)	median
1	5	[5]	[]	5.0

Step	add	low (max-heap)	high (min-heap)	median
2	15	[5]	[15]	10.0
3	1	[5, 1]	[15]	5.0
4	3	[3, 1]	[5, 15]	4.0

Median of a stream: add 5, 15, 1, 3.

Verification of step 4: after adding 3, the sorted data is 1, 3, 5, 15. The two middle values are 3 and 5, and their mean is 4.0. The printed heaps show exactly that: root of `low` is 3 and root of `high` is 5.

5.1.3 Java code

One program with every solution of the chapter: kth largest, top-k frequent (heap of map entries), k closest points (max-heap on squared distance), merge k sorted lists, task scheduler (the counting formula), the two-heap median, and a small demonstration of the `PriorityQueue` gotchas.

Main.java JAVA

```
import java.util.*;

public class Main {
    // Kth largest with a min-heap of size k, tracing heap state
    static int kthLargest(int[] nums, int k, boolean trace) {
        PriorityQueue<Integer> heap = new PriorityQueue<>();
        for (int x : nums) {
            String action;
            if (heap.size() < k) {
                heap.offer(x);
                action = "push";
            } else if (x > heap.peek()) {
                int out = heap.poll();
                heap.offer(x);
                action = "replace " + out;
            } else {
                action = "skip";
            }
            if (trace) System.out.println("x=" + x + " " + action + " heap=" + heap);
        }
        return heap.peek();
    }

    static int[] topKFrequent(int[] nums, int k) {
        Map<Integer, Integer> freq = new HashMap<>();
        for (int x : nums) freq.merge(x, 1, Integer::sum);
        PriorityQueue<Map.Entry<Integer, Integer>> heap =
            new PriorityQueue<>(Comparator.comparingInt(Map.Entry::getValue));
        for (var e : freq.entrySet()) {
            heap.offer(e);
            if (heap.size() > k) heap.poll();
        }
        int[] res = new int[k];
        for (int i = k - 1; i >= 0; i--) res[i] = heap.poll().getKey();
        return res;
    }

    static int[][] kClosest(int[][] pts, int k) {
        // max-heap on squared distance, keep k smallest
        PriorityQueue<int[]> heap = new PriorityQueue<>(
            (a, b) -> Integer.compare(b[0] * b[0] + b[1] * b[1], a[0] * a[0] + a[1] * a[1]));
        for (int[] p : pts) {
            heap.offer(p);
            if (heap.size() > k) heap.poll();
        }
        return heap.toArray(new int[0][]);
    }

    static class ListNode {
        int val; ListNode next;
        ListNode(int v) { val = v; }
    }

    static ListNode build(int... vals) {
        ListNode dummy = new ListNode(0), cur = dummy;
```

Main.java (continued) JAVA

```
        for (int v : vals) { cur.next = new ListNode(v); cur = cur.next; }
        return dummy.next;
    }

    static String show(ListNode n) {
        StringBuilder sb = new StringBuilder();
        for (; n != null; n = n.next) sb.append(n.val).append(n.next != null ? "->" : "");
        return sb.toString();
    }

    static ListNode mergeK(ListNode[] lists) {
        PriorityQueue<ListNode> heap = new PriorityQueue<>(Comparator.comparingInt(n -> n.val));
        for (ListNode l : lists) if (l != null) heap.offer(l);
        ListNode dummy = new ListNode(0), tail = dummy;
        while (!heap.isEmpty()) {
            ListNode n = heap.poll();
            tail.next = n; tail = n;
            if (n.next != null) heap.offer(n.next);
        }
        return dummy.next;
    }

    static int leastInterval(char[] tasks, int n) {
        int[] cnt = new int[26];
        for (char c : tasks) cnt[c - 'A']++;
        int max = 0;
        for (int c : cnt) max = Math.max(max, c);
        int numMax = 0;
        for (int c : cnt) if (c == max) numMax++;
        return Math.max(tasks.length, (max - 1) * (n + 1) + numMax);
    }

    static class MedianFinder {
        private final PriorityQueue<Integer> low = new PriorityQueue<>(Collections.reverseOrder());
        private final PriorityQueue<Integer> high = new PriorityQueue<>();
        void addNum(int x) {
            low.offer(x);
            high.offer(low.poll());
            if (high.size() > low.size()) low.offer(high.poll());
        }
        double findMedian() {
            return low.size() > high.size() ? low.peek() : (low.peek() + high.peek()) / 2.0;
        }
        String state() { return "low=" + low + " high=" + high; }
    }

    public static void main(String[] args) {
        int[] a = {3, 2, 1, 5, 6, 4};
        System.out.println("kth largest (k=2): " + kthLargest(a, 2, true));
        int[] top = topKFrequent(new int[]{1, 1, 1, 2, 2, 3}, 2);
        System.out.println("top 2 frequent: " + Arrays.toString(top));
        int[][] pts = {{1, 3}, {-2, 2}, {5, 8}, {0, 1}};
        System.out.println("2 closest: " + Arrays.deepToString(kClosest(pts, 2)));
        ListNode merged = mergeK(new ListNode[]{build(1, 4, 5), build(1, 3, 4), build(2, 6)});
        System.out.println("merged: " + show(merged));
        System.out.println("task scheduler: "
            + leastInterval("AAABBB".toCharArray(), 2) + " "
            + leastInterval("AAAAAABCDEFG".toCharArray(), 2));
        MedianFinder mf = new MedianFinder();
        for (int x : new int[]{5, 15, 1, 3}) {
            mf.addNum(x);
            System.out.println("add " + x + " " + mf.state() + " median=" + mf.findMedian());
        }
        // mechanics
        PriorityQueue<Integer> pq = new PriorityQueue<>(List.of(9, 4, 7, 1, 8, 2));
        System.out.println("array order: " + pq + " peek=" + pq.peek());
        StringBuilder sb = new StringBuilder();
        while (!pq.isEmpty()) sb.append(pq.poll()).append(' ');
        System.out.println("poll order: " + sb.toString().trim());
        int x = Integer.MIN_VALUE, y = 5;
        System.out.println("x - y > 0 ? " + (x - y > 0));
        System.out.println("compare says x < y ? " + (Integer.compare(x, y) < 0));
    }
}
```

Compile and run TERMINAL

```
$ java Main.java
x=3 push heap=[3]
x=2 push heap=[2, 3]
x=1 skip heap=[2, 3]
x=5 replace 2 heap=[3, 5]
x=6 replace 3 heap=[5, 6]
x=4 skip heap=[5, 6]
kth largest (k=2): 5
top 2 frequent: [1, 2]
2 closest: [[-2, 2], [0, 1]]
merged: 1->1->2->3->4->4->5->6
task scheduler: 8 16
add 5 low=[5] high=[] median=5.0
add 15 low=[5] high=[15] median=10.0
add 1 low=[5, 1] high=[15] median=5.0
add 3 low=[3, 1] high=[5, 15] median=4.0
array order: [1, 4, 2, 9, 8, 7] peek=1
poll order: 1 2 4 7 8 9
x - y > 0 ? true
compare says x < y ? true
```

What to notice

- `kthLargest` is the template unchanged. The trace line is the only extra.
- `topKFrequent` first counts with `freq.merge(x, 1, Integer::sum)`, then runs the same bounded min-heap, but ordered by count. The loop `for (int i = k - 1; ...)` fills the result from the back, so the most frequent value ends first.
- `kClosest` uses a MAX-heap on distance (the comparator flips `a` and `b`) so that the farthest of the k candidates is on top and gets evicted. It compares squared distances, so no `Math.sqrt` and no floating-point error.
- `mergeK` puts only the head of each list into the heap, so the heap never has more than k nodes. After polling a node, push its successor. This is the same trick as "merge k sorted arrays" and as the k-way merge in external sorting.
- `leastInterval` does not need a heap. The task with the highest count creates `max - 1` full gaps of length `n + 1`; the last row has `numMax` tasks. If there are enough other tasks to fill the gaps, no idle time is needed and the answer is just `tasks.length`. A heap version exists (always run the most frequent remaining task, park it in a cooldown queue), and interviewers sometimes ask for it, but the formula is the one to remember.
- `MedianFinder.addNum` always goes through `low` first. That one rule keeps both ordering and size invariants without any `if` about which side the number belongs to.

> **NOTE**
> Why the last line prints `true` for `x - y > 0`. With `x = Integer.MIN_VALUE` and `y = 5`, `x - y` overflows and wraps around to a large positive number. A comparator written as `(a, b) -> a - b` would then say that MIN_VALUE is bigger than 5. `Integer.compare(a, b)` never overflows. Use it.

5.1.4 Complexity and mistakes

Problem	Time	Space	Why
Kth largest in an array	O(n log k)	O(k)	n offers, each at most log k
Top k frequent	O(n log k)	O(n)	counting is O(n); the map holds up to n keys
K closest points	O(n log k)	O(k)	same bounded heap
Merge k sorted lists	O(N log k)	O(k)	N = total nodes, heap holds one per list
Median of a stream	O(log n) per add, O(1) median	O(n)	two heaps hold everything
Build a heap from n items	O(n)	O(n)	heapify, not n separate offers

Complexity of the heap solutions.

Building a heap by n `offer` calls is O(n log n), but `new PriorityQueue<>(collection)` runs heapify and takes O(n). It is a classic surprise: most nodes are near the bottom and sink only a short way.

Common mistakes:

- Using the wrong heap direction. The k LARGEST values require a MIN-heap. A max-heap containing all n items is also correct, but it needs O(n) memory and O(n log n) time and discards the advantage of the pattern.
- Writing a comparator as `a - b`. It overflows for large or negative values. Use `Integer.compare`, `Long.compare` or `Comparator.comparingInt`.
- Expecting iteration or `toString` to be sorted. Poll until empty, or copy and sort.
- Calling `remove(Object)` inside a loop. Each call costs O(n). When arbitrary deletion or update is required, lazy deletion (leaving the stale entry and discarding it when polled) or a `TreeMap`/`TreeSet` is appropriate.
- Changing an object after it is in the heap. The heap does not re-sort. If a priority changes, remove and re-insert, or push a new entry and ignore the stale one.
- Forgetting that `PriorityQueue` is not thread-safe and does not allow `null`.
- Ties. `PriorityQueue` is not stable: two equal elements may come out in any order. Add a sequence number to the comparator if order of arrival matters.

5.1.5 Questions to expect

1. Find the kth largest element in an unsorted array. (clue: "kth largest", approach: min-heap of size k, O(n log k); follow-up: quickselect gives O(n) average but changes the array.)
2. Design a class that returns the kth largest score after every new score. (clue: "stream"; approach: the same size-k min-heap kept as a field, `add` returns `peek()`.)
3. Return the k most frequent elements. (clue: "most frequent"; approach: count with a hash map, then a size-k min-heap on count. Follow-up: is O(n) attainable? Yes, by bucket sort on frequency, since a count is at most n.)
4. Return the k points closest to the origin. (approach: max-heap of size k on squared distance. Follow-up: what if k is close to n? Then sort, or keep a heap of the n-k farthest.)
5. Merge k sorted linked lists. (approach: heap of list heads, O(N log k). Alternative: divide and conquer pairwise merging, same time, O(1) extra heap memory.)
6. Schedule tasks with a cooldown n between equal tasks. (approach: formula from the maximum count, or heap plus a cooldown queue.)
7. Maintain the median of a stream. (approach: two heaps. Follow-up: what if all numbers are in 0 to 100? Use a count array. What if 99 percent are in 0 to 100? Count array plus two outlier counters.)
8. Simulation with a max-heap: "repeatedly smash the two heaviest stones" (Last Stone Weight). (approach: max-heap, poll twice, push the difference.)

INTERVIEW ANGLE

Why is a min-heap used to find the k largest elements? "The heap must expose the element that is eligible for eviction. Among the k retained candidates that element is the smallest, so a min-heap places it at the root: each new value is compared against the root in O(1) and replaces it in O(log k). A max-heap would place the best candidate at the root, which is never to be evicted. The total cost is O(n log k) time and O(k) space, and the method applies to streams because the complete data set is never required." Follow-up: "Why not sort?" Sorting costs O(n log n) and requires all data in memory. For n = 10^7 and k = 10 the heap performs roughly 7 times fewer comparisons per element (log2 of 10 is about 3, log2 of 10^7 is about 23) and occupies 10 slots instead of 10^7.

5.1.6 LeetCode practice

#	Title	Level	Key idea
703	Kth Largest Element in a Stream	Easy	size-k min-heap kept as a field; add returns the root
1046	Last Stone Weight	Easy	max-heap; poll two, push the difference if non-zero
215	Kth Largest Element in an Array	Medium	size-k min-heap, or quickselect for O(n) average
347	Top K Frequent Elements	Medium	count, then size-k heap on frequency (or bucket sort)
973	K Closest Points to Origin	Medium	max-heap of size k on squared distance
378	Kth Smallest Element in a Sorted Matrix	Medium	heap of row heads, pop k times (or binary search on the value)
621	Task Scheduler	Medium	max count decides the frame: (max-1)*(n+1)+numMax, at least n tasks
23	Merge k Sorted Lists	Hard	heap holds one node per list; push the successor after each poll
295	Find Median from Data Stream	Hard	max-heap for the lower half, min-heap for the upper half, keep sizes within 1

LeetCode problems for this pattern.

KEY POINTS

- A heap provides O(1) peek and O(log n) insertion and removal; it applies when the current best element is needed repeatedly.
- For the k LARGEST keep a MIN-heap of size k (and the reverse for k smallest); time O(n log k), space O(k).
- Java's `PriorityQueue` is a min-heap; reversal is done with a comparator, never with `a - b`. Its iteration order is not sorted.
- Merging k sorted sources puts one head per source in the heap; a running median uses a max-heap and a min-heap whose sizes differ by at most one.
- Updating or removing arbitrary items is O(n); prefer lazy deletion.

5.2 Graph Representation, DFS and BFS

A **graph** is a set of **vertices** (nodes) together with a set of **edges** connecting pairs of vertices. Road maps, social networks, course prerequisites, grids of cells and word-transformation puzzles are all instances. The typical questions are: which vertices are connected, how many disjoint connected groups exist, and what is the shortest route between two vertices.

Enumerating every path is infeasible because the number of paths grows exponentially. The standard remedy is to **visit each vertex at most once** and record the visited set, which bounds the work by the number of vertices plus edges, O(V + E). Two traversals realise this bound: depth-first search (DFS) descends as far as possible before backtracking, and breadth-first search (BFS) explores vertices in order of non-decreasing distance from the source.

5.2.1 The idea

Three ways to store a graph

The adjacency list is the default representation: for each vertex it stores the list of its neighbours. The matrix and the edge list serve specific purposes.

Representation	Memory	Is there an edge u-v?	All neighbours of u
Adjacency list	O(V + E)	O(degree of u)	O(degree of u)
Adjacency matrix	O(V squared)	O(1)	O(V)
Edge list	O(E)	O(E)	O(E)

Graph representations (V vertices, E edges).

The list is preferred for sparse graphs (E much smaller than V squared), which covers nearly all cases. A matrix is appropriate when V is small (a few hundred) and constant-time edge queries are needed, or when the input is already a matrix (LeetCode "Number of Provinces" supplies `isConnected[i][j]`). An edge list is the natural input format and the natural form for sorting edges (Kruskal; see the chapter on Union-Find). In an **undirected** graph each edge is stored in both directions; in a **directed** graph it is stored once.

In Java build the list with `List<List<Integer>>` filled in a loop, as shown in the program. Do not write `new ArrayList[n]` for generics reasons; it compiles with warnings and surprises people.

Grids are implicit graphs

In a grid every cell is a vertex, and its up to four neighbours (or eight with diagonals) are its edges. No list is built; neighbours are enumerated with a direction array: `int[][] DIRS = {{1,0},{-1,0},{0,1},{0,-1}}`. The bounds must be checked before `grid[nr][nc]` is accessed.

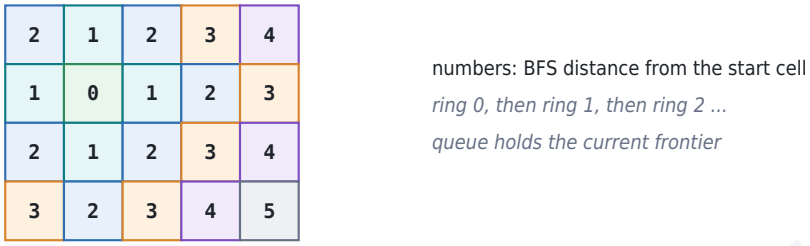


Figure 5.2. BFS spreads from the start in rings: all cells at distance 1, then all at distance 2, and so on. The first time a cell is reached is its shortest distance.

How to recognise it

- "number of islands / connected components / provinces / regions".
- "is there a path from A to B", "is it reachable", "is it connected".
- "minimum number of steps / moves / minutes" on an unweighted grid or graph (BFS).
- "spreads simultaneously from several sources" (multi-source BFS: rotting oranges, walls and gates).
- "change one letter / one move at a time" (word ladder, puzzle states): the states are vertices.
- "split into two groups with no edge inside a group" (bipartite check), "copy the structure" (clone).

DFS versus BFS

Both traversals establish reachability. **DFS** suits exhaustive exploration (components, flood fill, cycle checks, recursive formulations). **BFS** is required for the SHORTEST path in an unweighted graph: the first visit to a vertex occurs along a path with the fewest edges. DFS provides no such guarantee.

Template

Template.java JAVA

```java
// BFS skeleton: shortest distance from src in an unweighted graph
int[] dist = new int[n];
Arrays.fill(dist, -1);                      // -1 means "not visited yet"
Deque<Integer> q = new ArrayDeque<>();
dist[src] = 0;
q.offer(src);
while (!q.isEmpty()) {
    int u = q.poll();
    for (int v : g.get(u)) {
        if (dist[v] == -1) {                // mark when you ADD to the queue, not when you poll
            dist[v] = dist[u] + 1;
            q.offer(v);
        }
    }
}
```

sreeram

DfsTemplate.java JAVA

```java
// DFS skeleton (a method; seen and g are fields)
void dfs(int u) {
    seen[u] = true;
    for (int v : g.get(u)) if (!seen[v]) dfs(v);
}
```

WATCH OUT

A vertex must be marked as visited when it is enqueued, not when it is dequeued. Deferring the mark until `poll` allows the same vertex to be enqueued repeatedly by different neighbours, so the queue can grow far beyond V. On a dense grid this converts an O(V) algorithm into one that exceeds the time limit.

5.2.2 Dry run

Graph with 7 vertices and the undirected edges `0-1, 0-2, 1-3, 2-3, 3-4, 5-6`. Find the shortest distance from vertex 0 to every vertex. The adjacency list is `[[1, 2], [0, 3], [0, 3], [1, 2, 4], [3], [6], [5]]`. Vertices 5 and 6 are in a separate component, so they are never reached.

Step	Pop	Added to queue	Queue after	dist array
0	start	0 (dist 0)	[0]	[0, -1, -1, -1, -1, -1, -1]
1	0	1, 2	[1, 2]	[0, 1, 1, -1, -1, -1, -1]
2	1	3	[2, 3]	[0, 1, 1, 2, -1, -1, -1]
3	2	none (3 is already marked)	[3]	[0, 1, 1, 2, -1, -1, -1]
4	3	4	[4]	[0, 1, 1, 2, 3, -1, -1]
5	4	none	[]	[0, 1, 1, 2, 3, -1, -1]

BFS from vertex 0. Distances start as -1 (not visited).

In step 3, vertex 2 looks at neighbour 3 and sees `dist[3]` is already 2, so it does not add it again. That is the visited check doing its job: without it, 3 would enter the queue twice and the work would double on larger graphs. The queue is empty after step 5, and the vertices still showing -1 (5 and 6) are unreachable from 0. The DFS from 0 visits `[0, 1, 3, 2, 4]`: it goes deep along 0, 1, 3 before coming back for 2, which is why DFS order differs from BFS order.

A second, grid-based dry run, rotting oranges (multi-source BFS). Grid `[[2,1,1],[1,1,0],[0,1,1]]`, where 2 is rotten, 1 is fresh, 0 is empty. Put ALL rotten oranges into the queue at the start, then process the queue one whole level (one minute) at a time.

Minute	Cells that turn rotten	Fresh left	Frontier size
0	start: (0,0) is rotten	6	1
1	(0,1) and (1,0)	4	2
2	(0,2) and (1,1)	2	2
3	(2,1)	1	1
4	(2,2)	0	1

Rotting oranges: one row per minute.

All fresh oranges are gone after minute 4, so the answer is 4. If `fresh` were still above zero when the queue ran empty, some orange is unreachable and the answer is -1.

5.2.3 Java code

The program covers adjacency-list building, DFS, counting components, BFS distances (with the trace above), number of islands by flood fill, rotting oranges, a bipartite check by two-colouring, word ladder, and clone graph.

Main.java JAVA

```java
import java.util.*;

public class Main {
    static List<List<Integer>> buildGraph(int n, int[][] edges, boolean directed) {
        List<List<Integer>> g = new ArrayList<>();
        for (int i = 0; i < n; i++) g.add(new ArrayList<>());
        for (int[] e : edges) {
            g.get(e[0]).add(e[1]);
            if (!directed) g.get(e[1]).add(e[0]);
        }
        return g;
    }

    static void dfs(List<List<Integer>> g, int u, boolean[] seen, List<Integer> order) {
        seen[u] = true;
        order.add(u);
        for (int v : g.get(u)) if (!seen[v]) dfs(g, v, seen, order);
    }

    static int components(List<List<Integer>> g) {
        boolean[] seen = new boolean[g.size()];
        int count = 0;
        for (int s = 0; s < g.size(); s++) {
            if (!seen[s]) { count++; dfs(g, s, seen, new ArrayList<>()); }
        }
        return count;
    }

    static int[] bfsDist(List<List<Integer>> g, int src, boolean trace) {
        int[] dist = new int[g.size()];
        Arrays.fill(dist, -1);
        Deque<Integer> q = new ArrayDeque<>();
        dist[src] = 0;
        q.offer(src);
        int step = 0;
        while (!q.isEmpty()) {
            int u = q.poll();
            List<Integer> added = new ArrayList<>();
            for (int v : g.get(u)) {
                if (dist[v] == -1) { dist[v] = dist[u] + 1; q.offer(v); added.add(v); }
            }
            if (trace) System.out.println(++step + ": pop " + u + " add " + added + " queue=" + q
                + " dist=" + Arrays.toString(dist));
        }
        return dist;
    }

    static final int[][] DIRS = {{1, 0}, {-1, 0}, {0, 1}, {0, -1}};

    static int numIslands(char[][] grid) {
        int rows = grid.length, cols = grid[0].length, count = 0;
        for (int r = 0; r < rows; r++)
            for (int c = 0; c < cols; c++)
                if (grid[r][c] == '1') { count++; sink(grid, r, c); }
        return count;
    }

    static void sink(char[][] grid, int r, int c) {
        if (r < 0 || c < 0 || r >= grid.length || c >= grid[0].length || grid[r][c] != '1') return;
        grid[r][c] = '0';
        for (int[] d : DIRS) sink(grid, r + d[0], c + d[1]);
    }

    static int orangesRotting(int[][] grid, boolean trace) {
        int rows = grid.length, cols = grid[0].length, fresh = 0;
        Deque<int[]> q = new ArrayDeque<>();
        for (int r = 0; r < rows; r++)
            for (int c = 0; c < cols; c++) {
                if (grid[r][c] == 2) q.offer(new int[]{r, c});
                else if (grid[r][c] == 1) fresh++;
            }
        int minutes = 0;
        while (!q.isEmpty() && fresh > 0) {
            int size = q.size();
            for (int i = 0; i < size; i++) {
```

sreeram

Main.java (continued) JAVA

```
                int[] cur = q.poll();
                for (int[] d : DIRS) {
                    int nr = cur[0] + d[0], nc = cur[1] + d[1];
                    if (nr < 0 || nc < 0 || nr >= rows || nc >= cols || grid[nr][nc] != 1) continue;
                    grid[nr][nc] = 2;
                    fresh--;
                    q.offer(new int[]{nr, nc});
                }
            }
            minutes++;
            if (trace) System.out.println("minute " + minutes + ": fresh left=" + fresh
                + " frontier=" + q.size());
        }
        return fresh == 0 ? minutes : -1;
    }

    static boolean isBipartite(int[][] graph) {
        int n = graph.length;
        int[] color = new int[n]; // 0 = unvisited, 1 and -1 are the two sides
        for (int s = 0; s < n; s++) {
            if (color[s] != 0) continue;
            Deque<Integer> q = new ArrayDeque<>();
            q.offer(s);
            color[s] = 1;
            while (!q.isEmpty()) {
                int u = q.poll();
                for (int v : graph[u]) {
                    if (color[v] == 0) { color[v] = -color[u]; q.offer(v); }
                    else if (color[v] == color[u]) return false;
                }
            }
        }
        return true;
    }

    static int ladderLength(String begin, String end, List<String> words) {
        Set<String> dict = new HashSet<>(words);
        if (!dict.contains(end)) return 0;
        Deque<String> q = new ArrayDeque<>();
        q.offer(begin);
        Set<String> seen = new HashSet<>(List.of(begin));
        int steps = 1;
        while (!q.isEmpty()) {
            for (int i = q.size(); i > 0; i--) {
                String w = q.poll();
                if (w.equals(end)) return steps;
                char[] cs = w.toCharArray();
                for (int p = 0; p < cs.length; p++) {
                    char orig = cs[p];
                    for (char ch = 'a'; ch <= 'z'; ch++) {
                        cs[p] = ch;
                        String next = new String(cs);
                        if (dict.contains(next) && seen.add(next)) q.offer(next);
                    }
                    cs[p] = orig;
                }
            }
            steps++;
        }
        return 0;
    }

    static class Node {
        int val; List<Node> neighbors = new ArrayList<>();
        Node(int v) { val = v; }
    }

    static Node cloneGraph(Node start) {
        if (start == null) return null;
        Map<Node, Node> copy = new HashMap<>();
        Deque<Node> q = new ArrayDeque<>();
        copy.put(start, new Node(start.val));
        q.offer(start);
        while (!q.isEmpty()) {
            Node u = q.poll();
```

Main.java (continued) JAVA

```
            for (Node v : u.neighbors) {
                if (!copy.containsKey(v)) { copy.put(v, new Node(v.val)); q.offer(v); }
                copy.get(u).neighbors.add(copy.get(v));
            }
        }
        return copy.get(start);
    }

    public static void main(String[] args) {
        int[][] edges = {{0, 1}, {0, 2}, {1, 3}, {2, 3}, {3, 4}, {5, 6}};
        var g = buildGraph(7, edges, false);
        System.out.println("adjacency list: " + g);
        List<Integer> order = new ArrayList<>();
        dfs(g, 0, new boolean[7], order);
        System.out.println("dfs order from 0: " + order);
        System.out.println("components: " + components(g));
        System.out.println("--- BFS from 0 ---");
        int[] dist = bfsDist(g, 0, true);
        System.out.println("final dist: " + Arrays.toString(dist));

        char[][] grid = {
            "11000".toCharArray(),
            "11000".toCharArray(),
            "00100".toCharArray(),
            "00011".toCharArray()};
        System.out.println("islands: " + numIslands(grid));

        int[][] oranges = {{2, 1, 1}, {1, 1, 0}, {0, 1, 1}};
        System.out.println("rotting minutes: " + orangesRotting(oranges, true));
        int[][] stuck = {{2, 1, 1}, {0, 1, 1}, {1, 0, 1}};
        System.out.println("blocked: " + orangesRotting(stuck, false));

        System.out.println("bipartite square: "
            + isBipartite(new int[][]{{1, 3}, {0, 2}, {1, 3}, {0, 2}}));
        System.out.println("bipartite triangle: "
            + isBipartite(new int[][]{{1, 2}, {0, 2}, {0, 1}}));
        System.out.println("ladder: " + ladderLength("hit", "cog",
            List.of("hot", "dot", "dog", "lot", "log", "cog")));

        Node a = new Node(1), b = new Node(2), c = new Node(3);
        a.neighbors.addAll(List.of(b, c));
        b.neighbors.addAll(List.of(a, c));
        c.neighbors.addAll(List.of(a, b));
        Node ca = cloneGraph(a);
        System.out.println("clone: " + (ca != a) + " val=" + ca.val + " nbrs="
            + ca.neighbors.stream().map(n -> n.val).toList());
    }
}
```

Compile and run TERMINAL

```
$ java Main.java
adjacency list: [[1, 2], [0, 3], [0, 3], [1, 2, 4], [3], [6], [5]]
dfs order from 0: [0, 1, 3, 2, 4]
components: 2
--- BFS from 0 ---
1: pop 0 add [1, 2] queue=[1, 2] dist=[0, 1, 1, -1, -1, -1, -1]
2: pop 1 add [3] queue=[2, 3] dist=[0, 1, 1, 2, -1, -1, -1]
3: pop 2 add [] queue=[3] dist=[0, 1, 1, 2, -1, -1, -1]
4: pop 3 add [4] queue=[4] dist=[0, 1, 1, 2, 3, -1, -1]
5: pop 4 add [] queue=[] dist=[0, 1, 1, 2, 3, -1, -1]
final dist: [0, 1, 1, 2, 3, -1, -1]
islands: 3
minute 1: fresh left=4 frontier=2
minute 2: fresh left=2 frontier=2
minute 3: fresh left=1 frontier=1
minute 4: fresh left=0 frontier=1
rotting minutes: 4
blocked: -1
bipartite square: true
bipartite triangle: false
ladder: 5
clone: true val=1 nbrs=[2, 3]
```

Sheram

What to notice

- `numIslands` modifies the grid (sets visited land to `'0'`) instead of keeping a `boolean[][]`. That is fine when the caller allows it; say so in an interview and offer a `visited` array if the input must stay intact.
- `orangesRotting` records `q.size()` at the start of each level; this snapshot separates "minute k" from "minute k + 1" and is the precondition for counting minutes correctly.
- `isBipartite` colours with `1` and `-1`, so the opposite colour is just `-color[u]`. It loops over every start vertex, because the graph may be disconnected. Forgetting that loop is the classic bug.
- `ladderLength` generates neighbours by trying all 26 letters at each position. That costs O(26L) per word, better than comparing every pair of words (O(N squared L)). The `seen.add(next)` call both tests and marks in one step.
- `cloneGraph` keeps a map from original node to copy. The map serves simultaneously as the visited set and the lookup table. A copy is created when a node is first encountered, and edges are added afterwards.
- Recursion in `sink` goes as deep as the island is large. A 1000 by 1000 grid of land can cause `StackOverflowError`; convert to an explicit stack or BFS when the grid is big.

5.2.4 Complexity and mistakes

Every vertex is entered once and every edge is looked at once (twice if undirected), so BFS and DFS take **O(V + E)** time with an adjacency list and O(V) extra space for the visited set and the queue or recursion stack. On an R by C grid that is O(RC) time and space. On an adjacency matrix the neighbour scan is O(V) per vertex, so the traversal is O(V squared). Word ladder is O(26NL) for N words of length L. Clone graph is O(V + E).

Common mistakes:

- Marking visited on `poll` instead of on `offer` (duplicates in the queue, wasted time).
- Not checking the grid bounds before reading a neighbour, or checking them in the wrong order (`grid[r][c]` before `r >= 0`).
- Counting levels incorrectly in level-order BFS: omitting the `q.size()` snapshot, or adding an extra step after the last level. Rotting oranges additionally requires the loop condition `fresh > 0`; without it the result exceeds the correct value by one minute.
- Running a single traversal from vertex 0 on a graph that may be disconnected. Wrap it in a loop over all start vertices.
- Using DFS to find shortest paths. DFS finds A path, not the shortest.
- Treating a directed edge as undirected, or the reverse. Read the statement: `edges[i] = [a, b]` is sometimes one direction only.
- Using `==` on `Integer` objects from a list when comparing vertex ids. Use `int` loops or `.equals`.
- Cycle detection in an undirected graph requires the parent: a visited neighbour other than the immediate predecessor indicates a cycle. (For directed graphs see the chapter on topological sort.)

5.2.5 Questions to expect

1. Count the islands in a 2D grid of land and water. (clue: "connected groups of 1s"; approach: loop over cells, run DFS or BFS flood fill from each unvisited land cell, count the starts. Follow-up: cells arrive one at a time and the count is reported after each: Union-Find.)
2. Return the largest island's area. (same flood fill, return the size of each fill and keep the maximum.)
3. Rotting oranges / "walls and gates" / "01 Matrix". (clue: "spreads from every source at once", "minimum minutes"; approach: multi-source BFS, put all sources in the queue first.)
4. Shortest path in a binary matrix with 8 directions. (approach: BFS with 8 direction pairs; return the distance at which the target is first reached. Follow-up: weighted cells need Dijkstra.)
5. Clone a graph. (approach: map original to copy, BFS or DFS, create on first sight.)
6. Is the graph bipartite? Can the people be split into two groups that do not know each other? (approach: 2-colour with BFS or DFS; an edge between equal colours means an odd cycle, answer no. Run from every unvisited vertex.)
7. Word ladder: shortest sequence from one word to another, changing one letter at a time. (approach: BFS over words. Follow-up: bidirectional BFS searches from both ends and cuts the search a lot.)
8. Pacific Atlantic water flow. (approach: reverse the problem. Start a flood fill from each ocean's border going uphill, then intersect the two sets. Starting from every cell is O((RC) squared).)

sreeram

INTERVIEW ANGLE

When is BFS preferred over DFS, and conversely? "For distance or the minimum number of steps in an unweighted graph, BFS is used because it reaches every vertex by a path with the fewest edges. For reachability, component counting or backtracking, DFS is used because the recursive formulation is shorter and its memory is proportional to the depth. Both run in O(V + E). On very large or deep graphs recursive DFS is avoided in Java, since the default stack permits only some thousands of frames, and an explicit `Deque` serves as the stack." Follow-up: "How is the path itself reconstructed?" A `parent[]` array records `parent[v] = u` when `v` is first reached; the path is recovered by following parents back from the target.

5.2.6 LeetCode practice

#	Title	Level	Key idea
733	Flood Fill	Easy	DFS or BFS from the start cell; stop if the new colour equals the old one
1971	Find if Path Exists in Graph	Easy	build the adjacency list, traverse from source, check destination
200	Number of Islands	Medium	start a flood fill at every unvisited land cell, count the starts
695	Max Area of Island	Medium	flood fill returns the size; keep the maximum
133	Clone Graph	Medium	map original to copy; create on first sight
994	Rotting Oranges	Medium	multi-source BFS, process the queue level by level
542	01 Matrix	Medium	multi-source BFS from all zeros, distance fills outward
1091	Shortest Path in Binary Matrix	Medium	BFS with 8 directions, count cells on the path
785	Is Graph Bipartite?	Medium	2-colour with BFS or DFS from every unvisited vertex
417	Pacific Atlantic Water Flow	Medium	flood uphill from each ocean border, intersect the two results
127	Word Ladder	Hard	BFS over words, generate neighbours by changing one letter

LeetCode problems for this pattern.

KEY POINTS

- Build an adjacency list (O(V + E) memory); treat grids as implicit graphs with a direction array.
- BFS gives shortest paths in unweighted graphs and handles "spreads from many sources" with a multi-source start; DFS is for reachability, components and recursion.
- Mark visited at enqueue time; take a `q.size()` snapshot to count levels; iterate over all start vertices for disconnected graphs.
- Cost is O(V + E), or O(RC) on a grid; deep recursion can overflow the stack on big grids.
- If an unusual state space appears (words, puzzle boards), the states are the vertices and one move is an edge.

5.3 Topological Sort

Many tasks have precedence constraints: a course may require another as a prerequisite, a source file may require its imports to be compiled first. Representing each constraint as an arrow from the prerequisite to the dependent item yields a **directed graph**. A **topological order** is a linear arrangement of all vertices in which every arrow points forward. Such an order exists if and only if the graph contains no directed cycle (a **DAG**, directed acyclic graph).

Enumerating all n! permutations is infeasible. Two O(V + E) algorithms solve the problem, and both also decide whether a cycle exists.

5.3.1 The idea

Kahn's algorithm (BFS). The **in-degree** of a vertex is the number of arrows directed into it. A vertex with in-degree 0 has no unmet prerequisite and may be placed first. All such vertices are enqueued; the algorithm repeatedly removes one, appends it to the output and decrements the in-degree of each of its successors, enqueueing any successor whose in-degree reaches 0. If fewer than V vertices are output, the remaining vertices lie on or behind a cycle.

DFS version. A vertex is appended to a list when it is FINISHED, that is, after every vertex reachable from it has finished. The reverse of this list is a topological order, because a vertex always finishes after all of its successors. Cycle detection uses three states: 0 = unvisited, 1 = on the current DFS path, 2 = finished. Reaching a vertex in state 1 means that an ancestor on the current path has been reached again, which constitutes a cycle.

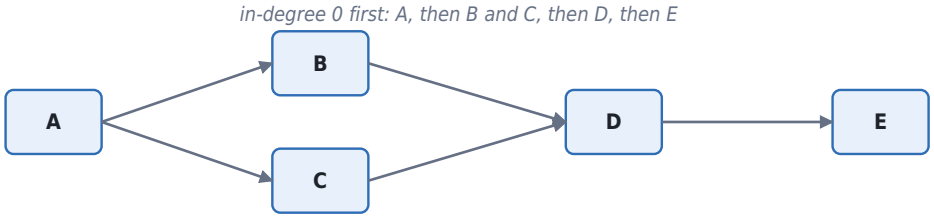


Figure 5.3. Kahn's algorithm peels off vertices with no remaining prerequisites, layer by layer; a cycle never reaches in-degree 0, so it is left over.

How to recognise it

- "prerequisites", "dependencies", "must come before", "build order", "can all tasks be finished".
- "return any valid order" or "is there a valid order" (cycle check).
- "derive the order from a sorted list" (alien dictionary: letters are vertices, the first differing letter of adjacent words is an edge).
- "longest chain / minimum semesters" over a DAG: topological layers.
- "which nodes are safe / eventually end in a terminal", computed from the end backwards.

Template (Kahn)

Template.java — JAVA

```java
int[] indeg = new int[n];
for (int[] e : edges) { g.get(e[0]).add(e[1]); indeg[e[1]]++; }   // e[0] must precede e[1]
Deque<Integer> q = new ArrayDeque<>();
for (int i = 0; i < n; i++) if (indeg[i] == 0) q.offer(i);
int done = 0;
while (!q.isEmpty()) {
    int u = q.poll();
    done++;                                  // append u to the order here
    for (int v : g.get(u)) if (--indeg[v] == 0) q.offer(v);
}
boolean hasCycle = done < n;
```

WATCH OUT

Read the direction of the edge in the statement. In LeetCode's Course Schedule, `prerequisites[i] = [a, b]` means that course b must precede course a, so the arrow is directed from b to a. Constructing the arrow in the opposite direction is a frequent error. The reversed graph still admits a valid order, but that order is the reverse of the required one.

5.3.2 Dry run

A build-order graph with 6 vertices. An edge `u -> v` means "u comes before v": `5->2, 5->0, 4->0, 4->1, 2->3, 3->1`. In-degrees are `[2, 2, 1, 1, 0, 0]`: vertex 0 has two incoming arrows (from 5 and 4), vertex 1 two (from 4 and 3), and so on. Vertices 4 and 5 start with in-degree 0, so the queue is `[4, 5]`.

Step	Pop	In-degrees after (index 0 to 5)	Queue after
0	start	`[2, 2, 1, 1, 0, 0]`	`[4, 5]`
1	4	`[1, 1, 1, 1, 0, 0]`	`[5]`

Step	Pop	In-degrees after (index 0 to 5)	Queue after
2	5	`[0, 1, 0, 1, 0, 0]`	`[2, 0]`
3	2	`[0, 1, 0, 0, 0, 0]`	`[0, 3]`
4	0	`[0, 1, 0, 0, 0, 0]`	`[3]`
5	3	`[0, 0, 0, 0, 0, 0]`	`[1]`
6	1	`[0, 0, 0, 0, 0, 0]`	`[]`

Kahn's algorithm on the 6-vertex graph.

The order is `[4, 5, 2, 0, 3, 1]`. Check it: 5 is before 2 and 0, 4 is before 0 and 1, 2 is before 3, and 3 is before 1. Every arrow points forward, so the order is valid. The answer is not unique: the DFS version produces `[5, 4, 2, 3, 1, 0]` for the same graph, which is also valid. Interviewers accept any valid order unless the problem demands the smallest one (then use a `PriorityQueue` instead of the plain queue).

In step 4, vertex 0 leaves, but its list of neighbours is empty, so nothing changes in the in-degrees. For the cyclic graph `0->1, 1->2, 2->0` every in-degree is 1, the queue starts empty, `done` stays 0, and `done < n` reports the cycle immediately.

5.3.3 Java code

The program has Kahn's algorithm with a trace, the DFS version with three states, Course Schedule I and II built on the same helper (note the swap of the edge direction in `findOrder`), and Alien Dictionary.

Main.java JAVA

```java
import java.util.*;

public class Main {
    // Kahn's algorithm. Edge {a, b} means a must come before b.
    static int[] kahn(int n, int[][] edges, boolean trace) {
        List<List<Integer>> g = new ArrayList<>();
        for (int i = 0; i < n; i++) g.add(new ArrayList<>());
        int[] indeg = new int[n];
        for (int[] e : edges) { g.get(e[0]).add(e[1]); indeg[e[1]]++; }
        Deque<Integer> q = new ArrayDeque<>();
        for (int i = 0; i < n; i++) if (indeg[i] == 0) q.offer(i);
        if (trace) System.out.println("start indeg=" + Arrays.toString(indeg) + " queue=" + q);
        int[] order = new int[n];
        int k = 0;
        while (!q.isEmpty()) {
            int u = q.poll();
            order[k++] = u;
            for (int v : g.get(u)) if (--indeg[v] == 0) q.offer(v);
            if (trace) System.out.println("pop " + u + " indeg=" + Arrays.toString(indeg)
                + " queue=" + q);
        }
        return k == n ? order : new int[0];   // empty array = cycle
    }

    // DFS version: 0 = new, 1 = on current path, 2 = finished
    static int[] state;
    static Deque<Integer> post;

    static boolean dfs(List<List<Integer>> g, int u) {
        state[u] = 1;
        for (int v : g.get(u)) {
            if (state[v] == 1) return false;                  // back edge: cycle
            if (state[v] == 0 && !dfs(g, v)) return false;
        }
        state[u] = 2;
        post.push(u);                                         // push after all descendants
        return true;
    }

    static int[] dfsOrder(int n, int[][] edges) {
        List<List<Integer>> g = new ArrayList<>();
        for (int i = 0; i < n; i++) g.add(new ArrayList<>());
        for (int[] e : edges) g.get(e[0]).add(e[1]);
        state = new int[n];
        post = new ArrayDeque<>();
```

Sreeram

Main.java (continued) JAVA

```
        for (int i = 0; i < n; i++) if (state[i] == 0 && !dfs(g, i)) return new int[0];
        return post.stream().mapToInt(Integer::intValue).toArray();
    }

    // LeetCode 210 wants prerequisites[i] = {course, prereq}
    static int[] findOrder(int numCourses, int[][] prerequisites) {
        int[][] edges = new int[prerequisites.length][];
        for (int i = 0; i < edges.length; i++)
            edges[i] = new int[]{prerequisites[i][1], prerequisites[i][0]};
        return kahn(numCourses, edges, false);
    }

    static String alienOrder(String[] words) {
        Map<Character, Set<Character>> g = new LinkedHashMap<>();
        Map<Character, Integer> indeg = new LinkedHashMap<>();
        for (String w : words) for (char c : w.toCharArray()) {
            g.putIfAbsent(c, new LinkedHashSet<>());
            indeg.putIfAbsent(c, 0);
        }
        for (int i = 0; i + 1 < words.length; i++) {
            String a = words[i], b = words[i + 1];
            if (a.length() > b.length() && a.startsWith(b)) return "";   // "abc" before "ab"
            for (int j = 0; j < Math.min(a.length(), b.length()); j++) {
                char x = a.charAt(j), y = b.charAt(j);
                if (x != y) {
                    if (g.get(x).add(y)) indeg.merge(y, 1, Integer::sum);
                    break;
                }
            }
        }
        Deque<Character> q = new ArrayDeque<>();
        indeg.forEach((c, d) -> { if (d == 0) q.offer(c); });
        StringBuilder sb = new StringBuilder();
        while (!q.isEmpty()) {
            char c = q.poll();
            sb.append(c);
            for (char nxt : g.get(c)) if (indeg.merge(nxt, -1, Integer::sum) == 0) q.offer(nxt);
        }
        return sb.length() == indeg.size() ? sb.toString() : "";
    }

    static boolean canFinish(int n, int[][] prereq) {
        return findOrder(n, prereq).length == n;
    }

    public static void main(String[] args) {
        // build-order style graph: 6 nodes
        int[][] edges = {{5, 2}, {5, 0}, {4, 0}, {4, 1}, {2, 3}, {3, 1}};
        System.out.println("--- Kahn ---");
        int[] order = kahn(6, edges, true);
        System.out.println("order: " + Arrays.toString(order));
        System.out.println("dfs order: " + Arrays.toString(dfsOrder(6, edges)));
        int[][] ring = {{0, 1}, {1, 2}, {2, 0}};
        System.out.println("cycle (kahn): " + Arrays.toString(kahn(3, ring, false)));
        System.out.println("cycle (dfs): " + Arrays.toString(dfsOrder(3, ring)));
        System.out.println("canFinish 2, [[1,0]]: " + canFinish(2, new int[][]{{1, 0}}));
        int[][] loop2 = {{1, 0}, {0, 1}};
        System.out.println("canFinish 2, [[1,0],[0,1]]: " + canFinish(2, loop2));
        System.out.println("findOrder 4: " + Arrays.toString(
            findOrder(4, new int[][]{{1, 0}, {2, 0}, {3, 1}, {3, 2}})));
        System.out.println("alien: " + alienOrder(new String[]{"wrt", "wrf", "er", "ett", "rftt"}));
        System.out.println("alien invalid: '" + alienOrder(new String[]{"abc", "ab"}) + "'");
    }
}
```

Compile and run TERMINAL

```
$ java Main.java
--- Kahn ---
start indeg=[2, 2, 1, 1, 0, 0] queue=[4, 5]
pop 4 indeg=[1, 1, 1, 1, 0, 0] queue=[5]
pop 5 indeg=[0, 1, 0, 1, 0, 0] queue=[2, 0]
pop 2 indeg=[0, 1, 0, 0, 0, 0] queue=[0, 3]
pop 0 indeg=[0, 1, 0, 0, 0, 0] queue=[3]
```

Compile and run (continued) TERMINAL

```
pop 3 indeg=[0, 0, 0, 0, 0, 0] queue=[1]
pop 1 indeg=[0, 0, 0, 0, 0, 0] queue=[]
order: [4, 5, 2, 0, 3, 1]
dfs order: [5, 4, 2, 3, 1, 0]
cycle (kahn): []
cycle (dfs): []
canFinish 2, [[1,0]]: true
canFinish 2, [[1,0],[0,1]]: false
findOrder 4: [0, 1, 2, 3]
alien: wertf
alien invalid: ''
```

What to notice

- `kahn` returns an empty array on a cycle, so the caller tests `order.length == n`. Returning `null` or a flag also works; just be consistent.
- `dfs` pushes the vertex AFTER the loop over neighbours. Pushing before the loop gives a wrong order. Using `Deque.push` puts the newest finished vertex at the front, which performs the reversal for free.
- `findOrder` converts LeetCode's `{course, prereq}` into the arrow `prereq -> course`. `canFinish` is just "does an order of length n exist".
- `alienOrder` has two traps: a prefix such as `"abc"` before `"ab"` is invalid input (return empty), and vertices must be created for ALL letters, including those that appear in no comparison, or they vanish from the answer. Also, add each edge only once (the `g.get(x).add(y)` check) so in-degrees are not counted twice. This problem is Premium on LeetCode, so it is not in the practice table, but it is a favourite in interviews.
- Recursive DFS can overflow the stack on a path graph with 100,000 vertices; Kahn's algorithm is not recursive and is therefore the safer default.

5.3.4 Complexity and mistakes

Both algorithms touch each vertex once and each edge once: **O(V + E)** time. Space is O(V + E) for the graph plus O(V) for the in-degree array and queue (or the recursion stack and state array in DFS). For Alien Dictionary, V is at most 26 and E is bounded by the number of adjacent word pairs, so the dominant cost is reading the words: O(total characters).

Common mistakes:

- Reversing the edge direction (see the warning above).
- Not creating vertices that have no edges. A course with no prerequisites and no dependents still belongs in the output.
- Checking for a cycle only from vertex 0 in DFS. Start from every unvisited vertex.
- Using a single `visited` array in DFS cycle detection. A finished vertex that is reached again from another branch is legitimate; only a vertex on the CURRENT path signals a cycle. The third state encodes this distinction.
- Counting a duplicate edge twice in the in-degree array but only once in the adjacency set (or the reverse). Count each edge exactly once on both sides, as `alienOrder` does with `g.get(x).add(y)`.
- Assuming that the topological order is unique and comparing the output with the expected one literally. The correct check validates every edge.

5.3.5 Questions to expect

1. Can all courses be finished given the prerequisite pairs? (clue: "can all be finished"; approach: Kahn, finished count equals n, or DFS with three states.)
2. Return an order in which to take the courses. (same, return the order or an empty array.)
3. Given a sorted list of words in an alien language, recover the alphabet. (approach: compare adjacent words, first differing letters give an edge, topological sort; watch the prefix case. Follow-up: when is the answer not unique? When the queue holds more than one vertex at some step.)
4. Given projects and dependencies, find a build order. (the classic "Build Order" question; same algorithm, report an error on a cycle.)
5. Find eventual safe states: nodes from which every path ends in a terminal node. (approach: reverse the graph and run Kahn from the terminals, or DFS with three states where "finished" means safe.)

sreeram

6. Find minimum height trees. (approach: peel leaves layer by layer, Kahn on an undirected tree, until one or two nodes remain.)
7. How many semesters are needed if any number of courses may be taken per semester? (approach: Kahn level by level; the number of levels is the answer. This is the level-counting technique used for minutes in BFS.)
8. Give the lexicographically smallest valid order. (approach: replace the queue with a min-heap.)

INTERVIEW ANGLE

How is a cycle in a directed graph detected? "Two methods apply. With Kahn's algorithm, vertices of in-degree zero are removed repeatedly; if fewer than V vertices are removed, the remaining ones lie on or behind a cycle. With DFS, three states are kept per vertex and a cycle is reported when a vertex still on the current recursion path is reached. Both run in O(V + E). Kahn's algorithm is preferable for deep graphs because it is not recursive; DFS is preferable when a value must also be computed bottom-up." Follow-up: "Why does a single visited array fail for DFS?" Because in a DAG two paths may legally reach the same vertex (a diamond shape); only reaching an ancestor on the current path is a cycle.

5.3.6 LeetCode practice

#	Title	Level	Key idea
207	Course Schedule	Medium	cycle detection: Kahn finishes all n vertices or DFS finds no back edge
210	Course Schedule II	Medium	Kahn's order, empty array if a cycle remains
802	Find Eventual Safe States	Medium	reverse graph and Kahn from terminals, or DFS with three states
310	Minimum Height Trees	Medium	peel leaves layer by layer; the last one or two nodes are the roots
2115	Find All Possible Recipes from Given Supplies	Medium	recipes are vertices, ingredients are in-edges; Kahn from supplies
1462	Course Schedule IV	Medium	topological order, propagate reachability sets (or Floyd-Warshall)
329	Longest Increasing Path in a Matrix	Hard	cells form a DAG by value; DFS with memo or Kahn layers

LeetCode problems for this pattern.

KEY POINTS

- A topological order exists only for a DAG; Kahn (in-degree queue) and reverse-DFS-finish-order both take O(V + E).
- If Kahn's output has fewer than V vertices, there is a cycle; in DFS use three states (new, on path, done).
- Check the edge direction in the statement, include isolated vertices, and run DFS from every unvisited vertex.
- The order is rarely unique; use a heap instead of a queue for the smallest order, and count BFS levels for "number of rounds".
- Alien Dictionary: edges come from the first differing letters of adjacent words, and a longer word before its own prefix is invalid.

5.4 Union-Find (Disjoint Set Union)

Consider a sequence of edges inserted one at a time, interleaved with queries of the form "are x and y in the same group?" or "how many groups exist?". Rebuilding the graph and running BFS after each insertion costs O(V + E) per query, which is prohibitive over many queries. **Union-Find**, also called **disjoint set union (DSU)**, maintains the groups as a forest and answers each query in nearly constant amortised time.

The structure supports two operations. `find(x)` returns the representative (the root) of the group containing `x`. `union(a, b)` merges the groups of `a` and `b`. Two items are in the same group if and only if their roots are equal. The structure only merges groups and cannot split them; this limitation determines when another approach is required.

5.4.1 The idea

Each element stores a parent reference; a root refers to itself. The operation `find` follows parent references to the root, and `union` makes one root the parent of the other. Without further measures the trees can degenerate into long chains, so two optimisations bound the height:

- **Path compression**: during `find`, redirect nodes closer to the root, so the next `find` on them is short.
- **Union by rank (or size)**: attach the shorter tree under the taller one, so trees stay shallow.

With both, the amortised cost per operation is O(alpha(n)), where alpha is the inverse Ackermann function, below 5 for any input that fits in a computer. Call it constant. With only one of the two tricks it is O(log n) amortised, which is still fine for interviews.

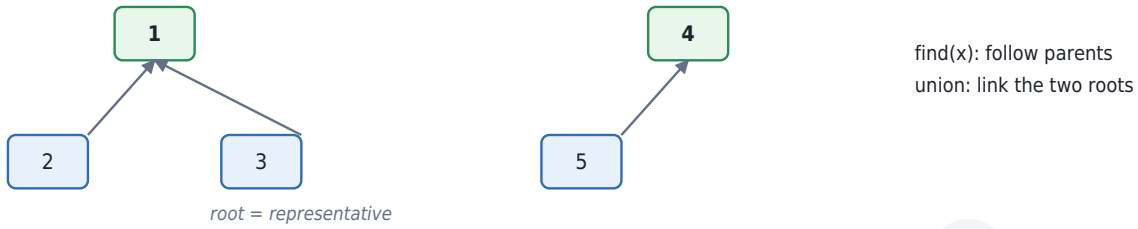


Figure 5.4. Union-Find as a forest: union attaches one root under the other, and find follows parent links to the root, flattening the path on the way.

How to recognise it

- "connected components" in a graph where edges arrive over time, or where only the COUNT or the same-group test.
- "are these two in the same group", "merge groups", "number of groups after each operation".
- "find the edge that creates a cycle" (an edge whose endpoints already share a root).
- "merge accounts / sets that share an element" (accounts merge, equality equations, similar strings).
- "minimum spanning tree" or "cheapest way to connect everything" (Kruskal).

Template

DsuTemplate.java — JAVA

```java
class DSU {
    final int[] parent, rank;
    int sets;                                   // number of groups
    DSU(int n) {
        parent = new int[n]; rank = new int[n]; sets = n;
        for (int i = 0; i < n; i++) parent[i] = i;
    }
    int find(int x) {
        while (parent[x] != x) {
            parent[x] = parent[parent[x]];      // path halving
            x = parent[x];
        }
        return x;
    }
    boolean union(int a, int b) {               // false if already in the same group
        int ra = find(a), rb = find(b);
        if (ra == rb) return false;
        if (rank[ra] < rank[rb]) { int t = ra; ra = rb; rb = t; }
        parent[rb] = ra;
        if (rank[ra] == rank[rb]) rank[ra]++;
        sets--;
        return true;
    }
}
```

The loop in `find` is **path halving**, an iterative form of path compression: every visited node is pointed at its grandparent. It needs no recursion, so it cannot overflow the stack on a long chain. The Boolean returned by `union` is the key to most interview problems: `false` means "these two were already connected", which is exactly a cycle in an undirected graph.

sreeram

5.4.2 Dry run

Problem (Redundant Connection): a tree with nodes 1 to 3 had one extra edge added. The edges are `[1,2]`, `[1,3]`, `[2,3]`. Return the edge that closes a cycle. The DSU has `n + 1 = 4` slots (index 0 is unused because nodes start at 1). Initially `parent = [0, 1, 2, 3]` and `rank = [0, 0, 0, 0]`.

Step	Edge	find results	parent array and rank after	Merged?
1	1-2	find(1)=1, find(2)=2	parent [0,1,1,3] rank [0,1,0,0]	yes: 2 hangs under 1
2	1-3	find(1)=1, find(3)=3	parent [0,1,1,1] rank [0,1,0,0]	yes: 3 hangs under 1
3	2-3	find(2)=1, find(3)=1	parent [0,1,1,1] rank [0,1,0,0]	no: same root, cycle

Redundant connection, edges processed in order.

At step 1 both ranks are 0, so the rule `rank[ra] < rank[rb]` is false, node 1 (the first argument) becomes the root, and its rank grows to 1. At step 2 the root 1 has rank 1 and node 3 has rank 0, so 3 goes under 1 and the rank stays 1. At step 3 both `find` calls return 1. Everything is already connected, so `[2, 3]` is the redundant edge. If several edges could be removed, the problem wants the one that appears last in the input, which is why the first edge that fails to merge during an in-order scan is returned.

The same code also shows the effect of path halving. A chain `0 <- 1 <- 2 <- ... <- 7` (each node's parent is the previous one) becomes `[0, 0, 1, 1, 3, 3, 5, 5]` after a single `find(7)`. The path from 7 now needs about half the steps, and repeated finds flatten it further.

5.4.3 Java code

The program has the DSU class, Redundant Connection with the trace, Number of Provinces (components of an adjacency matrix), Accounts Merge (union accounts that share an email), and Kruskal's minimum spanning tree.

Main.java JAVA

```java
import java.util.*;

public class Main {
    static class DSU {
        final int[] parent, rank;
        int sets;
        DSU(int n) {
            parent = new int[n]; rank = new int[n]; sets = n;
            for (int i = 0; i < n; i++) parent[i] = i;
        }
        int find(int x) {
            while (parent[x] != x) {
                parent[x] = parent[parent[x]];   // path halving
                x = parent[x];
            }
            return x;
        }
        boolean union(int a, int b) {
            int ra = find(a), rb = find(b);
            if (ra == rb) return false;
            if (rank[ra] < rank[rb]) { int t = ra; ra = rb; rb = t; }
            parent[rb] = ra;
            if (rank[ra] == rank[rb]) rank[ra]++;
            sets--;
            return true;
        }
    }

    static int[] redundant(int[][] edges, boolean trace) {
        DSU dsu = new DSU(edges.length + 1);
        for (int[] e : edges) {
            boolean merged = dsu.union(e[0], e[1]);
            if (trace) System.out.println("union(" + e[0] + "," + e[1] + ") merged=" + merged
                + " parent=" + Arrays.toString(dsu.parent) + " rank=" + Arrays.toString(dsu.rank));
            if (!merged) return e;
        }
        return new int[0];
    }
```

Main.java (continued) JAVA

```
    static int provinces(int[][] isConnected) {
        int n = isConnected.length;
        DSU dsu = new DSU(n);
        for (int i = 0; i < n; i++)
            for (int j = i + 1; j < n; j++)
                if (isConnected[i][j] == 1) dsu.union(i, j);
        return dsu.sets;
    }

    static List<List<String>> accountsMerge(List<List<String>> accounts) {
        DSU dsu = new DSU(accounts.size());
        Map<String, Integer> owner = new HashMap<>();
        for (int i = 0; i < accounts.size(); i++) {
            for (String email : accounts.get(i).subList(1, accounts.get(i).size())) {
                Integer prev = owner.putIfAbsent(email, i);
                if (prev != null) dsu.union(prev, i);
            }
        }
        Map<Integer, TreeSet<String>> groups = new TreeMap<>();
        for (var e : owner.entrySet())
            groups.computeIfAbsent(dsu.find(e.getValue()), k -> new TreeSet<>()).add(e.getKey());
        List<List<String>> res = new ArrayList<>();
        for (var g : groups.entrySet()) {
            List<String> row = new ArrayList<>();
            row.add(accounts.get(g.getKey()).get(0));
            row.addAll(g.getValue());
            res.add(row);
        }
        return res;
    }

    // Kruskal: sort edges {u, v, w} by weight, take an edge if it joins two components
    static int kruskal(int n, int[][] edges, boolean trace) {
        int[][] sorted = edges.clone();
        Arrays.sort(sorted, Comparator.comparingInt(e -> e[2]));
        DSU dsu = new DSU(n);
        int total = 0, used = 0;
        for (int[] e : sorted) {
            boolean take = dsu.union(e[0], e[1]);
            if (trace) System.out.println("edge " + e[0] + "-" + e[1] + " w=" + e[2]
                + (take ? " take" : " skip (cycle)"));
            if (take) { total += e[2]; used++; }
        }
        return used == n - 1 ? total : -1;
    }

    public static void main(String[] args) {
        System.out.println("--- redundant connection ---");
        System.out.println("answer: " + Arrays.toString(
            redundant(new int[][]{{1, 2}, {1, 3}, {2, 3}}, true)));
        System.out.println("answer: " + Arrays.toString(
            redundant(new int[][]{{1, 2}, {2, 3}, {3, 4}, {1, 4}, {1, 5}}, false)));
        System.out.println("provinces: " + provinces(new int[][]{{1, 1, 0}, {1, 1, 0}, {0, 0, 1}}));
        var accounts = List.of(
            List.of("John", "johnsmith@mail.com", "john_newyork@mail.com"),
            List.of("John", "johnsmith@mail.com", "john00@mail.com"),
            List.of("Mary", "mary@mail.com"),
            List.of("John", "johnnybravo@mail.com"));
        for (var row : accountsMerge(accounts)) System.out.println("merged: " + row);
        System.out.println("--- Kruskal ---");
        int[][] edges = {{0, 1, 4}, {0, 2, 3}, {1, 2, 1}, {1, 3, 2}, {2, 3, 4}, {3, 4, 2}};
        System.out.println("MST weight: " + kruskal(5, edges, true));
        // path compression effect on a chain built the worst way (no rank)
        int n = 8;
        int[] parent = new int[n];
        for (int i = 0; i < n; i++) parent[i] = Math.max(i - 1, 0);
        System.out.println("chain before: " + Arrays.toString(parent));
        int x = n - 1;
        while (parent[x] != x) { parent[x] = parent[parent[x]]; x = parent[x]; }
        System.out.println("after find(7) with halving: " + Arrays.toString(parent));
    }
}
```

freeram

Compile and run TERMINAL

```
$ java Main.java
--- redundant connection ---
union(1,2) merged=true parent=[0, 1, 1, 3] rank=[0, 1, 0, 0]
union(1,3) merged=true parent=[0, 1, 1, 1] rank=[0, 1, 0, 0]
union(2,3) merged=false parent=[0, 1, 1, 1] rank=[0, 1, 0, 0]
answer: [2, 3]
answer: [1, 4]
provinces: 2
merged: [John, john00@mail.com, john_newyork@mail.com, johnsmith@mail.com]
merged: [Mary, mary@mail.com]
merged: [John, johnnybravo@mail.com]
--- Kruskal ---
edge 1-2 w=1 take
edge 1-3 w=2 take
edge 3-4 w=2 take
edge 0-2 w=3 take
edge 0-1 w=4 skip (cycle)
edge 2-3 w=4 skip (cycle)
MST weight: 8
chain before: [0, 0, 1, 2, 3, 4, 5, 6]
after find(7) with halving: [0, 0, 1, 1, 3, 3, 5, 5]
```

What to notice

- `provinces` returns `dsu.sets`, the counter maintained inside `union`; counting roots at the end is unnecessary.
- `accountsMerge` does not union emails. It maps each email to the FIRST account index that owned it; the next time the same email shows up, it unions the two account indices. Then it groups emails by `find(account)`. The `TreeSet` gives the sorted emails the problem asks for, and the `TreeMap` keeps groups in a stable order.
- `kruskal` sorts the edge list by weight with a comparator that uses `Integer.compare` semantics through `comparingInt`, then takes every edge whose endpoints are in different groups. A **minimum spanning tree** (MST) connects all V vertices with V - 1 edges and the smallest total weight. Here the weight is 1 + 2 + 2 + 3 = 8. If fewer than V - 1 edges are taken, the graph is disconnected and there is no spanning tree.
- Index mapping: when the input uses labels (emails, letters, coordinates), map them to integers first, or write a DSU over a `HashMap`. Stone problems (LeetCode 947) union row `r` with column `c + 10001`, so rows and columns share one array.

> **NOTE**
> Union by rank is optional in an interview. Path compression alone is O(log n) amortised and is what most people write. Mention union by rank, show it, and move on. The failure to avoid is a `find` without any compression, which degrades to O(n) per call.

5.4.4 Complexity and mistakes

Creating the DSU is O(n). Each `find` and `union` is O(alpha(n)), effectively constant, amortised over the sequence. So processing E edges takes about O(E alpha(V)) and O(V) memory. Kruskal is dominated by sorting: O(E log E). For comparison, BFS over a static graph is O(V + E) once, so DSU wins when edges arrive online or when only the connectivity test is required; it loses when paths, distances or the explicit members of each component are required without extra bookkeeping.

Common mistakes:

- Calling `union(a, b)` without `find`: setting `parent[a] = b` directly links two non-root nodes and silently splits groups.
- Forgetting that elements may be numbered from 1 (use size `n + 1`) or that labels need an index map.
- Reading `parent[x]` as the group id after a union. Only `find(x)` is guaranteed to be the current root.
- Recursive `find` on a long chain: `StackOverflowError`. The iterative version above avoids it.
- Counting groups by `n - number of successful unions` and then also decrementing in another place. Keep one counter, updated only in `union` when it returns true.
- Expecting to delete an edge or split a group. Plain DSU cannot. Reverse the process (process deletions backwards as additions), or use another approach.
- In Kruskal, sorting with `(a, b) -> a[2] - b[2]`: fine for small weights, a bug if weights can overflow. Prefer `Integer.compare`.

5.4.5 Questions to expect

1. Count the connected components / provinces. (clue: "groups of directly or indirectly connected"; approach: union every edge, return the counter. DFS also works, but DSU is shorter and extends to the next questions.)
2. Which edge makes a tree into a graph with a cycle? (clue: "redundant edge"; approach: the first edge whose endpoints already share a root. Follow-up: in a DIRECTED graph (Redundant Connection II) a node can also have two parents, which needs extra case analysis.)
3. Merge accounts that share an email. (approach: map email to first owner, union owners, group emails by root, sort.)
4. Are all the equality and inequality equations satisfiable? (`a==b`, `b!=c`) (approach: union all `==` first, then check that no `!=` pair shares a root.)
5. Minimum cost to connect all points / cities. (approach: Kruskal on all pair distances, or Prim with a heap. Follow-up: how many edges does an MST have? V - 1.)
6. How many cables must be moved to connect a network? (approach: if edges < V - 1 return -1, else answer = components - 1.)
7. Number of islands II (islands appear one cell at a time, report the count after each; this one is Premium). (approach: DSU over cells, add a cell, union with land neighbours, report `sets`.)
8. Swap characters at given index pairs to get the smallest string. (approach: DSU groups indices that can reach each other; sort the characters within each group and put them back in order.)

INTERVIEW ANGLE

Why use Union-Find instead of DFS for connected components? "For a single static graph the costs are identical, O(V + E), and the simpler method is chosen. DSU is superior when edges arrive incrementally or when connectivity queries are interleaved with insertions, because each query costs almost O(1) instead of a new traversal. It also detects cycles in an undirected graph directly: an edge whose endpoints already share a root closes a cycle. It cannot split a group, produce a path or report distances." Follow-up: "What do path compression and union by rank achieve?" Without them a chain makes `find` cost O(n). With both, the amortised cost per operation is the inverse Ackermann function, at most 4 or 5 for any realistic input.

5.4.6 LeetCode practice

#	Title	Level	Key idea
547	Number of Provinces	Medium	union every pair with a 1 in the matrix; the answer is the set counter
684	Redundant Connection	Medium	return the first edge whose endpoints already share a root
1319	Number of Operations to Make Network Connected	Medium	need at least n-1 cables; answer = components - 1
990	Satisfiability of Equality Equations	Medium	union all equalities, then check each inequality
721	Accounts Merge	Medium	union accounts sharing an email, group and sort emails
947	Most Stones Removed with Same Row or Column	Medium	union row with column; answer = stones - components
1202	Smallest String With Swaps	Medium	components of indices; sort characters inside each
399	Evaluate Division	Medium	weighted DSU or graph search over ratios
1584	Min Cost to Connect All Points	Medium	Kruskal over all pairs with Manhattan distance

LeetCode problems for this pattern.

sreeram

KEY POINTS

- Union-Find tracks groups under merging: `find` returns the root, `union` joins two roots, same root means same group.
- Path compression plus union by rank gives near-constant amortised time per operation (inverse Ackermann).
- `union` returning false means "already connected", which is cycle detection in an undirected graph and the heart of Kruskal's MST.
- Keep one `sets` counter updated inside `union`; map labels to integer ids before using arrays.
- It cannot split groups or produce paths; BFS or DFS applies when those are required.

5.5 Shortest Paths

BFS yields shortest paths when every edge has unit cost. When edges carry weights (distances, prices, delays), the path with the fewest edges is not necessarily the cheapest. Enumerating all paths is exponential. Shortest-path algorithms instead exploit optimal substructure: the best route to a vertex is the best route to a predecessor extended by one edge. The choice of algorithm depends on the edge weights.

Situation	Algorithm	Time	Note
All edges equal	BFS	O(V + E)	levels are distances
Weights are 0 or 1	0-1 BFS (deque)	O(V + E)	push 0-edges to the front
Non-negative weights	Dijkstra (heap)	O((V + E) log V)	the default answer
Limit on number of edges	Bellman-Ford rounds	O(KE)	also handles negative weights
Negative weights, no limit	Bellman-Ford	O(VE)	detects negative cycles
All pairs, V small	Floyd-Warshall	O(V cubed)	three nested loops

Choosing a shortest-path algorithm.

5.5.1 The idea

Dijkstra's algorithm maintains for every vertex a tentative distance `dist[v]`, initialised to infinity except `dist[source] = 0`. It repeatedly extracts the unfinished vertex with the smallest tentative distance, using a `PriorityQueue` in O(log n). For each edge `u -> v` of weight `w`, if `dist[u] + w < dist[v]`, the value `dist[v]` is lowered and the pair is inserted into the queue. This step is called **relaxation**.

Why it works: when the smallest tentative distance is popped, no unfinished vertex can offer a shorter route to it, because every other route must pass through a vertex whose distance is at least as large, and adding a non-negative edge cannot make it smaller. That argument is exactly why **Dijkstra fails with negative edges**: a later negative edge could improve a vertex that has already been finalised.

Java's `PriorityQueue` has no decrease-key operation. The standard workaround inserts a NEW entry `(vertex, newDistance)` on each improvement and discards an extracted entry as stale when its distance exceeds the current `dist[vertex]`. The queue holds at most E entries, which preserves the bound O(E log V).

How to recognise it

- "minimum cost / time / distance" with edge weights, travel between places.
- "time for a signal to reach every node" (the maximum over the shortest distances).
- "path where the cost is the maximum (or product) along the path" (a Dijkstra variant: change the combine step).
- "at most k stops/edges" (Bellman-Ford rounds, or BFS with a state of (node, stops used)).
- "cells with different move costs", "break at most k walls": a grid with weights.
- "shortest distance between all pairs, n up to about 100": Floyd-Warshall.

Template (Dijkstra)

DijkstraTemplate.java JAVA

```java
int[] dist = new int[n];
Arrays.fill(dist, INF);                                  // INF = Integer.MAX_VALUE / 2
dist[src] = 0;
PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> Integer.compare(a[1], b[1]));
pq.offer(new int[]{src, 0});                             // {vertex, distance}
while (!pq.isEmpty()) {
    int[] top = pq.poll();
    int u = top[0], d = top[1];
    if (d > dist[u]) continue;                           // stale entry
    for (int[] e : g.get(u)) {                           // e = {neighbour, weight}
        if (d + e[1] < dist[e[0]]) {
            dist[e[0]] = d + e[1];
            pq.offer(new int[]{e[0], dist[e[0]]});
        }
    }
}
```

WATCH OUT

Use `Integer.MAX_VALUE / 2` (or `Long.MAX_VALUE / 2`) as infinity, never `Integer.MAX_VALUE`. `INF + weight` would overflow to a huge negative number and look like the best path ever. The program below prints `INF + INF` and `Integer.MAX_VALUE + 1` to show the difference. Also check `dist[u] < INF` before relaxing from a vertex in Bellman-Ford.

5.5.2 Dry run

Directed weighted graph with 5 vertices (edges as `u -> v : w`): `0->1:4, 0->2:1, 2->1:2, 1->3:1, 2->3:5, 3->4:3`. Find the shortest distance from vertex 0. The direct edge `0 -> 1` costs 4, but going through 2 costs 1 + 2 = 3, so the obvious edge is not the best. Each row is one pop from the priority queue; `dist` is printed after relaxing the popped vertex's edges, `inf` means not reached yet.

Step	Pop (vertex, d)	dist after (vertices 0 to 4)	Queue size
1	(0, 0)	[0, 4, 1, inf, inf]	2
2	(2, 1)	[0, 3, 1, 6, inf]	3
3	(1, 3)	[0, 3, 1, 4, inf]	3
4	(1, 4)	unchanged: stale, skip (3 < 4)	-
5	(3, 4)	[0, 3, 1, 4, 7]	2
6	(3, 6)	unchanged: stale, skip (4 < 6)	-
7	(4, 7)	[0, 3, 1, 4, 7]	0

Dijkstra from vertex 0.

Step 1 sets `dist[1] = 4` and `dist[2] = 1`. Step 2 pops the cheaper vertex 2 first. It improves vertex 1 from 4 to 3 and gives vertex 3 its first value, 1 + 5 = 6. Step 3 pops (1, 3): vertex 3 improves from 6 to 4. The queue still holds the old entries (1, 4) and (3, 6); they are popped later and ignored because their distance exceeds `dist` for that vertex (steps 4 and 6). Final answer: `[0, 3, 1, 4, 7]`.

Now the "at most k stops" case with Bellman-Ford. Flights `0->1:100, 1->2:100, 2->0:100, 1->3:600, 2->3:200`, from 0 to 3. A round relaxes every edge using the distances from the PREVIOUS round, so after round `r` the array holds the best cost using at most `r` flights.

Rounds done	dist for vertices 0 to 3
0	[0, inf, inf, inf]
1	[0, 100, inf, inf]
2	[0, 100, 200, 700]
3	[0, 100, 200, 400]

Bellman-Ford with edge limit.

With k = 1 stop, 2 flights are permitted (k + 1 edges), so the answer is `dist[3]` after 2 rounds: 700 (via 0, 1, 3). With k = 2 stops, 3 flights are allowed and the cheaper route 0, 1, 2, 3 costs 400. Dijkstra would answer 400 regardless of stops, which is why it is the wrong tool when the number of edges is limited.

5.5.3 Java code

The program has Dijkstra with a trace, Network Delay Time, Cheapest Flights Within K Stops (Bellman-Ford), Path With Minimum Effort (Dijkstra where the path cost is the MAXIMUM step), a 0-1 BFS for the minimum obstacles to remove, and Floyd-Warshall.

Main.java JAVA

```
import java.util.*;

public class Main {
    static final int INF = Integer.MAX_VALUE / 2;

    // edges {u, v, w}, directed
    static int[] dijkstra(int n, int[][] edges, int src, boolean trace) {
        List<List<int[]>> g = new ArrayList<>();
        for (int i = 0; i < n; i++) g.add(new ArrayList<>());
        for (int[] e : edges) g.get(e[0]).add(new int[]{e[1], e[2]});
        int[] dist = new int[n];
        Arrays.fill(dist, INF);
        dist[src] = 0;
        PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> Integer.compare(a[1], b[1]));
        pq.offer(new int[]{src, 0});
        int step = 0;
        while (!pq.isEmpty()) {
            int[] top = pq.poll();
            int u = top[0], d = top[1];
            if (d > dist[u]) {
                if (trace) System.out.println(++step + ": pop (" + u + "," + d + ") stale, skip");
                continue;
            }
            for (int[] e : g.get(u)) {
                if (d + e[1] < dist[e[0]]) {
                    dist[e[0]] = d + e[1];
                    pq.offer(new int[]{e[0], dist[e[0]]});
                }
            }
            if (trace) System.out.println(++step + ": pop (" + u + "," + d + ") dist="
                + Arrays.toString(dist).replace("" + INF, "inf") + " pq size=" + pq.size());
        }
        return dist;
    }

    static int networkDelay(int[][] times, int n, int k) {
        int[] dist = dijkstra(n + 1, times, k, false);
        int max = 0;
        for (int i = 1; i <= n; i++) max = Math.max(max, dist[i]);
        return max >= INF ? -1 : max;
    }

    // Bellman-Ford limited to k+1 edges
    static int cheapest(int n, int[][] flights, int src, int dst, int k, boolean trace) {
        int[] dist = new int[n];
        Arrays.fill(dist, INF);
        dist[src] = 0;
        for (int round = 0; round <= k; round++) {
            int[] next = dist.clone();                  // read old, write new
            for (int[] f : flights)
                if (dist[f[0]] < INF && dist[f[0]] + f[2] < next[f[1]])
                    next[f[1]] = dist[f[0]] + f[2];
            dist = next;
            if (trace) System.out.println("after " + (round + 1) + " edge(s): "
                + Arrays.toString(dist).replace("" + INF, "inf"));
        }
        return dist[dst] >= INF ? -1 : dist[dst];
    }

    static int minEffort(int[][] h) {
        int rows = h.length, cols = h[0].length;
        int[][] best = new int[rows][cols];
        for (int[] row : best) Arrays.fill(row, INF);
```

Sreeram

Main.java (continued) JAVA

```
        best[0][0] = 0;
        PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> Integer.compare(a[0], b[0]));
        pq.offer(new int[]{0, 0, 0});
        int[][] dirs = {{1, 0}, {-1, 0}, {0, 1}, {0, -1}};
        while (!pq.isEmpty()) {
            int[] t = pq.poll();
            int e = t[0], r = t[1], c = t[2];
            if (e > best[r][c]) continue;
            if (r == rows - 1 && c == cols - 1) return e;
            for (int[] d : dirs) {
                int nr = r + d[0], nc = c + d[1];
                if (nr < 0 || nc < 0 || nr >= rows || nc >= cols) continue;
                int ne = Math.max(e, Math.abs(h[nr][nc] - h[r][c]));   // path cost is a MAX
                if (ne < best[nr][nc]) { best[nr][nc] = ne; pq.offer(new int[]{ne, nr, nc}); }
            }
        }
        return 0;
    }

    // 0-1 BFS: grid cell 1 = obstacle (cost 1), 0 = free (cost 0)
    static int minObstacles(int[][] grid) {
        int rows = grid.length, cols = grid[0].length;
        int[][] dist = new int[rows][cols];
        for (int[] row : dist) Arrays.fill(row, INF);
        Deque<int[]> dq = new ArrayDeque<>();
        dist[0][0] = 0;
        dq.offerFirst(new int[]{0, 0});
        int[][] dirs = {{1, 0}, {-1, 0}, {0, 1}, {0, -1}};
        while (!dq.isEmpty()) {
            int[] cur = dq.pollFirst();
            for (int[] d : dirs) {
                int nr = cur[0] + d[0], nc = cur[1] + d[1];
                if (nr < 0 || nc < 0 || nr >= rows || nc >= cols) continue;
                int w = grid[nr][nc];
                if (dist[cur[0]][cur[1]] + w < dist[nr][nc]) {
                    dist[nr][nc] = dist[cur[0]][cur[1]] + w;
                    int[] cell = {nr, nc};
                    if (w == 0) dq.offerFirst(cell); else dq.offerLast(cell);
                }
            }
        }
        return dist[rows - 1][cols - 1];
    }

    static int[][] floyd(int n, int[][] edges) {
        int[][] d = new int[n][n];
        for (int[] row : d) Arrays.fill(row, INF);
        for (int i = 0; i < n; i++) d[i][i] = 0;
        for (int[] e : edges) d[e[0]][e[1]] = Math.min(d[e[0]][e[1]], e[2]);
        for (int k = 0; k < n; k++)               // intermediate vertex MUST be the outer loop
            for (int i = 0; i < n; i++)
                for (int j = 0; j < n; j++)
                    if (d[i][k] + d[k][j] < d[i][j]) d[i][j] = d[i][k] + d[k][j];
        return d;
    }

    public static void main(String[] args) {
        int[][] edges = {{0, 1, 4}, {0, 2, 1}, {2, 1, 2}, {1, 3, 1}, {2, 3, 5}, {3, 4, 3}};
        System.out.println("--- Dijkstra from 0 ---");
        int[] dist = dijkstra(5, edges, 0, true);
        System.out.println("final: " + Arrays.toString(dist));
        System.out.println("network delay: "
            + networkDelay(new int[][]{{2, 1, 1}, {2, 3, 1}, {3, 4, 1}}, 4, 2));
        System.out.println("--- Bellman-Ford ---");
        int[][] flights = {{0, 1, 100}, {1, 2, 100}, {2, 0, 100}, {1, 3, 600}, {2, 3, 200}};
        System.out.println("cheapest k=1: " + cheapest(4, flights, 0, 3, 1, false));
        System.out.println("cheapest k=2: " + cheapest(4, flights, 0, 3, 2, true));
        int[][] heights = {{1, 2, 2}, {3, 8, 2}, {5, 3, 5}};
        System.out.println("min effort: " + minEffort(heights));
        System.out.println("min obstacles: "
            + minObstacles(new int[][]{{0, 1, 1}, {1, 1, 0}, {1, 1, 0}}));
        int[][] fw = floyd(4, new int[][]{{0, 1, 3}, {1, 2, 1}, {2, 3, 1}, {0, 3, 10}});
        for (int[] row : fw) System.out.println(Arrays.toString(row).replace("" + INF, "inf"));
        int a = INF + INF;
```

Main.java (continued) JAVA

```
        System.out.println("INF + INF = " + a + " (no overflow because INF = MAX/2)");
        System.out.println("MAX + 1 = " + (Integer.MAX_VALUE + 1));
    }
}
```

Compile and run TERMINAL

```
$ java Main.java
--- Dijkstra from 0 ---
1: pop (0,0) dist=[0, 4, 1, inf, inf] pq size=2
2: pop (2,1) dist=[0, 3, 1, 6, inf] pq size=3
3: pop (1,3) dist=[0, 3, 1, 4, inf] pq size=3
4: pop (1,4) stale, skip
5: pop (3,4) dist=[0, 3, 1, 4, 7] pq size=2
6: pop (3,6) stale, skip
7: pop (4,7) dist=[0, 3, 1, 4, 7] pq size=0
final: [0, 3, 1, 4, 7]
network delay: 2
--- Bellman-Ford ---
cheapest k=1: 700
after 1 edge(s): [0, 100, inf, inf]
after 2 edge(s): [0, 100, 200, 700]
after 3 edge(s): [0, 100, 200, 400]
cheapest k=2: 400
min effort: 2
min obstacles: 2
[0, 3, 4, 5]
[inf, 0, 1, 2]
[inf, inf, 0, 1]
[inf, inf, inf, 0]
INF + INF = 2147483646 (no overflow because INF = MAX/2)
MAX + 1 = -2147483648
```

What to notice

- In `dijkstra` the check `d > dist[u]` is what skips stale entries (steps 4 and 6 of the trace). Without it the answer is still correct, but the work can blow up.
- `networkDelay` runs Dijkstra from the source and takes the maximum distance over all nodes. If any node is still `INF`, the signal never arrives: return -1. The vertices are numbered from 1, so it allocates `n + 1` slots.
- `cheapest` copies the array (`dist.clone()`) at the start of every round. Relaxing in place would allow one round to chain several edges and violate the stop limit; this is the most common error for this problem.
- `minEffort` is Dijkstra with a different "plus": the cost of a path is the largest height difference on it, so the new cost is `Math.max(e, diff)`, not `e + diff`. This works because the max is also monotone: extending a path never lowers its cost.
- `minObstacles` uses a `Deque`: a free cell (weight 0) goes to the FRONT, an obstacle (weight 1) to the BACK. The deque then behaves like a priority queue with only two priorities, at O(V + E) instead of a log factor.
- In `floyd` the loop over the intermediate vertex `k` must be the outermost. Putting it innermost gives wrong answers, and the error is easy to miss in a small example.

REAL WORLD
Production routing, maps and network protocols use Dijkstra-like algorithms. OSPF, the routing protocol inside many company networks, runs Dijkstra on the link-state graph. Map applications use A-star (Dijkstra plus a guess of the remaining distance) and preprocessing tricks, because plain Dijkstra on a continent-size road graph is too slow for each request.

5.5.4 Complexity and mistakes

Dijkstra with a binary heap does at most E pushes and E pops of O(log E) each, which is O((V + E) log V) time and O(V + E) space. Bellman-Ford with K rounds is O(KE); with K = V - 1 rounds it is O(VE) and also finds the shortest paths when negative edges exist (one extra round that still improves something reveals a negative cycle). Floyd-Warshall is O(V cubed) time and O(V squared) space, so it is only practical for V up to a few hundred. 0-1 BFS is O(V + E).

Common mistakes:

- Using Dijkstra with negative edges. Switch to Bellman-Ford (or SPFA) instead.

- `Integer.MAX_VALUE` as infinity, then overflowing when adding a weight.
- Using a plain `Queue` instead of a `PriorityQueue` for weighted edges (that is BFS and gives wrong costs).
- Marking a vertex finished when it is first PUSHED. In Dijkstra it is final only when it is POPPED with its best distance; a later push may improve it.
- Comparator written as `a[1] - b[1]`: overflow for large distances. Use `Integer.compare` (or `Long.compare` for long distances).
- Not noticing that the answer needs `long`. Sums of many large weights (for instance 10^5 edges of 10^9) overflow `int`.
- Cheapest flights: relaxing in place instead of from a copy, and off-by-one between "k stops" and "k + 1 edges".
- Treating an undirected edge as directed once. Add it in both directions.

5.5.5 Questions to expect

1. How long until a signal sent from node k reaches all n nodes? (clue: "minimum time for all nodes"; approach: Dijkstra from k, answer is the maximum distance, -1 if any node is unreachable.)
2. Cheapest price from src to dst with at most k stops. (clue: "at most k"; approach: Bellman-Ford for k + 1 rounds with a copied array, or a Dijkstra/BFS whose state is (city, stops). Why not plain Dijkstra? It would ignore the limit and could return a path with too many stops.)
3. In a grid of heights find the route that minimises the largest height difference between consecutive cells. (approach: Dijkstra on cells with `max` as the combine step. Alternative: binary search the answer plus BFS, or Union-Find adding edges in order of difference.)
4. Of all paths with the highest success probability, return that probability. (approach: Dijkstra with a max-heap and multiplication, since probabilities in [0,1] never increase along a path.)
5. Minimum walls to remove to get from the top-left to the bottom-right. (clue: costs are only 0 or 1; approach: 0-1 BFS.)
6. Find the city with the fewest neighbours within a distance threshold. (clue: all-pairs distances and V at most 100; approach: Floyd-Warshall or Dijkstra from each city.)
7. Count the number of shortest paths. (approach: Dijkstra plus a `ways[]` array: when `d + w < dist[v]` reset `ways[v] = ways[u]`; when it equals, add `ways[u]`. Use `long` and a modulus.)
8. Report the actual path. (approach: store `parent[v] = u` whenever an edge is relaxed, then walk back from the target.)

INTERVIEW ANGLE

Why does Dijkstra not work with negative edge weights, and which algorithm applies instead? "Dijkstra finalises a vertex when it is extracted, which is correct only if all edge weights are non-negative: no other route can then be shorter. A negative edge discovered later can lower the distance of an already finalised vertex, so the invariant fails. Bellman-Ford is used instead; it relaxes all edges V - 1 times in O(VE) and detects a negative cycle if a further round still improves some distance. For a DAG, relaxing edges in topological order solves the problem in O(V + E)." Follow-up: "What is the complexity of Dijkstra?" With a binary heap, O((V + E) log V). With a Fibonacci heap, O(E + V log V), which is rarely used in practice.

5.5.6 LeetCode practice

#	Title	Level	Key idea
743	Network Delay Time	Medium	Dijkstra from k; answer is the largest distance, -1 if any is infinite
787	Cheapest Flights Within K Stops	Medium	Bellman-Ford for k + 1 rounds using a copy of the array
1631	Path With Minimum Effort	Medium	Dijkstra where the path cost is the max step
1514	Path with Maximum Probability	Medium	Dijkstra with a max-heap and multiplication
1334	Find the City With the Smallest Number of Neighbors at a Threshold Distance	Medium	Floyd-Warshall, count cities within the threshold
1976	Number of Ways to Arrive at Destination	Medium	Dijkstra plus a ways array, use long and mod
778	Swim in Rising Water	Hard	Dijkstra with max as the combine step (or binary search plus BFS)
2290	Minimum Obstacle Removal to Reach Corner	Hard	0-1 BFS: free cell to the front, obstacle to the back

LeetCode problems for this pattern.

KEY POINTS

- Pick the algorithm from the weights: BFS (equal), 0-1 BFS (0 and 1), Dijkstra (non-negative), Bellman-Ford (negative or an edge limit), Floyd-Warshall (all pairs, small V).
- Dijkstra = priority queue of (vertex, distance) + relax edges + skip stale entries; O((V + E) log V).
- Use `Integer.MAX_VALUE / 2` as infinity, `Integer.compare` in comparators, and `long` when sums can be large.
- Bellman-Ford with a stop limit must relax from a copy of the previous round.
- If the path cost is a max (or a product), keep Dijkstra's structure and change only the combine step.

sreeram

PART 6

Dynamic Programming, Greedy and Bits

Three families of optimisation problems: memoised recursion (dynamic programming), locally optimal choice (greedy) and direct manipulation of the binary representation (bits).

In this part

- **6.1** Dynamic programming: the method
- **6.2** DP on grids and paths
- **6.3** Knapsack patterns
- **6.4** DP on strings and sequences
- **6.5** DP on intervals, trees and states
- **6.6** Greedy
- **6.7** Bit manipulation

6.1 Dynamic programming: the method

Dynamic programming (DP) is a method for problems whose recursive formulation evaluates the same sub-problem repeatedly. A DP solution stores each sub-problem result once and reuses it, which reduces the running time from exponential to polynomial in the number of distinct sub-problems. A correct brute-force recursion is therefore the starting point; the remaining work is the identification of the state, the recurrence and a valid evaluation order.

The cost of the naive recursion arises because many formulations express the answer as "the best completion from the current state", and distinct decision sequences reach the same state. Naive Fibonacci evaluates `fib(3)` several million times when computing `fib(35)`. Storing each result at first computation and retrieving it afterwards changes the running time from exponential to linear. The rest of this chapter is concerned with identifying such states.

6.1.1 The idea

A problem admits a DP solution when it has two properties. **Overlapping sub-problems**: the recursion visits the same inputs repeatedly. **Optimal substructure**: the answer for a state is a function of the answers for smaller states (the best path to a cell is derived from the best paths to its predecessors). Together they permit a table of answers indexed by a compact description of the sub-problem, called the **state**.

Every DP solution is specified by three components, which should be stated explicitly before any code is written:

- **State**: the meaning of each table index, stated as a precise sentence, for example "`dp[i]` is the maximum loot obtainable from the first `i` houses". Without such a statement the formulation is not yet a DP.
- **Transition** (recurrence): how a state is computed from smaller states. "`dp[i] = max(dp[i-1], dp[i-2] + nums[i])`."
- **Base cases**: the states whose answer is known without recursion. "`dp[0] = nums[0]`."

The final code is reached through four refinements of increasing efficiency. They are applied in order, because each form serves as the reference for validating the next.

Form	Content	Implementation cost	Typical extra space
1. Plain recursion	Recurrence as a function. Correct but repeats work.	Easiest	Call stack
2. Memoisation (top-down)	Same function plus a cache checked at the top.	One extra array	Cache + call stack
3. Tabulation (bottom-up)	Loops that fill the table from base cases upward.	Need the right loop order	Table
4. Space optimisation	Keep only the table cells the recurrence still reads.	Needs care	Often O(1) or one row

The four forms of one DP solution.

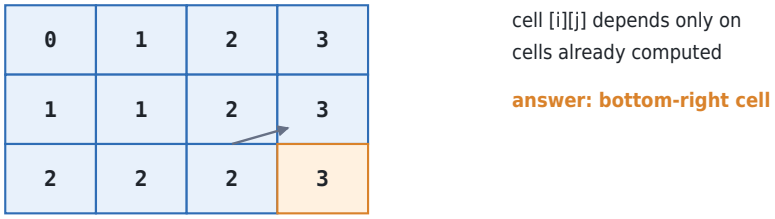


Figure 6.1. Reading a DP table: each cell is computed only from cells already filled (arrows), so the loop order must visit dependencies first.

6.1.2 How to recognise it

- "Count the number of ways to ..." (ways to climb, decode, partition).
- "Find the minimum / maximum cost / length / profit of ..." where a greedy choice is not obviously safe.
- "Is it possible to ...?" with choices at each step (reachability, feasibility of a split).
- The brute force is "try every subset / every path / every split", and the input is small enough that a table of `n` or `n × m` cells is feasible.
- Each decision only depends on a small summary of the past (the index so far, the remaining amount), not on the full history.

6.1.3 The template

Memoisation template (top-down). Start here when the recurrence is clear but the loop order is not.

Template.java JAVA

```java
// Top-down: define solve(state), cache it, return the cached value when present.
static int solve(int i, int[] memo) {
    if (i < 0) return BASE_VALUE;            // base case(s)
    if (memo[i] != UNSET) return memo[i];    // already computed
    int best = /* combine solve(i - 1), solve(i - 2), ... with the item at i */ 0;
    return memo[i] = best;                   // store, then return
}

// Bottom-up: same recurrence, explicit loop. Dependencies must already be filled.
int[] dp = new int[n + 1];
dp[0] = BASE_VALUE;
for (int i = 1; i <= n; i++) {
    dp[i] = /* combine dp[i - 1], dp[i - 2], ... */ 0;
}
```

6.1.4 Dry run

The example is House Robber: house `i` holds `nums[i]` units of money, two adjacent houses must not both be selected, and the total is to be maximised. Input: `nums = [2, 7, 9, 3, 1]`.

State: `dp[i]` is the best total from houses `0..i`. At house `i` exactly two choices exist. **Skip** it: the value is `dp[i-1]`. **Take** it: house `i-1` must have been skipped, so the value is `dp[i-2] + nums[i]`. The larger of the two is stored.

Step	skip = dp[i-1]	take = dp[i-2] + nums[i]	dp[i]	Decision
i=0	-	-	2	base: rob house 0
i=1	2	7	7	rob house 1 instead of 0
i=2	7	11	11	rob houses 0 and 2 (2+9)
i=3	11	10	11	skip house 3 (10 < 11)
i=4	11	12	12	rob house 4 on top of 0 and 2 (2+9+1)

House Robber trace for nums = [2, 7, 9, 3, 1] (real output of the program below).

Sreeram

The result is `dp[4] = 12`, obtained from houses 0, 2 and 4. At `i=3` no look-ahead occurs: the recurrence retains the larger of two values that are already known. Since `dp[i]` depends only on the two preceding cells, two scalar variables may replace the array.

6.1.5 Java code

The four forms are first applied to Fibonacci, which isolates the technique from problem-specific detail. The counter records the work removed by memoisation.

Main.java JAVA

```java
import java.util.Arrays;

public class Main {
    static long calls;

    // Step 1: plain recursion. Correct, but exponential.
    static long fibRec(int n) {
        calls++;
        if (n <= 1) return n;
        return fibRec(n - 1) + fibRec(n - 2);
    }

    // Step 2: the same recursion plus a memo table (top-down DP).
    static long fibMemo(int n, long[] memo) {
        calls++;
        if (n <= 1) return n;
        if (memo[n] != -1) return memo[n];
        return memo[n] = fibMemo(n - 1, memo) + fibMemo(n - 2, memo);
    }

    // Step 3: tabulation (bottom-up DP): fill the table from the base cases upward.
    static long fibTab(int n) {
        if (n <= 1) return n;
        long[] dp = new long[n + 1];
        dp[1] = 1;
        for (int i = 2; i <= n; i++) dp[i] = dp[i - 1] + dp[i - 2];
        return dp[n];
    }

    // Step 4: space optimisation: dp[i] only needs the previous two values.
    static long fibO1(int n) {
        long prev = 0, cur = 1;
        for (int i = 2; i <= n; i++) {
            long next = prev + cur;
            prev = cur;
            cur = next;
        }
        return n == 0 ? 0 : cur;
    }

    public static void main(String[] args) {
        int n = 35;
        calls = 0;
        long a = fibRec(n);
        System.out.println("recursion  fib(" + n + ") = " + a + "  calls = " + calls);

        long[] memo = new long[n + 1];
        Arrays.fill(memo, -1);
        calls = 0;
        long b = fibMemo(n, memo);
        System.out.println("memo       fib(" + n + ") = " + b + "  calls = " + calls);

        System.out.println("tabulation fib(" + n + ") = " + fibTab(n));
        System.out.println("O(1) space fib(" + n + ") = " + fibO1(n));
        System.out.println("fib(90) = " + fibO1(90) + "  (fits in long; fib(93) overflows)");
    }
}
```

sreeram

Run it TERMINAL

```
$ java Main.java
recursion  fib(35) = 9227465  calls = 29860703
memo       fib(35) = 9227465  calls = 69
tabulation fib(35) = 9227465
O(1) space fib(35) = 9227465
fib(90) = 2880067194370816120  (fits in long; fib(93) overflows)
```

Observations. Plain recursion performed 29,860,703 calls for `fib(35)`; memoisation performed 69, because each value is computed once and subsequently retrieved. The constant-space version applies the same recurrence with a sliding pair of variables. The type `long` overflows at `fib(93)`, which matters for problems that count ways.

House Robber follows: the table version with a trace, the two-variable version, and the circular variant (House Robber II), which invokes the linear solver twice.

Main.java JAVA

```java
import java.util.Arrays;

public class Main {
    // dp[i] = best loot using houses 0..i, never robbing two neighbours.
    static int robTable(int[] nums, boolean trace) {
        int n = nums.length;
        if (n == 1) return nums[0];
        int[] dp = new int[n];
        dp[0] = nums[0];
        dp[1] = Math.max(nums[0], nums[1]);
        if (trace) {
            System.out.println("i=0 skip=- take=- dp[0]=" + dp[0]);
            System.out.println("i=1 skip=" + nums[0] + " take=" + nums[1] + " dp[1]=" + dp[1]);
        }
        for (int i = 2; i < n; i++) {
            int skip = dp[i - 1];
            int take = dp[i - 2] + nums[i];
            dp[i] = Math.max(skip, take);
            if (trace) {
                System.out.println("i=" + i + " skip=" + skip + " take=" + take
                    + " dp[" + i + "]=" + dp[i]);
            }
        }
        return dp[n - 1];
    }

    // Same recurrence, two variables instead of an array.
    static int rob01(int[] nums) {
        int prev2 = 0, prev1 = 0;      // best up to i-2 and up to i-1
        for (int x : nums) {
            int cur = Math.max(prev1, prev2 + x);
            prev2 = prev1;
            prev1 = cur;
        }
        return prev1;
    }

    // House Robber II: the street is a circle, so house 0 and house n-1 are neighbours.
    static int robCircle(int[] nums) {
        int n = nums.length;
        if (n == 1) return nums[0];
        int a = rob01(Arrays.copyOfRange(nums, 0, n - 1));   // houses 0..n-2
        int b = rob01(Arrays.copyOfRange(nums, 1, n));       // houses 1..n-1
        return Math.max(a, b);
    }

    public static void main(String[] args) {
        int[] nums = {2, 7, 9, 3, 1};
        System.out.println("nums = " + Arrays.toString(nums));
        System.out.println("answer (table) = " + robTable(nums, true));
        System.out.println("answer (O(1) space) = " + rob01(nums));
        System.out.println("circle [2,3,2] = " + robCircle(new int[]{2, 3, 2}));
        System.out.println("circle [1,2,3,1] = " + robCircle(new int[]{1, 2, 3, 1}));
    }
}
```

Sriram

Run it TERMINAL

```
$ java Main.java
nums = [2, 7, 9, 3, 1]
i=0 skip=- take=- dp[0]=2
i=1 skip=2 take=7 dp[1]=7
i=2 skip=7 take=11 dp[2]=11
i=3 skip=11 take=10 dp[3]=11
i=4 skip=11 take=12 dp[4]=12
answer (table) = 12
answer (O(1) space) = 12
circle [2,3,2] = 3
circle [1,2,3,1] = 4
```

In the circular variant the first and last elements are adjacent and cannot both be selected. The problem is therefore solved twice, once excluding the last element and once excluding the first, and the larger result is returned.

The last program covers climbing stairs, minimum-cost climbing stairs and decode ways. In decode ways `dp[i]` counts the decodings of the first `i` characters; the base case `dp[0] = 1` (the empty prefix has one decoding) exists solely to make the recurrence uniform.

Main.java JAVA

```java
public class Main {
    // Climbing stairs: ways(n) = ways(n-1) + ways(n-2).
    static int climb(int n) {
        int a = 1, b = 1;                 // ways(0), ways(1)
        for (int i = 2; i <= n; i++) {
            int c = a + b;
            a = b;
            b = c;
        }
        return b;
    }

    // Min cost climbing stairs: cost[i] is paid when you leave step i; you may start at 0 or 1.
    static int minCost(int[] cost) {
        int n = cost.length;
        int a = 0, b = 0;                 // best cost to stand on step i-2 and i-1
        for (int i = 2; i <= n; i++) {
            int c = Math.min(b + cost[i - 1], a + cost[i - 2]);
            a = b;
            b = c;
        }
        return b;                         // standing "on top", one past the last step
    }

    // Decode ways: dp[i] = number of ways to decode the first i characters of s.
    static int decode(String s, boolean trace) {
        int n = s.length();
        int[] dp = new int[n + 1];
        dp[0] = 1;                        // empty prefix: one way (decode nothing)
        dp[1] = s.charAt(0) == '0' ? 0 : 1;
        if (trace) System.out.println("i=1 one='" + s.charAt(0) + "' dp[1]=" + dp[1]);
        for (int i = 2; i <= n; i++) {
            int one = s.charAt(i - 1) - '0';          // last digit alone
            int two = (s.charAt(i - 2) - '0') * 10 + one;   // last two digits
            if (one >= 1) dp[i] += dp[i - 1];
            if (two >= 10 && two <= 26) dp[i] += dp[i - 2];
            if (trace) {
                System.out.println("i=" + i + " one=" + one + " two=" + two
                    + " dp[" + i + "]=" + dp[i]);
            }
        }
        return dp[n];
    }

    public static void main(String[] args) {
        System.out.println("climb(5) = " + climb(5));
        System.out.println("minCost([10,15,20]) = " + minCost(new int[]{10, 15, 20}));
        System.out.println("minCost([1,100,1,1,1,100,1,1,100,1]) = "
            + minCost(new int[]{1, 100, 1, 1, 1, 100, 1, 1, 100, 1}));
        System.out.println("decode(\"226\") = " + decode("226", true));
```

sreeram

Main.java (continued) JAVA

```
        System.out.println("decode(\"06\") = " + decode("06", false));
        System.out.println("decode(\"1201234\") = " + decode("1201234", false));
    }
}
```

Run it TERMINAL

```
$ java Main.java
climb(5) = 8
minCost([10,15,20]) = 15
minCost([1,100,1,1,1,100,1,1,100,1]) = 6
i=1 one='2' dp[1]=1
i=2 one=2 two=22 dp[2]=2
i=3 one=6 two=26 dp[3]=3
decode("226") = 3
decode("06") = 0
decode("1201234") = 3
```

6.1.6 Complexity and mistakes

The time of a DP is (number of states) × (work per state). House Robber has `n` states and O(1) work per state, so O(n) time. The space is the table size, O(n), reduced to O(1) when only a fixed window of earlier states is read. For memoised recursion add the call stack: depth up to `n`, which can overflow the Java stack around a few thousand to tens of thousands of frames, so prefer tabulation when `n` is large (10^5 and above).

- **Imprecise state.** If the meaning of `dp[i]` cannot be stated in one sentence, the recurrence is likely incorrect. "Best using the first `i` items" and "best ending exactly at `i`" are distinct states with distinct base cases.
- **Incorrect base case or indexing.** Accessing `dp[i-2]` at `i = 1` throws `ArrayIndexOutOfBoundsException`. The table must be consistently indexed either by length (size `n + 1`) or by position (size `n`).
- **Wrong fill order in tabulation.** Each cell must be computed after the cells it reads. Draw an arrow from dependency to cell; loops must follow the arrows.
- **Overflow.** Counting problems blow past `int` quickly. Check whether the problem asks for the answer modulo `10^9 + 7`; if so apply `% MOD` after every addition, using `long`.
- **Overwriting a cell that is still required** during space compression. The update order must preserve old values until their last use (see the knapsack chapter for the downward loop).
- **Misclassifying greedy and DP problems.** If the locally best choice is provably safe, greedy is simpler and faster. If a counter-example exists (see the greedy chapter), DP is required.

GOTCHA

Recursive memoisation with a `HashMap<Integer, Long>` and `computeIfAbsent` that recurses into the same map throws `ConcurrentModificationException` in `HashMap`. Use an array cache, or check `containsKey` then `put` after the recursive call returns.

6.1.7 Questions to expect

1. "Count the ways to climb `n` stairs taking 1 or 2 steps." Pattern clue: "number of ways" and the last step came from one of two earlier states. Approach: `dp[n] = dp[n-1] + dp[n-2]`. Follow-up: steps of 1, 2 or 3 (tribonacci), or any step size from a given set (loop over the set).
2. "Maximum loot without robbing adjacent houses." Clue: take-or-skip with a neighbour constraint. Approach: the recurrence above. Follow-ups: houses in a circle (two linear runs), houses in a tree (DP on trees, covered later in this Part).
3. "Minimum cost to reach the top of a staircase where each step has a cost." Clue: minimise over two predecessors. Approach: `dp[i] = min(dp[i-1] + cost[i-1], dp[i-2] + cost[i-2])`.
4. "Number of ways to decode a digit string into letters." Clue: counting ways, each position uses one or two characters. Approach: the table above; handle `'0'` carefully.
5. "Maximum sum of a contiguous subarray." Clue: best subarray ending at `i` either extends the previous one or restarts. Approach: Kadane's algorithm is a one-variable DP: `cur = max(nums[i], cur + nums[i])`.
6. "Maximum profit from one buy and one sell." Clue: best profit selling today depends on the cheapest price so far. Approach: track the minimum so far, a one-variable DP.

7. "Maximum product subarray." Clue: like Kadane, but a negative number can turn the smallest product into the largest, so track both the running maximum and minimum.

INTERVIEW ANGLE

"How is the choice between memoisation and tabulation made?" Memoisation is preferred when the recurrence is clear but the iteration order is not, or when only a small fraction of the states is reachable, since only required states are computed. Tabulation is preferred when the recursion depth could overflow the stack, when every state is needed, or when memory is to be compressed to a single row. The asymptotic time is identical; tabulation removes recursion overhead and stack risk. A sound procedure is to write the recursion, add a cache for correctness, convert to loops, and then reduce space.

6.1.8 LeetCode practice

#	Title	Level	Key idea
509	Fibonacci Number	Easy	The four forms on the simplest recurrence; two variables suffice.
70	Climbing Stairs	Easy	Ways to reach step n = ways(n-1) + ways(n-2).
1137	N-th Tribonacci Number	Easy	Same idea with three predecessors; keep three variables.
746	Min Cost Climbing Stairs	Easy	Minimise over the two previous steps; answer is one past the last step.
121	Best Time to Buy and Sell Stock	Easy	Track the minimum price so far; best sale today = price - minimum.
198	House Robber	Medium	Take or skip: max(dp[i-1], dp[i-2] + nums[i]).
53	Maximum Subarray	Medium	Best sum ending at i: extend the previous run or restart at i.
213	House Robber II	Medium	Circular street: solve twice, excluding first then last house.
91	Decode Ways	Medium	One- or two-digit decodings; zeros only valid as part of 10 or 20.
152	Maximum Product Subarray	Medium	Track running max and min because negatives swap them.

LeetCode problems for this pattern.

KEY POINTS

Dynamic programming is recursion plus a table of answers to repeated sub-problems.

- The state, the transition and the base cases are stated in words before any code is written.
- The refinements are applied in order (plain recursion, memoisation, tabulation, space optimisation); each serves as the reference for the next.
- If the recurrence reads only a fixed number of earlier cells, that many variables replace the table.
- Time = number of states × work per state; watch `int` overflow and recursion depth.

6.2 DP on grids and paths

Many DP problems are defined on a grid. A path starts at the top-left cell and ends at the bottom-right cell, with moves restricted to right and down. Enumerating all paths costs C(m+n-2, m-1), which is 48,620 for a 10×10 grid and exceeds 10^27 for a 50×50 grid. However, the best path to a cell can arrive only from the cell above or the cell to the left, so the optimal values of those two predecessors suffice.

6.2.1 The idea

State: `dp[r][c]` is the answer for the sub-grid that ends at cell `(r, c)` (best cost, number of paths, size of the biggest square whose corner is here). Transition: combine `dp[r-1][c]` and `dp[r][c-1]` (and sometimes the diagonal `dp[r-1][c-1]`) with the value in the cell. Base: the first row and first column have only one possible predecessor.

Clues in the statement:

- "Robot / path / grid / matrix" with movement restricted to right and down (or down and diagonal in a triangle).
- "Minimum path sum", "number of unique paths", "largest square / rectangle of ones".
- "From the top to the bottom", "from the top-left to the bottom-right".
- A grid of size up to about 1000 × 1000 (10^6 cells is fine; 10^9 is not).

GridTemplate.java JAVA

```java
int rows = grid.length, cols = grid[0].length;
int[][] dp = new int[rows][cols];
dp[0][0] = grid[0][0];
for (int c = 1; c < cols; c++) dp[0][c] = dp[0][c - 1] + grid[0][c];   // first row
for (int r = 1; r < rows; r++) dp[r][0] = dp[r - 1][0] + grid[r][0];   // first column
for (int r = 1; r < rows; r++) {
    for (int c = 1; c < cols; c++) {
        dp[r][c] = Math.min(dp[r - 1][c], dp[r][c - 1]) + grid[r][c];  // from above or from left
    }
}
// answer: dp[rows - 1][cols - 1]
```

A variation arises when the constraint propagates from the end (Dungeon Game): the table is then filled backwards. The invariant is unchanged: every dependency is computed before it is read.

6.2.2 Dry run

Minimum path sum on this grid, moving only right or down:

Input grid

```
1 3 1
1 5 1
4 2 1
```

For each cell the program prints the best total cost to arrive there.

Cell	Value	Came from	dp = min(above, left) + value	dp
(0,0)	1	start	1	1
(0,1)	3	left (1)	1 + 3	4
(0,2)	1	left (4)	4 + 1	5
(1,0)	1	above (1)	1 + 1	2
(1,1)	5	above 4, left 2: take 2	2 + 5	7
(1,2)	1	above 5, left 7: take 5	5 + 1	6
(2,0)	4	above (2)	2 + 4	6
(2,1)	2	above 7, left 6: take 6	6 + 2	8
(2,2)	1	above 6, left 8: take 6	6 + 1	7

Min path sum, cell by cell (real trace).

The answer is `dp[2][2] = 7`. The path is recovered by traversing the table backwards: at `(2,2)` the best predecessor is above `(1,2)` with 6; at `(1,2)` it is above `(0,2)` with 5; at `(0,2)` it is left `(0,1)`, then `(0,0)`. The path is 1 → 3 → 1 → 1 → 1, total 7. This is the only path with cost 7; the other five cost 9 to 12. This backward traversal reconstructs an actual path when one is required in addition to the cost.

sreeram

6.2.3 Java code

The first listing solves minimum path sum (two-dimensional, then compressed to one row) and unique paths with obstacles, where the recurrence sums instead of minimising. In the compressed version `row[c]` still holds the value of the row above until it is overwritten, which is exactly the "above" predecessor.

Main.java JAVA

```
import java.util.Arrays;

public class Main {
    // dp[r][c] = cheapest cost of a path from (0,0) to (r,c), moving only right or down.
    static int minPathSum(int[][] g, boolean show) {
        int rows = g.length, cols = g[0].length;
        int[][] dp = new int[rows][cols];
        for (int r = 0; r < rows; r++) {
            for (int c = 0; c < cols; c++) {
                if (r == 0 && c == 0) dp[r][c] = g[r][c];
                else if (r == 0) dp[r][c] = dp[r][c - 1] + g[r][c];
                else if (c == 0) dp[r][c] = dp[r - 1][c] + g[r][c];
                else dp[r][c] = Math.min(dp[r - 1][c], dp[r][c - 1]) + g[r][c];
                if (show) {
                    System.out.println("(" + r + "," + c + ") cell=" + g[r][c]
                        + " -> dp=" + dp[r][c]);
                }
            }
        }
        return dp[rows - 1][cols - 1];
    }

    // One row is enough: row[c] holds dp of the row above until we overwrite it.
    static int minPathSum1D(int[][] g) {
        int cols = g[0].length;
        int[] row = new int[cols];
        Arrays.fill(row, Integer.MAX_VALUE);
        row[0] = 0;
        for (int[] line : g) {
            row[0] += line[0];
            for (int c = 1; c < cols; c++) row[c] = Math.min(row[c], row[c - 1]) + line[c];
        }
        return row[cols - 1];
    }

    // Unique paths with obstacles (1 = blocked). Counting paths adds instead of taking min.
    static int uniquePathsObstacles(int[][] g) {
        int cols = g[0].length;
        int[] ways = new int[cols];
        ways[0] = 1;
        for (int[] line : g) {
            for (int c = 0; c < cols; c++) {
                if (line[c] == 1) ways[c] = 0;
                else if (c > 0) ways[c] += ways[c - 1];
            }
        }
        return ways[cols - 1];
    }

    public static void main(String[] args) {
        int[][] grid = {{1, 3, 1}, {1, 5, 1}, {4, 2, 1}};
        System.out.println("grid = " + Arrays.deepToString(grid));
        System.out.println("minPathSum = " + minPathSum(grid, true));
        System.out.println("minPathSum (1 row) = " + minPathSum1D(grid));
        int[][] blocked = {{0, 0, 0}, {0, 1, 0}, {0, 0, 0}};
        System.out.println("unique paths with obstacle = " + uniquePathsObstacles(blocked));
        System.out.println("unique paths 3x7 no obstacle = "
            + uniquePathsObstacles(new int[3][7]));
    }
}
```

Run it TERMINAL

```
$ java Main.java
grid = [[1, 3, 1], [1, 5, 1], [4, 2, 1]]
(0,0) cell=1 -> dp=1
(0,1) cell=3 -> dp=4
(0,2) cell=1 -> dp=5
```

Sreeram

Run it (continued) TERMINAL

```
(1,0) cell=1 -> dp=2
(1,1) cell=5 -> dp=7
(1,2) cell=1 -> dp=6
(2,0) cell=4 -> dp=6
(2,1) cell=2 -> dp=8
(2,2) cell=1 -> dp=7
minPathSum = 7
minPathSum (1 row) = 7
unique paths with obstacle = 2
unique paths 3x7 no obstacle = 28
```

The second listing contains three further grid problems. **Maximal Square** defines `dp[r][c]` as the side of the largest all-ones square whose *bottom-right* corner is `(r, c)`; it is limited by the smallest of its three neighbours plus one. **Triangle** is easiest bottom-up, which removes the edge-case checks. **Dungeon Game** must be solved *backwards*, because a forward state ("max health so far") does not capture what is needed later.

Main.java JAVA

```java
import java.util.Arrays;
import java.util.List;

public class Main {
    // Maximal square of '1's. dp[r][c] = side of the biggest square with BOTTOM-RIGHT corner (r,c).
    static int maximalSquare(char[][] m) {
        int rows = m.length, cols = m[0].length, best = 0;
        int[][] dp = new int[rows + 1][cols + 1];          // padding row/column of zeros
        for (int r = 1; r <= rows; r++) {
            for (int c = 1; c <= cols; c++) {
                if (m[r - 1][c - 1] == '1') {
                    dp[r][c] = 1 + Math.min(dp[r - 1][c - 1], Math.min(dp[r - 1][c], dp[r][c - 1]));
                    best = Math.max(best, dp[r][c]);
                }
            }
        }
        return best * best;                                // area, not side
    }

    // Triangle: bottom-up, reusing one array. dp[j] = best sum from here down to the last row.
    static int triangle(List<List<Integer>> t) {
        int n = t.size();
        int[] dp = new int[n];
        for (int j = 0; j < n; j++) dp[j] = t.get(n - 1).get(j);
        for (int i = n - 2; i >= 0; i--) {
            for (int j = 0; j <= i; j++) dp[j] = t.get(i).get(j) + Math.min(dp[j], dp[j + 1]);
        }
        return dp[0];
    }

    // Dungeon game: walk from top-left to bottom-right, health must stay >= 1 at all times.
    // Work BACKWARDS: need[r][c] = minimum health required when ENTERING cell (r,c).
    static int dungeon(int[][] d) {
        int rows = d.length, cols = d[0].length;
        int[][] need = new int[rows + 1][cols + 1];
        for (int[] row : need) Arrays.fill(row, Integer.MAX_VALUE);
        need[rows][cols - 1] = 1;                          // virtual cell below the princess
        need[rows - 1][cols] = 1;
        for (int r = rows - 1; r >= 0; r--) {
            for (int c = cols - 1; c >= 0; c--) {
                int best = Math.min(need[r + 1][c], need[r][c + 1]);
                need[r][c] = Math.max(1, best - d[r][c]);
            }
        }
        return need[0][0];
    }

    public static void main(String[] args) {
        char[][] m = {
            "10100".toCharArray(), "10111".toCharArray(),
            "11111".toCharArray(), "10010".toCharArray()};
        System.out.println("maximalSquare area = " + maximalSquare(m));
        List<List<Integer>> t = List.of(
            List.of(2), List.of(3, 4), List.of(6, 5, 7), List.of(4, 1, 8, 3));
        System.out.println("triangle min path = " + triangle(t));
```

sreeram

Main.java (continued) JAVA

```
        int[][] d = {{-2, -3, 3}, {-5, -10, 1}, {10, 30, -5}};
        System.out.println("dungeon min starting health = " + dungeon(d));
    }
}
```

Run it TERMINAL

```
$ java Main.java
maximalSquare area = 4
triangle min path = 11
dungeon min starting health = 7
```

6.2.4 Complexity and mistakes

Time O(rows × cols), one constant-time update per cell. Space O(rows × cols) for the full table, or O(cols) when each row depends only on the row above (and the current row's left neighbour), which holds for every problem in this chapter. For the triangle with `n` rows the space is O(n), the size of one row.

- **Row/column mix-up.** `grid.length` is the number of rows, `grid[0].length` the number of columns. Using `dp[c][r]` on a non-square grid gives wrong answers or exceptions.
- **First row and first column.** They have only one predecessor. Padding the table with an extra row and column of zeros (as in Maximal Square) removes the special cases, but then every index shifts by one; keep the shift consistent.
- **Obstacle on the start or end cell.** In Unique Paths II a blocked start yields 0; the base case must not assign `dp[0][0] = 1` unconditionally.
- **Forward DP where backward is needed.** If a constraint is "never drop below X along the way" (Dungeon Game), the state must describe the requirement from here to the end.
- **Loss of a dependency during space compression.** One-row updates are valid only when the left neighbour has already been updated for the current row and the cell still holds the "above" value; the diagonal predecessor in Maximal Square requires a saved copy.
- **Returning the side instead of the area** in Maximal Square (the answer is `side * side`).

6.2.5 Questions to expect

1. "Robot moves right or down in an `m × n` grid; count the paths." Clue: counts, two predecessors. Approach: `dp[r][c] = dp[r-1][c] + dp[r][c-1]`; or the closed form C(m+n-2, m-1) provided that overflow is handled. Follow-up: obstacles.
2. "Minimum cost path from top-left to bottom-right." Clue: minimum, restricted moves. Approach: the table above. Follow-up: what if moves can go up and left too? Then it is a shortest-path problem; use Dijkstra instead (see the shortest paths chapter).
3. "Largest square of 1s in a binary matrix." Clue: "largest square", cell is a corner. Approach: `1 + min(left, up, diagonal)`. Follow-up: largest rectangle of 1s needs the histogram technique (monotonic stack).
4. "Minimum path sum from top to bottom of a triangle." Clue: each step goes to one of two cells below. Approach: bottom-up with an array of size `n`.
5. "Knight / princess starting health" (Dungeon Game). Clue: constraint that must hold at every step. Approach: fill backwards with `max(1, min(next) - cell)`.
6. "Longest increasing path in a matrix." Clue: moves in four directions but strictly increasing values, so no cycles. Approach: DFS with memoisation per cell (or process cells in sorted order).
7. "Minimum falling path sum." Clue: from each cell the move is to one of up to three cells in the next row. Approach: row-by-row DP with `min` of three.

INTERVIEW ANGLE

"Can the space be reduced?" Yes. Each cell depends only on the cell above and the cell to the left, so a single array of length `cols` suffices. While row `r` is processed, `dp[c]` still holds the value of row `r-1` (the above predecessor) until it is overwritten, and `dp[c-1]` has already been updated for row `r` (the left predecessor). Space decreases from O(rows × cols) to O(cols). If the path itself must be reconstructed, the full table (or parent pointers) is required, because the compressed table discards history.

6.2.6 LeetCode practice

#	Title	Level	Key idea
62	Unique Paths	Medium	Number of paths into a cell is the sum of the cell above and to the left.
63	Unique Paths II	Medium	Same, but set the count to 0 on blocked cells.
64	Minimum Path Sum	Medium	Cheaper of above and left, plus the cell value.
120	Triangle	Medium	Go bottom-up so every cell has exactly two children to choose from.
931	Minimum Falling Path Sum	Medium	Row by row, take the minimum of up to three cells in the row above.
221	Maximal Square	Medium	Side of the square at a corner = 1 + min of three neighbours.
329	Longest Increasing Path in a Matrix	Hard	DFS from every cell with a memo; strictly increasing means no cycles.
174	Dungeon Game	Hard	Fill backwards: health needed to enter a cell is max(1, best next need - cell).

LeetCode problems for this pattern.

KEY POINTS

Grid DP has one cell per sub-problem and a recurrence over a few neighbouring cells.

- Define `dp[r][c]` precisely (cost to arrive, ways to arrive, side of the square ending here).
- Fill order must respect dependencies: forward for "arrive at", backward for "needed from here".
- One row suffices when only the previous row is read; path reconstruction requires the full table.
- The first row and column, obstacles, and the row/column distinction are the usual sources of error.

6.3 Knapsack patterns

The knapsack problem selects items, each with a weight and a value, subject to a capacity bound. Exhaustive search examines 2^n subsets and is infeasible beyond n of about 30. The family is central because many problems are reformulations of it: feasibility of a sum, the number of ways to reach a sum, and the minimum number of items reaching a sum. Once the reduction is identified, the implementation is short.

6.3.1 The idea

The state is two-dimensional in principle: (items considered, capacity or sum used). It is almost always compressed to one dimension, `dp[s]`, and correctness then depends on the **loop direction** and the **loop order**:

- **0/1 knapsack** (each item used at most once): for each item, loop `s` from high to low. Going downwards guarantees `dp[s - x]` still reflects the state *before* this item was considered.
- **Unbounded knapsack** (each item reusable): loop `s` from low to high. Now `dp[s - x]` may already include the current item, which is exactly "use it again".
- **Combinations versus permutations**: with items in the outer loop and sums inside, each multiset is counted once (order ignored). With sums in the outer loop and items inside, each ordered sequence is counted.

Clue phrases:

- "Partition into two groups with equal sum" (subset sum with target = total / 2).
- "Number of ways to reach a target sum using these numbers" (counting version).
- "Minimum number of coins / items to make an amount" (minimising version).
- "Each number can be used once" versus "unlimited supply" (decides the loop direction).
- "Target sum with plus and minus signs" (reduces to subset sum).
- A word and a dictionary: "can the string be segmented?" (the same shape, with substrings as items).

Variant	Recurrence	Loop over sums	Initial value
Subset sum (can we?)	`can[s] = can[s] or can[s - x]`	`high to low (0/1)`	can[0] = true
Count ways (0/1)	`ways[s] += ways[s - x]`	`high to low`	ways[0] = 1
Count combinations (unbounded)	`ways[a] += ways[a - coin]`	`low to high, coins outer`	ways[0] = 1
Count permutations (unbounded)	`ways[a] += ways[a - coin]`	`amounts outer, coins inner`	ways[0] = 1
Fewest items (unbounded)	`dp[a] = min(dp[a], dp[a - c] + 1)`	`low to high`	dp[0] = 0, rest = infinity

The knapsack variants at a glance.

6.3.2 Dry run

Coin Change: coins `[1, 2, 5]`, amount `11`. State: `dp[a]` is the minimum number of coins summing exactly to `a`. Transition: `dp[a] = min over coins c <= a of dp[a - c] + 1`. Base: `dp[0] = 0`; every other cell is initialised to a sentinel, `amount + 1`, which exceeds any feasible answer.

a	Best last coin (from dp[a - coin] + 1)	dp[a]
1	1 (dp[0] + 1)	1
2	2 (dp[0] + 1)	1
3	1 or 2 (dp[2] + 1 = dp[1] + 1)	2
4	2 (dp[2] + 1)	2
5	5 (dp[0] + 1)	1
6	5 or 1 (dp[1] + 1 = dp[5] + 1)	2
7	5 (dp[2] + 1)	2
8	any of 1, 2, 5 (dp[7], dp[6], dp[3] are 2, 2, 2)	3
9	5 (dp[4] + 1)	3
10	5 (dp[5] + 1)	2
11	5 (dp[6] + 1)	3

Coin change dp table for amount 1 to 11 (real trace; dp[0] = 0).

Answer `dp[11] = 3` (5 + 5 + 1). The program below also shows why greedy fails: with coins `[1, 3, 4]` and amount 6, taking the largest coin first gives 4 + 1 + 1 = three coins, but 3 + 3 is two. DP never commits early; it tries every last coin.

6.3.3 Java code

Coin change with the trace that produced the table above:

Main.java JAVA

```java
import java.util.Arrays;

public class Main {
    // Coin change (unbounded): fewest coins that sum to amount, or -1.
    // dp[a] = fewest coins for amount a. INF marks "unreachable".
    static int coinChange(int[] coins, int amount, boolean trace) {
        int INF = amount + 1;                       // larger than any real answer
        int[] dp = new int[amount + 1];
        Arrays.fill(dp, INF);
        dp[0] = 0;
        for (int a = 1; a <= amount; a++) {
            for (int coin : coins) {
                if (coin <= a && dp[a - coin] + 1 < dp[a]) dp[a] = dp[a - coin] + 1;
            }
            if (trace) System.out.println("a=" + a + " dp=" + dp[a]);
        }
        return dp[amount] >= INF ? -1 : dp[amount];
```

Main.java (continued) JAVA

```
    }

    public static void main(String[] args) {
        int[] coins = {1, 2, 5};
        System.out.println("coins = " + Arrays.toString(coins) + ", amount = 11");
        System.out.println("answer = " + coinChange(coins, 11, true));
        System.out.println("coins [2], amount 3 -> " + coinChange(new int[]{2}, 3, false));
        System.out.println("greedy trap: coins [1,3,4], amount 6 -> "
            + coinChange(new int[]{1, 3, 4}, 6, false) + " (greedy 4+1+1 would use 3)");
    }
}
```

Run it TERMINAL

```
$ java Main.java
coins = [1, 2, 5], amount = 11
a=1 dp=1
a=2 dp=1
a=3 dp=2
a=4 dp=2
a=5 dp=1
a=6 dp=2
a=7 dp=2
a=8 dp=3
a=9 dp=3
a=10 dp=2
a=11 dp=3
answer = 3
coins [2], amount 3 -> -1
greedy trap: coins [1,3,4], amount 6 -> 2 (greedy 4+1+1 would use 3)
```

The second listing collects the remaining members of the family. The direction of the inner loop and the identity of the outer loop are the only differences between several of the problems.

Main.java JAVA

```
import java.util.Arrays;
import java.util.List;

public class Main {
    // 0/1 subset sum: can some subset (each item used at most once) add up to target?
    // Iterate the sum DOWNWARDS so an item cannot be reused within the same pass.
    static boolean subsetSum(int[] nums, int target) {
        boolean[] can = new boolean[target + 1];
        can[0] = true;
        for (int x : nums) {
            for (int s = target; s >= x; s--) can[s] |= can[s - x];
        }
        return can[target];
    }

    // Partition equal subset sum = subset sum with target = total / 2.
    static boolean canPartition(int[] nums) {
        int total = Arrays.stream(nums).sum();
        return total % 2 == 0 && subsetSum(nums, total / 2);
    }

    // Target sum (+/- signs): P - N = target and P + N = total, so P = (total + target) / 2.
    static int targetSum(int[] nums, int target) {
        int total = Arrays.stream(nums).sum();
        if (Math.abs(target) > total || (total + target) % 2 != 0) return 0;
        int want = (total + target) / 2;
        int[] ways = new int[want + 1];
        ways[0] = 1;
        for (int x : nums) {
            for (int s = want; s >= x; s--) ways[s] += ways[s - x];
        }
        return ways[want];
    }

    // Coin change II: number of COMBINATIONS. Coins in the OUTER loop => order does not matter.
    static int combinations(int[] coins, int amount) {
        int[] ways = new int[amount + 1];
        ways[0] = 1;
```

sreeram

Main.java (continued) JAVA

```java
        for (int coin : coins) {
            for (int a = coin; a <= amount; a++) ways[a] += ways[a - coin];
        }
        return ways[amount];
    }

    // Same loops swapped: amount in the OUTER loop counts ORDERED sequences (permutations).
    static int permutations(int[] coins, int amount) {
        int[] ways = new int[amount + 1];
        ways[0] = 1;
        for (int a = 1; a <= amount; a++) {
            for (int coin : coins) if (coin <= a) ways[a] += ways[a - coin];
        }
        return ways[amount];
    }

    // Word break: dp[i] = true if s[0..i) can be split into dictionary words.
    static boolean wordBreak(String s, List<String> dict) {
        boolean[] dp = new boolean[s.length() + 1];
        dp[0] = true;
        for (int i = 1; i <= s.length(); i++) {
            for (String w : dict) {
                int start = i - w.length();
                if (start >= 0 && dp[start] && s.startsWith(w, start)) {
                    dp[i] = true;
                    break;
                }
            }
        }
        return dp[s.length()];
    }

    public static void main(String[] args) {
        int[] sample = {3, 34, 4, 12, 5, 2};
        System.out.println("subsetSum [3,34,4,12,5,2], 9 = " + subsetSum(sample, 9));
        System.out.println("canPartition [1,5,11,5] = " + canPartition(new int[]{1, 5, 11, 5}));
        System.out.println("canPartition [1,2,3,5] = " + canPartition(new int[]{1, 2, 3, 5}));
        System.out.println("targetSum [1,1,1,1,1], 3 = " + targetSum(new int[]{1, 1, 1, 1, 1}, 3));
        int[] coins = {1, 2, 5};
        System.out.println("combinations amount 5 = " + combinations(coins, 5));
        System.out.println("permutations amount 5 = " + permutations(coins, 5));
        System.out.println("wordBreak leetcode = "
            + wordBreak("leetcode", List.of("leet", "code")));
        System.out.println("wordBreak catsandog = "
            + wordBreak("catsandog", List.of("cats", "dog", "sand", "and", "cat")));
    }
}
```

Run it TERMINAL

```
$ java Main.java
subsetSum [3,34,4,12,5,2], 9 = true
canPartition [1,5,11,5] = true
canPartition [1,2,3,5] = false
targetSum [1,1,1,1,1], 3 = 5
combinations amount 5 = 4
permutations amount 5 = 9
wordBreak leetcode = true
wordBreak catsandog = false
```

Observations. `combinations(coins, 5)` returns 4 (5; 2+2+1; 2+1+1+1; 1+1+1+1+1), whereas `permutations(coins, 5)` returns 9, because ordered sequences such as 1+2+2, 2+1+2 and 2+2+1 are counted separately. The two functions differ only in which loop is outer. Target Sum reduces the sign assignment to subset sum: if `P` is the sum of the numbers assigned a plus sign and `N` the sum of those assigned a minus sign, then `P - N = target` and `P + N = total`, hence `P = (total + target) / 2`.

6.3.4 Complexity and mistakes

Subset-sum problems require O(n × target) time and O(target) space. Coin change requires O(amount × coins). Word break requires O(n × dictionary size × average word length) in the implementation above, or O(n^2) substring checks with a `HashSet`. The bound depends on the numeric value of the target rather than on the input size, so the algorithms are pseudo-polynomial: practical for targets up to about 10^4 to 10^5 and impractical for 10^9.

- **Incorrect loop direction.** An upward loop in 0/1 knapsack permits reuse of an item and overestimates the result; a downward loop in unbounded knapsack omits combinations. This is the most frequent defect.
- **Incorrect initial values.** For counting, `ways[0] = 1`. For minimisation, unreachable cells hold a large sentinel rather than 0, and `Integer.MAX_VALUE` must not be used if 1 is subsequently added to it (the sum overflows to a negative value).
- **Confusion of combinations and permutations.** If the statement treats [1,2] and [2,1] as identical, the items form the outer loop; if they are distinct sequences, the sums form the outer loop.
- **Negative targets and odd totals.** In Target Sum, check `abs(target) > total` and parity before indexing the array.
- **Forgetting the empty prefix** in Word Break (`dp[0] = true`).
- **Overflow in counting.** Some problems count combinations that exceed `int`; the statement then says modulo 10^9 + 7 or guarantees the answer fits.

GOTCHA

Evaluating `dp[a - coin] + 1` when `dp[a - coin]` equals `Integer.MAX_VALUE` overflows to `Integer.MIN_VALUE`, which then appears to be the best value. A sentinel such as `amount + 1` avoids this, or the case must be excluded before the addition.

6.3.5 Questions to expect

1. "Can this array be split into two subsets with equal sum?" Clue: "equal sum", each element once. Approach: total must be even; run 0/1 subset sum with target total / 2. Follow-up: print the two subsets (keep a 2-D table or parent pointers).
2. "Fewest coins for an amount; return -1 if impossible." Clue: unbounded, minimise. Approach: the table above. Follow-up: why greedy fails and when it works (canonical coin systems like 1, 5, 10, 25).
3. "Number of ways to make change." Clue: unbounded, count, order irrelevant. Approach: coins outer, amounts inner. Follow-up: if order matters, swap the loops.
4. "Assign + or - to each number to reach a target; count the assignments." Clue: signs reduce to a subset with a computed sum. Approach: the reduction `(total + target) / 2` followed by 0/1 counting.
5. "Can the string be segmented into dictionary words?" Clue: prefix feasibility. Approach: `dp[i]` = prefix of length `i` is segmentable. Follow-up: return all segmentations (backtracking with memo).
6. "Fewest perfect squares that sum to n." Clue: unbounded minimise where the "coins" are squares. Approach: coin change with coins 1, 4, 9, ...
7. "Largest subset of binary strings with at most m zeros and n ones." Clue: two capacities. Approach: 0/1 knapsack with a two-dimensional dp, both loops downwards.
8. "Minimise the difference between two groups' sums" (Last Stone Weight II). Clue: split into two groups. Approach: subset sum up to half the total; answer = total - 2 × (best reachable sum).

INTERVIEW ANGLE

"Why is the capacity iterated backwards in 0/1 knapsack?" With a one-dimensional array, `dp[s]` is updated from `dp[s - x]`. In an upward loop, `dp[s - x]` has already been updated for the same item, so the item may be counted more than once. In a downward loop every `dp[s - x]` read still holds its value from before the current item, which enforces "at most once". Equivalently, the two-dimensional formulation reads from the previous row, and the one-dimensional formulation reproduces this if and only if only cells not yet overwritten are read.

6.3.6 LeetCode practice

#	Title	Level	Key idea
416	Partition Equal Subset Sum	Medium	Subset sum with target total / 2; 0/1, loop downwards.
322	Coin Change	Medium	Unbounded minimise; dp[a] = min(dp[a - c]) + 1; sentinel for impossible.
518	Coin Change II	Medium	Unbounded count of combinations; coins in the outer loop.
139	Word Break	Medium	dp[i] = some dictionary word ends at i and dp[start] is true.
279	Perfect Squares	Medium	Coin change where the coins are squares.
377	Combination Sum IV	Medium	Count ordered sequences; amount in the outer loop.
494	Target Sum	Medium	Reduce signs to subset sum with P = (total + target) / 2.
474	Ones and Zeroes	Medium	0/1 knapsack with two capacities; both loops go downwards.
1049	Last Stone Weight II	Medium	Split into two groups; best subset sum not above half the total.

LeetCode problems for this pattern.

KEY POINTS

Most "sum" problems on arrays are a knapsack: choose items to hit a target.

- 0/1 (each once) loops the sum downwards; unbounded (reusable) loops it upwards.
- Items outer counts combinations; sums outer counts ordered sequences.
- Initialise `dp[0]` correctly (true, 1 or 0) and use a safe sentinel for "impossible".
- Cost is pseudo-polynomial: O(items × target), fine only for modest targets.

6.4 DP on strings and sequences

Two strings, or one string against itself. Questions: how similar are they? What is the longest thing they share? How many edits turn one into the other? Brute force tries every subsequence of one string against the other: 2^n subsequences. The DP table for two strings has `(n + 1) × (m + 1)` cells, and each cell answers the question for a pair of **prefixes**. That one change of viewpoint, from whole strings to pairs of prefixes, solves the whole family.

6.4.1 The idea

State: `dp[i][j]` is the answer for the first `i` characters of string `a` and the first `j` characters of string `b`. Row 0 and column 0 correspond to the empty prefix, which makes the base cases natural (edit distance from empty to `j` characters is `j`). Transition: compare the last characters `a[i-1]` and `b[j-1]`.

- If the characters match, the diagonal is usually extended: `dp[i][j] = dp[i-1][j-1] + 1` (LCS), or `dp[i-1][j-1]` (edit distance, zero cost).
- If they differ, the best of three predecessors is taken (drop a character from `a`, from `b`, or from both): the cells above, to the left and on the diagonal.

Clues in the statement:

- "Longest common subsequence / substring" of two strings.
- "Minimum number of operations (insert, delete, replace) to convert one string into another."
- "Longest increasing subsequence" in one array; "longest palindromic subsequence" in one string.
- "Number of distinct subsequences of s equal to t."
- "Does the string match the pattern with `?`, `*`, `.`" (pattern matching DP).
- Strings of length up to about 1000 to 5000 (an n × m table is feasible; n × m of 10^9 is not).

A **subsequence** keeps the order but may skip characters; a **substring** is contiguous. Longest common substring uses a different transition: reset to 0 on mismatch, and track the best cell.

6.4.2 Dry run

Longest Common Subsequence of `a = "abcde"` and `b = "ace"`. `dp[i][j]` = LCS length of the first `i` characters of `a` and the first `j` of `b`. Row 0 and column 0 are all zero (an empty prefix shares nothing).

dp	""	a	ac	ace
""	0	0	0	0
a	0	1	1	1
ab	0	1	1	1
abc	0	1	2	2
abcd	0	1	2	2
abcde	0	1	2	3

LCS table (real output; row label is the prefix of abcde, column label the prefix of ace).

Interpretation. The cell (row `abc`, column `ac`) is 2: the last characters are both `c`, so the value is the diagonal cell (`ab`, `a`) = 1, plus 1. The cell (row `abcd`, column `ac`) is 2: `d` and `c` differ, so the maximum of the cell above (`abc`, `ac` = 2) and the cell to the left (`abcd`, `a` = 1) is taken. The bottom-right cell is 3, the length of the LCS `ace`. The string itself is recovered by traversing back from that corner: on a match the move is diagonal and the character is recorded; on a mismatch the move is towards the larger neighbour.

Longest Increasing Subsequence admits a faster method. A list `tails` is maintained in which `tails[k]` is the smallest possible last value of an increasing subsequence of length `k + 1`. For each element, binary search locates the first tail not smaller than it, which is then overwritten; if no such tail exists, the element is appended.

x	pos	tails after	meaning
10	0	[10]	first element starts a length-1 subsequence
9	0	[9]	a smaller ending for length 1
2	0	[2]	smaller again
5	1	[2, 5]	extends: length 2 ends at 5
3	1	[2, 3]	better length-2 ending (3 < 5)
7	2	[2, 3, 7]	length 3
101	3	[2, 3, 7, 101]	length 4
18	3	[2, 3, 7, 18]	better length-4 ending

LIS with tails on [10, 9, 2, 5, 3, 7, 101, 18] (real trace).

The final list has four entries, so the LIS length is 4. The list `tails` is not itself the subsequence: its entries may originate from different subsequences, and only its length is meaningful (here [2, 3, 7, 18] happens to be a valid subsequence).

6.4.3 Java code

LCS and edit distance share the same skeleton. The program prints the full tables, which can be compared with the table above.

Main.java JAVA

```java
public class Main {
    // dp[i][j] = length of the LCS of a[0..i) and b[0..j).
    static int[][] lcsTable(String a, String b) {
        int[][] dp = new int[a.length() + 1][b.length() + 1];
        for (int i = 1; i <= a.length(); i++) {
            for (int j = 1; j <= b.length(); j++) {
                if (a.charAt(i - 1) == b.charAt(j - 1)) dp[i][j] = dp[i - 1][j - 1] + 1;
                else dp[i][j] = Math.max(dp[i - 1][j], dp[i][j - 1]);
            }
        }
        return dp;
```

Main.java (continued) JAVA

```
    }

    // Walk back from the corner to recover one actual LCS string.
    static String lcsString(String a, String b, int[][] dp) {
        StringBuilder sb = new StringBuilder();
        int i = a.length(), j = b.length();
        while (i > 0 && j > 0) {
            if (a.charAt(i - 1) == b.charAt(j - 1)) {
                sb.append(a.charAt(i - 1));
                i--;
                j--;
            } else if (dp[i - 1][j] >= dp[i][j - 1]) i--;
            else j--;
        }
        return sb.reverse().toString();
    }

    // Edit distance: dp[i][j] = fewest insert/delete/replace ops to turn a[0..i) into b[0..j).
    static int[][] editTable(String a, String b) {
        int[][] dp = new int[a.length() + 1][b.length() + 1];
        for (int i = 0; i <= a.length(); i++) dp[i][0] = i;     // delete i characters
        for (int j = 0; j <= b.length(); j++) dp[0][j] = j;     // insert j characters
        for (int i = 1; i <= a.length(); i++) {
            for (int j = 1; j <= b.length(); j++) {
                if (a.charAt(i - 1) == b.charAt(j - 1)) dp[i][j] = dp[i - 1][j - 1];
                else dp[i][j] = 1 + Math.min(dp[i - 1][j - 1],            // replace
                                   Math.min(dp[i - 1][j], dp[i][j - 1])); // delete, insert
            }
        }
        return dp;
    }


    public static void main(String[] args) {
        String a = "abcde", b = "ace";
        int[][] t = lcsTable(a, b);
        System.out.println("LCS of " + a + " and " + b
            + " (rows: _" + a + ", columns: _" + b + ")");
        for (int[] row : t) System.out.println(java.util.Arrays.toString(row));
        System.out.println("length = " + t[a.length()][b.length()]
            + ", one LCS = " + lcsString(a, b, t));

        String x = "horse", y = "ros";
        int[][] e = editTable(x, y);
        System.out.println("edit distance " + x + " -> " + y
            + " (rows: _" + x + ", columns: _" + y + ")");
        for (int[] row : e) System.out.println(java.util.Arrays.toString(row));
        System.out.println("answer = " + e[x.length()][y.length()]);
    }
}
```

Run it TERMINAL

```
$ java Main.java
LCS of abcde and ace (rows: _abcde, columns: _ace)
[0, 0, 0, 0]
[0, 1, 1, 1]
[0, 1, 1, 1]
[0, 1, 2, 2]
[0, 1, 2, 2]
[0, 1, 2, 3]
length = 3, one LCS = ace
edit distance horse -> ros (rows: _horse, columns: _ros)
[0, 1, 2, 3]
[1, 1, 2, 3]
[2, 2, 1, 2]
[3, 2, 2, 2]
[4, 3, 3, 2]
[5, 4, 4, 3]
answer = 3
```

Observations. In the edit distance table, row 0 and column 0 are initialised to `0..m` and `0..n`: converting the empty string to `j` characters requires `j` insertions. For a mismatch the three predecessors correspond to replace (diagonal),

delete from `a` (up) and insert into `a` (left), each at cost 1. The conversion `horse` to `ros` requires 3 edits (replace h with r, delete r, delete e).

The second listing contains four more problems: LIS in O(n log n), longest palindromic subsequence (which is the LCS of a string and its reverse, written directly on one string), distinct subsequences with a one-dimensional array, and wildcard matching.

Main.java JAVA

```java
import java.util.ArrayList;
import java.util.Arrays;
import java.util.Collections;
import java.util.List;

public class Main {
    // LIS in O(n log n): tails.get(k) = smallest possible last value of an increasing
    // subsequence of length k+1. The list stays sorted, so binary search finds the slot.
    static int lis(int[] nums, boolean trace) {
        List<Integer> tails = new ArrayList<>();
        for (int x : nums) {
            int pos = Collections.binarySearch(tails, x);
            if (pos < 0) pos = -pos - 1;               // insertion point: first tail >= x
            if (pos == tails.size()) tails.add(x);     // x extends the longest subsequence
            else tails.set(pos, x);                    // x is a better (smaller) ending
            if (trace) System.out.println("x=" + x + " pos=" + pos + " tails=" + tails);
        }
        return tails.size();
    }

    // Longest palindromic subsequence: dp[i][j] for the substring s[i..j].
    static int lps(String s) {
        int n = s.length();
        int[][] dp = new int[n][n];
        for (int i = n - 1; i >= 0; i--) {
            dp[i][i] = 1;
            for (int j = i + 1; j < n; j++) {
                if (s.charAt(i) == s.charAt(j)) dp[i][j] = dp[i + 1][j - 1] + 2;
                else dp[i][j] = Math.max(dp[i + 1][j], dp[i][j - 1]);
            }
        }
        return dp[0][n - 1];
    }

    // Distinct subsequences: ways[j] = number of ways s so far contains t[0..j) as a subsequence.
    static long distinct(String s, String t) {
        long[] ways = new long[t.length() + 1];
        ways[0] = 1;
        for (char ch : s.toCharArray()) {
            for (int j = t.length(); j >= 1; j--) {      // downwards: each char of s used once
                if (t.charAt(j - 1) == ch) ways[j] += ways[j - 1];
            }
        }
        return ways[t.length()];
    }

    // Wildcard matching: '?' = any one char, '*' = any sequence (including empty).
    static boolean wildcard(String s, String p) {
        boolean[][] dp = new boolean[s.length() + 1][p.length() + 1];
        dp[0][0] = true;
        for (int j = 1; j <= p.length(); j++) dp[0][j] = dp[0][j - 1] && p.charAt(j - 1) == '*';
        for (int i = 1; i <= s.length(); i++) {
            for (int j = 1; j <= p.length(); j++) {
                char pc = p.charAt(j - 1);
                if (pc == '*') dp[i][j] = dp[i][j - 1] || dp[i - 1][j];  // empty / eat one char
                else dp[i][j] = dp[i - 1][j - 1] && (pc == '?' || pc == s.charAt(i - 1));
            }
        }
        return dp[s.length()][p.length()];
    }

    public static void main(String[] args) {
        int[] nums = {10, 9, 2, 5, 3, 7, 101, 18};
        System.out.println("nums = " + Arrays.toString(nums));
        System.out.println("LIS length = " + lis(nums, true));
        System.out.println("LPS(\"bbbab\") = " + lps("bbbab"));
        System.out.println("distinct(\"rabbbit\", \"rabbit\") = " + distinct("rabbbit", "rabbit"));
```

Main.java (continued) JAVA

```
        System.out.println("wildcard(\"adceb\", \"*a*b\") = " + wildcard("adceb", "*a*b"));
        System.out.println("wildcard(\"acdcb\", \"a*c?b\") = " + wildcard("acdcb", "a*c?b"));
    }
}
```

Run it TERMINAL

```
$ java Main.java
nums = [10, 9, 2, 5, 3, 7, 101, 18]
x=10 pos=0 tails=[10]
x=9 pos=0 tails=[9]
x=2 pos=0 tails=[2]
x=5 pos=1 tails=[2, 5]
x=3 pos=1 tails=[2, 3]
x=7 pos=2 tails=[2, 3, 7]
x=101 pos=3 tails=[2, 3, 7, 101]
x=18 pos=3 tails=[2, 3, 7, 18]
LIS length = 4
LPS("bbbab") = 4
distinct("rabbbit", "rabbit") = 3
wildcard("adceb", "*a*b") = true
wildcard("acdcb", "a*c?b") = false
```

In wildcard matching, `*` matches the empty sequence (`dp[i][j-1]`: the star is ignored) or consumes one further character of `s` (`dp[i-1][j]`: the star remains active). Regular-expression matching (`.` and `*` meaning "zero or more of the preceding element") uses the same table, but the star depends on the preceding pattern character: `dp[i][j] = dp[i][j-2]` (zero copies) or, if that character matches `s[i-1]`, `dp[i-1][j]`. The difficulty lies in the case analysis rather than in a new idea.

6.4.4 Complexity and mistakes

LCS, edit distance, interleaving and matching problems require O(n × m) time and space. Space reduces to O(min(n, m)) with two rows when only the value is required; reconstruction of the edit script or of the LCS string requires the full table. LPS on a single string requires O(n^2). LIS with the `tails` technique requires O(n log n) time and O(n) space, compared with O(n^2) for the `dp[i]` formulation (best LIS ending at `i`), which remains adequate for n up to about 5000 and extends more readily to counting and reconstruction.

- **Prefix versus index confusion.** `dp[i][j]` refers to *prefixes*, so the characters compared are `a.charAt(i - 1)` and `b.charAt(j - 1)`. Using `charAt(i)` is the commonest bug.
- **Forgetting the base row and column.** For edit distance they are not 0; they are `i` and `j`.
- **Strict versus non-strict LIS.** For strictly increasing, find the first tail `>= x`; for non-decreasing, the first tail `> x`. `Collections.binarySearch` returns `-(insertion point) - 1` when not found; decode it with `-pos - 1`.
- **Treating `tails` as the answer sequence.** It gives the length only.
- **Subsequence versus substring.** A mismatch in a substring problem resets the cell to 0.
- **Counting problems overflow `int`.** Distinct subsequences can be huge; the code uses `long` (LeetCode guarantees the answer fits in a 32-bit integer for the official tests, but intermediate cells in some inputs can overflow, so prefer `long`).
- **Iterating the 1-D array upwards** in distinct subsequences double-uses the same character of `s` (same rule as 0/1 knapsack).

6.4.5 Questions to expect

1. "Length of the longest common subsequence of two strings." Clue: "common subsequence". Approach: the table above, O(n × m). Follow-up: print one LCS (walk back); use O(min(n, m)) space (two rows).
2. "Minimum operations to convert word1 to word2 using insert, delete, replace." Clue: "minimum edits". Approach: edit distance table. Follow-up: only insert and delete allowed (answer = n + m - 2 × LCS, which is Delete Operation for Two Strings).
3. "Length of the longest strictly increasing subsequence." Clue: "increasing subsequence". Approach: O(n^2) DP first, then `tails` plus binary search for O(n log n). Follow-up: Russian dolls envelopes (sort by width ascending, height descending, then LIS on heights).
4. "Longest palindromic subsequence." Clue: palindrome, may skip characters. Approach: interval DP `dp[i][j]`, or LCS of the string with its reverse. Follow-up: minimum insertions to make it a palindrome = n - LPS.

5. "Count palindromic substrings." Clue: contiguous palindromes. Approach: expand around each centre in O(n^2), simpler than DP (see the matrix and string patterns chapter).
6. "How many times does t occur as a subsequence of s?" Clue: count of subsequences. Approach: `dp[j]` over prefixes of `t`, loop `j` downwards.
7. "Does string s match pattern p with `?` and `*`?" Clue: pattern with wildcards. Approach: table of prefix pairs; `*` is the interesting case. Follow-up: a greedy two-pointer solution exists for `?` and `*` in O(n × m) worst case with O(1) space.
8. "Is s3 an interleaving of s1 and s2?" Clue: merge preserving each string's order. Approach: `dp[i][j]` = first `i + j` characters of s3 come from the first `i` of s1 and `j` of s2.

INTERVIEW ANGLE

"How is the answer reconstructed, and not only its length?" The full table must be retained. Starting at the bottom-right cell, the procedure repeats: if the characters match and the cell equals the diagonal value plus one, the character is emitted and the move is diagonal; otherwise the move is to the neighbour holding the same value. For edit distance, at each cell the predecessor whose value plus operation cost equals the current value determines whether an insertion, deletion or replacement is emitted. The reconstruction costs O(n + m) additional time but requires O(n × m) space.

6.4.6 LeetCode practice

#	Title	Level	Key idea
1143	Longest Common Subsequence	Medium	dp over prefix pairs: match extends the diagonal, else max of up and left.
647	Palindromic Substrings	Medium	Expand around each of the 2n - 1 centres, or a boolean interval table.
300	Longest Increasing Subsequence	Medium	O(n^2) dp first; then tails array with binary search for O(n log n).
583	Delete Operation for Two Strings	Medium	Only deletions: answer = n + m - 2 * LCS.
516	Longest Palindromic Subsequence	Medium	Interval dp, or LCS of the string with its reverse.
72	Edit Distance	Medium	Three neighbours: replace (diagonal), delete (up), insert (left).
97	Interleaving String	Medium	dp[i][j]: s3's first i + j characters from s1[0..i) and s2[0..j).
115	Distinct Subsequences	Hard	Count ways t's prefix appears; downward 1-D loop or 2-D table.
44	Wildcard Matching	Hard	Star matches empty (left) or one more character (up).
10	Regular Expression Matching	Hard	Star applies to the previous pattern char: zero copies or one more match.
1312	Minimum Insertion Steps to Make a String Palindrome	Hard	Answer = length - longest palindromic subsequence.

LeetCode problems for this pattern.

KEY POINTS

Two-string DP is a table of prefix pairs; one-string DP is a table of intervals or of "ending at i".

- Compare `a[i-1]` and `b[j-1]` because row and column 0 mean the empty prefix.
- Match: extend the diagonal. Mismatch: best of up, left (and diagonal for replace).
- LIS in O(n log n) keeps the smallest tail for each length; the list is not the subsequence.
- Rebuild answers from the full table; compress to two rows only when the length is enough.

6.5 DP on intervals, trees and states

Some problems have a state that is neither a prefix nor a grid cell. The state may be a **range** (the best result on the segment from `l` to `r`), a **tree node** (the best result in the subtree), a **mode** (for example, whether a share is currently held) or a **set** (the items used so far, encoded as a bitmask). The method is unchanged: state definition, transition, base cases and evaluation order. Only the content of the state differs.

6.5.1 The idea

Interval DP. `dp[l][r]` is the answer for the segment `l..r`. Transition: choose a split point or a "last action" `k` inside the segment, combine `dp[l][k]` and `dp[k][r]`. Fill order: by increasing segment length, so that shorter segments are ready. Clue: "merge / burst / remove in some order", "cost depends on neighbours at the time of the action", "minimum cost to multiply a chain of matrices", palindromes by ends.

Tree DP. The recursion returns a small tuple per node, for example "best if this node is taken, best if it is skipped". It is a post-order DFS; every node is visited once. Clue: a binary tree plus a rule linking parent and child choices (no two adjacent nodes, diameter through a node).

State-machine DP. The day (or index) is the loop; the extra dimension is a small *mode* such as `holding / not holding / cooling down`, or the number of transactions used. Each day's value for each mode is computed from yesterday's values for the modes that may precede it. Clue: "at most k transactions", "cooldown", "fee", "alternating".

Bitmask DP. When `n` is at most about 20, a subset of items can be encoded as the bits of an integer `mask` (`1 << i` means item `i` is used). `dp[mask]` or `dp[mask][last]` then stores the best result for exactly that subset. There are 2^n masks, so n = 20 gives about a million states. Clue: "n <= 20", "visit every city once", "assign tasks to workers", "can the first player force a win using numbers 1..n".

6.5.2 Dry run

Burst Balloons: balloons carry numbers `[3, 1, 5, 8]`. Bursting balloon `i` earns `left * nums[i] * right` coins, where left and right are its *current* neighbours (an out-of-range neighbour counts as 1). Find the best order of bursting.

The naive state "which balloons remain" has 2^n values, and the neighbours of a balloon change as others are removed. The formulation is changed to select the **last** balloon burst in a segment. If `k` is the last balloon burst strictly between `l` and `r`, then `l` and `r` are still present at that moment, so the gain is `a[l] * a[k] * a[r]`, and the segments `(l, k)` and `(k, r)` are independent sub-problems. The array is padded to `[1, 3, 1, 5, 8, 1]` so that the borders are valid indices, and the table is filled in order of increasing `r - l`.

len	Segment (l, r)	Best last balloon k	dp[l][r]
2	(0, 2)	a[1] = 3	3
2	(1, 3)	a[2] = 1	15
2	(2, 4)	a[3] = 5	40
2	(3, 5)	a[4] = 8	40
3	(0, 3)	a[1] = 3	30
3	(1, 4)	a[3] = 5	135
3	(2, 5)	a[4] = 8	48
4	(0, 4)	a[1] = 3	159
4	(1, 5)	a[4] = 8	159
5	(0, 5)	a[4] = 8	167

Burst balloons on [3,1,5,8], padded a = [1,3,1,5,8,1] (real trace).

Verification of one cell: `dp[2][4]` covers only the balloon `a[3] = 5` between `a[2] = 1` and `a[4] = 8`, giving `1 * 5 * 8 = 40`. The final cell, segment `(0, 5)` containing all four balloons, selects `a[4] = 8` as the last balloon: `dp[0][4] + 1 * 8 * 1 + dp[4][5]` = `159 + 8 + 0 = 167`. The result is 167.

6.5.3 Java code

The first program has Burst Balloons (interval), Best Time to Buy and Sell Stock with Cooldown (state machine with three modes) and the general "at most k transactions" stock solution (also a state machine: `buy[j]` and `sell[j]` for the j-th transaction).

Main.java JAVA

```
import java.util.Arrays;

public class Main {
    // Burst balloons. Pad the array with 1 at both ends. dp[l][r] = best coins from bursting
    // every balloon STRICTLY between l and r. Trick: pick k as the LAST balloon to burst
    // in (l, r); then its neighbours at that moment are exactly l and r.
    static int maxCoins(int[] nums, boolean trace) {
        int n = nums.length;
        int[] a = new int[n + 2];
        a[0] = a[n + 1] = 1;
        System.arraycopy(nums, 0, a, 1, n);
        int[][] dp = new int[n + 2][n + 2];
        for (int len = 2; len <= n + 1; len++) {             // distance r - l
            for (int l = 0; l + len <= n + 1; l++) {
                int r = l + len;
                int best = 0, bestK = -1;
                for (int k = l + 1; k < r; k++) {
                    int coins = dp[l][k] + a[l] * a[k] * a[r] + dp[k][r];
                    if (coins > best) {
                        best = coins;
                        bestK = k;
                    }
                }
                dp[l][r] = best;
                if (trace) {
                    System.out.println("len=" + len + " (l=" + l + ",r=" + r + ") last=a[" + bestK
                        + "]=" + a[bestK] + " dp=" + best);
                }
            }
        }
        return dp[0][n + 1];
    }

    // Stock with cooldown: three states per day, each a max profit so far.
    static int stockCooldown(int[] p, boolean trace) {
        int hold = -p[0];      // currently own a share
        int sold = 0;          // sold today (tomorrow must rest)
        int rest = 0;          // own nothing and free to buy
        for (int i = 1; i < p.length; i++) {
            int newHold = Math.max(hold, rest - p[i]);
            int newSold = hold + p[i];
            int newRest = Math.max(rest, sold);
            hold = newHold;
            sold = newSold;
            rest = newRest;
            if (trace) System.out.println("day " + i + " price=" + p[i] + " hold=" + hold
                + " sold=" + sold + " rest=" + rest);
        }
        return Math.max(sold, rest);
    }

    // Stock with at most k transactions. buy[j] / sell[j] = best profit in the j-th transaction.
    static int stockK(int k, int[] p) {
        int[] buy = new int[k + 1];
        int[] sell = new int[k + 1];
        Arrays.fill(buy, Integer.MIN_VALUE / 2);
        for (int price : p) {
            for (int j = 1; j <= k; j++) {
                buy[j] = Math.max(buy[j], sell[j - 1] - price);
                sell[j] = Math.max(sell[j], buy[j] + price);
            }
        }
        return sell[k];
    }

    public static void main(String[] args) {
        int[] balloons = {3, 1, 5, 8};
        System.out.println("balloons = " + Arrays.toString(balloons));
```

Main.java (continued) JAVA

```
        System.out.println("max coins = " + maxCoins(balloons, true));
        int[] prices = {1, 2, 3, 0, 2};
        System.out.println("prices = " + Arrays.toString(prices));
        System.out.println("cooldown profit = " + stockCooldown(prices, true));
        int[] p2 = {3, 3, 5, 0, 0, 3, 1, 4};
        System.out.println("k=2 on [3,3,5,0,0,3,1,4] = " + stockK(2, p2));
        System.out.println("k=1 on [7,1,5,3,6,4] = " + stockK(1, new int[]{7, 1, 5, 3, 6, 4}));
    }
}
```

Run it TERMINAL

```
$ java Main.java
balloons = [3, 1, 5, 8]
len=2 (l=0,r=2) last=a[1]=3 dp=3
len=2 (l=1,r=3) last=a[2]=1 dp=15
len=2 (l=2,r=4) last=a[3]=5 dp=40
len=2 (l=3,r=5) last=a[4]=8 dp=40
len=3 (l=0,r=3) last=a[1]=3 dp=30
len=3 (l=1,r=4) last=a[3]=5 dp=135
len=3 (l=2,r=5) last=a[4]=8 dp=48
len=4 (l=0,r=4) last=a[1]=3 dp=159
len=4 (l=1,r=5) last=a[4]=8 dp=159
len=5 (l=0,r=5) last=a[4]=8 dp=167
max coins = 167
prices = [1, 2, 3, 0, 2]
day 1 price=2 hold=-1 sold=1 rest=0
day 2 price=3 hold=-1 sold=2 rest=1
day 3 price=0 hold=1 sold=-1 rest=2
day 4 price=2 hold=1 sold=3 rest=2
cooldown profit = 3
k=2 on [3,3,5,0,0,3,1,4] = 6
k=1 on [7,1,5,3,6,4] = 5
```

Observations. In the cooldown computation, `sold` is reachable only from `hold`, and `rest` takes the maximum of `rest` and the previous day's `sold` (the cooldown day). The three new values are computed from the three old values using temporaries, because in-place updates would mix values of different days. The `k`-transaction version is the same state machine with the transaction count as an additional dimension. When `k` is at least `n / 2` the limit is non-binding and the problem reduces to the unlimited case (sum of all positive day-to-day differences), which avoids a large table.

The second program shows tree DP (House Robber III, using a `record` for the nodes) and the two bitmask classics: Travelling Salesman and the first-player-wins number game.

Main.java JAVA

```
import java.util.Arrays;

public class Main {
    record Node(int val, Node left, Node right) {}

    // House Robber III: each node returns {best if we skip it, best if we rob it}.
    static int[] robTree(Node node) {
        if (node == null) return new int[]{0, 0};
        int[] left = robTree(node.left());
        int[] right = robTree(node.right());
        int skip = Math.max(left[0], left[1]) + Math.max(right[0], right[1]);
        int rob = node.val() + left[0] + right[0];     // robbing me forces children to be skipped
        return new int[]{skip, rob};
    }

    // Bitmask DP: shortest tour visiting every city exactly once (Travelling Salesman, n <= ~20).
    // dp[mask][last] = cheapest cost to have visited the cities in mask, standing at `last`.
    static int tsp(int[][] dist) {
        int n = dist.length, full = (1 << n) - 1, INF = 1_000_000_000;
        int[][] dp = new int[1 << n][n];
        for (int[] row : dp) Arrays.fill(row, INF);
        dp[1][0] = 0;                                  // start at city 0, only city 0 visited
        for (int mask = 1; mask <= full; mask++) {
            for (int last = 0; last < n; last++) {
                if (dp[mask][last] == INF || (mask & (1 << last)) == 0) continue;
                for (int next = 0; next < n; next++) {
                    if ((mask & (1 << next)) != 0) continue;       // already visited
                    int nm = mask | (1 << next);
```

Main.java (continued) JAVA

```
                    dp[nm][next] = Math.min(dp[nm][next], dp[mask][last] + dist[last][next]);
                }
            }
        }
        int best = INF;
        for (int last = 1; last < n; last++) best = Math.min(best, dp[full][last] + dist[last][0]);
        return best;
    }

    // Can I Win: players alternately pick an unused number 1..max; first to reach total wins.
    static boolean canIWin(int max, int total) {
        if (max * (max + 1) / 2 < total) return false;
        if (total <= 0) return true;
        Boolean[] memo = new Boolean[1 << max];           // memo[usedMask] = does the mover win?
        return win(max, total, 0, memo);
    }

    static boolean win(int max, int remaining, int used, Boolean[] memo) {
        if (memo[used] != null) return memo[used];
        for (int i = 1; i <= max; i++) {
            int bit = 1 << (i - 1);
            if ((used & bit) != 0) continue;
            if (i >= remaining || !win(max, remaining - i, used | bit, memo)) {
                return memo[used] = true;
            }
        }
        return memo[used] = false;
    }

    public static void main(String[] args) {
        Node root = new Node(3, new Node(2, null, new Node(3, null, null)),
                                new Node(3, null, new Node(1, null, null)));
        int[] r = robTree(root);
        System.out.println("robTree: skip root=" + r[0] + ", rob root=" + r[1]
            + " -> best " + Math.max(r[0], r[1]));
        int[][] dist = {{0, 10, 15, 20}, {10, 0, 35, 25}, {15, 35, 0, 30}, {20, 25, 30, 0}};
        System.out.println("tsp = " + tsp(dist));
        System.out.println("canIWin(10, 11) = " + canIWin(10, 11));
        System.out.println("canIWin(10, 0)  = " + canIWin(10, 0));
        System.out.println("canIWin(4, 6)   = " + canIWin(4, 6));
    }
}
```

Run it TERMINAL

```
$ java Main.java
robTree: skip root=6, rob root=7 -> best 7
tsp = 80
canIWin(10, 11) = false
canIWin(10, 0)  = true
canIWin(4, 6)   = true
```

In `robTree` each node returns `{best if skipped, best if taken}`. Skipping a node allows each child to choose freely (the maximum of its two values); taking it forces both children to their "skipped" value. No memo map is needed because each node is visited once, giving O(n) in total. In the number game, `memo[used]` is sufficient because `remaining` is a function of `used` (the target minus the sum of the used numbers).

6.5.4 Complexity and mistakes

- Interval DP: O(n^2) states with O(n) split points each, so O(n^3) time and O(n^2) space. Fine for n up to about 500.
- Tree DP: O(n) time and O(h) recursion stack for tree height `h`. A skewed tree of 10^5 nodes can overflow the default stack; an iterative post-order traversal or a thread with a larger stack is then required.
- State machine: O(n × states) time, O(states) space after compression.
- Bitmask DP: O(2^n × n) states and O(n) per transition for TSP gives O(2^n × n^2); for n = 20 this is about 4 × 10^8 simple operations, which is borderline. The memory is the larger problem (2^20 × 20 ints is 80 MB).

Mistakes:

- **Choosing the first action instead of the last** in interval problems (Burst Balloons). The first balloon changes its neighbours' neighbours; the last one does not.

- **Wrong segment order.** Filling by `l` only (without growing the length) reads cells that are still zero.
- **Updating the state-machine values in place** and mixing today's with yesterday's.
- **Forgetting the sentinel padding** (the two `1`s) or reading `a[l - 1]` out of range.
- **Operator precedence in bit tests.** The expression `mask & 1 << i` parses as `mask & (1 << i)` in Java (shift binds tighter than `&`), whereas `mask & (1 << i) != 0` applies `!=` first and does not compile. The form `(mask & (1 << i)) != 0` is therefore required.
- **Using `1 << i` with `i >= 31`** overflows `int`; use `1L << i` and a `long` mask for n between 32 and 63 (then DP over masks is impractical anyway).

6.5.5 Questions to expect

1. "Maximum coins from bursting balloons in the best order." Clue: removal order with neighbour-dependent score. Approach: interval DP on the last balloon, O(n^3).
2. "Best Time to Buy and Sell Stock with cooldown." Clue: mode changes with a forced rest. Approach: three-state machine.
3. "At most k transactions." Clue: count of transactions limited. Approach: `buy[j]`, `sell[j]`; if `k >= n / 2` fall back to the unlimited version.
4. "Rob houses arranged in a binary tree; two directly linked houses cannot both be robbed." Clue: tree plus adjacency rule. Approach: return a pair per node.
5. "Minimum cost to multiply a chain of matrices" (matrix-chain). Clue: parenthesisation. Approach: `dp[i][j] = min over k of dp[i][k] + dp[k+1][j] + p[i] * p[k+1] * p[j+1]`.
6. "Shortest route that visits every city exactly once and returns." Clue: n at most 15 to 20 and "every". Approach: bitmask DP (Held-Karp), O(2^n × n^2).
7. "Can the first player force a win?" (Can I Win). Clue: two-player game, state described by used numbers. Approach: memoise on the bitmask of used numbers.
8. "Partition an array into k subsets with equal sum." Clue: n at most 16. Approach: bitmask over used elements, filling one bucket at a time.

INTERVIEW ANGLE

"How is interval DP recognised?" The answer for a range depends on how the range is split or which element is handled last, and the cost of an action depends on the boundaries of the range. Typical statements are burst balloons, merge stones, minimum cost to cut a stick and longest palindromic subsequence. The table is filled by increasing length; the complexity is O(n^3) with a loop over split points. The essential step is the choice of the enumerated quantity: usually the last action in the range, because the sub-problems are then independent.

6.5.6 LeetCode practice

#	Title	Level	Key idea
122	Best Time to Buy and Sell Stock II	Medium	Unlimited transactions: add every positive day-to-day rise.
714	Best Time to Buy and Sell Stock with Transaction Fee	Medium	Two states, hold and free; pay the fee when selling.
309	Best Time to Buy and Sell Stock with Cooldown	Medium	Three states: hold, sold, rest.
337	House Robber III	Medium	Post-order DFS returning a (skip, rob) pair for every node.
464	Can I Win	Medium	Memoise on the bitmask of used numbers.
698	Partition to K Equal Sum Subsets	Medium	Bitmask over used elements; fill one bucket at a time.
123	Best Time to Buy and Sell Stock III	Hard	State machine with exactly two buy/sell pairs.
188	Best Time to Buy and Sell Stock IV	Hard	buy[j]/sell[j] for up to k pairs; k >= n / 2 means unlimited.
312	Burst Balloons	Hard	Interval dp on the LAST balloon to burst in each segment.

LeetCode problems for this pattern.

KEY POINTS

When prefixes and grids do not fit, choose a state that does: a range, a tree node, a mode, or a bitmask.

- Interval DP enumerates the last action in `l..r` and fills by increasing length (O(n^3)).
- Tree DP returns a small tuple per node from a post-order DFS (O(n)).
- State-machine DP keeps one value per mode and updates them together each step.
- Bitmask DP needs n at most about 20; always parenthesise bit tests.

6.6 Greedy

A greedy algorithm constructs a solution incrementally and, at each step, selects the locally best admissible choice without revising it. When applicable it is efficient, typically a sort followed by a single pass, O(n log n) or O(n), with little auxiliary memory. Its applicability is restricted to problems with specific structure, and a proof of correctness is required. The dynamic-programming chapters evaluate all alternatives; greedy is valid only when the local choice can be justified.

Exhaustive search for a scheduling problem examines every subset of jobs (2^n) or every order (n!). The greedy insight is that a specific ordering rule makes all other choices at each step provably no better.

6.6.1 The idea

Greedy requires two properties. The **greedy-choice property**: some optimal solution contains the locally best first choice. **Optimal substructure**: after that choice, the remainder is an instance of the same problem. Correctness is usually established by an **exchange argument**: take any optimal solution that does not contain the greedy choice, substitute the greedy choice, and show that the result is no worse.

The argument for interval scheduling (maximum number of non-overlapping intervals) is as follows. Consider the interval with the earliest end. Any optimal solution has a first interval; replacing it by the earliest-ending interval cannot introduce an overlap, because the replacement ends no later. The solution size is unchanged, so selecting the earliest-ending interval first is safe. The argument is then applied to the remaining intervals.

Clues in the statement:

- "Minimum number of ... to cover / remove / schedule" on intervals, jumps, platforms, arrows.
- "Maximum number of non-overlapping ...", "earliest", "latest", "smallest", "largest first".
- A choice that can be made with a sort on a single key, then one pass.
- "Can you reach the end?", "fewest jumps" where each position offers a reach.
- Rearrangement without adjacent duplicates (use a heap of counts).
- Distribution problems: cookies to children, candy by ratings.

Distinguishing greedy from DP: a counter-example is sought on a small input of two or three items. If the natural rule fails (coin change with coins 1, 3, 4 and amount 6), DP is required. If no counter-example exists and an exchange argument can be constructed, greedy is appropriate.

GreedySkeleton.java JAVA

```java
// Typical shape: sort by the key the exchange argument justifies, then scan once.
Arrays.sort(items, (a, b) -> Integer.compare(a.key(), b.key()));
int chosen = 0;
State state = initial;
for (Item item : items) {
    if (canTake(item, state)) {      // the locally best option that is still allowed
        take(item, state);
        chosen++;
    }
}
```

6.6.2 Dry run

Jump Game II: `nums[i]` is the maximum jump length from index `i`. Starting at index 0, find the fewest jumps to reach the last index. Input `nums = [2, 3, 1, 1, 4]`.

The problem is analysed in levels, as in breadth-first search. After 0 jumps the position is index 0; after 1 jump any index in `1..2` is reachable; after 2 jumps any index up to the farthest reach from that window. The scan maintains `end` (the last index of the current window) and `farthest` (the maximum reach observed inside it). When `i` reaches `end`, another jump is required, and the new window ends at `farthest`.

i	nums[i]	farthest	end	jumps	What happened
0	2	2	2	1	i equals old end (0): take a jump; window is now 1..2
1	3	4	2	1	inside the window; reach grows to 4 (index 1 plus 3)
2	1	4	4	2	i equals end: second jump; window now reaches index 4
3	1	4	4	2	nothing new; the end is already covered

Jump Game II on [2, 3, 1, 1, 4] (real trace; stops before the last index).

The result is 2 (index 0 to index 1, then index 1 to index 4). The algorithm never selects a landing index; it records only the maximum reach of the window, which is why a single pass suffices.

6.6.3 Java code

The first program contains four problems that maintain a running optimum: Jump Game (reachability), Jump Game II (above, with the trace), Gas Station (the start candidate is reset whenever the tank becomes negative) and Interval Scheduling (sort by end time).

Main.java JAVA

```java
import java.util.Arrays;

public class Main {
    // Jump Game: can we reach the last index? Track the farthest index reachable so far.
    static boolean canJump(int[] nums) {
        int farthest = 0;
        for (int i = 0; i < nums.length; i++) {
            if (i > farthest) return false;              // stranded: i is unreachable
            farthest = Math.max(farthest, i + nums[i]);
        }
        return true;
    }

    // Jump Game II: fewest jumps. Think in BFS levels: all indices reachable with j jumps
    // form a window [.., end]. The next window ends at the farthest point seen inside this one.
    static int minJumps(int[] nums, boolean trace) {
        int jumps = 0, end = 0, farthest = 0;
        for (int i = 0; i < nums.length - 1; i++) {
            farthest = Math.max(farthest, i + nums[i]);
            boolean jump = (i == end);
            if (jump) {
                jumps++;
                end = farthest;
            }
            if (trace) {
                System.out.println("i=" + i + " nums[i]=" + nums[i] + " farthest=" + farthest
                    + " end=" + end + " jumps=" + jumps + (jump ? "  <- jump" : ""));
            }
        }
        return jumps;
    }

    // Gas station: if total gas >= total cost a start exists; restart after any negative tank.
    static int gasStation(int[] gas, int[] cost) {
        int total = 0, tank = 0, start = 0;
        for (int i = 0; i < gas.length; i++) {
            int diff = gas[i] - cost[i];
            total += diff;
            tank += diff;
```

Main.java (continued) JAVA

```
            if (tank < 0) {              // cannot reach i+1 from start; nothing in between works
                start = i + 1;
                tank = 0;
            }
        }
        return total >= 0 ? start : -1;
    }

    // Interval scheduling: the most non-overlapping intervals. Sort by END time, take greedily.
    static int maxNonOverlapping(int[][] iv) {
        Arrays.sort(iv, (x, y) -> Integer.compare(x[1], y[1]));   // not x[1] - y[1]: can overflow
        int count = 0, lastEnd = Integer.MIN_VALUE;
        for (int[] cur : iv) {
            if (cur[0] >= lastEnd) {
                count++;
                lastEnd = cur[1];
            }
        }
        return count;
    }

    public static void main(String[] args) {
        int[] jumps = {2, 3, 1, 1, 4};
        System.out.println("nums = " + Arrays.toString(jumps));
        System.out.println("minJumps = " + minJumps(jumps, true));
        System.out.println("canJump [3,2,1,0,4] = " + canJump(new int[]{3, 2, 1, 0, 4}));
        System.out.println("canJump [2,3,1,1,4] = " + canJump(jumps));
        int[] gas = {1, 2, 3, 4, 5}, cost = {3, 4, 5, 1, 2};
        System.out.println("gasStation = " + gasStation(gas, cost));
        System.out.println("gasStation impossible = "
            + gasStation(new int[]{2, 3, 4}, new int[]{3, 4, 3}));
        int[][] iv = {{1, 3}, {2, 4}, {3, 5}, {0, 6}, {5, 7}, {8, 9}, {5, 9}};
        int kept = maxNonOverlapping(iv);
        System.out.println("intervals kept = " + kept + ", removed = " + (iv.length - kept));
    }
}
```

Run it TERMINAL

```
$ java Main.java
nums = [2, 3, 1, 1, 4]
i=0 nums[i]=2 farthest=2 end=2 jumps=1  <- jump
i=1 nums[i]=3 farthest=4 end=2 jumps=1
i=2 nums[i]=1 farthest=4 end=4 jumps=2  <- jump
i=3 nums[i]=1 farthest=4 end=4 jumps=2
minJumps = 2
canJump [3,2,1,0,4] = false
canJump [2,3,1,1,4] = true
gasStation = 3
gasStation impossible = -1
intervals kept = 4, removed = 3
```

Observations. In Gas Station, if the trip starts at `s` and fuel is exhausted at station `i`, no station between `s` and `i` can be a valid start either, since each would arrive at `i` with no more fuel; the next candidate is therefore `i + 1`. If total gas is at least total cost, one candidate must be valid. Interval scheduling sorts by end time with `Integer.compare`, not subtraction (`x[1] - y[1]` can overflow). The program keeps 4 intervals and removes 3, which is also the result for "Non-overlapping Intervals".

The second program contains distribution and rearrangement problems, and a greedy rule that fails, for contrast:

Main.java JAVA

```
import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;
import java.util.PriorityQueue;

public class Main {
    // Assign cookies: sort both, give each child the smallest cookie that satisfies it.
    static int assignCookies(int[] greed, int[] size) {
        Arrays.sort(greed);
        Arrays.sort(size);
        int child = 0;
        for (int cookie = 0; cookie < size.length && child < greed.length; cookie++) {
```

Main.java (continued) JAVA

```
            if (size[cookie] >= greed[child]) child++;
        }
        return child;
    }

    // Candy: two passes. Left-to-right fixes "higher than left neighbour", right-to-left the other.
    static int candy(int[] rating) {
        int n = rating.length;
        int[] c = new int[n];
        Arrays.fill(c, 1);
        for (int i = 1; i < n; i++) if (rating[i] > rating[i - 1]) c[i] = c[i - 1] + 1;
        for (int i = n - 2; i >= 0; i--) {
            if (rating[i] > rating[i + 1]) c[i] = Math.max(c[i], c[i + 1] + 1);
        }
        return Arrays.stream(c).sum();
    }

    // Reorganize string: no two equal neighbours. Always place the most frequent letter that
    // is not the one we just placed (a max-heap of counts).
    static String reorganize(String s) {
        int[] freq = new int[26];
        for (char ch : s.toCharArray()) freq[ch - 'a']++;
        PriorityQueue<int[]> pq = new PriorityQueue<>((x, y) -> y[1] - x[1]);   // max-heap by count
        for (int i = 0; i < 26; i++) if (freq[i] > 0) pq.add(new int[]{i, freq[i]});
        StringBuilder sb = new StringBuilder();
        int[] held = null;                                  // letter we must skip for one turn
        while (!pq.isEmpty() || held != null) {
            if (pq.isEmpty()) return "";                    // only the held letter is left
            int[] top = pq.poll();
            sb.append((char) ('a' + top[0]));
            top[1]--;
            if (held != null) pq.add(held);
            held = top[1] > 0 ? top : null;
        }
        return sb.toString();
    }

    // Partition labels: cut as early as possible so each letter lives in exactly one piece.
    static List<Integer> partitionLabels(String s) {
        int[] last = new int[26];
        for (int i = 0; i < s.length(); i++) last[s.charAt(i) - 'a'] = i;
        List<Integer> sizes = new ArrayList<>();
        int start = 0, end = 0;
        for (int i = 0; i < s.length(); i++) {
            end = Math.max(end, last[s.charAt(i) - 'a']);
            if (i == end) {
                sizes.add(end - start + 1);
                start = i + 1;
            }
        }
        return sizes;
    }

    // Greedy that FAILS: coin change with {1, 3, 4}. Take the biggest coin first.
    static int greedyCoins(int[] coins, int amount) {
        int[] sorted = coins.clone();
        Arrays.sort(sorted);
        int count = 0;
        for (int i = sorted.length - 1; i >= 0; i--) {
            count += amount / sorted[i];
            amount %= sorted[i];
        }
        return amount == 0 ? count : -1;
    }

    public static void main(String[] args) {
        System.out.println("cookies = " + assignCookies(new int[]{1, 2, 3}, new int[]{1, 1}));
        System.out.println("candy [1,0,2] = " + candy(new int[]{1, 0, 2}));
        System.out.println("candy [1,2,2] = " + candy(new int[]{1, 2, 2}));
        System.out.println("reorganize aab = " + reorganize("aab"));
        System.out.println("reorganize aaab = '" + reorganize("aaab") + "'");
        System.out.println("reorganize vvvlo = " + reorganize("vvvlo"));
        System.out.println("labels = " + partitionLabels("ababcbacadefegdehijhklij"));
```

Sieran

Main.java (continued) JAVA

```
        System.out.println("greedy coins {1,3,4} for 6 = " + greedyCoins(new int[]{1, 3, 4}, 6)
            + " (optimal is 2: 3+3)");
    }
}
```

Run it TERMINAL

```
$ java Main.java
cookies = 1
candy [1,0,2] = 5
candy [1,2,2] = 4
reorganize aab = aba
reorganize aaab = ''
reorganize vvvlo = vlvov
labels = [9, 7, 8]
greedy coins {1,3,4} for 6 = 3 (optimal is 2: 3+3)
```

Observations. `candy` performs two passes: left to right enforces "a higher rating than the left neighbour receives more", right to left enforces the same for the right neighbour, and `max` preserves both. `reorganize` uses a max-heap of letter counts and withholds the letter just placed for one turn so that it cannot repeat; `aaab` is infeasible (a letter may occupy at most half of the positions, rounded up), and the code detects this when only the withheld letter remains. The final line shows that greedy coin change returns 3 coins where 2 is optimal.

6.6.4 Complexity and mistakes

Most greedy solutions are dominated by a sort, so O(n log n) time and O(1) or O(n) extra space; single-pass ones (Jump Game, Gas Station) are O(n). The heap in Reorganize String gives O(n log k) for `k` distinct letters (at most 26).

- **Sorting by the wrong key.** Interval scheduling sorts by *end* time. Sorting by start time or by length gives wrong answers on easy counter-examples (one long interval overlapping many short ones).
- **No proof, no counter-example search.** Try small inputs by hand, or write a brute force and compare on random tests.
- **Comparator overflow.** The comparator `(a, b) -> a[1] - b[1]` fails when values are near the `int` limits; `Integer.compare` is correct.
- **Off-by-one on touching intervals.** `[1,3]` and `[3,5]` overlap or not depending on the statement. Check whether the boundary is inclusive (`cur[0] >= lastEnd` above treats touching as fine).
- **Looping to the last index in Jump Game II.** Stop at `n - 2`: counting a jump at the last index over-counts by one.
- **Forgetting the impossible case** (Gas Station total negative, Reorganize String with a dominant letter).
- **Using a `Stack` or a `PriorityQueue` of the wrong order.** `PriorityQueue` is a min-heap; for "largest count first" pass a reversed comparator.

GOTCHA

A comparator of the form `(a, b) -> b[1] - a[1]` is incorrect for values that may be large or negative. `Integer.compare(b[1], a[1])` is correct; negation-based alternatives such as `Comparator.comparingInt(a -> -a[1])` also overflow for `Integer.MIN_VALUE`.

6.6.5 Questions to expect

1. "Determine whether the last index is reachable if each cell gives a maximum jump length." Clue: reach. Approach: single pass tracking the farthest reachable index; fail if `i > farthest`.
2. "Fewest jumps to reach the end." Clue: minimum jumps with ranges. Approach: level-by-level window as in the dry run; O(n).
3. "Circular route with gas and cost arrays: which station can start the trip?" Clue: circular, running balance. Approach: total check plus restart on negative tank.
4. "Maximum number of meetings / activities without overlap; or minimum removals to make intervals non-overlapping." Clue: intervals plus a count. Approach: sort by end. Follow-up: minimum arrows to burst all balloons (the same sort, count groups of overlapping intervals).
5. "Distribute candies by ratings so that higher-rated neighbours get more." Clue: two-sided neighbour constraint. Approach: two passes. Follow-up: O(1) space via slope counting.

Sheeram

6. "Rearrange a string so no two adjacent characters are equal." Clue: adjacency constraint with counts. Approach: max-heap on counts, or fill even then odd positions with the most frequent letter first.
7. "Split a string into as many parts as possible so each letter appears in only one part." Clue: last occurrence. Approach: record last index per letter, cut when `i` reaches the current maximum last index.
8. "Valid parentheses string with `*` wildcards." Clue: balance may be a range. Approach: track the minimum and maximum possible open count; O(n) and O(1).

INTERVIEW ANGLE

"How is the correctness of a greedy solution established?" An exchange argument is sought: assume an optimal solution that differs from the greedy choice at the first step, substitute the greedy choice, and show that the solution is not worse. The rule is also tested on small inputs, because a single counter-example invalidates it (coin change with coins 1, 3, 4 and amount 6 refutes "take the largest coin"). If no proof is found or a counter-example exists, dynamic programming, which examines all options at the cost of a table, is used instead.

6.6.6 LeetCode practice

#	Title	Level	Key idea
455	Assign Cookies	Easy	Sort both; give each child the smallest cookie that satisfies them.
55	Jump Game	Medium	Track the farthest reachable index; fail when stranded.
45	Jump Game II	Medium	BFS levels in one pass: jump when you reach the end of the window.
134	Gas Station	Medium	If total gas >= total cost, restart the candidate after any negative tank.
435	Non-overlapping Intervals	Medium	Sort by end; count how many you keep; removals = total - kept.
452	Minimum Number of Arrows to Burst Balloons	Medium	Sort by end; shoot at the end of the first balloon; skip all it pops.
763	Partition Labels	Medium	Last occurrence of each letter; cut when i reaches the running maximum.
767	Reorganize String	Medium	Max-heap of counts; never place the same letter twice in a row.
678	Valid Parenthesis String	Medium	Keep the range [lowest, highest] of possible open parentheses.
846	Hand of Straights	Medium	Take the smallest card; it must start a group; use a TreeMap of counts.
135	Candy	Hard	Two passes (left to right, right to left), keep the maximum.

LeetCode problems for this pattern.

KEY POINTS

Greedy commits to the locally best choice and never looks back; it is fast but needs a justification.

- Sort-then-scan structure is the typical form; correctness is justified by an exchange argument.
- The rule is tested against a small counter-example (coin change with {1, 3, 4} is the standard one).
- Interval problems sort by end time; compare with `Integer.compare`, not subtraction.
- If the greedy rule fails or cannot be proved, dynamic programming is used.

sreeram

6.7 Bit manipulation

Integers are stored as fixed-width bit patterns, and the operations that act directly on those bits (AND, OR, XOR, shifts) are among the cheapest processor instructions. Bit techniques replace auxiliary data structures with arithmetic on the representation: "find the element without a pair", "count the set bits", "enumerate all subsets". Solutions typically use O(1) memory where a straightforward approach uses O(n).

6.7.1 The idea

Java's `int` has 32 bits in two's complement representation and `long` has 64. Negative numbers have the sign bit set, and `-x` equals `~x + 1`. The operators are:

Operation	Meaning	Typical use
AND	1 where both bits are 1	test a bit, clear bits, mask
OR (the pipe operator)	1 where either bit is 1	set a bit, combine flags
XOR	1 where the bits differ	cancel pairs, toggle, add without carry
NOT (tilde)	flips every bit	build a mask: n & ~(1 << k) clears bit k
Shift left	shift left, fill with 0	multiply by 2^k, build 1 << k
Shift right (signed)	arithmetic shift right (copies the sign)	divide by 2^k rounding toward negative infinity
Shift right (unsigned)	logical shift right (fills with 0)	treat the int as unsigned, extract high bits

Java bit operators: & (AND), | (OR), ^ (XOR), ~ (NOT), << >> >>> (shifts).

Four identities underlie most solutions:

- `x ^ x = 0`, `x ^ 0 = x`, and XOR is commutative and associative. So XOR of a list cancels every value that appears twice.
- `n & (n - 1)` clears the lowest set bit of `n`. Therefore `n & (n - 1) == 0` for `n > 0` means a power of two, and repeating it counts set bits (Brian Kernighan).
- `n & -n` isolates the lowest set bit.
- `(mask >> i) & 1` reads bit `i`; `mask | (1 << i)` sets it; `mask & ~(1 << i)` clears it; `mask ^ (1 << i)` toggles it.

Clue phrases: "without using extra space", "appears twice except one", "power of two", "number of 1 bits", "all subsets" (n at most about 20), "without using + or -", "XOR", "parity", "bit by bit".

6.7.2 Dry run

The example counts the set bits of `n = 44` (binary `101100`) with Brian Kernighan's method. Each iteration computes `n & (n - 1)`. Subtracting 1 clears the lowest set bit and sets all lower bits; the conjunction with the original value therefore clears exactly the lowest set bit. The loop executes once per set bit rather than once per bit position.

Step	n	n - 1	n & (n - 1)	count
1	101100	101011	101000	1
2	101000	100111	100000	2
3	100000	011111	000000	3

Kernighan on 44 (real trace, 6-bit binary).

`n` reached 0 after three steps, so `44` has three set bits. The related "counting bits" recurrence is `bits[i] = bits[i >> 1] + (i & 1)`: the count for `i` is the count for `i` without its last bit, plus that last bit. For `i` from 0 to 8 it gives `[0, 1, 1, 2, 1, 2, 2, 3, 1]` (real output below). Check `i = 5` (binary 101): `bits[2] + 1 = 1 + 1 = 2` ✓.

6.7.3 Java code

The first program demonstrates the operators with printed binary values, including sign and shift edge cases.

Main.java JAVA

```java
public class Main {
    static String bin(int x, int width) {
        String s = Integer.toBinaryString(x);
        return "0".repeat(Math.max(0, width - s.length())) + s;
    }

    // Brian Kernighan: n & (n - 1) clears the lowest set bit, so the loop runs once per 1-bit.
    static int popcount(int n, boolean trace) {
        int count = 0;
        while (n != 0) {
            int next = n & (n - 1);
            if (trace) {
                System.out.println("n=" + bin(n, 6) + "  n-1=" + bin(n - 1, 6)
                    + "  n&(n-1)=" + bin(next, 6));
            }
            n = next;
            count++;
        }
        return count;
    }

    public static void main(String[] args) {
        System.out.println("popcount(44) via Kernighan:");
        System.out.println("count = " + popcount(44, true));

        int a = 0b1100, b = 0b1010;
        System.out.println("a=" + bin(a, 4) + " b=" + bin(b, 4));
        System.out.println("a & b = " + bin(a & b, 4) + ", a | b = " + bin(a | b, 4)
            + ", a ^ b = " + bin(a ^ b, 4) + ", ~a & 15 = " + bin(~a & 15, 4));
        int n = 0b101100;
        System.out.println("check bit 2: " + ((n >> 2) & 1) + ", set bit 0: " + bin(n | 1, 6));
        System.out.println("clear bit 3: " + bin(n & ~(1 << 3), 6)
            + ", toggle bit 5: " + bin(n ^ (1 << 5), 6));
        System.out.println("lowest set bit of 44: " + (44 & -44)
            + "  (-44 = " + Integer.toBinaryString(-44) + ")");
        System.out.println("isPowerOfTwo: 16 -> " + isPow2(16) + ", 18 -> " + isPow2(18)
            + ", 0 -> " + isPow2(0) + ", MIN_VALUE -> " + isPow2(Integer.MIN_VALUE));
        System.out.println("-8 >> 1 = " + (-8 >> 1) + ", -8 >>> 1 = " + (-8 >>> 1));
        System.out.println("1 << 31 = " + (1 << 31) + ", 1 << 32 = " + (1 << 32)
            + ", 1L << 32 = " + (1L << 32));
        System.out.println("Integer.bitCount(44) = " + Integer.bitCount(44)
            + ", Integer.reverse(1) = " + Integer.reverse(1));
    }

    static boolean isPow2(int n) {
        return n > 0 && (n & (n - 1)) == 0;           // the n > 0 check matters: 0 and MIN_VALUE
    }
}
```

Run it TERMINAL

```
$ java Main.java
popcount(44) via Kernighan:
n=101100  n-1=101011  n&(n-1)=101000
n=101000  n-1=100111  n&(n-1)=100000
n=100000  n-1=011111  n&(n-1)=000000
count = 3
a=1100 b=1010
a & b = 1000, a | b = 1110, a ^ b = 0110, ~a & 15 = 0011
check bit 2: 1, set bit 0: 101101
clear bit 3: 100100, toggle bit 5: 001100
lowest set bit of 44: 4  (-44 = 11111111111111111111111111010100)
isPowerOfTwo: 16 -> true, 18 -> false, 0 -> false, MIN_VALUE -> false
-8 >> 1 = -4, -8 >>> 1 = 2147483644
1 << 31 = -2147483648, 1 << 32 = 1, 1L << 32 = 4294967296
Integer.bitCount(44) = 3, Integer.reverse(1) = -2147483648
```

Observations. `44 & -44 = 4`: in two's complement `-44` has the same lowest set bit as `44` and the complement of every higher bit, so the conjunction retains only that bit. `-8 >> 1` is `-4` (sign copied), whereas `-8 >>> 1` is `2147483644` (zero fill, bits treated as unsigned). `1 << 32` equals `1`: Java uses only the low 5 bits of the shift count for `int` (low 6 for

`long`), so a shift by 32 equals a shift by 0; `1L << 32` is required for wide masks. The test `n > 0` in `isPow2` is necessary: zero and `Integer.MIN_VALUE` both satisfy `n & (n - 1) == 0` without being powers of two.

The second program implements the standard set of problems.

Main.java JAVA

```java
import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

public class Main {
    // Single number: every value appears twice except one. x ^ x = 0 and x ^ 0 = x.
    static int singleNumber(int[] nums) {
        int acc = 0;
        for (int x : nums) acc ^= x;
        return acc;
    }

    // Single Number III: two values appear once. XOR of everything = u ^ v. Any set bit of
    // that XOR separates u from v, so split the array by that bit and XOR each half.
    static int[] singleNumberIII(int[] nums) {
        int xor = 0;
        for (int x : nums) xor ^= x;
        int lowBit = xor & -xor;
        int u = 0, v = 0;
        for (int x : nums) {
            if ((x & lowBit) == 0) u ^= x;
            else v ^= x;
        }
        return new int[]{u, v};
    }

    // Counting bits: bits[i] = bits[i / 2] + (i & 1). Shifting right drops the last bit.
    static int[] countBits(int n) {
        int[] bits = new int[n + 1];
        for (int i = 1; i <= n; i++) bits[i] = bits[i >> 1] + (i & 1);
        return bits;
    }

    // Missing number in 0..n: XOR all indices and all values; pairs cancel, the gap remains.
    static int missingNumber(int[] nums) {
        int acc = nums.length;
        for (int i = 0; i < nums.length; i++) acc ^= i ^ nums[i];
        return acc;
    }

    // Reverse the 32 bits of an int: shift the answer left, feed it the lowest bit of n.
    static int reverseBits(int n) {
        int result = 0;
        for (int i = 0; i < 32; i++) {
            result = (result << 1) | (n & 1);
            n >>>= 1;                                   // unsigned shift, or the sign bit smears
        }
        return result;
    }

    // All subsets: each mask from 0 to 2^n - 1 is one subset; bit i says "take nums[i]".
    static List<List<Integer>> subsets(int[] nums) {
        List<List<Integer>> all = new ArrayList<>();
        for (int mask = 0; mask < (1 << nums.length); mask++) {
            List<Integer> pick = new ArrayList<>();
            for (int i = 0; i < nums.length; i++) {
                if ((mask & (1 << i)) != 0) pick.add(nums[i]);
            }
            all.add(pick);
        }
        return all;
    }

    // Add without +: XOR is the sum ignoring carries, (a & b) << 1 is the carries.
    static int add(int a, int b) {
        while (b != 0) {
            int carry = (a & b) << 1;
            a ^= b;
            b = carry;
        }
```

Sreeram

Main.java (continued) JAVA

```
        return a;
    }

    public static void main(String[] args) {
        System.out.println("singleNumber [4,1,2,1,2] = " + singleNumber(new int[]{4, 1, 2, 1, 2}));
        System.out.println("singleNumberIII [1,2,1,3,2,5] = "
            + Arrays.toString(singleNumberIII(new int[]{1, 2, 1, 3, 2, 5})));
        System.out.println("countBits(8) = " + Arrays.toString(countBits(8)));
        System.out.println("missingNumber [3,0,1] = " + missingNumber(new int[]{3, 0, 1}));
        System.out.println("reverseBits(43261596) = " + reverseBits(43261596)
            + " (expected 964176192)");
        System.out.println("subsets [1,2,3] = " + subsets(new int[]{1, 2, 3}));
        System.out.println("add(5, 7) = " + add(5, 7) + ", add(-3, 5) = " + add(-3, 5));
    }
}
```

Run it TERMINAL

```
$ java Main.java
singleNumber [4,1,2,1,2] = 4
singleNumberIII [1,2,1,3,2,5] = [5, 3]
countBits(8) = [0, 1, 1, 2, 1, 2, 2, 3, 1]
missingNumber [3,0,1] = 2
reverseBits(43261596) = 964176192 (expected 964176192)
subsets [1,2,3] = [[], [1], [2], [1, 2], [3], [1, 3], [2, 3], [1, 2, 3]]
add(5, 7) = 12, add(-3, 5) = 2
```

Observations. `singleNumberIII`: the XOR of the whole array equals `u ^ v` for the two unique values. Since they differ, the XOR has a set bit where they disagree; `xor & -xor` selects one such bit, and partitioning the array by it separates `u` from `v`, with every other value paired within one group. `missingNumber` XORs each index with its value; present values cancel against their indices, leaving the missing one. `reverseBits` requires `>>>=` on the input: `>>=` would propagate sign bits for negative values. The loop over masks from 0 to `2^n - 1` enumerates all `2^n` subsets without recursion. `add` combines XOR (sum without carry) with a shifted AND (carries), and is valid for negative operands because Java integers wrap in two's complement.

6.7.4 Complexity and mistakes

Single-number, power-of-two and set-bit-count problems require O(n) or O(1) time and O(1) space (Kernighan's method executes once per set bit, at most 32 times). Subset enumeration by mask requires O(2^n × n), since each mask scans its bits. The library methods `Integer.bitCount`, `Integer.highestOneBit`, `Integer.numberOfTrailingZeros`, `Integer.reverse` and `Long.bitCount` are intrinsified (compiled to single instructions) on most platforms; the manual versions remain necessary when an exercise prohibits library calls.

- **Precedence.** `==` and `!=` bind tighter than `&`, `^`, `|`. `n & 1 == 0` does not compile (it tries `n & (1 == 0)`); write `(n & 1) == 0`. Shifts bind tighter than `&`, so `x & 1 << k` works but parentheses make intent clear.
- `>>` **versus** `>>>`**.** `>>>` is required whenever the value is treated as unsigned (bit reversal, hash mixing, extraction of high bits).
- **Shift counts.** A shift by 32 or more wraps for `int`; `long` and `1L <<` are required for wider masks.
- **Negative numbers and** `%`**.** `-7 % 2` is `-1` in Java; `(n & 1) == 1` is the safe odd test for any sign.
- `Integer.MIN_VALUE` **edge cases.** `Math.abs(Integer.MIN_VALUE)` is still negative, and `-Integer.MIN_VALUE` equals itself.
- **Unsigned input.** Some problems pass an unsigned 32-bit value as a Java `int`; the bit pattern is what matters, so use `>>>` and `Integer.toBinaryString`, and do not compare with `<`.
- **Assuming that XOR cancellation applies when duplicates appear three times.** They do not; Single Number II needs bit counting (count each bit position modulo 3).

WATCH OUT

The condition `if (n & 1 == 0)` does not compile in Java, because it parses as `n & (1 == 0)`. In C-like languages the same expression compiles and yields an incorrect result. Every bit test should be parenthesised.

6.7.5 Questions to expect

1. "Every number appears twice except one; find it in O(n) time and O(1) space." Clue: pairs cancel. Approach: XOR everything. Follow-ups: two unique numbers (split by a set bit of the XOR), or every number appears three times except one (sum each bit position modulo 3).
2. "Count the 1 bits in an integer." Clue: popcount. Approach: Kernighan loop; or `Integer.bitCount`. Follow-up: count for every number 0..n (the `bits[i >> 1] + (i & 1)` recurrence is a DP in O(n)).
3. "Is a number a power of two?" Clue: exactly one set bit. Approach: `n > 0 && (n & (n - 1)) == 0`. Follow-up: power of four (add a mask check `n & 0x55555555`).
4. "Find the missing number from 0..n in an array of n numbers." Clue: values are a permutation minus one. Approach: XOR with indices, or sum formula `n(n+1)/2` (watch overflow) minus the array sum.
5. "Generate all subsets of a set of up to 20 elements." Clue: "all subsets", small n. Approach: mask from 0 to `(1 << n) - 1`, or backtracking. Follow-up: only subsets of size k (check `Integer.bitCount(mask) == k`).
6. "Add two integers without `+`." Clue: forbidden operators. Approach: XOR for the sum, `(a & b) << 1` for the carry, repeat until the carry is 0.
7. "Reverse the bits of a 32-bit integer." Clue: bit order. Approach: loop 32 times shifting; follow-up: how to speed it up if called many times (divide-and-conquer swapping halves, or cache bytes in a lookup table).
8. "Hamming distance between two integers" or "AND of all numbers in a range [m, n]". Clue: compare bits. Approach: `Integer.bitCount(x ^ y)`; the range AND is the common binary prefix of `m` and `n`, found by shifting both right until equal.

INTERVIEW ANGLE

"How are two unique numbers found when all others appear exactly twice?" All numbers are XORed. Pairs cancel, so the result `x` equals `u ^ v`. Since `u != v`, `x` has at least one set bit; the lowest is isolated with `x & -x`. Each number either has that bit or lacks it, which partitions the array into two groups; `u` and `v` fall into different groups, and the two copies of every other value fall into the same group. XORing each group separately yields `u` and `v`. Time is O(n) and space is O(1). If `x` equals `Integer.MIN_VALUE`, the lowest set bit is the sign bit and `x & -x` remains correct.

6.7.6 LeetCode practice

#	Title	Level	Key idea
136	Single Number	Easy	XOR the whole array; pairs cancel.
191	Number of 1 Bits	Easy	Kernighan: n & (n - 1) removes the lowest set bit.
231	Power of Two	Easy	n > 0 and n & (n - 1) == 0.
268	Missing Number	Easy	XOR indices with values, or arithmetic sum.
338	Counting Bits	Easy	bits[i] = bits[i >> 1] + (i & 1).
461	Hamming Distance	Easy	Count set bits of x ^ y.
190	Reverse Bits	Easy	Shift the result left, feed it n's lowest bit, use >>> on n.
371	Sum of Two Integers	Medium	XOR is the sum without carry; (a & b) << 1 is the carry.
78	Subsets	Medium	Each mask from 0 to 2^n - 1 is one subset (or use backtracking).
260	Single Number III	Medium	Split by one set bit of the total XOR.
137	Single Number II	Medium	Count each bit position modulo 3, or use two state variables.
201	Bitwise AND of Numbers Range	Medium	The answer is the common binary prefix of the two endpoints.

LeetCode problems for this pattern.

sreeram

KEY POINTS

Bit techniques replace extra memory with arithmetic on the bit pattern.

- XOR cancels pairs; `n & (n - 1)` clears the lowest set bit; `n & -n` isolates it.
- A mask from 0 to `2^n - 1` enumerates every subset when n is small.
- `>>>` gives unsigned shifts, `1L <<` gives wide masks, and every bit test is parenthesised.
- `int` edge cases include `Integer.MIN_VALUE`, shift counts of 32 or more, and negative `%`.

sreeram

PART 7

Revision and Interview Playbook

Everything from the earlier Parts, compressed into tables, questions, three full mock interviews and a study plan.

In this part

- **7.1** Pattern recognition cheat sheet
- **7.2** Mixed interview questions
- **7.3** Mock interview walkthroughs
- **7.4** Complexity and Java cheat sheet
- **7.5** A 14-week study plan
- **7.6** Final checklist and how to practise on LeetCode

7.1 Pattern recognition cheat sheet

Most interview problems are instances of a small set of established techniques. Effective preparation consists of reading the statement, identifying two or three clue phrases, and naming the applicable pattern within one to two minutes, rather than memorising individual solutions. This chapter serves as a consolidated reference for revision.

7.1.1 Use of the table

First identify the input type (array, string, tree, graph, grid, stream) and the required output (count, longest, minimum, all possibilities, feasibility). Then locate the clue in the table below. The table lists the usual pattern, the supporting data structure and the target complexity. A clue is a heuristic, not a proof; it must be confirmed on a small example before implementation.

Clue in the statement	Pattern	Data structure	Typical complexity
Sorted array, find a pair or triple with a target	Two pointers	Array, two indices	O(n) after sorting, O(n^2) for 3Sum
Contiguous subarray or substring, longest or shortest with a condition	Sliding window	Array or String plus a count map	O(n)
Sum of a range, many queries, or subarray sum equals k	Prefix sum (+ hash map)	long[] prefix, HashMap	O(n) build, O(1) per query
Count, group, find duplicates, first non-repeating	Hash map or set, frequency counting	HashMap, HashSet, int[26]	O(n) average
Sorted input, or "O(log n)" is demanded	Binary search	Array	O(log n)
Minimise the maximum or maximise the minimum, "least capacity", "smallest speed"	Binary search on the answer	Predicate function plus loop	O(n log range)
Overlapping ranges, meetings, merge, schedule	Sort by start, then sweep	List of int[2], sort	O(n log n)
Largest sum subarray, best day to buy and sell	Kadane or a single greedy scan	A few variables	O(n)
In-place, O(1) extra space, numbers in range 1..n	Cyclic sort, index as hash, swap	The array itself	O(n)
Cycle in a list, middle of a list, k-th from the end	Fast and slow pointers	ListNode	O(n), O(1) space
Reverse a list or part of it, merge sorted lists	Dummy node and pointer rewiring	ListNode	O(n)
Matching brackets, undo, nested structure, evaluate expression	Stack	ArrayDeque used as a stack	O(n)

sreeram

Clue in the statement	Pattern	Data structure	Typical complexity
Next greater or smaller element, span, histogram, trapped water	Monotonic stack	ArrayDeque of indices	O(n)
Maximum or minimum of every window of size k	Monotonic deque	ArrayDeque of indices	O(n)
Top k, k-th largest or smallest, merge k sorted, running median	Heap	PriorityQueue	O(n log k)
All subsets, permutations, combinations, every valid arrangement	Backtracking	Recursion plus a path list	O(2^n) or O(n!)
Tree: depth, path, balanced, same, symmetric, LCA	DFS (recursion returning a value)	TreeNode	O(n), O(h) stack
Tree: by level, right view, shortest depth	BFS with a queue	ArrayDeque	O(n), O(width)
Binary search tree property, k-th smallest, validate	Inorder traversal, bounds	TreeNode	O(n) or O(h)
Prefix of words, autocomplete, dictionary search	Trie	Array or map children	O(L) per word
Grid of 0 and 1, islands, flood fill, shortest steps	BFS or DFS on a grid	Queue, visited array	O(rows x cols)
Dependencies, order of tasks, prerequisites, "can finish all"	Topological sort	Adjacency list, in-degree array	O(V + E)
Connected components, merge groups, redundant edge	Union-Find (DSU)	parent[] and rank[]	O(n alpha(n))
Shortest path with weights	Dijkstra (non-negative), Bellman-Ford (limited stops)	PriorityQueue of pairs	O((V + E) log V)
Count the ways, minimum cost, maximum value, "is it possible" with choices that overlap	Dynamic programming	1D or 2D array, or a map	O(states x transitions)
Pick items to reach a target sum or capacity	Knapsack DP	dp[] over amounts	O(n x target)
Two strings, edit, common subsequence	String DP	2D table	O(n x m)
Locally best choice seems safe, intervals, jumps, scheduling	Greedy (prove with exchange argument)	Sort or heap	O(n log n)
Single number, powers of two, subsets of a small set (n up to 20)	Bit manipulation, bitmask	int or long	O(n) or O(2^n)

Clue phrase in the statement, the pattern it points to, the data structure, and the usual target complexity.

7.1.2 Decision procedure

When no clue phrase applies, the following questions are evaluated in order. They usually identify the pattern within seconds.

1. Is the input sorted, or can it be sorted without losing required information? If so, consider binary search, two pointers or intervals.
2. Are all solutions required, or a single value? Enumeration suggests backtracking; a single value suggests dynamic programming, greedy selection or a data structure.
3. Do sub-problems overlap (the same state is reached by different choices)? If so, memoisation applies, which is dynamic programming.
4. Is there an implicit graph (grid cells, courses, words differing by one letter, accounts sharing an email)? The nodes and edges should be stated explicitly.

5. Do the constraints bound the complexity? n up to 10^5 requires O(n log n) or better; n up to 20 permits exponential time; n up to 500 permits O(n^3).

TIP

When two patterns both apply, select the one whose invariant can be stated in one sentence, for example "the window always contains distinct characters". An invariant that cannot be stated precisely is difficult to maintain correctly.

KEY POINTS

- Identify the input type and the required output first; clue phrases then narrow the pattern to one or two candidates.
- A pattern comes with a data structure and a target complexity; check both against the constraints.
- Confirm the pattern on a small example before coding, and state the invariant in one sentence.
- When no pattern is evident, check: sorted input, enumeration versus single value, overlapping sub-problems, implicit graph.

7.2 Mixed interview questions

This chapter lists 40 questions grouped by the style of company that tends to ask them. The grouping reflects tendencies reported by candidates, not rules; reported questions vary considerably even within one company, and hiring processes change. The groups are intended to balance practice, not to predict an interview. Each question names the pattern and gives a one-line approach; the full treatment appears in the pattern chapters. The approach should be formulated before the answer is read.

7.2.1 Service and campus-placement style

These rounds test fundamentals and code quality. Problems are typically easy to easy-medium, often two or three in 60 minutes.

1. Reverse a string or an array in place. Two pointers from both ends, swap until they meet.
2. Check whether a string is a palindrome, ignoring case and non-alphanumerics. Two pointers, skipping characters that do not count.
3. Find the second largest element in one pass. Track the largest and second largest; decide how to treat duplicates.
4. Two Sum on an unsorted array. Hash map from value to index, check the complement before inserting.
5. Valid Parentheses. Stack of opening brackets; each closing bracket must match the top.
6. Merge two sorted linked lists. Dummy node plus a tail pointer; attach the smaller head each step.
7. Find the missing number in 0..n. Sum formula with long, or XOR of indices and values.
8. Find the first non-repeating character. Count frequencies in int[26] or int[128], then scan the string again.

7.2.2 Amazon style: practical and design-flavoured

Amazon-style rounds are widely reported to favour realistic problems (rankings, grids, caches, logs) and to combine coding with behavioural questions about ownership. Edge cases and scale should be discussed.

1. Number of Islands. BFS or DFS over the grid, marking visited cells; count the starts.
2. Top K Frequent Elements. Frequency map, then a min-heap of size k or bucket sort by frequency.
3. LRU Cache. HashMap plus doubly linked list, or LinkedHashMap with access order; both operations O(1).
4. Rotting Oranges. Multi-source BFS; the answer is the number of BFS levels.
5. Merge Intervals. Sort by start, extend the last interval while they overlap.
6. K Closest Points to Origin. Max-heap of size k on squared distance (no square root needed).
7. Group Anagrams. Key each word by its sorted letters or by a 26-count signature; map key to list.
8. Course Schedule. Directed graph; Kahn's algorithm, and the order is complete only if every node is removed.

sreeram

7.2.3 Google style: algorithmic depth

Reports describe more graph, dynamic programming and binary-search-on-the-answer problems, with follow-ups such as an improved complexity, a streaming variant or a proof of correctness.

1. Word Ladder. Treat words as nodes, one-letter changes as edges; BFS for the shortest chain.
2. Koko Eating Bananas, or any "minimum speed" problem. Binary search on the answer with a monotonic feasibility check.
3. Longest Increasing Subsequence in O(n log n). Keep a tails array and binary search the insertion point.
4. Median of a data stream. Two heaps: a max-heap for the lower half and a min-heap for the upper half.
5. Network Delay Time. Dijkstra from the source; the answer is the largest finishing distance.
6. Edit Distance. 2D DP over prefixes; insert, delete, replace from neighbouring cells.
7. Alien Dictionary. Compare adjacent words to build edges between letters, then topological sort; detect cycles and invalid prefixes.
8. Trapping Rain Water. Two pointers with running left and right maxima, or a monotonic stack.

7.2.4 Meta style: speed on mediums

Reports describe two medium problems in 45 minutes, so speed and defect-free code are important. Arrays, strings, trees and graph traversal dominate.

1. Subarray Sum Equals K. Prefix sums plus a hash map of how often each prefix appeared.
2. Minimum Window Substring. Variable sliding window with need and have counters; shrink while valid.
3. Binary Tree Right Side View. Level-order BFS, keep the last node of each level; or DFS visiting right first.
4. Lowest Common Ancestor of a Binary Tree. Recursive DFS: if both sides return a node, the current node is the answer.
5. Product of Array Except Self. Prefix products going right, suffix products going left, no division.
6. Valid Palindrome II (delete at most one character). Two pointers; on the first mismatch try skipping either side.
7. Add Two Numbers (digits in linked lists). Walk both lists with a carry, build the result behind a dummy node.
8. Random Pick with Weight, or Sparse Matrix Multiplication. Prefix sums with binary search; or loop over non-zero entries only.

7.2.5 Microsoft style: classics and data structures

Reports describe classic data-structure manipulation problems (lists, trees, design) and emphasis on communicating reasoning.

1. Reverse a linked list, iteratively and recursively. Three pointers; or recurse to the end and rewire on the way back.
2. Detect and find the start of a cycle in a linked list. Floyd's slow and fast pointers, then restart one pointer from the head.
3. Validate Binary Search Tree. Inorder strictly increasing, or recursion carrying (low, high) bounds.
4. Serialize and Deserialize a Binary Tree. Preorder with null markers; rebuild with a queue of tokens.
5. Implement a Trie. Node with child links and an end-of-word flag; insert, search, startsWith.
6. Maximum Subarray. Kadane: best ending here = max(x, best ending here + x).
7. Container With Most Water. Two pointers; always move the shorter side, because only that can improve the area.
8. Coin Change. DP over amounts, dp[a] = min over coins of dp[a - c] + 1, with a sentinel for unreachable.

INTERVIEW ANGLE

How should a list like this be practised? Approaches should be concealed, not read. Select eight questions per week from different groups, set a 25-minute timer, and state the pattern, the brute-force method, the improved method and the complexity aloud before coding. Questions that could not be solved are re-solved from scratch after three days.

KEY POINTS

- The forty questions cover most patterns in this book; each should be understood well enough to explain, not only to code.
- Company styles are tendencies derived from candidate reports; practice should span all groups.
- For every question, formulate the sequence: pattern, brute force, improved method, complexity.

sreeram

7.3 Mock interview walkthroughs

Reading solutions is a passive activity. This chapter presents three complete interviews in step form, each with a medium problem, illustrating the sequence of a well-structured session: clarify, examples, brute force, optimise, code, test, complexity. The code in each was executed on JDK 21 and the printed output is real. The candidate lines should be concealed and answered independently first.

7.3.1 Mock 1: Longest Substring Without Repeating Characters (LeetCode 3, Medium)

Interviewer: Given a string, find the length of the longest substring that contains no repeated character.

Step 1: clarify. *Candidate:* "Substring means contiguous, not a subsequence, correct? What is the alphabet: only ASCII, or any Unicode? Can the string be empty? And do you want the length only, or the substring itself?" *Interviewer:* "Contiguous. Assume any char. Empty is possible. Length only."

Step 2: examples. *Candidate:* "For `abcabcbb` the answer is 3 (`abc`). For `bbbbb` it is 1. For `pwwkew` it is 3 (`wke`), and note that `pwke` would be a subsequence, which does not count. For an empty string it is 0."

Step 3: brute force. *Candidate:* "For every start index, extend to the right until I meet a repeat. Tracking seen characters with a boolean array, that is O(n x k) where k is the alphabet size, which is O(n^2) in the worst case for a large alphabet. With n up to about 10^5 that is too slow."

Step 4: optimise. *Candidate:* "Clue: longest contiguous range with a condition, so a sliding window. I keep `left` and `right`. A map stores the last index where each character appeared. When the character at `right` was last seen inside the window, I move `left` to just after that index. The invariant: the window `[left, right]` never has a repeat. Each index enters once, so it is O(n) time, and O(min(n, alphabet)) space."

The essential condition is `prev >= left`: a character last seen outside the window must not move `left` backwards. The dry run on `abcabcbb`, produced by the program below, illustrates this.

right	c	left	window	best	note
0	a	0	a	1	first sight
1	b	0	ab	2	grow
2	c	0	abc	3	grow
3	a	1	bca	3	a last at 0, so left = 1
4	b	2	cab	3	b last at 1, so left = 2
5	c	3	abc	3	c last at 2, so left = 3
6	b	5	cb	3	b last at 4, so left = 5
7	b	7	b	3	b last at 6, so left = 7

Dry run of the window on abcabcbb (real trace).

Step 5: code.

Main.java JAVA

```java
import java.util.HashMap;
import java.util.Map;

public class Main {
    static int bruteForce(String s) {
        int best = 0;
        for (int i = 0; i < s.length(); i++) {
            boolean[] seen = new boolean[128];
            int len = 0;
            for (int j = i; j < s.length() && !seen[s.charAt(j)]; j++) {
                seen[s.charAt(j)] = true;
                len++;
            }
```

Main.java (continued) JAVA

```
            best = Math.max(best, len);
        }
        return best;
    }

    static int lengthOfLongest(String s, boolean trace) {
        Map<Character, Integer> last = new HashMap<>();
        int left = 0, best = 0;
        for (int right = 0; right < s.length(); right++) {
            char c = s.charAt(right);
            Integer prev = last.get(c);
            if (prev != null && prev >= left) {
                left = prev + 1;
            }
            last.put(c, right);
            best = Math.max(best, right - left + 1);
            if (trace) {
                System.out.println("right=" + right + " c=" + c + " left=" + left
                    + " window=" + s.substring(left, right + 1) + " best=" + best);
            }
        }
        return best;
    }

    public static void main(String[] args) {
        lengthOfLongest("abcabcbb", true);
        String[] tests = {"", "a", "bbbbb", "pwwkew", "abba", "tmmzuxt"};
        for (String t : tests) {
            System.out.println("\"" + t + "\" -> " + lengthOfLongest(t, false)
                + " (brute " + bruteForce(t) + ")");
        }
    }
}
```

Step 6: test. *Candidate:* "I will run my edge cases: empty, one character, all the same, and `abba`, which is the case that breaks a solution that forgets the `prev >= left` check."

Run the program TERMINAL

```
$ java Main.java
right=0 c=a left=0 window=a best=1
right=1 c=b left=0 window=ab best=2
right=2 c=c left=0 window=abc best=3
right=3 c=a left=1 window=bca best=3
right=4 c=b left=2 window=cab best=3
right=5 c=c left=3 window=abc best=3
right=6 c=b left=5 window=cb best=3
right=7 c=b left=7 window=b best=3
"" -> 0 (brute 0)
"a" -> 1 (brute 1)
"bbbbb" -> 1 (brute 1)
"pwwkew" -> 3 (brute 3)
"abba" -> 2 (brute 2)
"tmmzuxt" -> 5 (brute 5)
```

Step 7: complexity and follow-ups. *Candidate:* "O(n) time, O(k) space for k distinct characters. If the alphabet is only lowercase letters, an `int[26]` replaces the map and is faster." *Interviewer:* "What if the input is a stream you cannot store?" *Candidate:* "The map holds last indices, so I only need the current window's characters; I would keep a bounded buffer of the window, which can grow up to the alphabet size."

GOTCHA

`Map.get` returns `Integer`, which may be `null`. The statement `int prev = last.get(c)` throws a `NullPointerException` on the first occurrence of a character. The boxed type with a `null` test, or `getOrDefault(c, -1)`, avoids this.

7.3.2 Mock 2: Number of Islands (LeetCode 200, Medium)

Interviewer: You get a grid of characters `'1'` (land) and `'0'` (water). Count the islands. An island is land connected horizontally or vertically.

Step 1: clarify. *Candidate:* "Are diagonals connected? Can I modify the input grid? How large can it be, and can it be empty?" *Interviewer:* "Four directions only. You may modify it. Up to 300 x 300; treat an empty grid as zero."

Step 2: examples. *Candidate:* "The grid `11000 / 11000 / 00100 / 00011` has three islands: the top-left block, the single middle cell, and the bottom-right pair. The grid `101 / 010 / 101` has five single-cell islands, because the diagonal neighbours do not connect."

Step 3: brute force. *Candidate:* "A union of all land cells by comparing every pair is O(L^2) for L land cells, which is wasteful. The grid is already a graph: cells are nodes and neighbouring land cells are edges. Counting islands equals counting connected components."

Step 4: optimise. *Candidate:* "Scan every cell. When I find unvisited land, I count one island and flood the whole island so I never count it again. BFS with a queue avoids deep recursion; a 300 x 300 all-land grid would make recursive DFS 90,000 frames deep, and the default Java stack can overflow. I mark visited by overwriting `'1'` with `'0'`, since modification is allowed. If it were not, I would use a boolean array. Time O(rows x cols), because each cell is enqueued at most once. Space O(min(rows, cols)) for the queue frontier in the typical case, O(rows x cols) in the worst case."

Step 5: code.

Main.java JAVA

```java
import java.util.ArrayDeque;
import java.util.Deque;

public class Main {
    static int countIslands(char[][] grid) {
        if (grid.length == 0) return 0;
        int rows = grid.length, cols = grid[0].length, count = 0;
        int[][] dirs = {{1, 0}, {-1, 0}, {0, 1}, {0, -1}};
        for (int r = 0; r < rows; r++) {
            for (int c = 0; c < cols; c++) {
                if (grid[r][c] != '1') continue;
                count++;
                grid[r][c] = '0';
                Deque<int[]> queue = new ArrayDeque<>();
                queue.add(new int[]{r, c});
                while (!queue.isEmpty()) {
                    int[] cur = queue.poll();
                    for (int[] d : dirs) {
                        int nr = cur[0] + d[0], nc = cur[1] + d[1];
                        if (nr >= 0 && nr < rows && nc >= 0 && nc < cols && grid[nr][nc] == '1') {
                            grid[nr][nc] = '0';
                            queue.add(new int[]{nr, nc});
                        }
                    }
                }
            }
        }
        return count;
    }

    static char[][] parse(String... rows) {
        char[][] g = new char[rows.length][];
        for (int i = 0; i < rows.length; i++) g[i] = rows[i].toCharArray();
        return g;
    }

    public static void main(String[] args) {
        System.out.println(countIslands(parse("11000", "11000", "00100", "00011")));
        System.out.println(countIslands(parse("111", "010", "111")));
        System.out.println(countIslands(parse("0")));
        System.out.println(countIslands(parse("101", "010", "101")));
        System.out.println(countIslands(new char[0][0]));
    }
}
```

Step 6: test. *Candidate:* "Expected: 3, 1, 0, 5, 0. The second grid is a C-shaped ring of land with a water cell in the middle, so it is one island."

sriram

Run the program — TERMINAL

```
$ java Main.java
3
1
0
5
0
```

Step 7: complexity and follow-ups. *Candidate:* "O(rows x cols) time. The key correctness point is marking a cell visited when I enqueue it, not when I dequeue it; otherwise the same cell can be queued several times." *Interviewer:* "How would you handle land being added over time, with a count after each addition?" *Candidate:* "That is the classic Union-Find use: each new land cell merges with its land neighbours, and the component count goes up by one and down by one per successful union."

WATCH OUT

Marking a cell as visited on dequeue instead of on enqueue still yields the correct count but can place one cell in the queue up to four times, which degrades speed and increases memory use on large grids.

7.3.3 Mock 3: Coin Change (LeetCode 322, Medium)

Interviewer: Given coin denominations and a target amount, return the fewest coins that make the amount exactly, or -1 if impossible. You have unlimited coins of each kind.

Step 1: clarify. *Candidate:* "Are all denominations positive? Can the amount be zero? Can the amount be large?" *Interviewer:* "Positive. Zero is possible and needs zero coins. Amount up to 10,000."

Step 2: examples. *Candidate:* "Coins `{1, 2, 5}`, amount 11: answer 3 (5 + 5 + 1). Coins `{2}`, amount 3: impossible, so -1. Coins `{1, 3, 4}`, amount 6: the answer is 2 (3 + 3), but a greedy approach that always takes the largest coin picks 4 + 1 + 1 and gets 3. So greedy is wrong here."

Step 3: brute force. *Candidate:* "Recursion: `f(amount) = 1 + min f(amount - c)` over all coins. It is correct but recomputes the same amounts again and again, which is exponential. The sub-problems overlap, so this is dynamic programming."

Step 4: optimise. *Candidate:* "State: `dp[a]` is the fewest coins for amount `a`. Base: `dp[0] = 0`. Transition: `dp[a] = min(dp[a - c] + 1)` for every coin `c <= a`. I fill the array with `amount + 1` as an 'infinity' marker, because no valid answer can use more than `amount` coins (all coins are at least 1). That avoids `Integer.MAX_VALUE + 1` overflow. Time O(amount x coins), space O(amount)." The table for coins `{1, 2, 5}` and amount 11 is just the array printed by the program, with the index as the amount.

0	1	2	3	4	5	6	7	8	9	10	11
0	1	1	2	2	1	2	2	3	3	2	3

Real dp array for coins 1, 2, 5 (row 1 = amount, row 2 = fewest coins).

Read one cell: `dp[7] = 2` because `dp[7-5] + 1 = dp[2] + 1 = 2` (5 + 2). The cell `dp[11] = 3` comes from `dp[10] + 1` where `dp[10] = 2` (5 + 5).

Step 5: code.

Main.java — JAVA

```
import java.util.Arrays;

public class Main {
    static int brute(int[] coins, int amount) {
        if (amount == 0) return 0;
        if (amount < 0) return -1;
        int best = Integer.MAX_VALUE;
        for (int c : coins) {
            int sub = brute(coins, amount - c);
            if (sub >= 0) best = Math.min(best, sub + 1);
        }
        return best == Integer.MAX_VALUE ? -1 : best;
    }

    static int coinChange(int[] coins, int amount, boolean trace) {
        int[] dp = new int[amount + 1];
        Arrays.fill(dp, amount + 1);
```

Main.java (continued) JAVA

```java
        dp[0] = 0;
        for (int a = 1; a <= amount; a++) {
            for (int c : coins) {
                if (c <= a) dp[a] = Math.min(dp[a], dp[a - c] + 1);
            }
        }
        if (trace) System.out.println(Arrays.toString(dp));
        return dp[amount] > amount ? -1 : dp[amount];
    }

    public static void main(String[] args) {
        System.out.println(coinChange(new int[]{1, 2, 5}, 11, true));
        System.out.println(coinChange(new int[]{2}, 3, false));
        System.out.println(coinChange(new int[]{1}, 0, false));
        System.out.println(coinChange(new int[]{186, 419, 83, 408}, 6249, false));
        System.out.println(brute(new int[]{1, 2, 5}, 11));
        System.out.println(coinChange(new int[]{1, 3, 4}, 6, false));
    }
}
```

Step 6: test. *Candidate:* "I expect the dp array above and then 3, -1, 0, a large case, 3 from the brute force as a cross-check, and 2 for the greedy trap."

Run the program TERMINAL

```
$ java Main.java
[0, 1, 1, 2, 2, 1, 2, 2, 3, 3, 2, 3]
3
-1
0
20
3
2
```

Step 7: complexity and follow-ups. *Candidate:* "O(amount x number of coins) time and O(amount) space. The brute force is exponential in the amount." *Interviewer:* "How many ways are there to make the amount?" *Candidate:* "That is Coin Change II, a different DP: loop over coins on the outside and amounts on the inside so that each combination is counted once, not once per ordering."

INTERVIEW ANGLE

What do interviewers evaluate in these sessions? Typically four criteria: problem solving (a correct and efficient idea), coding (clean, correct, idiomatic), verification (testing the candidate's own code unprompted) and communication (reasoning aloud and responding to hints). A correct solution with no testing and no explanation often scores lower than a slightly slower solution that is explained and verified.

KEY POINTS

- Follow the same seven steps every time: clarify, examples, brute force, optimise, code, test, complexity.
- State the invariant or the DP state in one sentence before writing code.
- Run edge cases unprompted (empty, single element, all equal, and a discriminating case such as `abba`).
- Prepare for follow-ups: streaming input, sorted input, counting the ways, incremental updates.

7.4 Complexity and Java cheat sheet

This chapter is reference material: costs that should be known without derivation. Unless stated otherwise, the figures describe standard behaviour of the JDK collections and the usual textbook algorithms; the Javadoc of each class states the exact guarantees.

7.4.1 Sorting algorithms

Algorithm	Best	Average	Worst	Extra space	Stable? and notes
Bubble, insertion sort	O(n)	O(n^2)	O(n^2)	O(1)	Stable. Insertion sort is fast on tiny or nearly sorted input
Selection sort	O(n^2)	O(n^2)	O(n^2)	O(1)	Not stable; does only O(n) swaps
Merge sort	O(n log n)	O(n log n)	O(n log n)	O(n)	Stable; good for linked lists and external sorting
Quick sort	O(n log n)	O(n log n)	O(n^2)	O(log n) stack	Not stable; bad pivots give the worst case
Heap sort	O(n log n)	O(n log n)	O(n log n)	O(1)	Not stable; poor cache behaviour
Counting sort	O(n + k)	O(n + k)	O(n + k)	O(k)	Stable; for small integer ranges k
Radix sort	O(d(n + k))	O(d(n + k))	O(d(n + k))	O(n + k)	Stable; d digits

Sorting algorithms at a glance.

In Java, `Arrays.sort(int[])` and the other primitive overloads use a dual-pivot quicksort variant (sorting stability is meaningless for primitives). `Arrays.sort(T[])`, `Collections.sort` and `List.sort` use TimSort, a stable merge-sort hybrid that is O(n) on already sorted input and O(n log n) otherwise. Because the object sort is stable, sorting by one key and then by another preserves the earlier order among ties.

GOTCHA
A comparator such as `(a, b) -> a - b` overflows when the values have opposite signs and large magnitude. `Integer.compare(a, b)` is correct; `Long.compare` applies to `long` values.

7.4.2 Collections cost table

Structure	Access / get	Insert	Delete	Search / contains
`ArrayList`	O(1)	O(1) amortised at end, O(n) in middle	O(n) except last	O(n)
`LinkedList`	O(n)	O(1) at ends or at a held node	O(1) at ends	O(n)
`ArrayDeque`	O(1) at ends	O(1) amortised at ends	O(1) at ends	O(n)
`HashMap, HashSet`	O(1) average	O(1) average	O(1) average	O(1) average
`TreeMap, TreeSet`	O(log n)	O(log n)	O(log n)	O(log n)
`PriorityQueue`	peek O(1)	offer O(log n)	poll O(log n)	remove(Object) and contains O(n)

Typical costs of the Java collection operations (n elements).

Points that are commonly stated in interviews:

- A `HashMap` is O(1) on average, but a poor `hashCode` collapses a bucket. Since Java 8 a large bucket becomes a balanced tree, so the worst case per operation is O(log n) for keys that are comparable.
- `PriorityQueue` is a binary min-heap. Building it from n elements in one go (the constructor taking a collection) is O(n); adding n elements one by one is O(n log n). Iterating a `PriorityQueue` does not give sorted order.
- Use `ArrayDeque` for both stacks and queues; `Stack` and `LinkedList` are legacy choices with extra overhead.
- `TreeMap` gives `floorKey`, `ceilingKey`, `firstKey` and range views in O(log n), which a hash map cannot do.

sreeram

7.4.3 Common recurrences

The running time of a recursive algorithm is analysed with a recurrence. The following recurrences cover nearly every interview problem.

Recurrence	Result	Example
`T(n) = T(n/2) + O(1)`	`O(log n)`	Binary search
`T(n) = T(n/2) + O(n)`	`O(n)`	Quickselect on average, geometric sum
`T(n) = 2T(n/2) + O(1)`	`O(n)`	Visiting every node of a balanced tree
`T(n) = 2T(n/2) + O(n)`	`O(n log n)`	Merge sort, quicksort with good pivots
`T(n) = T(n-1) + O(1)`	`O(n)`	Linear recursion, linked list recursion
`T(n) = T(n-1) + O(n)`	`O(n^2)`	Quicksort with worst pivots, selection sort
`T(n) = 2T(n-1) + O(1)`	`O(2^n)`	All subsets, naive recursion with two branches
`T(n) = T(n-1) + T(n-2)`	`O(phi^n), about 1.62^n`	Naive Fibonacci
`T(n) = n x T(n-1)`	`O(n!)`	All permutations

Recurrences and what they solve to.

7.4.4 Other numbers worth knowing

- Reading constraints: n up to 10 allows O(n!); up to 20 allows O(2^n); up to 500 allows O(n^3); up to 5,000 allows O(n^2); up to 10^5 or 10^6 needs O(n log n) or O(n); above 10^8 needs O(log n) or a formula.
- `int` holds up to 2,147,483,647 (about 2.1 x 10^9). Sums of 10^5 values of 10^5 each overflow `int`; use `long`.
- Strings: concatenation in a loop is O(n^2) overall; use `StringBuilder`. `substring` copies, so it costs O(length).
- Recursion depth: the default Java thread stack handles roughly ten thousand frames of a simple method (the exact number depends on frame size); a skewed tree of 10^5 nodes can overflow it, so prefer an explicit stack or BFS when depth can be large.
- Space of recursion counts: DFS on a tree is O(height), which is O(log n) when balanced and O(n) when skewed.

KEY POINTS

- The sorting table, the collections table and the nine recurrences answer most complexity questions directly and should be memorised.
- Use `Integer.compare`, not subtraction, in comparators.
- Account for hidden costs: `substring`, `String` concatenation, `PriorityQueue.remove(Object)`, `LinkedList.get(i)`.

7.5 A 14-week study plan

This plan assumes about two hours per day, six days per week, with one lighter day for rest or review. It maps weeks to the Parts and chapters of this book by name. The problem counts are targets for solved problems from the practice tables, not quotas to be met at the expense of understanding. If time is limited, reduce volume; the revision weeks must be retained.

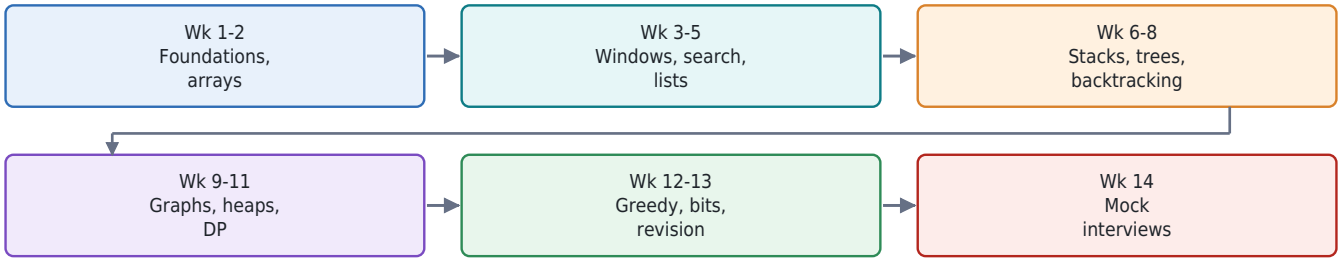


Figure 7.1. The 14-week route from foundations through patterns to mock interviews; revision weeks are deliberate pauses, not optional extras.

7.5.1 Daily rhythm

A two-hour session is divided into three blocks.

- 20 minutes: revise the previous day's problem or recall one pattern template from memory.
- 80 minutes: study the chapter or solve two or three new problems (read the pattern chapter early in the week, then devote the remaining days to problems).
- 20 minutes: record mistakes in the mistake log (see the final chapter of this Part).

For each problem, set a timer: 20 minutes of independent attempt for an easy problem, 35 for a medium. If no solution is found, read the idea only (not the code), close it, and implement it independently. Re-solve the problem after three days.

7.5.2 The plan

Week	Chapters (by name)	Focus	Problems
1	Foundations: how interviews work, complexity analysis, the dry-run method, the Java toolkit, core data structures	Learn the method, write every toolkit snippet yourself	10 easy
2	Arrays and Strings Patterns: prefix sum and difference array; two pointers	Templates for both; sorted-array pairs, 3Sum	14
3	Arrays and Strings Patterns: sliding window (fixed and variable); hash map and frequency counting	Window invariants; anagram and frequency problems	14
4	Arrays and Strings Patterns: binary search; binary search on the answer	Lower and upper bound templates; predicate thinking	14
5	Arrays and Strings Patterns: merge intervals and sorting; Kadane and in-place tricks; matrix and string patterns	Sweep line, greedy scans, Dutch national flag, spiral	14
6	Revision week 1: re-solve the weakest ten problems from weeks 2 to 5; first timed mock	No new patterns; fix gaps; mock Meta-style pair	10 re-solves, 2 mocks
7	Linked Lists, Stacks and Queues: reversal and dummy node; fast and slow pointers; stack; monotonic stack; queue, deque and design	Pointer drawing; monotonic invariants	16
8	Recursion, Backtracking and Trees: recursion and divide and conquer; subsets, permutations, combinations; grids and constraints	Choose, explore, unchoose	12
9	Recursion, Backtracking and Trees: binary tree DFS; BFS and level order; BST and construction; tries	Recursion that returns values	16
10	Heaps and Graphs: heaps and top-K; graph representation, DFS and BFS; topological sort	Grid and graph traversal	14
11	Heaps and Graphs: Union-Find; shortest paths. Revision of weeks 7 to 10	DSU and Dijkstra templates	12
12	Dynamic Programming, Greedy and Bits: DP method; grids and paths; knapsack	State, transition, base case	16
13	Dynamic Programming, Greedy and Bits: strings and sequences; intervals, trees and states; greedy; bit manipulation	Hard DP is optional; greedy proofs	14
14	Revision and Interview Playbook: mixed questions, three mocks, cheat sheets, behavioural prep	Timed random problems, no pattern hints	3 mocks, 10 mixed

Week-by-week plan. Problem targets are solved problems that week.

sreeram

The total is roughly 160 problems plus re-solves, a reasonable base for most software-engineer interviews. With additional time, add the later entries of each practice table and the hard problems. With less time, omit the hard problems first; the revision weeks and the mock interviews must be retained.

7.5.3 Rules for the revision and mock weeks

- Revision means re-solving from a blank editor, not rereading earlier code. Select problems from the mistake log.
- Mock interviews: use a 45-minute timer, select a random medium problem not seen before, speak aloud, and write code in a plain editor without autocomplete. A peer or a self-recorded video are both suitable.
- In week 14, do not select problems by pattern; use random problems so that the pattern must be recognised independently, as in a real interview.

KEY POINTS

- Two hours per day for 14 weeks yields about 160 solved problems with revision included.
- Study the patterns in the order of the book; each builds on the earlier ones.
- Revision weeks and mock interviews convert familiarity into recall under time pressure.

7.6 Final checklist and how to practise on LeetCode

Problem solving yields little benefit when practised poorly. Solving many problems by reading solutions, without revisiting them and without practising verbal explanation, does not reliably produce interview performance. This chapter describes a practice method that builds skill systematically.

7.6.1 Practice as a loop

1. **Attempt** the problem independently with a timer.
2. **Read** only the idea of an editorial or discussion if no progress is made, not the code.
3. **Implement** the solution again from scratch, without copying.
4. **Log** the problem in a notebook or spreadsheet: problem, date, pattern, key insight in one line, and the mistake made.
5. **Revisit** the problem with spaced repetition: after 1, 3, 7 and 21 days. After two clean solutions, retire it.

7.6.2 Tag your mistakes

The mistake log is more informative than the count of solved problems. Each failure is tagged with one of the categories below, and the counts are reviewed monthly; the most frequent tag determines the next practice topic.

Tag	Typical cause and fix
Did not recognise the pattern	Review the cheat sheet; record the clue phrase beside the pattern in the log
Off-by-one or boundary	Dry run the smallest input; write the invariant for loop bounds; prefer half-open ranges
Overflow or type	Use long for sums and products; use Integer.compare; compare boxed values with equals
Missed edge case	Run the checklist: empty, one element, all equal, negatives, duplicates, maximum size
Wrong complexity	Re-read the constraints before coding; state the target in advance
Implementation slip	Slow down; write the skeleton first; name variables clearly
Gave up too soon	Use the 20/35 minute rule; try a smaller example by hand before reading anything

Mistake tags and what to do about each.

sreeram

7.6.3 Contests and notes

Weekly timed contests train speed and composure. They are treated as practice, not as a rating exercise: after each contest, problems that were not completed are solved within a day or two (upsolving) and added to the log. Maintain one page of notes per pattern containing the template, the clue phrases and two or three canonical problems; reread these pages on the day before an interview instead of attempting new problems.

7.6.4 Before the interview: a checklist

- Binary search (lower bound), a sliding window, BFS on a grid, DFS on a tree, Dijkstra, Union-Find and a basic DP can be written from memory without errors.
- The complexity of every Java collection operation in use, and the hidden costs from the cheat sheet, is known.
- At least three full mock interviews have been completed aloud, with a timer and without autocomplete.
- Reasoning can be presented in the order: clarify, examples, brute force, optimise, code, test, complexity.
- A routine for edge cases exists: empty input, one element, duplicates, negatives, maximum values.

7.6.5 On the day and the behavioural round

Preparation on the day includes adequate sleep, a working microphone and a familiar plain editor. During the interview, prolonged silence should be avoided; the current line of reasoning is stated even when it proves to be a dead end. When no progress is made, state this and return to a small example; interviewers usually offer a hint and assess how it is used. After completing the solution, state the test cases and the complexity without waiting to be asked.

Behavioural rounds are assessed on evidence. Five or six accounts drawn from projects, internships, team work or hackathons should be prepared, each structured as Situation, Task, Action, Result (STAR), with one measurable outcome where one exists. Typical themes are a conflict or disagreement, a failure and the lesson drawn, an occasion of taking the lead, a deadline under pressure, and a technical decision that was made and defended. Accounts must be accurate, since follow-up questions readily expose invented details. Every project on the resume should be explainable down to its design choices, because candidates are commonly asked why a particular database, framework or approach was selected.

INTERVIEW ANGLE

Tell me about a bug you introduced and how you fixed it. A strong answer names a concrete project, a specific cause (not "a small mistake"), the method of diagnosis (logs, a failing test, a debugger), the change made, and the measure taken to prevent recurrence (a test, a review check). It demonstrates ownership without attributing blame to others.

KEY POINTS

- Practise as a loop: attempt, learn the idea, re-implement, log, revisit with spaced repetition.
- Tag mistakes; the most frequent tag identifies the next practice topic.
- Prepare the interview routine and five or six accurate STAR accounts with the same rigour as the algorithms.
- Consistent two-hour sessions with revision outperform occasional long sessions.